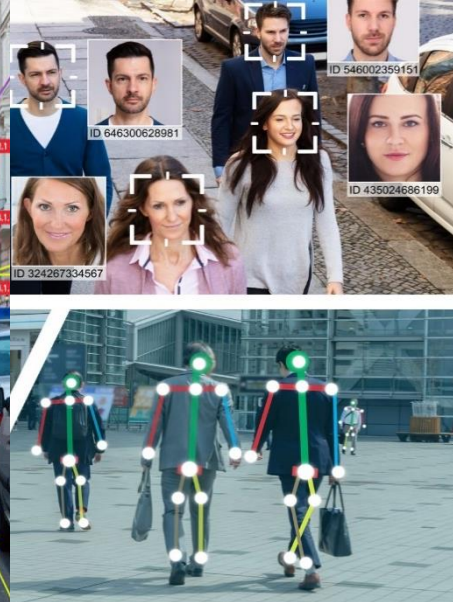
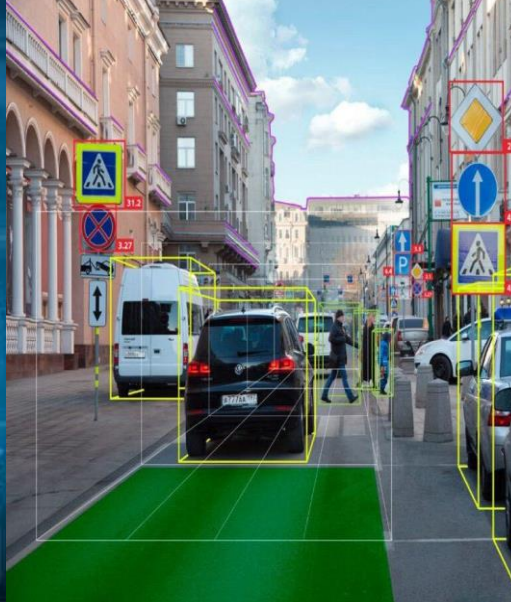




Computer vision is the use of digital processing and intelligent algorithms to interpret meaning from images or video

Knowledge requirement

- **Signal and image processing**
 - Image representation and filtering, colorimetric representation, etc.
- **Programming experience**
 - C++, OpenCV, Phyton, Matlab (curves visualization), etc.
 - IA basic tools
- **Linear algebra**
- **Probability basis**

- **Introduction to Computer Vision**
Motivation, applications and challenges
- **Computer Vision processing techniques**
 - Object characterization
 - Object detection,
 - Object tracking
 - Object Recognition
 - Motion Analysis
 - 3D reconstruction

Part 1

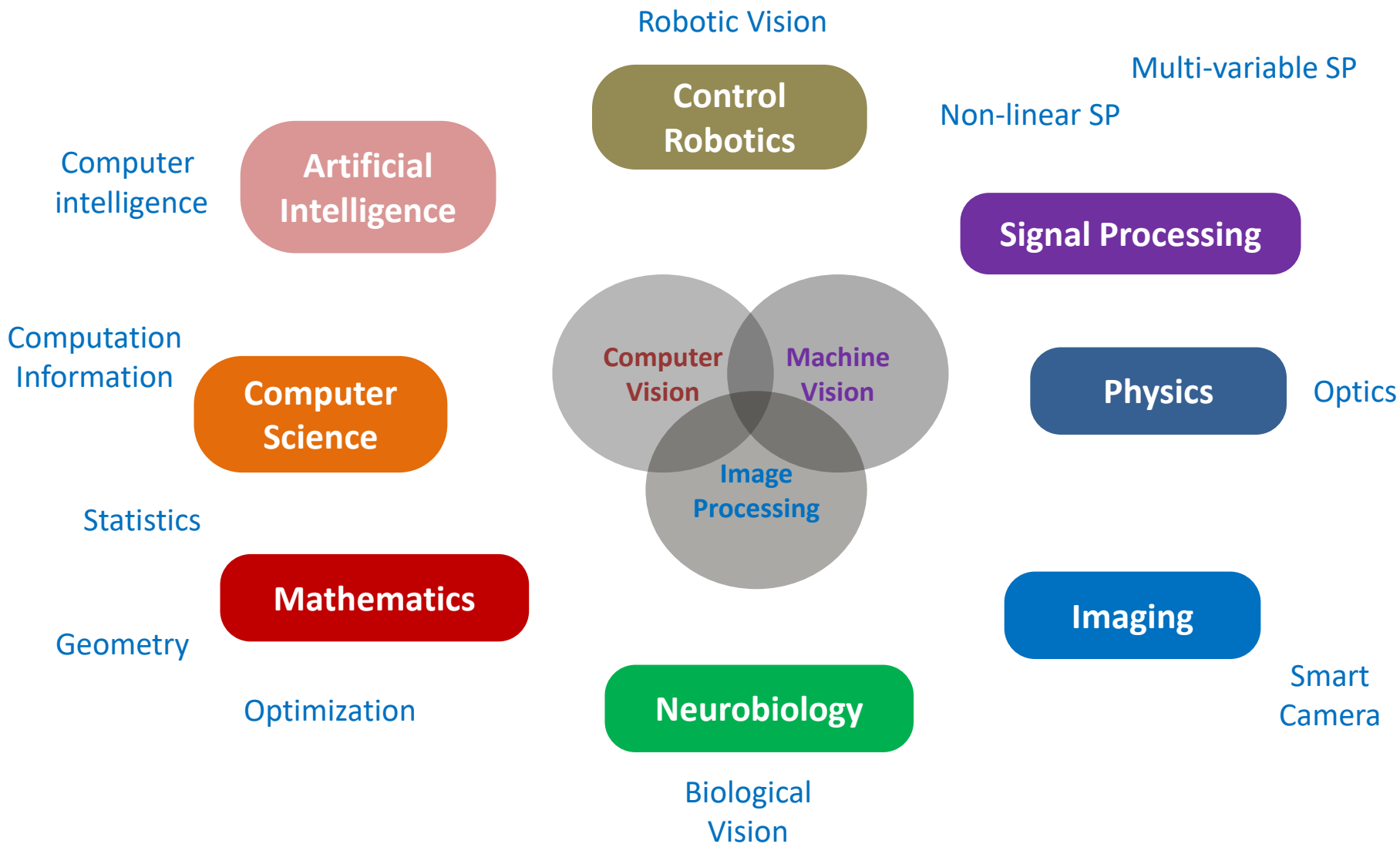
Introduction to Computer Vision

Why Computer Vision ?

Images and video are everywhere!
Images are worth a thousand words

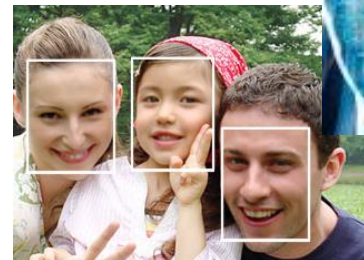
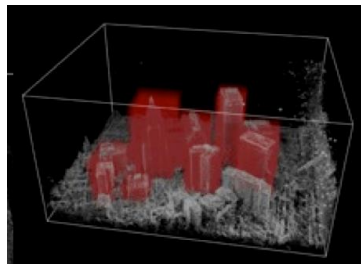
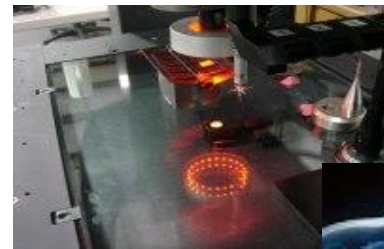
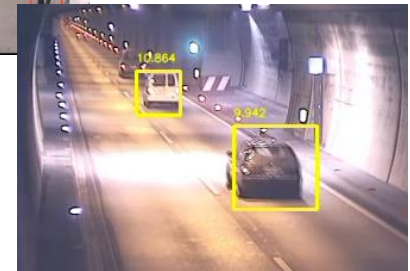
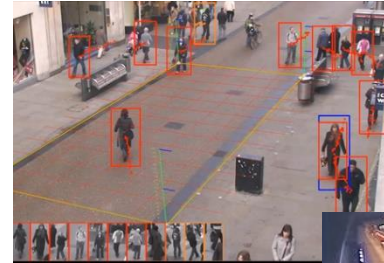


Connection to other techniques



Computer Vision applications classification

- Surveillance and Security
- Automotive & Transport
- Industrial Inspection and Quality Control
- Medical Image Analysis
- Entertainment & Virtual Reality
- Space Applications



Automotive and Transport applications



Driver Assistance



Somnolence Detection

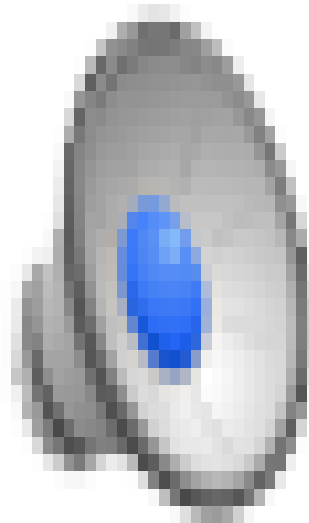


Autonomous Vehicle

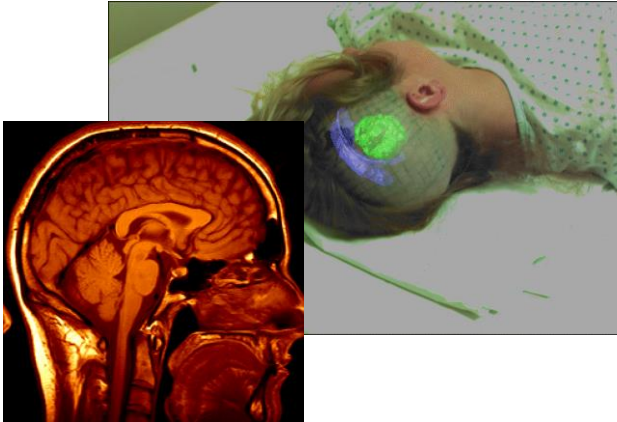


License Plate Recognition

Automotive and Transport applications



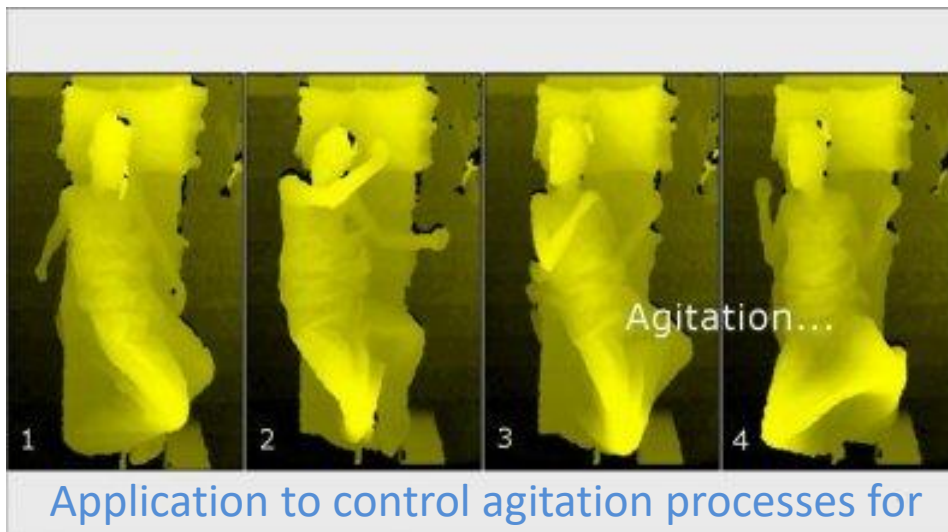
Medical Image Analysis Applications



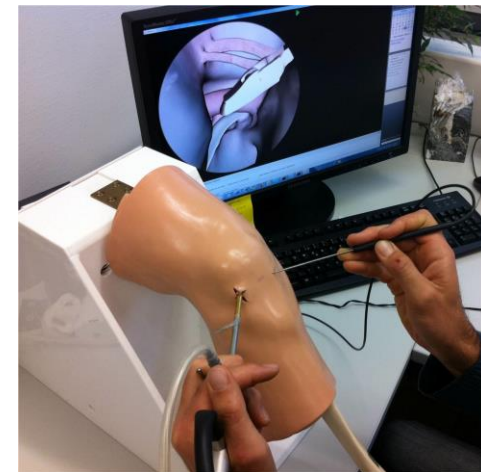
Medical Imaging



Assistive technologies



Application to control agitation processes for ICU's patients.



Medical researches



CHIMAERA

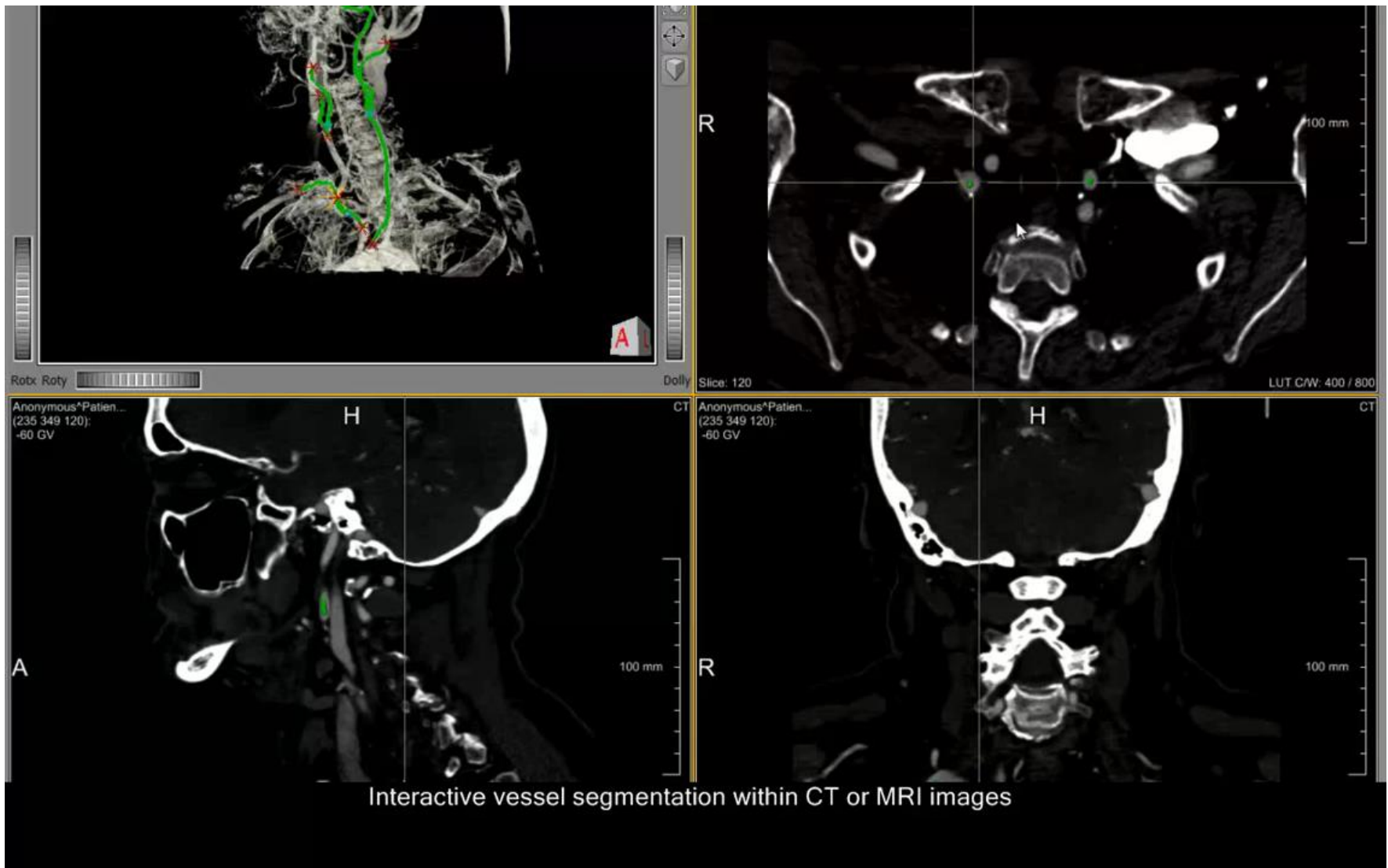
Interactive Segmentation

Examples:

1. MRI Brain Tumor
2. CT Lung Segmentation

© Chimaera GmbH

Medical Image Analysis Applications



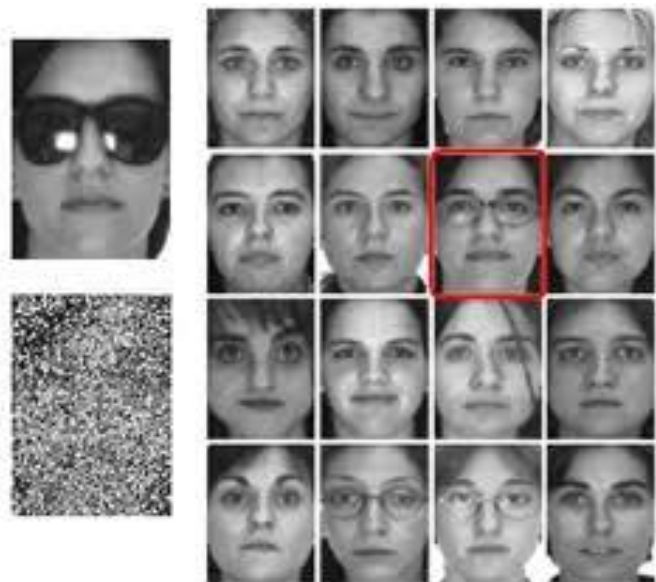
Surveillance and Security Applications



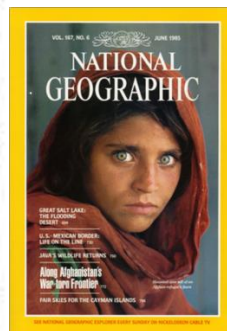
Video Surveillance



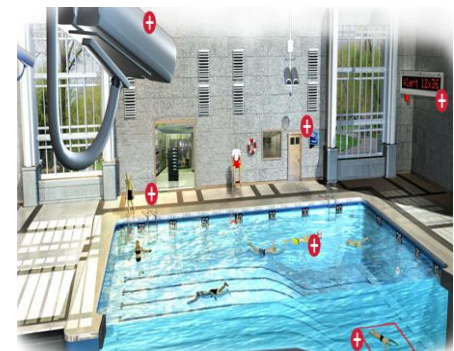
Smoke and Fire detection



Face detection and recognition

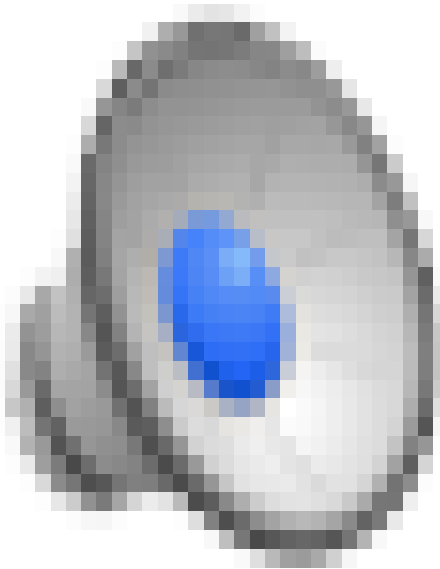


The Afghan Girl story

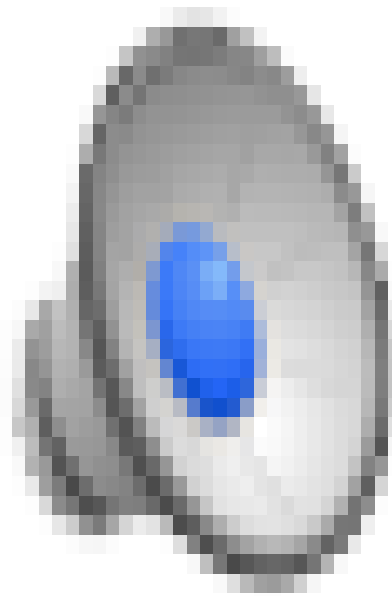


Pool surveillance

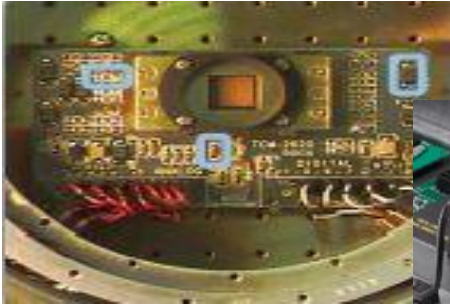
Surveillance and Security Applications



Surveillance and Security Applications



Industrial Inspection and Quality Control



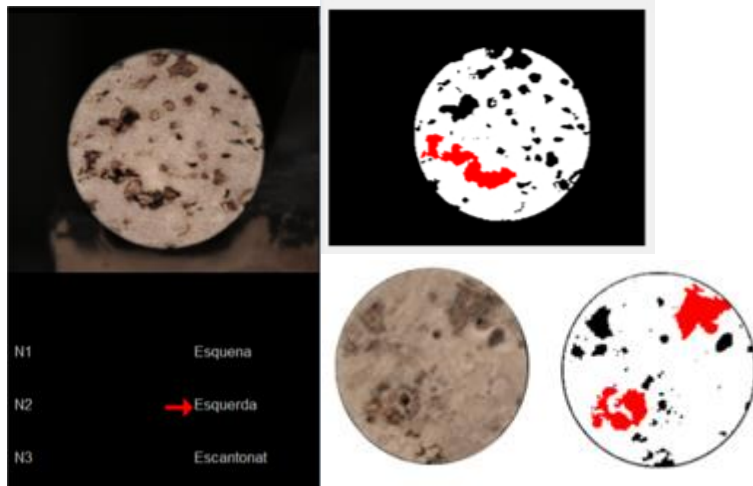
Process Inspection



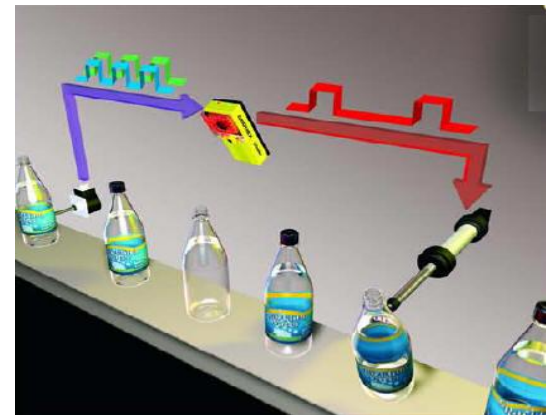
Default Detection



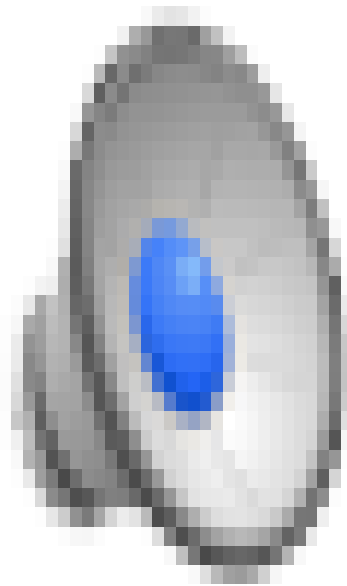
Accurate Metrology



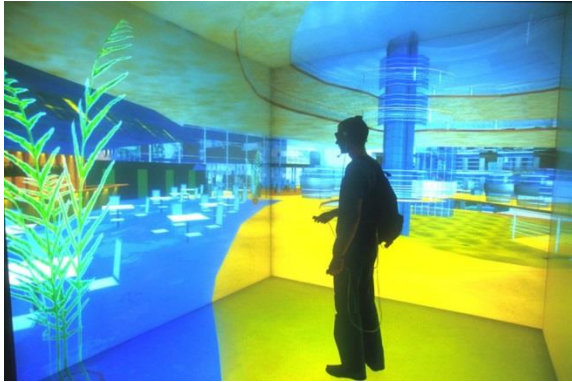
Cork stoppers inspection and classification system



Factory inspection



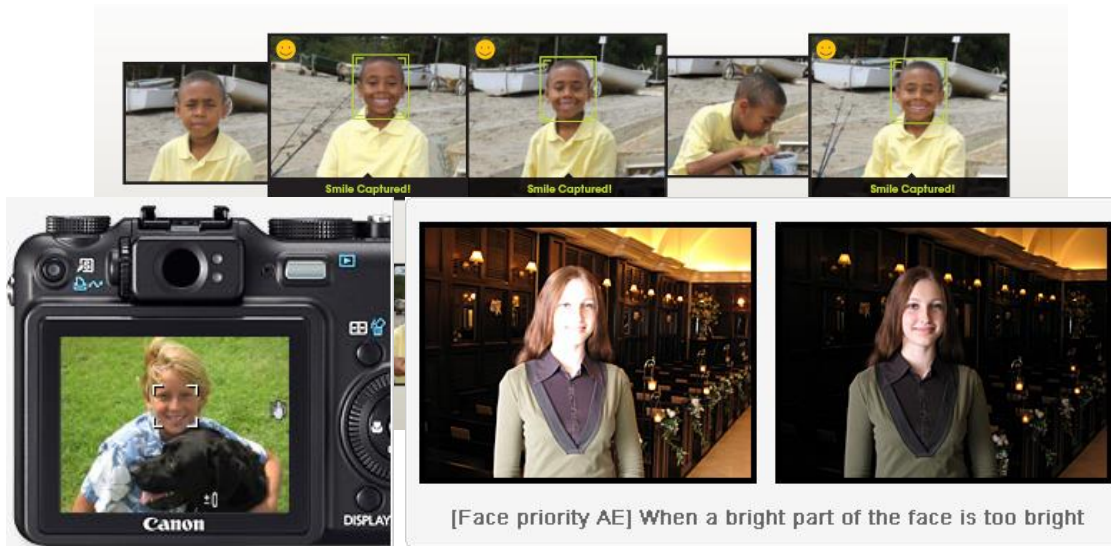
Entertainment & Virtual Reality Applications



Virtual reality



Movie special effects



Digital cameras
(face detection for setting focus, exposure)



Avatars



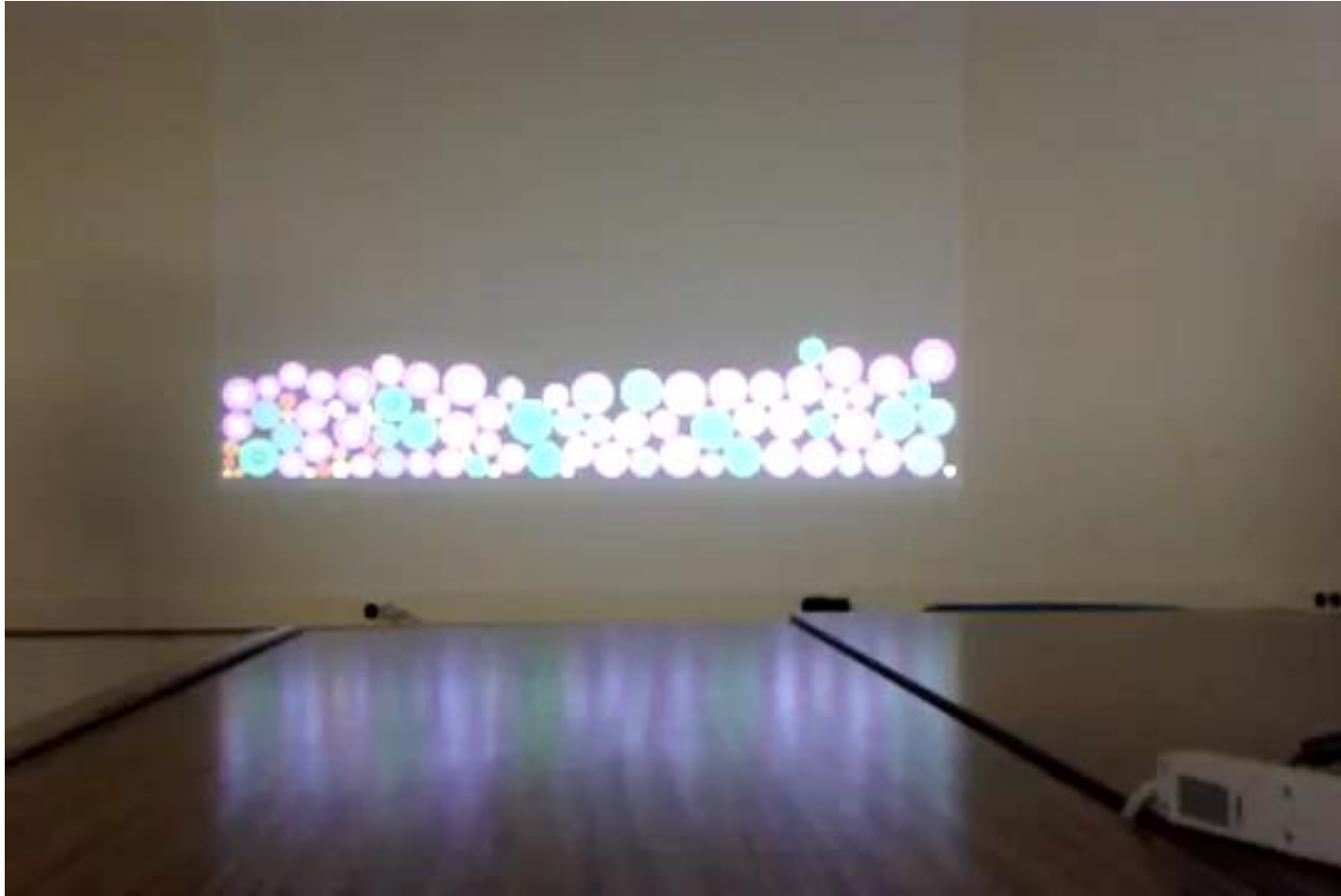
Gaming

A screenshot of a Kinect demo title screen. The background is solid black. In the center, the words "KINECT DEMO" are written in a large, bold, white, sans-serif font. Below this, in a smaller font, is the instruction "HIT SPACE TO START". The word "SPACE" is highlighted in a bright cyan color, while "HIT" and "TO START" are in white.

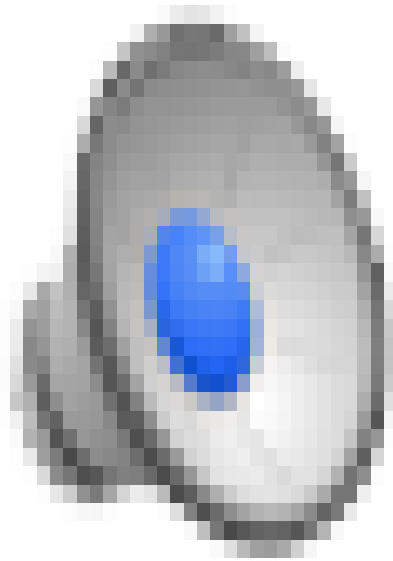
KINECT DEMO

HIT **SPACE** TO START

Entertainment & Virtual Reality Applications



Entertainment & Virtual Reality Applications



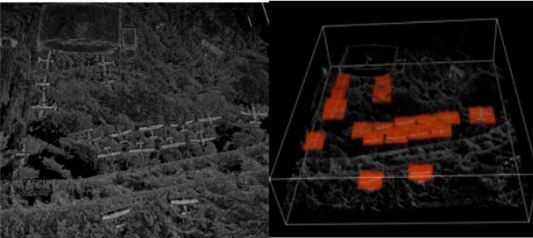
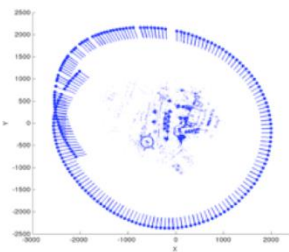
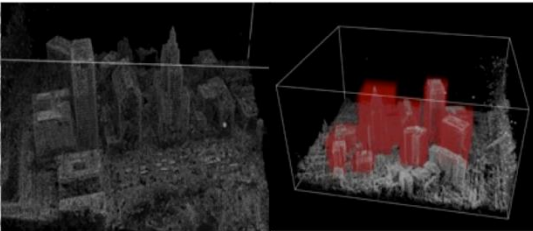
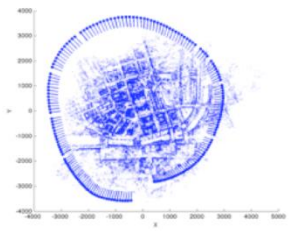
Space Applications



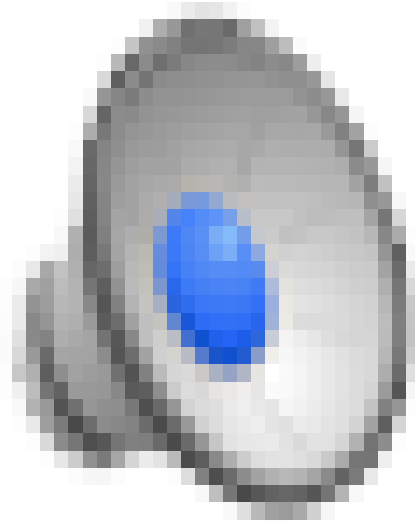
Drones Vision



Satellite Vision



Space Applications



The goal of Computer Vision

- To perceive the “world behind the picture”
- What exactly does this mean?
 - Vision as a source of metric information (computing)
 - Vision as a source of semantic information (recognizing)
 - Vision as an enhancement of information (enhancing)

Vision as a source of metric information

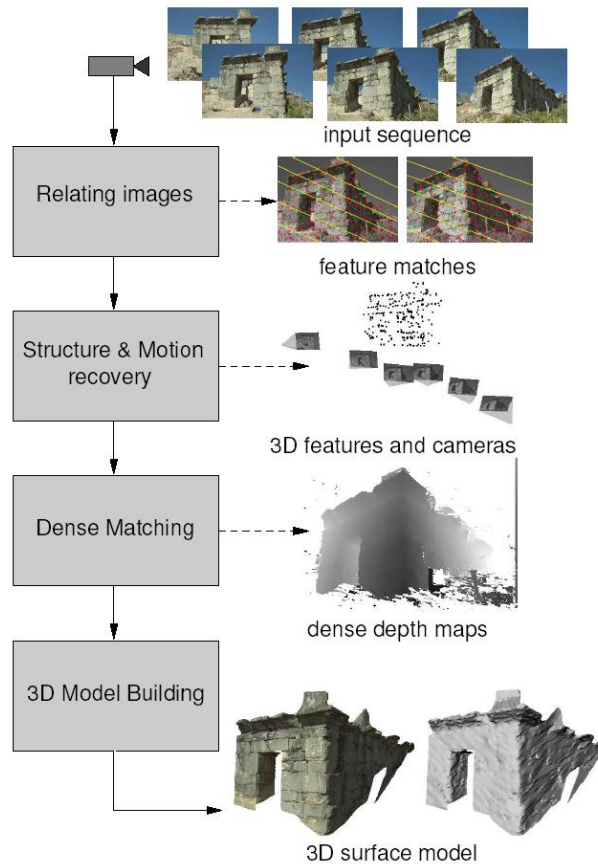
Structure from motion



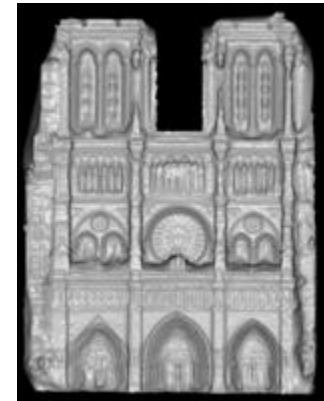
NASA Mars Rover



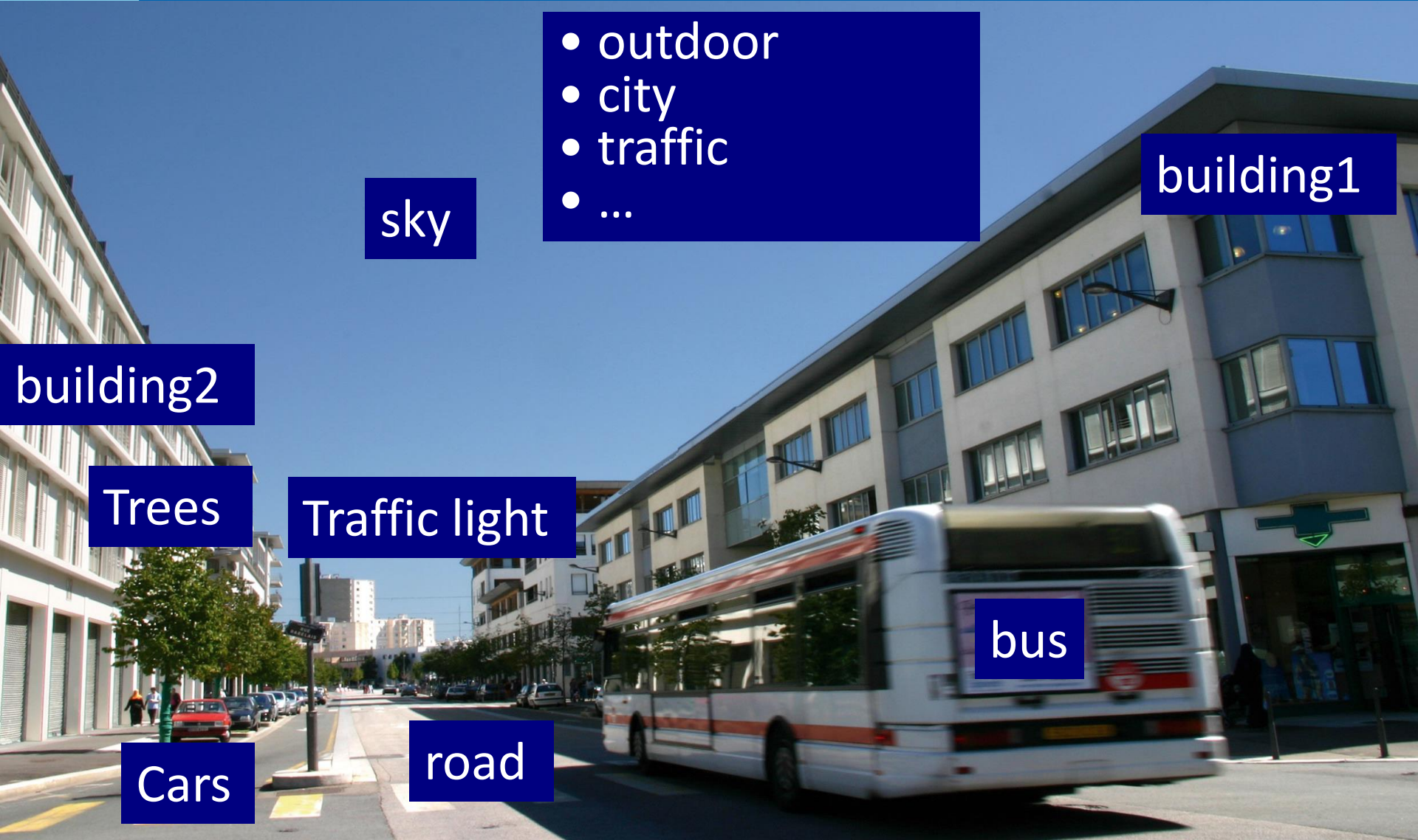
Real-time stereo



Multi-view stereo for community photo collections



Vision as a source of semantic information



Vision as an enhancement of information



Super-resolution



Texture synthesis / increased field of view (uncropping)



Inpainting / image completion



denoising



Forensics

The goal of Computer Vision

The goals of computer vision (**what + where + how**) are in terms of what humans care about

The Computer Vision challenges

- View point variation
- Illumination
- Scale
- Occlusion
- object intra-class variation
- Etc.

Computer Vision Challenges : View point variation



Computer Vision Challenges : illumination

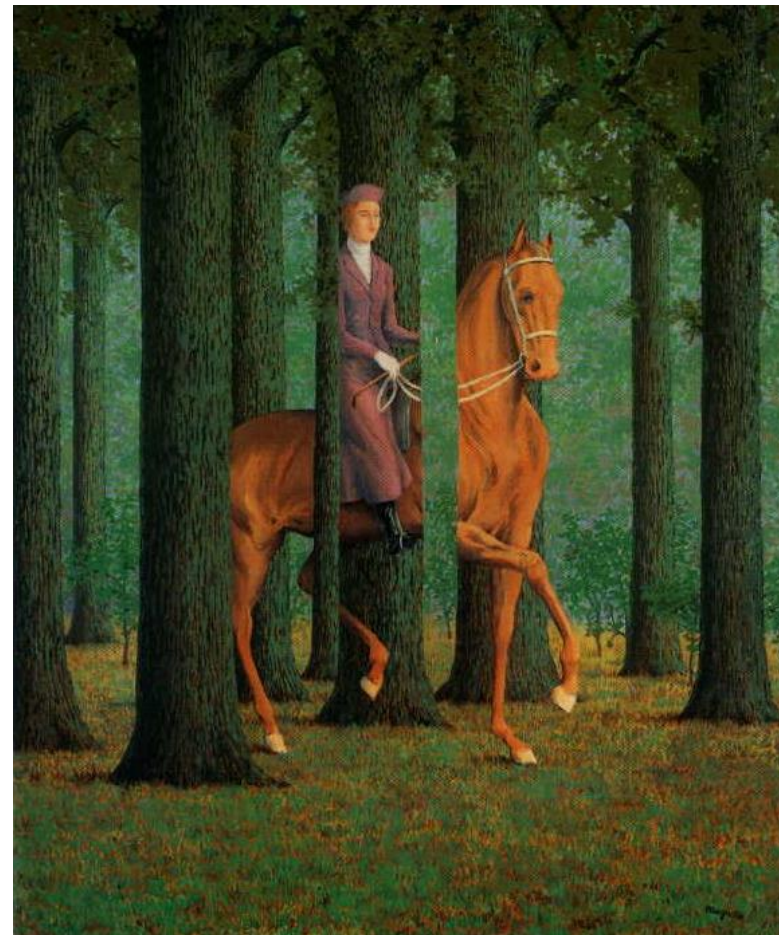
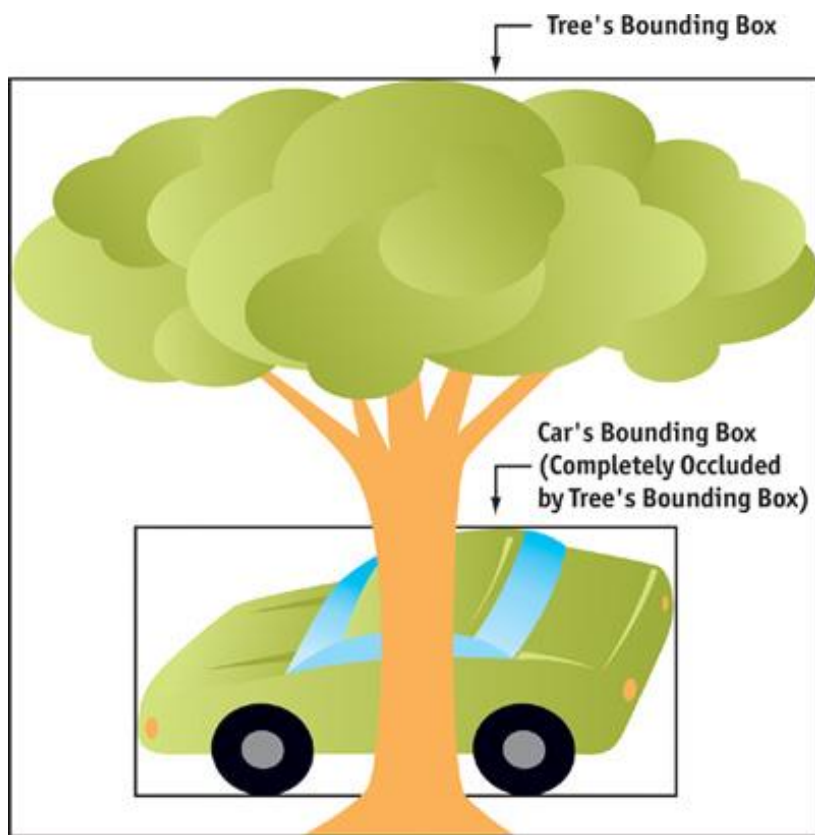


illumination problems for Face detection

Computer Vision Challenges : scale



Computer Vision Challenges : occlusion



Computer Vision Challenges : object intra-class variation

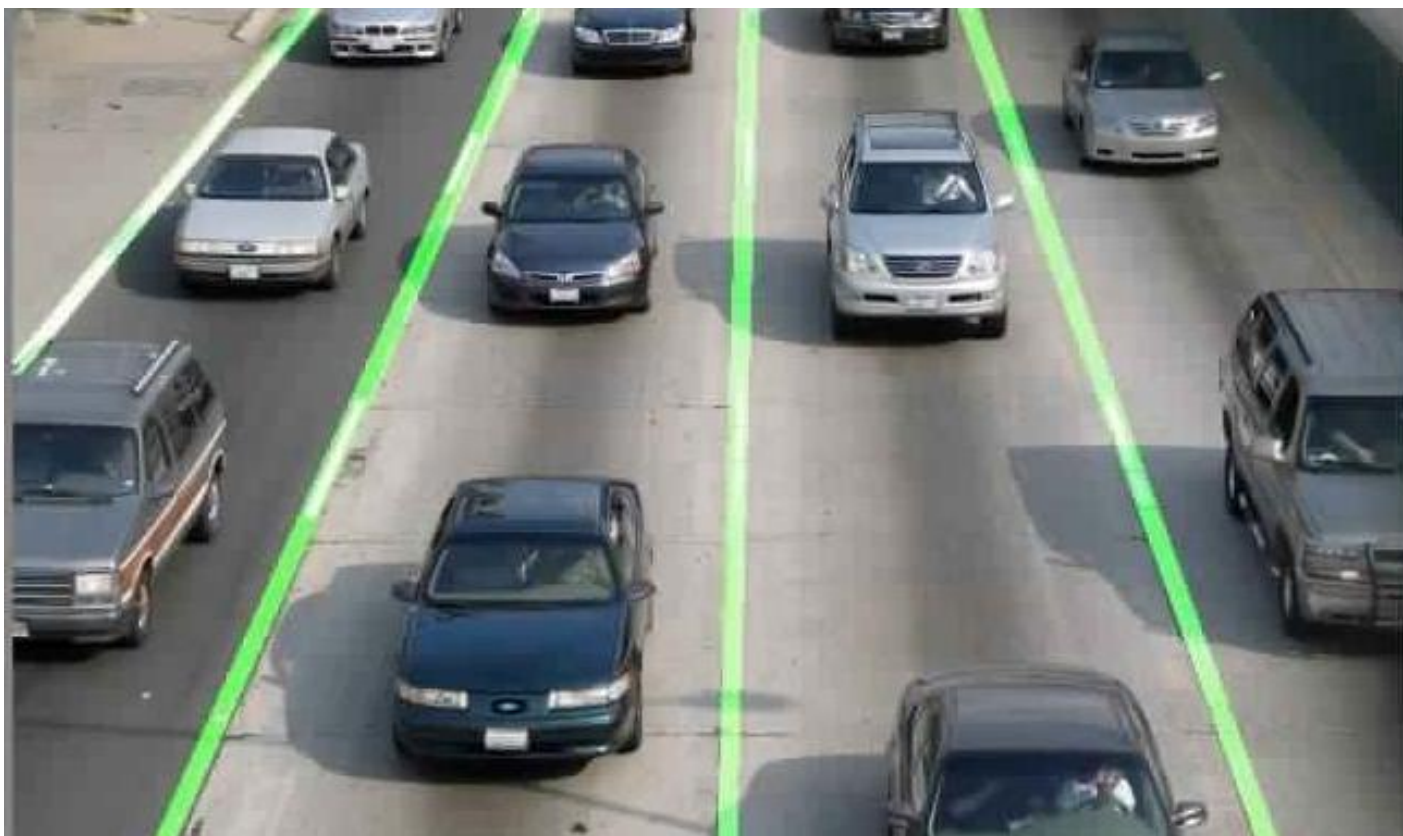


Challenges or opportunities?

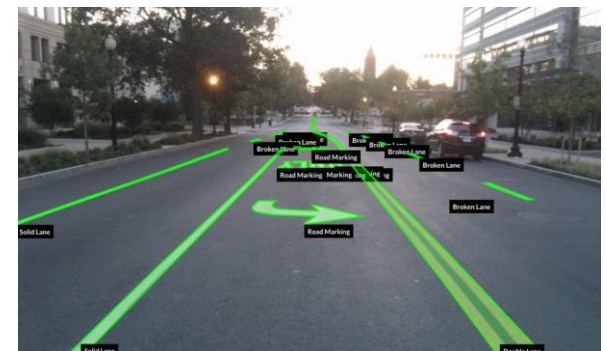
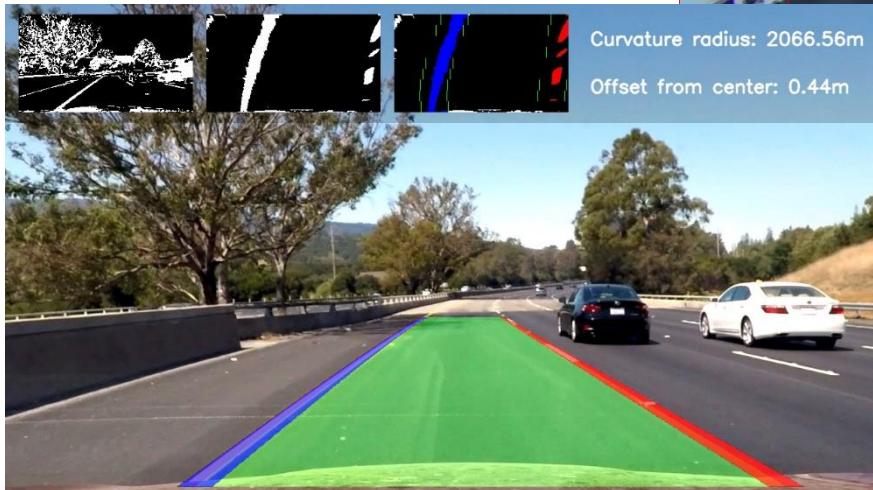
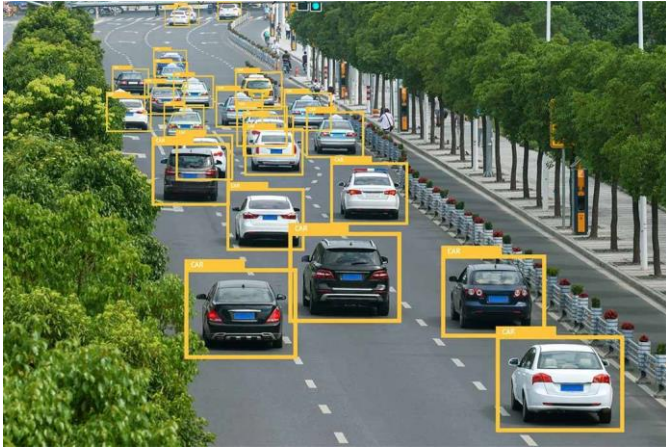
- Images are confusing, but they also reveal the structure of the world through numerous cues
- Our job is to interpret the cues!

Reflexion

What do we want to extract and what are the Challenges ?



Results

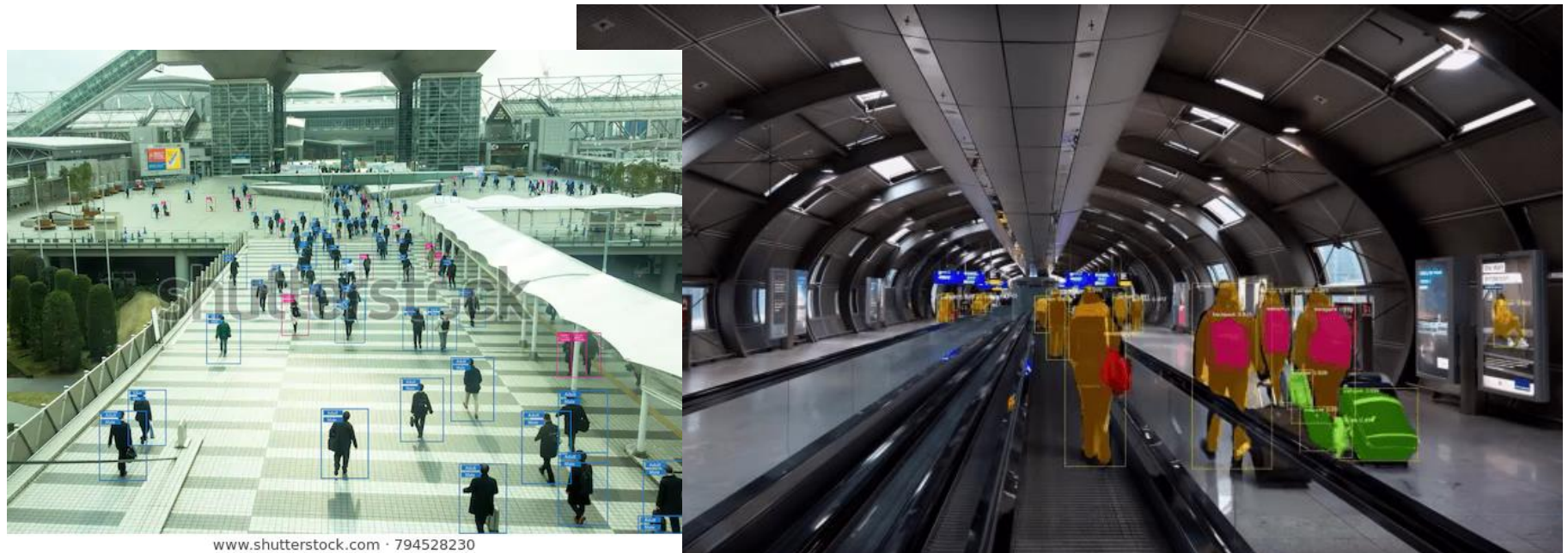
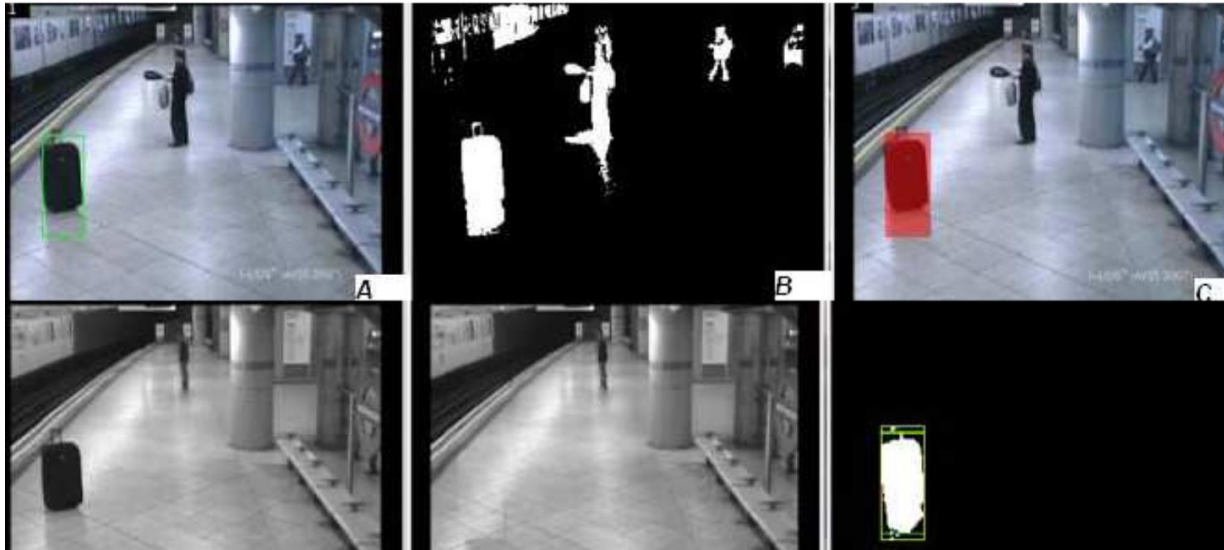


Reflexion

What do we want to extract and what are the Challenges ?



Results

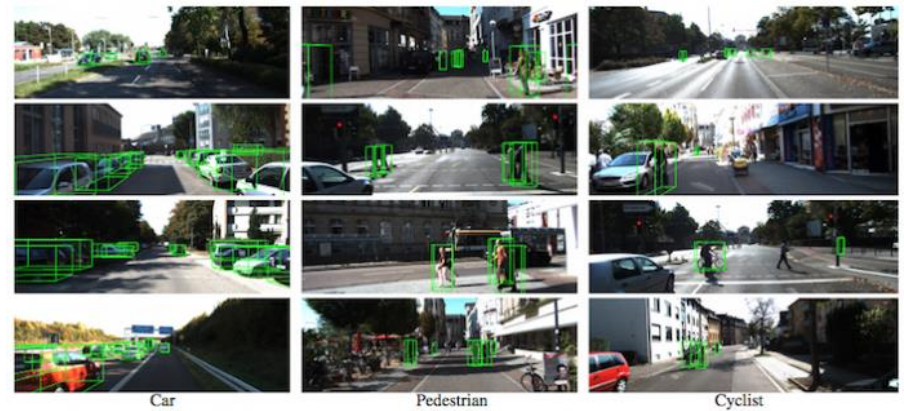
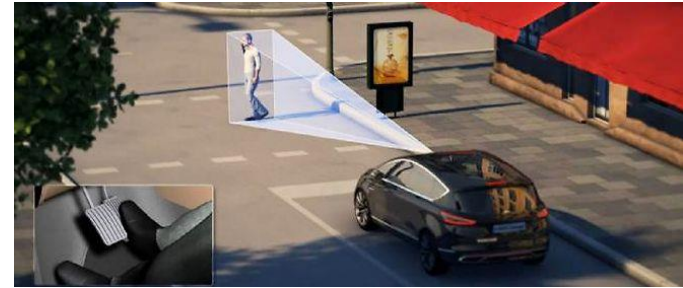
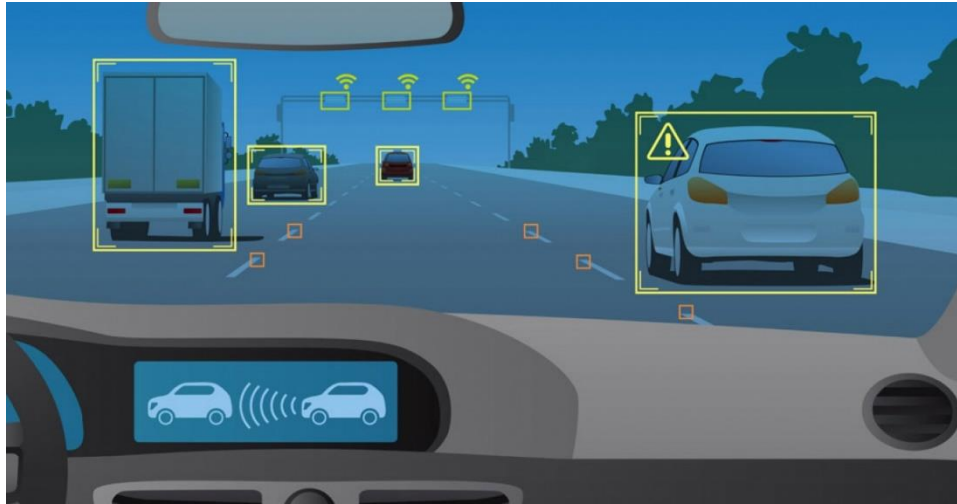


Reflexion

What do we want to extract and what are the Challenges ?



Results





Part 2

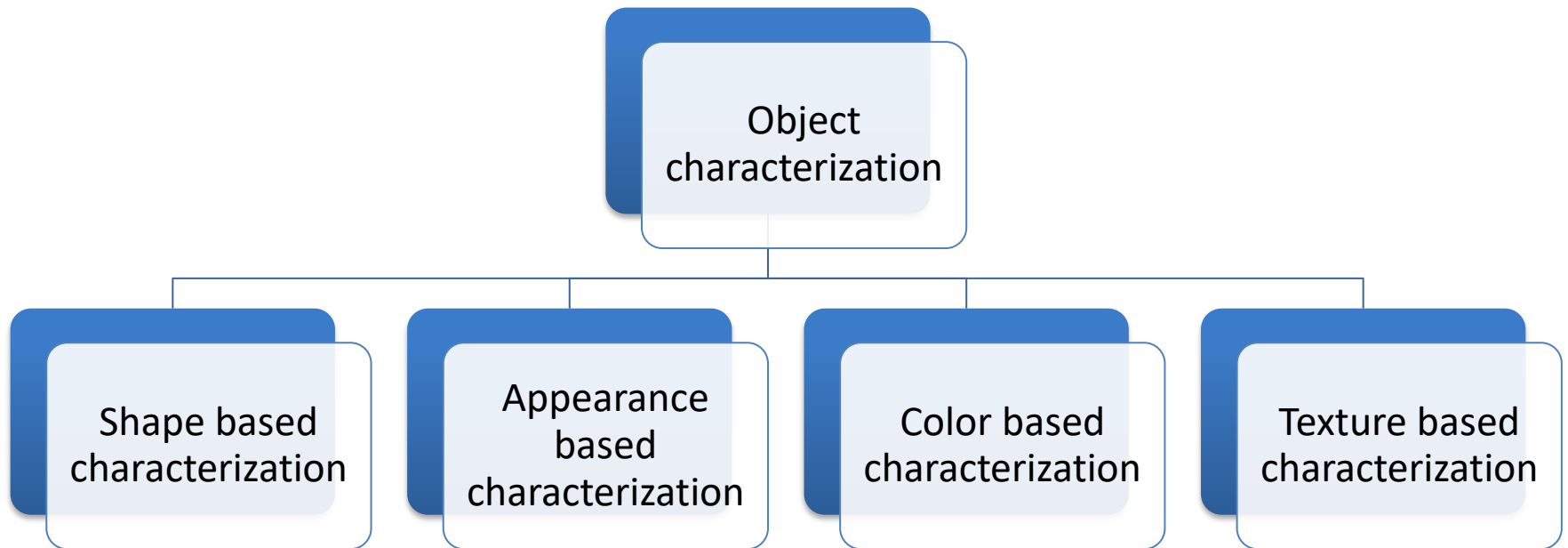
Computer Vision processing techniques

Computer Vision concepts and tools

- **Object characterization**
- **Object detection**
- **Object tracking**
- **Object Recognition**
- **Motion Analysis**

Object characterization

Object characterization



Object characterization : shape

Points



Centroid

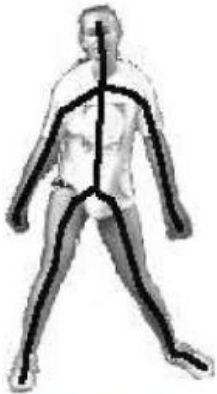


Multiple Points

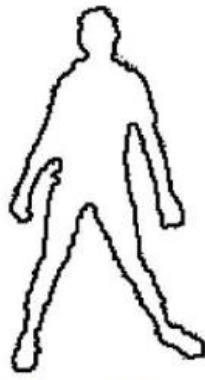


Control Points

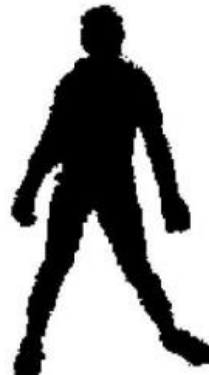
Contours



Skeletal Model



Complete Contour



Silhouette

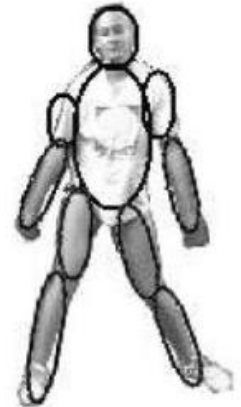
Patches



Rectangular Patch



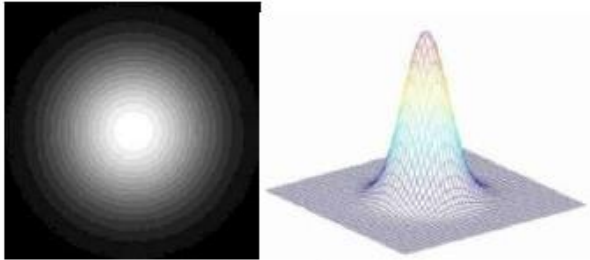
Elliptical Patch



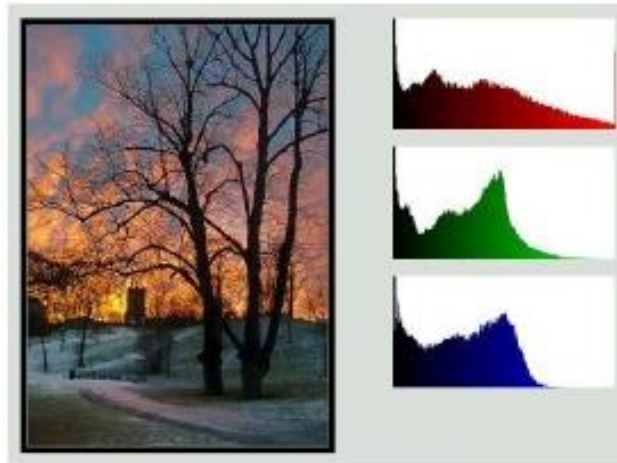
Multiple Patches

Object characterization : Appearance

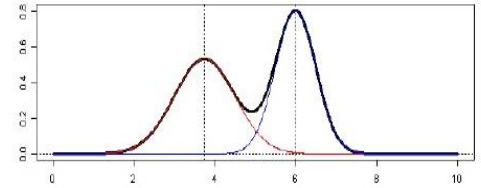
Probability densities



Gaussian distribution



Histogram distribution



Mixture of Gaussians

Models

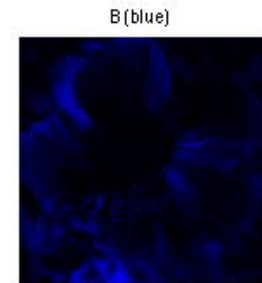


Geometric template



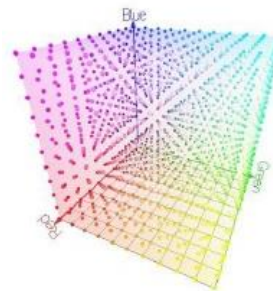
Multi-view Appearance

Object characterization : Color

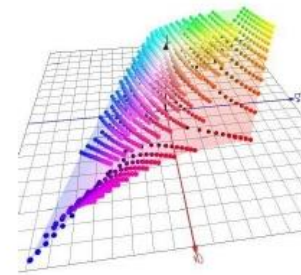


Color Spaces:

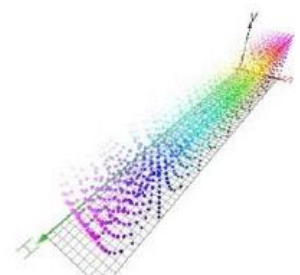
- RGB family (RGB, CMY).
- YCbCr.
- HSV.
- LAB.



RGB



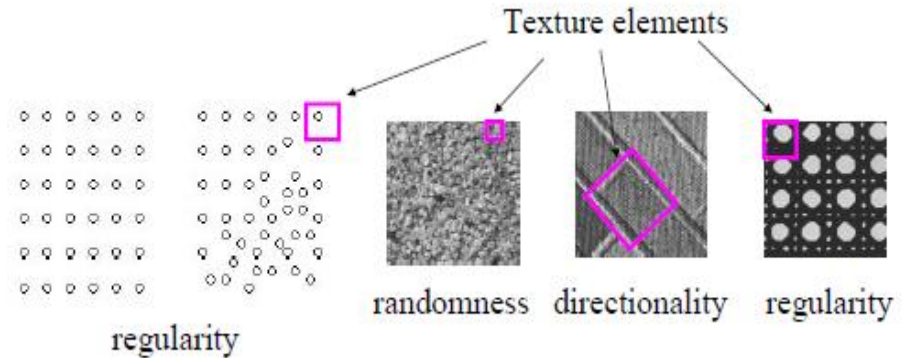
LAB



HSV

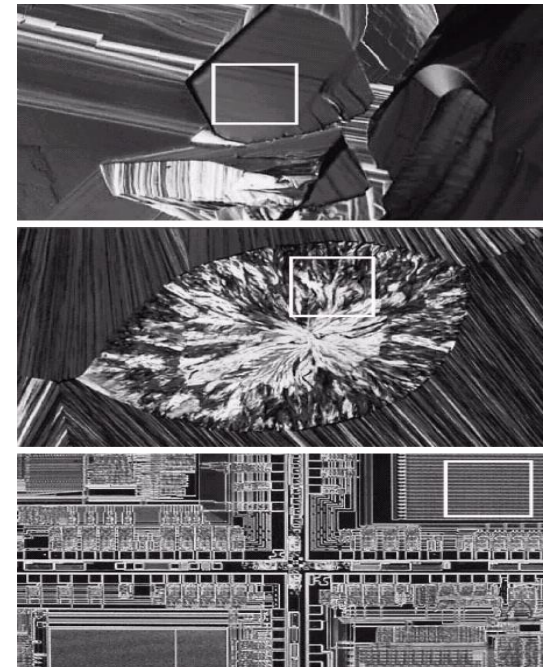
Object characterization : texture

- Defined as the structural pattern of surfaces *which is homogeneous in spite of fluctuations in brightness and color*
- Most important visual cue in identifying regions
- Main categories: regularity, randomness and directionality



Texture description techniques

- Statistical approach
 - From first order statistics (histogram)
 - From second order statistics (grey-level cooccurrence matrix)
- Spectral approach -Fourier power spectrum
- Gabor filter
- Multiresolution wavelet



Feature selection

- Which feature can be easily distinguished ?
- Good features should be robust to all sorts of nastiness that can occur between images.

=> Choose the features according to :

1/ the nature of the object

2/ the application

3/ the robustness of the feature to all computer vision challenges

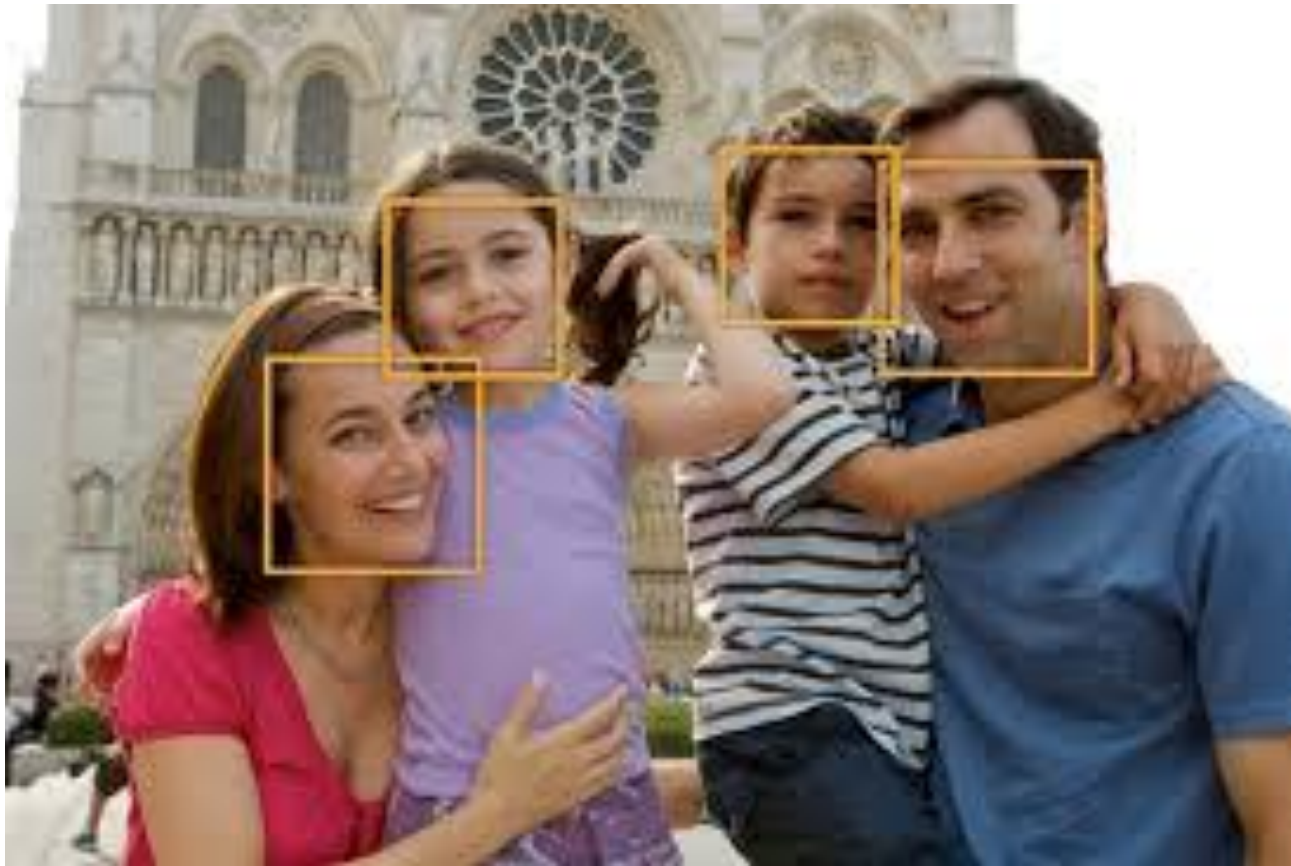
Reflexion

How we can characterize the object to extract it?



Reflexion

How we can characterize the object to extract it?



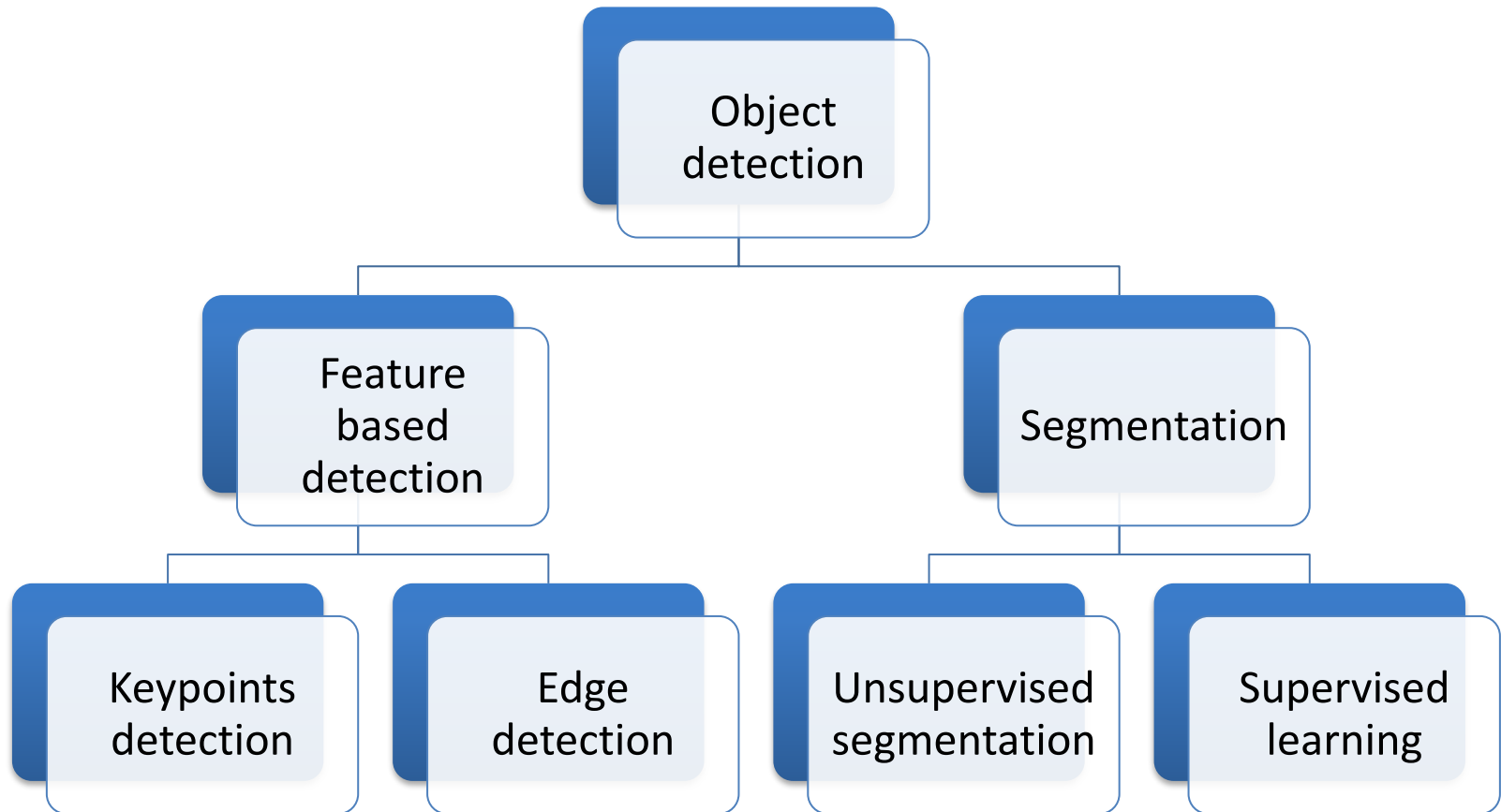
Reflexion

How we can characterize the object to extract it?



Object detection

Object detection

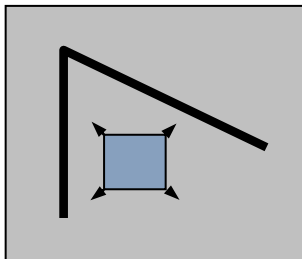
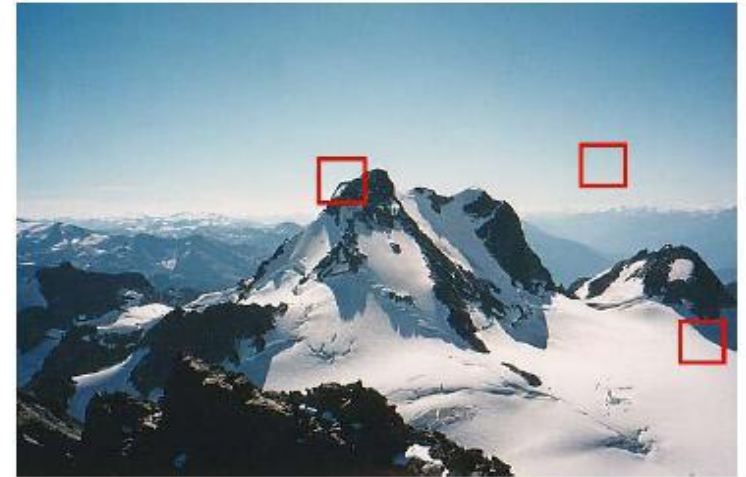


Keypoints detection

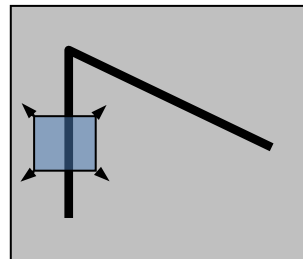
Operator	Accuracy	Number of detected points	Complexity and computing time	Invariance
DOG	Medium	Low	Low	Affine
Harris	Medium	Low	Low	Affine and Rotation
Harris-Laplace	High	Low	Medium	Affine, rotation and scale
SIFT	High	High	High	Affine, rotation and scale
SURF	High	High	Less than SIFT	Affine, rotation and scale

Keypoint Detection: Harris Operator

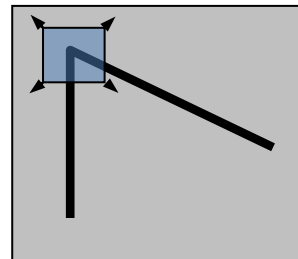
- How to describe a local region ?
- Some patches can be localized with match accuracy than the others
- Finding Corners : in the region around a corner, image gradient has two or more dominant directions
- Corners are repeatable and **distinctive**
- *Idea*



“flat” region:
no change in
all directions



“edge”: no change
along the edge
direction

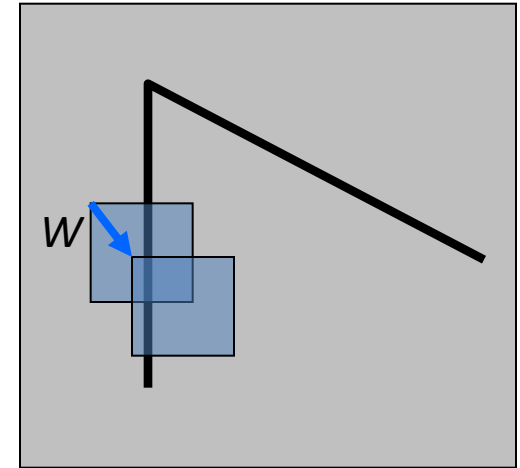


“corner”:
significant change in
all directions

Harris Operator: the Math

Consider shifting the window W by (u, v)

- how do the pixels in W change?
- compare each pixel before and after by summing up the squared differences (SSD)
- this defines an SSD “error” $E(u, v)$:



$$E(u, v) = \sum_{(x, y) \in W} (I(x + u, y + v) - I(x, y))^2$$

Taylor Series

$$I(x + u, y + v) = I(x, y) + \frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v + \text{higher order terms}$$

$$\begin{aligned} I(x + u, y + v) &\approx I(x, y) + \frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v \\ &\approx I(x, y) + [I_x \ I_y] \begin{bmatrix} u \\ v \end{bmatrix} \end{aligned}$$

shorthand: $I_x = \frac{\partial I}{\partial x}$

Harris Operator: the Math

$$\begin{aligned}
 E(u, v) &= \sum_{(x,y) \in W} (I(x+u, y+v) - I(x, y))^2 \\
 &\approx \sum_{(x,y) \in W} (I(x, y) + I_x(x, y)u + I_y(x, y)v - I(x, y))^2 \\
 &\approx \sum_{(x,y) \in W} (I_x(x, y)u + I_y(x, y)v)^2 \\
 &\approx \sum_{(x,y) \in W} (I_x^2 u^2 + 2I_x I_y uv + I_y^2 v^2) \\
 &\approx Au^2 + 2Buv + Cv^2
 \end{aligned}$$

$$\left\{ \begin{aligned} A &= \sum_{(x,y) \in W} I_x^2 \\ B &= \sum_{(x,y) \in W} I_x I_y \\ C &= \sum_{(x,y) \in W} I_y^2 \end{aligned} \right.$$

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \underbrace{\begin{bmatrix} A & B \\ B & C \end{bmatrix}}_H \begin{bmatrix} u \\ v \end{bmatrix}$$

Ellipse equation

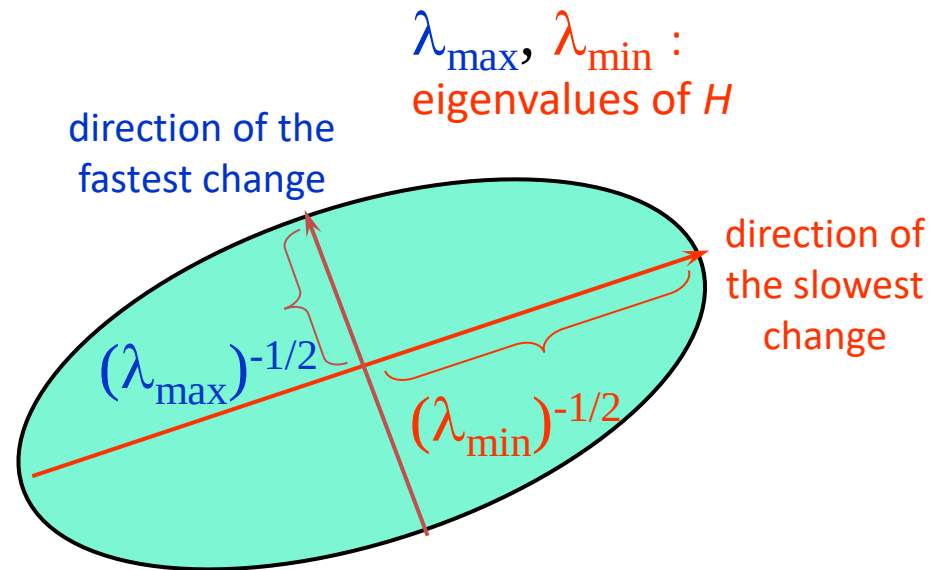
Harris Operator: the Math

- Ellipse equation:

We can visualize H as an ellipse with axis lengths determined by the *eigenvalues* of H and orientation determined by the *eigenvectors* of H

Eigenvalues and eigenvectors of H

- $x_{\max}$ = direction of largest increase in E
- $\lambda_{\max}$ = amount of increase in direction $x_{\max}$
- $x_{\min}$ = direction of smallest increase in E
- $\lambda_{\min}$ = amount of increase in direction $x_{\min}$

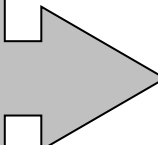
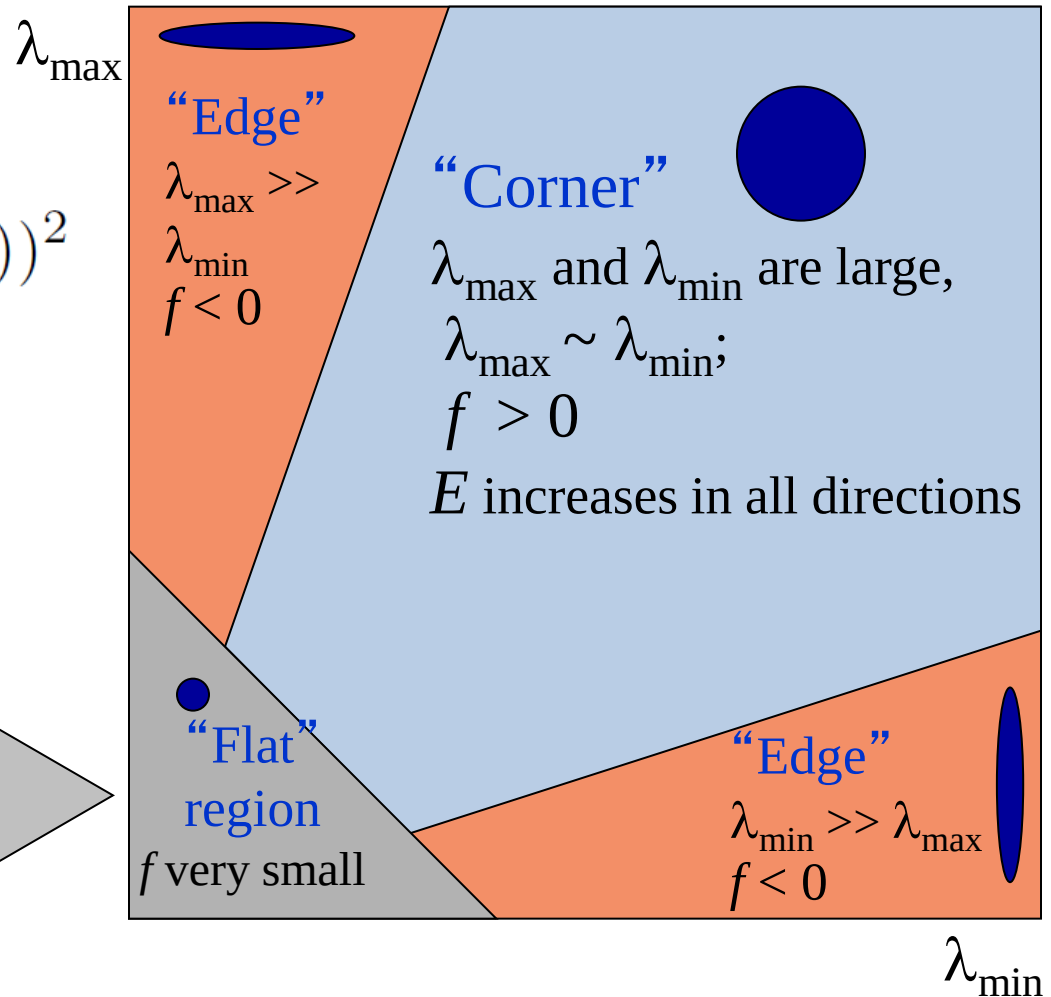


=> We can find the E variation by looking at the eigenvectors and eigenvalues of H

Harris Operator: the Math

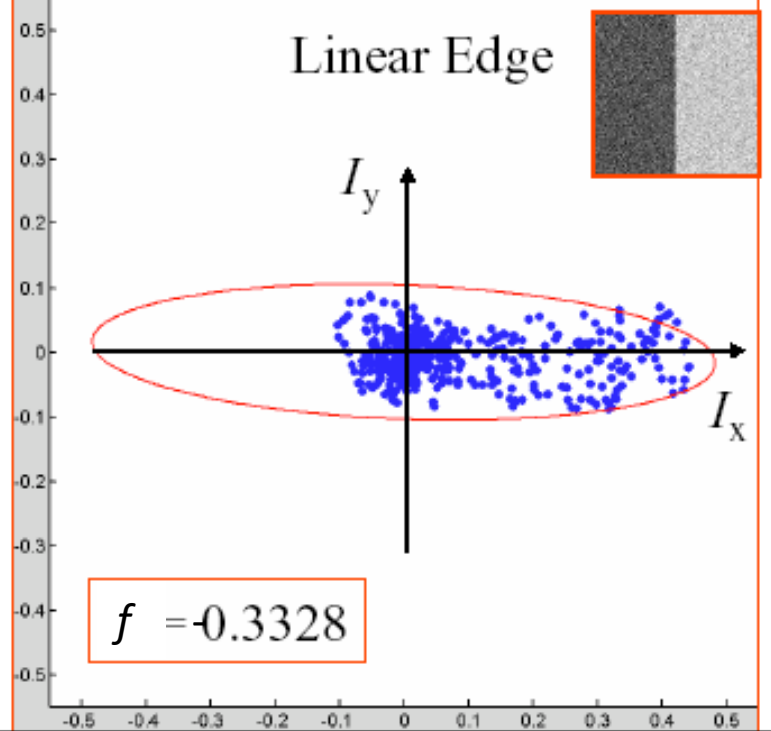
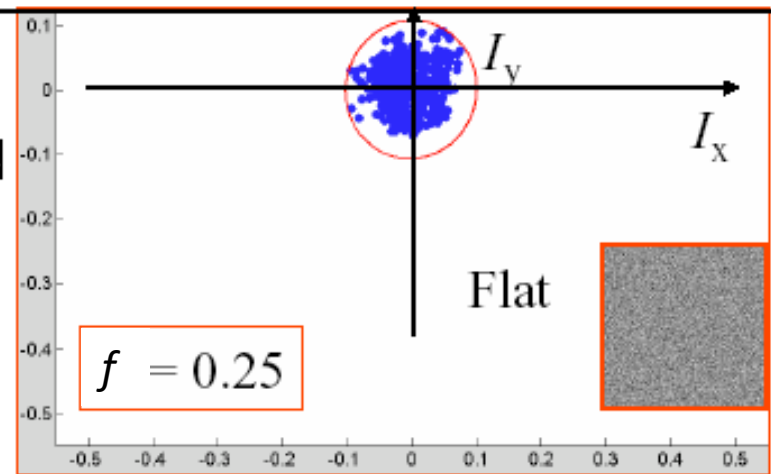
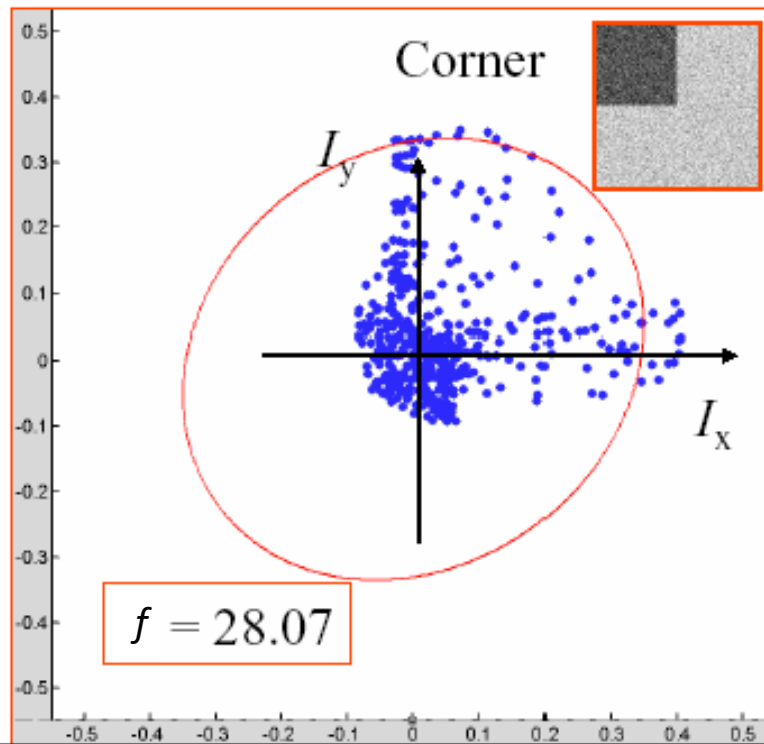
$$f = \det(H) - k(\text{tr}(H))^2$$

$\lambda_{\max}$ and $\lambda_{\min}$ are small; E is almost constant in all directions

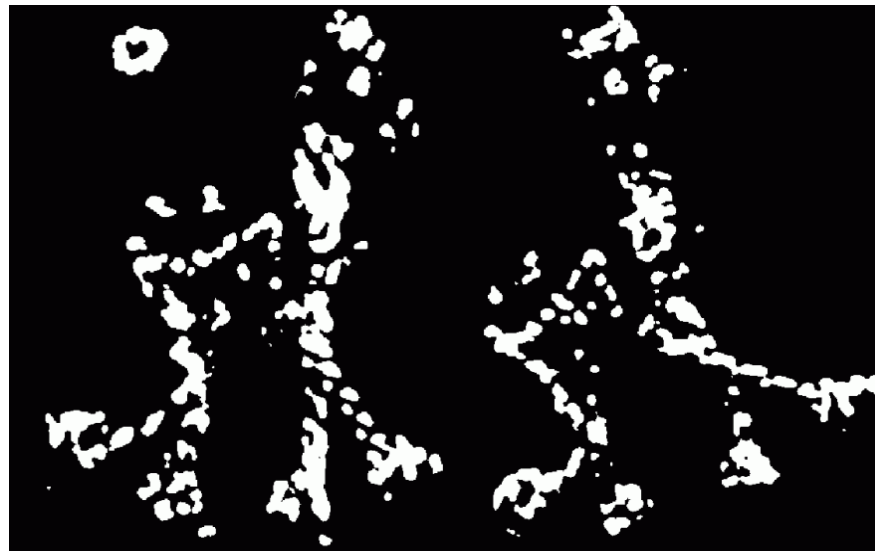
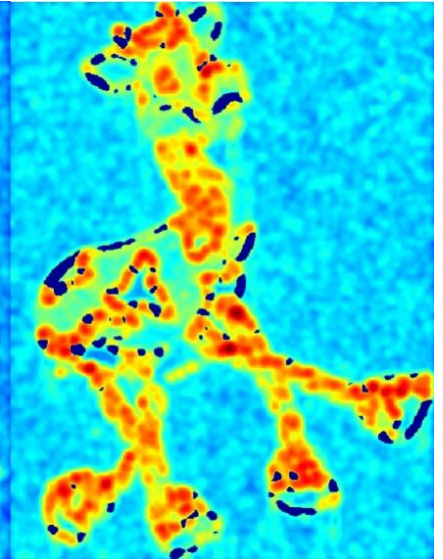
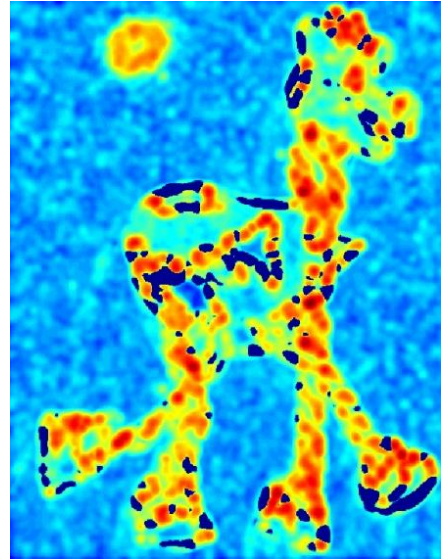
Harris Operator: the Math

The distribution of x and y derivatives can be characterized by the shape and size of the principal component ellipse



- Auto-correlation matrix (SSD)
 - captures the structure of the local neighborhood
 - measure based on **eigenvalues** of H
 - 2 strong eigenvalues $\Rightarrow$ interest point
 - 1 strong eigenvalue $\Rightarrow$ contour
 - 0 eigenvalue $\Rightarrow$ uniform region
- Interest point detection
 - threshold on the eigenvalues
 - local maximum for localization

Harris Operator detection

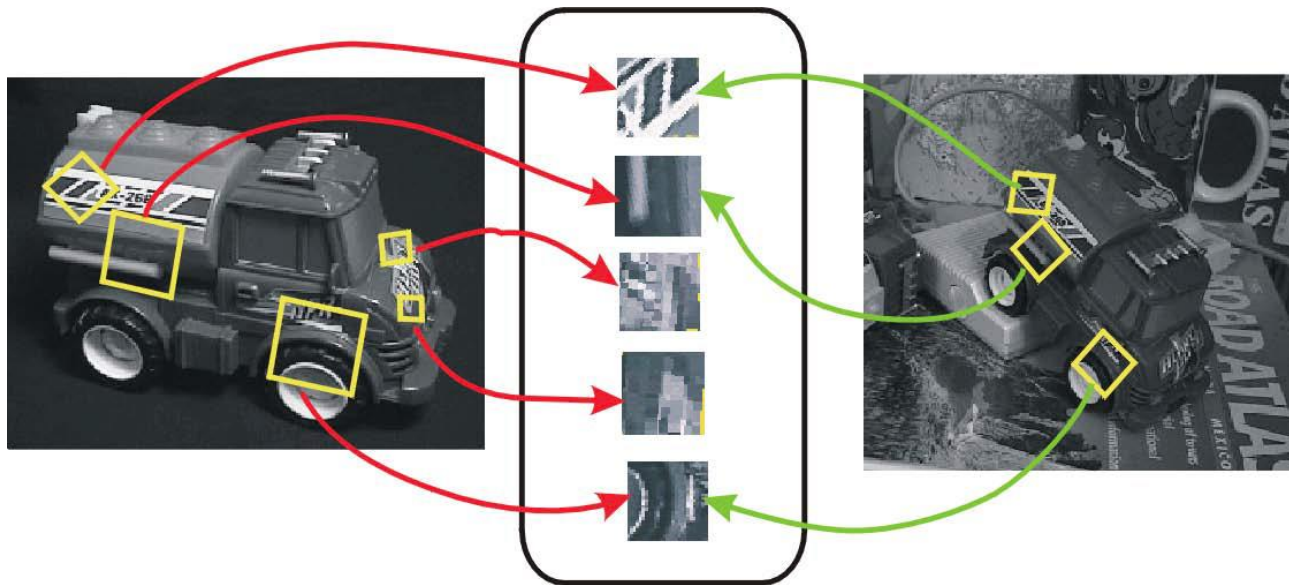


Harris Operator detection



Keypoint Detection: SIFT motivation

- Generates image features, “keypoints”
 - invariant to image scaling, translation and rotation
 - partially invariant to change in illumination and 3D camera viewpoint
 - many can be extracted from typical images
 - highly distinctive



SIFT algorithm main steps

- 1/ Scale-space extrema detection
 - Potential location for the finding features: Search over multiple scales and image locations.
- 2/ Keypoint localization
 - Accurately locating the feature keypoints: Selection based on a measure of stability.
- 3/ Orientation assignment
 - Assigning orientation to the keypoints.
- 4/ Keypoint descriptor
 - Describing the keypoint as a high dimensional vector: Using local image gradients at selected scale and rotation.

1/ Scale-space extrema detection

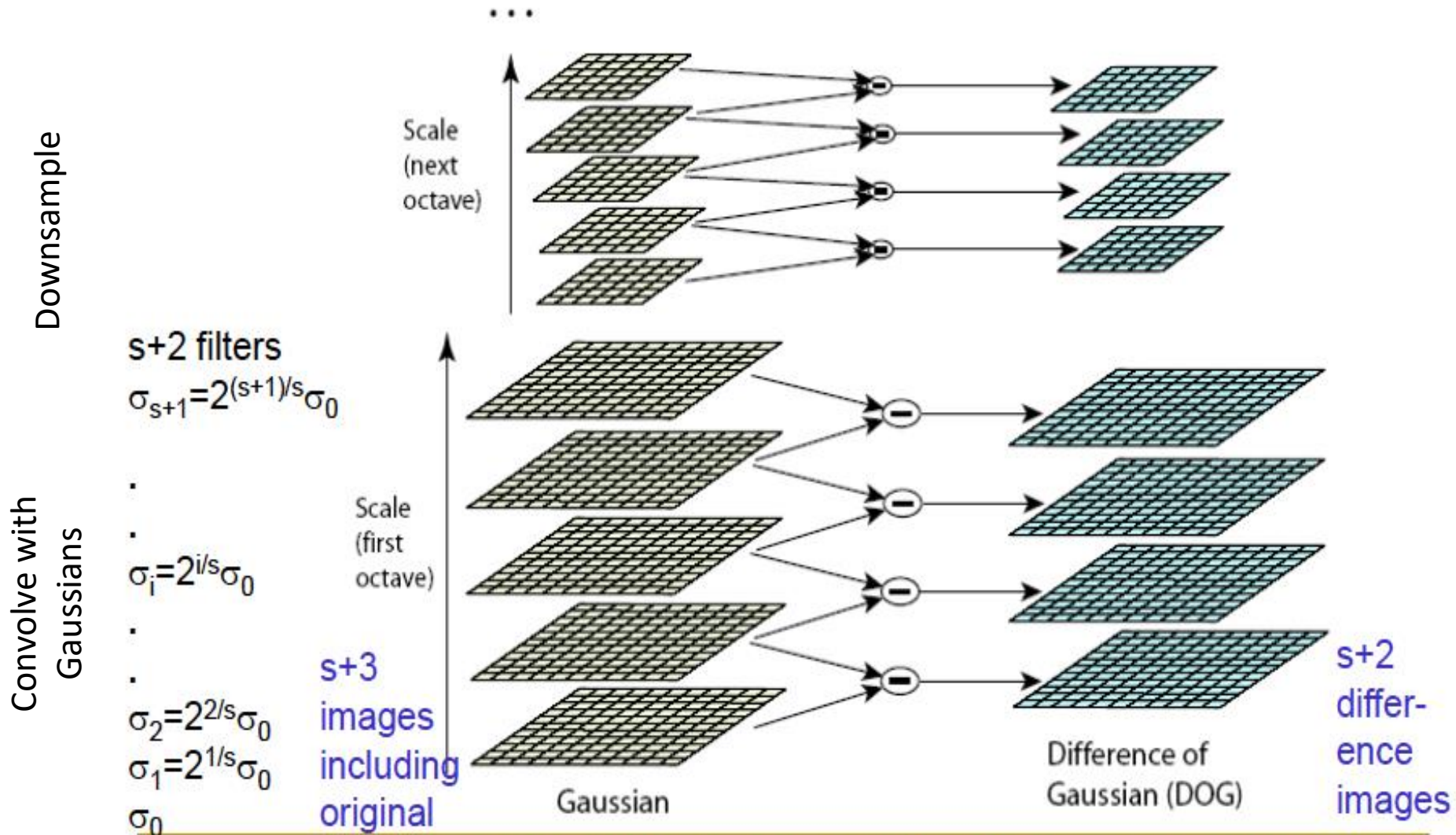
- Definition: $L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$
where $G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$
- Keypoints are detected using scale-space extrema in Laplacian-of-Gaussian $\sigma^2 \nabla^2 G$
- Close approximation to scale-normalized Laplacian of Gaussian (heat equation)

$$G(x, y, k\sigma) - G(x, y, \sigma) \approx (k - 1)\sigma^2 \nabla^2 G$$

- D definition:

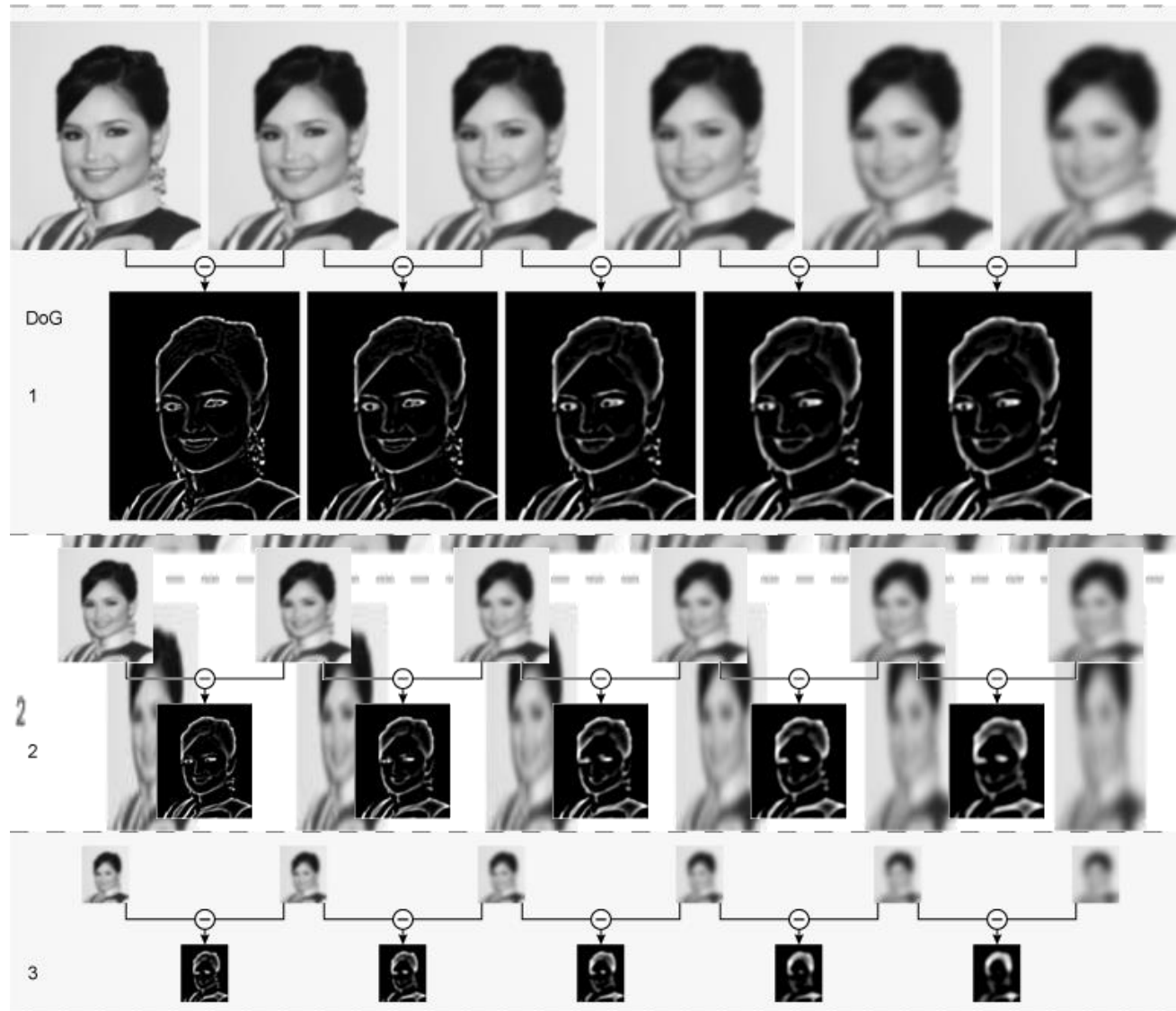
$$\begin{aligned} D(x, y, \sigma) &= (G(x, y, k\sigma) - G(x, y, \sigma)) * I(x, y) \\ &= L(x, y, k\sigma) - L(x, y, \sigma) \end{aligned}$$

Finding Keypoints



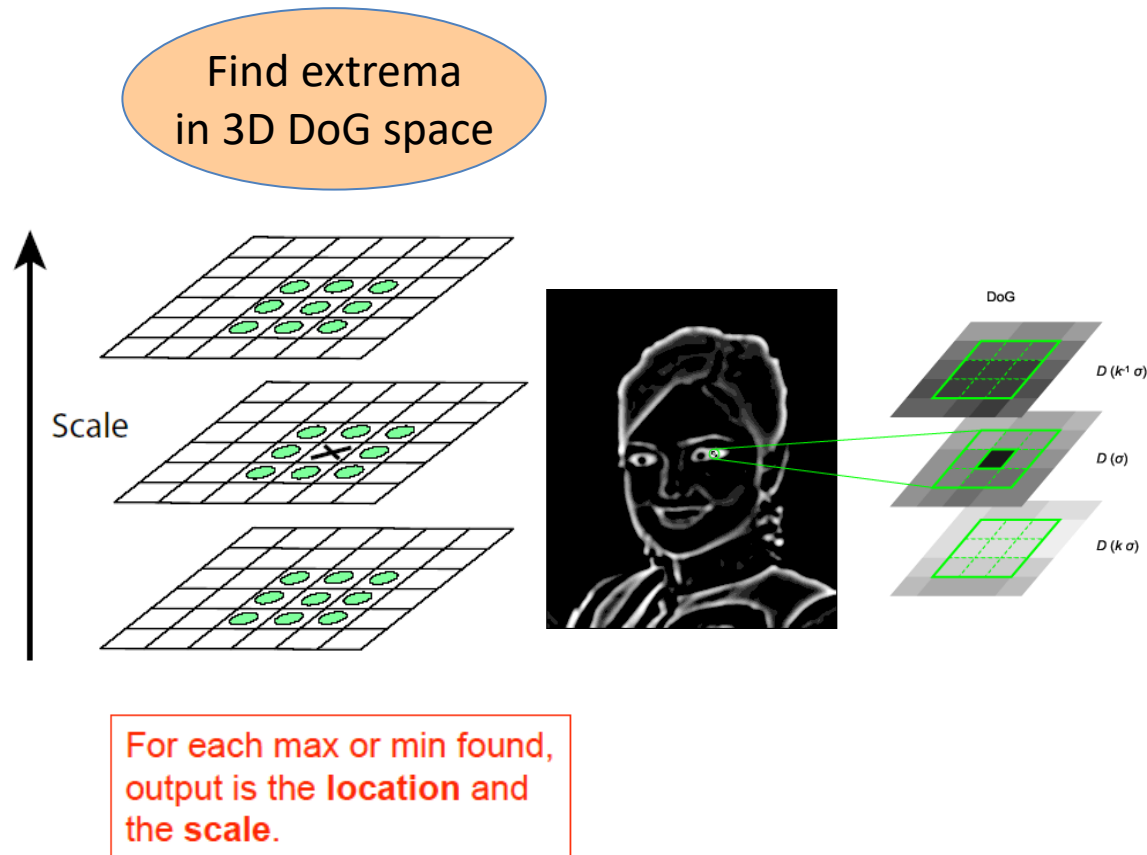
The parameter s determines the number of images per octave.

Finding Keypoints



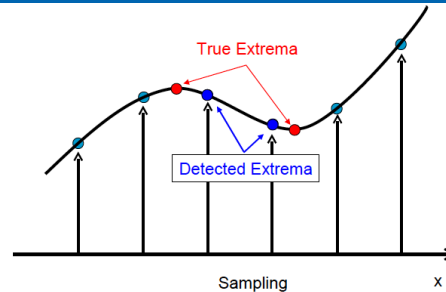
Finding Keypoints

- Detect maxima and minima of difference-of-Gaussian in scale space
- Each point is compared to its 8 neighbors in the current image and 9 neighbors each in the scales above and below



However still a lot of points including bad points !!

2/ Adjusting Localization



- Start with Taylor expansion with sample point as the origin

$$D(X) = D + \frac{\partial D^T}{\partial X} X + \frac{1}{2} X^T \frac{\partial^2 D}{\partial X^2} X$$

— where $X = (x, y, \sigma)^T$

- Take the derivative with respect to X , and set it to 0, giving

$$0 = \frac{\partial D}{\partial X} + \frac{\partial^2 D}{\partial X^2} \hat{X}$$

- $\hat{X} = -\frac{\partial^2 D^{-1}}{\partial X^2} \frac{\partial D}{\partial X}$ is the true location of the keypoint
- This is a 3x3 linear system

Filtering

- Contrast :
 - If $|D(X)| < 0.03$, throw points with bad contrast
- Edge-iness:
 - Like Harris - using Trace and Determinant of Hessian
 - Use ratio of principal curvatures to throw out poorly defined peaks
 - Curvatures come from Hessian:
 - Ratio of $Trace(H)^2$ and $Determinant(H)$

$$H = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$

$$Tr(H) = D_{xx} + D_{yy}$$

$$Det(H) = D_{xx}D_{yy} - (D_{xy})^2$$

$$R = \frac{Tr(H)^2}{Det(H)} = \frac{(r+1)^2}{r} \quad \text{with} \quad r = \frac{l_{\max}}{l_{\min}}$$

- If ratio $> (r+1)^2/(r)$, throw it out (SIFT uses $r=10$)

Filtering



3/ Orientation assignment

- Now we have set of good points
- Choose a region around each point
- Use scale of point to choose correct image:

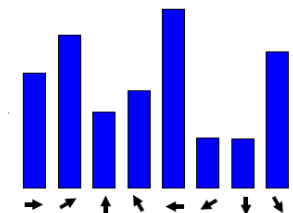
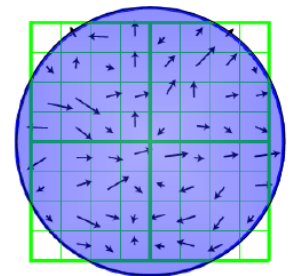
$$L(x, y) = G(x, y, \sigma) * I(x, y)$$

- Compute gradient magnitude and orientation using finite differences:

$$m(x, y) = \sqrt{(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2}$$

$$\theta(x, y) = \tan^{-1} \left(\frac{L(x, y+1) - L(x, y-1)}{L(x+1, y) - L(x-1, y)} \right)$$

- Create gradient histogram (36 bins)
- Any peak within 80% of the highest peak is used to create a keypoint with that orientation

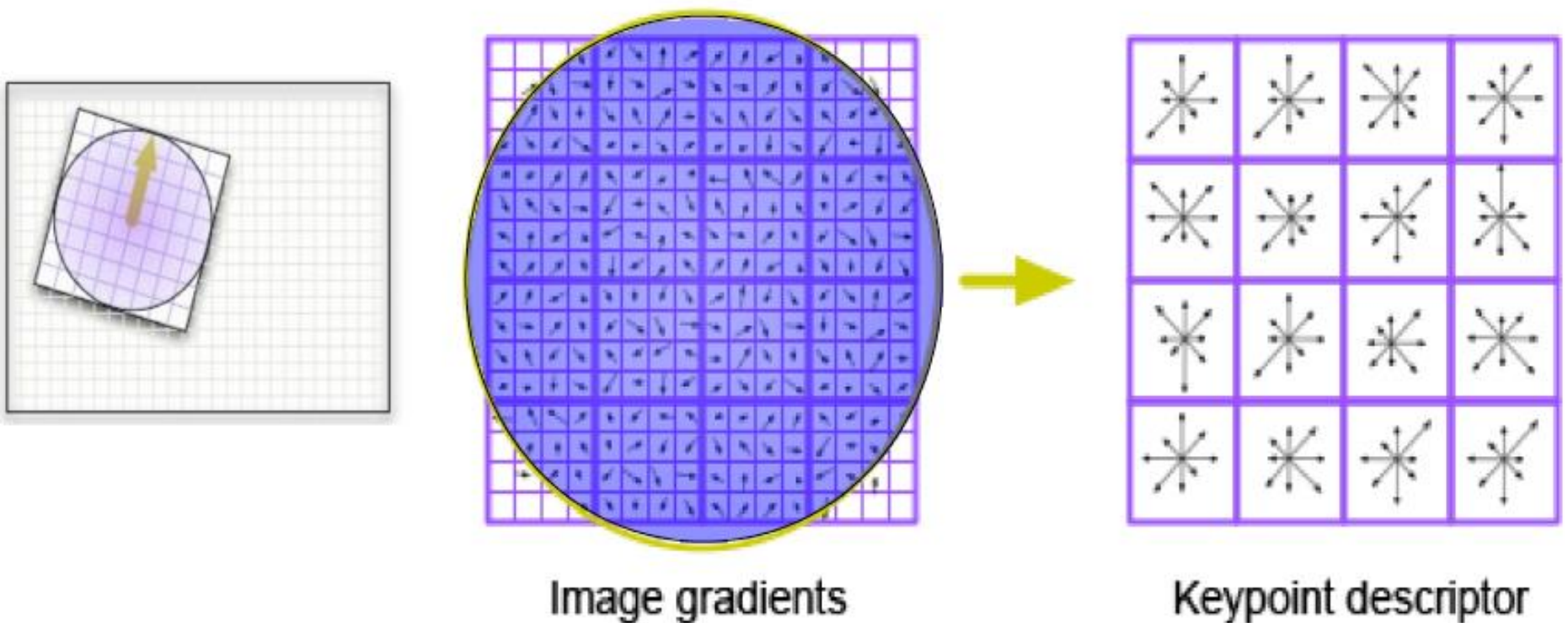


4/ Keypoint Descriptor

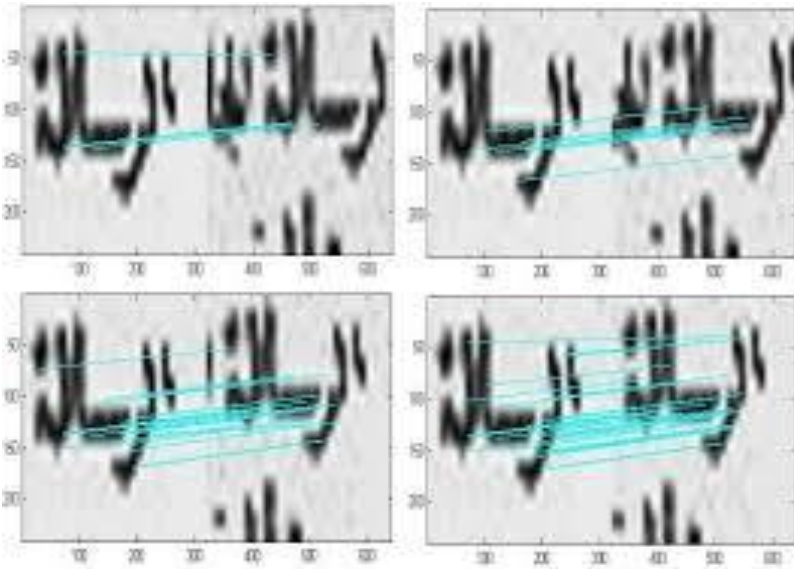
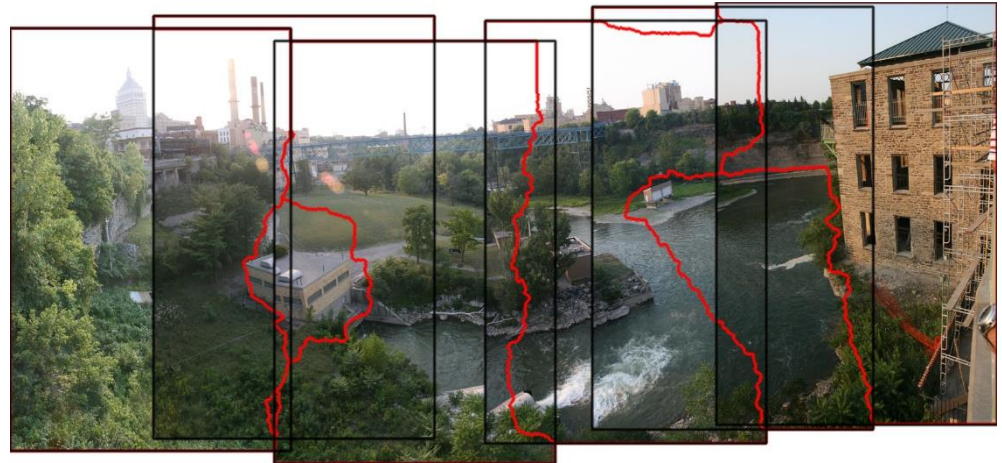
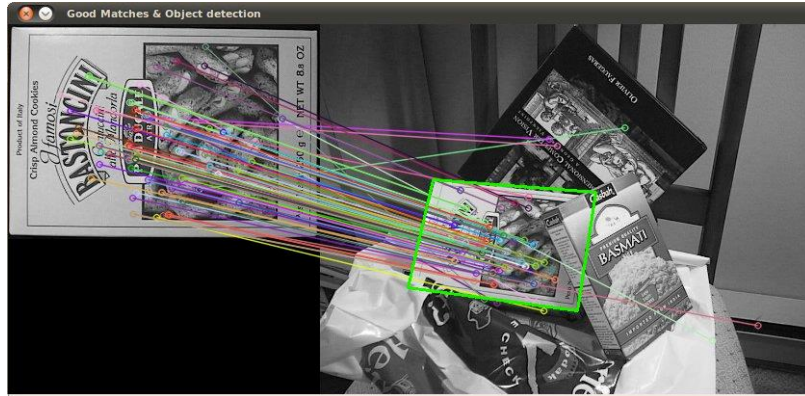
- At this point, each keypoint has
 - location
 - scale
 - orientation
- Next is to compute a descriptor for the local image region about each keypoint that is
 - highly distinctive
 - invariant as possible to variations such as changes in viewpoint and illumination

Descriptor

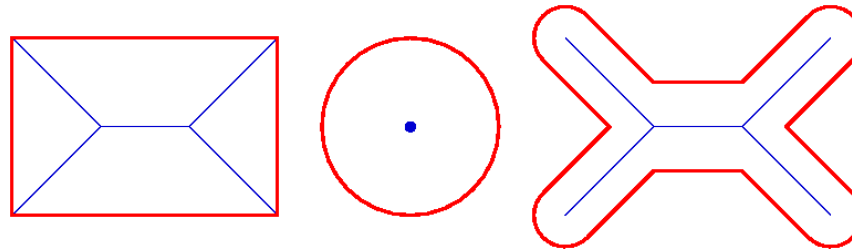
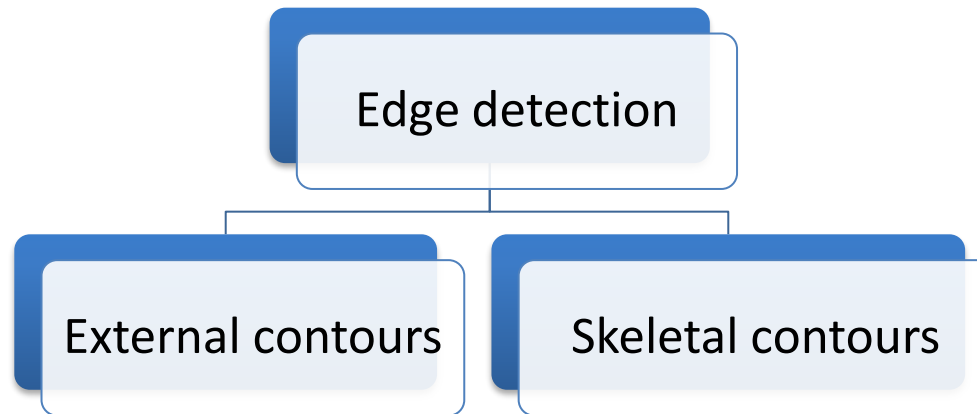
- 4x4 Gradient window
- Histogram of 4x4 samples per window in 8 directions
- Gaussian weighting around center(σ is 0.5 times that of the scale of a keypoint)
- 4x4x8 = 128 dimensional feature vector



Keypoint Detection



Edge Detection



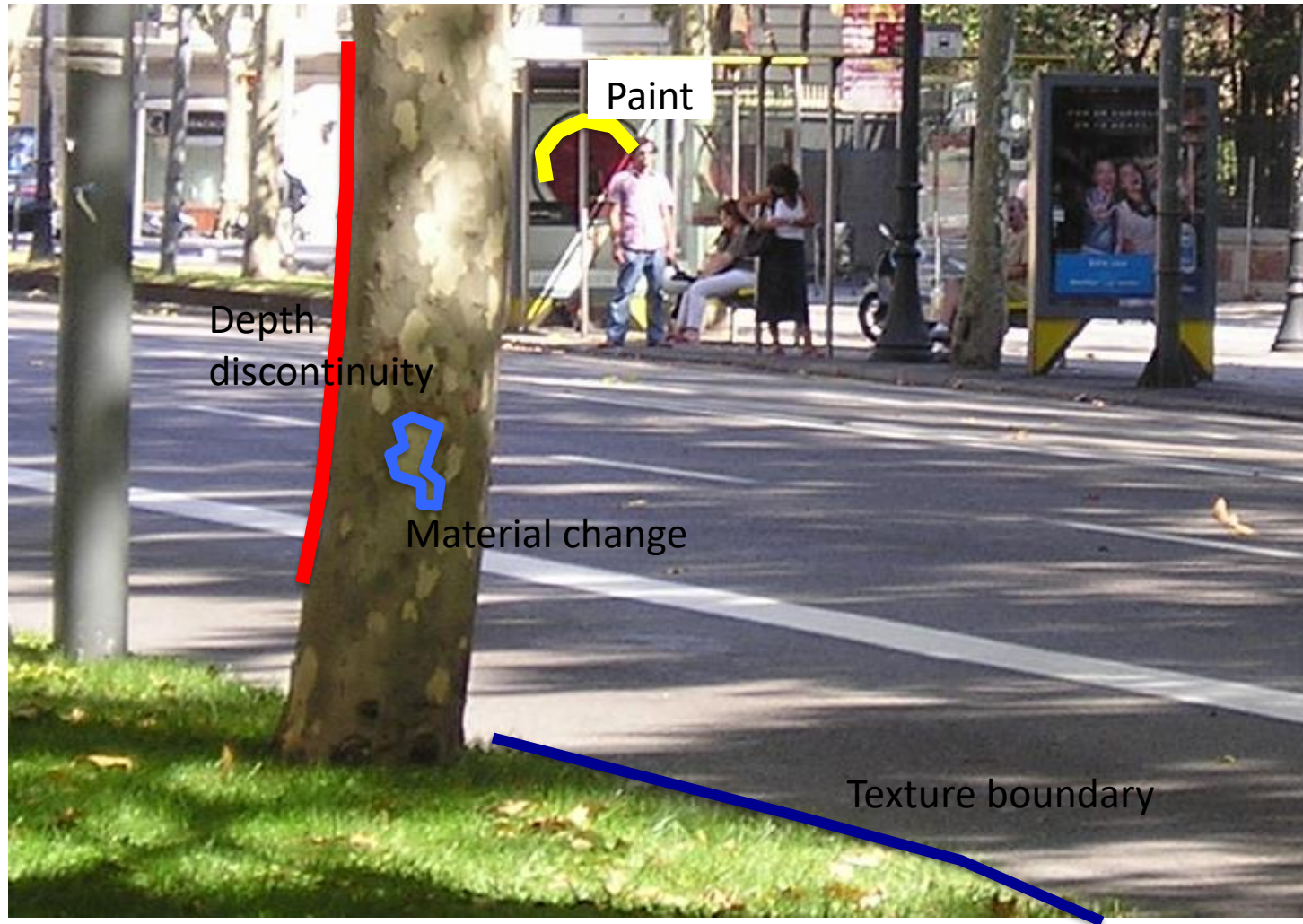
Edge Detection techniques classification

- **External contours**
 - Derivatives based techniques
 - **Gradient based techniques**
 - Sobel, Prewitt, **Canny edge detector**, etc.
 - Laplacian based techniques
 - LOG, Differential edge detection, etc.
 - Frequency and orientation based techniques
 - Gabor filters, Log Polar Gabor filters, etc.
 - **Morphological techniques**
 - **Morphological gradient**, Top hat, Watershed, etc.
 - Active contours
 - Snakes, level sets, etc.
- **Skeletal contours**
 - **Morphological detectors**
 - Using curve evolutions
 - Using the distance function
 - Using the level sets

Edge Detection: External contours

Operator	Accuracy	Complexity and computing time	Closed contours
Derivatives based techniques	Medium	Low	Not usually
Frequency and orientation based techniques	High	High	NO
Morphological techniques	Medium	Low	NO
Active contours	High	High	Yes

What is an edge?



Characterizing edges

- An edge is a place of rapid change in the image intensity function

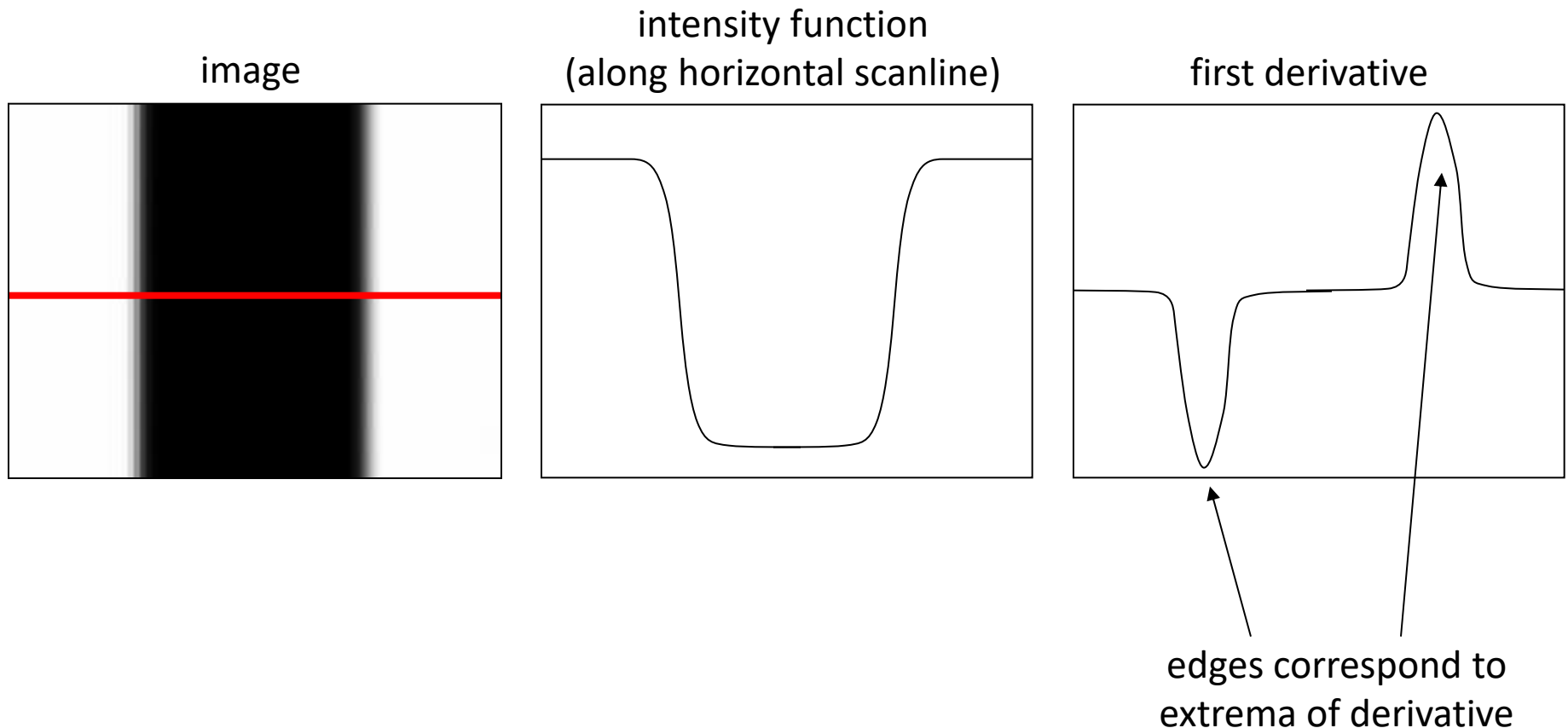
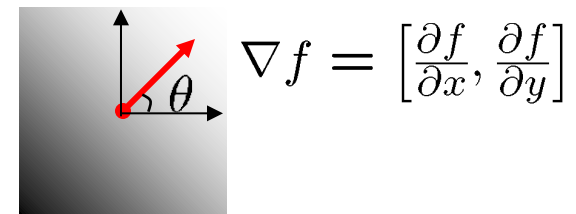
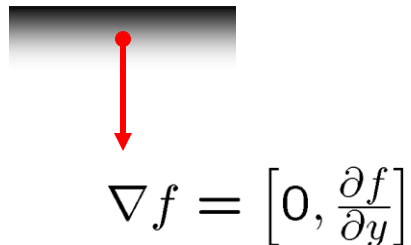
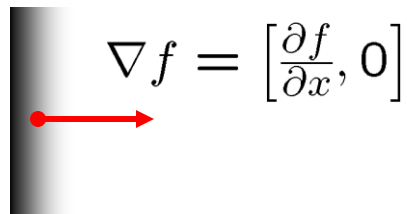


Image gradient

The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$



The gradient points in the direction of most rapid increase in intensity

The gradient direction is given by $\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

The edge strength is given by the gradient magnitude $\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}$

Differentiation and convolution

Recall, for 2D function,
 $f(x,y)$:

$$\frac{\partial f}{\partial x} = \lim_{\varepsilon \rightarrow 0} \left(\frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon} \right)$$

This is linear and shift invariant, so must be the result of a convolution.

We could approximate this as

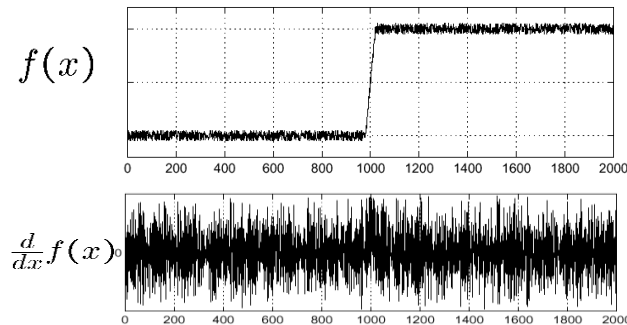
$$\frac{\partial f}{\partial x} \approx \frac{f(x_{n+1}, y) - f(x_n, y)}{\Delta x}$$

(which is obviously a convolution)

-1	1
----	---

Edge detection challenges

Noise !!



Where is the edge?



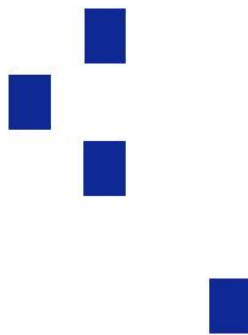
How do we link the edge points to form curves?

Criteria for an “optimal” edge detector

- **Good detection:** the optimal detector must minimize the probability of false positives (detecting spurious edges caused by noise), as well as that of false negatives (missing real edges)
- **Good localization:** the edges detected must be as close as possible to the true edges
- **Single response:** the detector must return one point only for each true edge point; that is, minimize the number of local maxima around the true edge



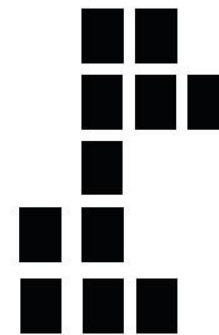
True
edge



Poor robustness
to noise



Poor
localisation

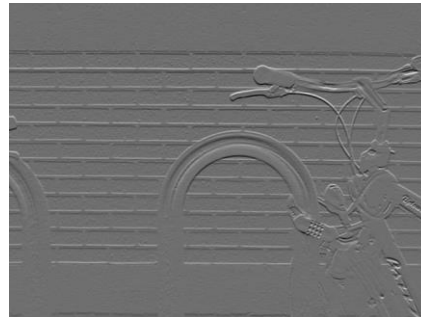


Too many
responses

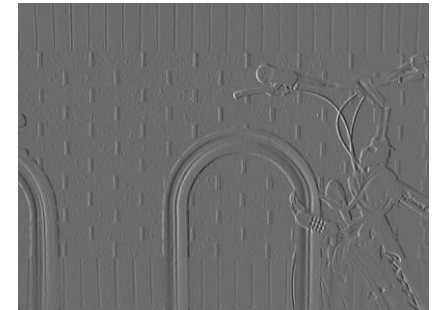
Edge Detection basic: Sobel operator

- Gradient based technique, mask approximation
- Looks for edges in both horizontal and vertical directions, then combine the information into a single metric.
- The masks are as follows:

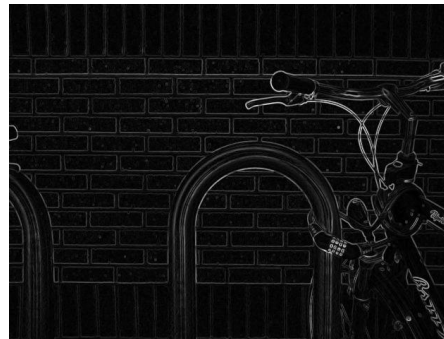
$$y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$



$$x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$



Edge Magnitude $\sqrt{x^2 + y^2}$

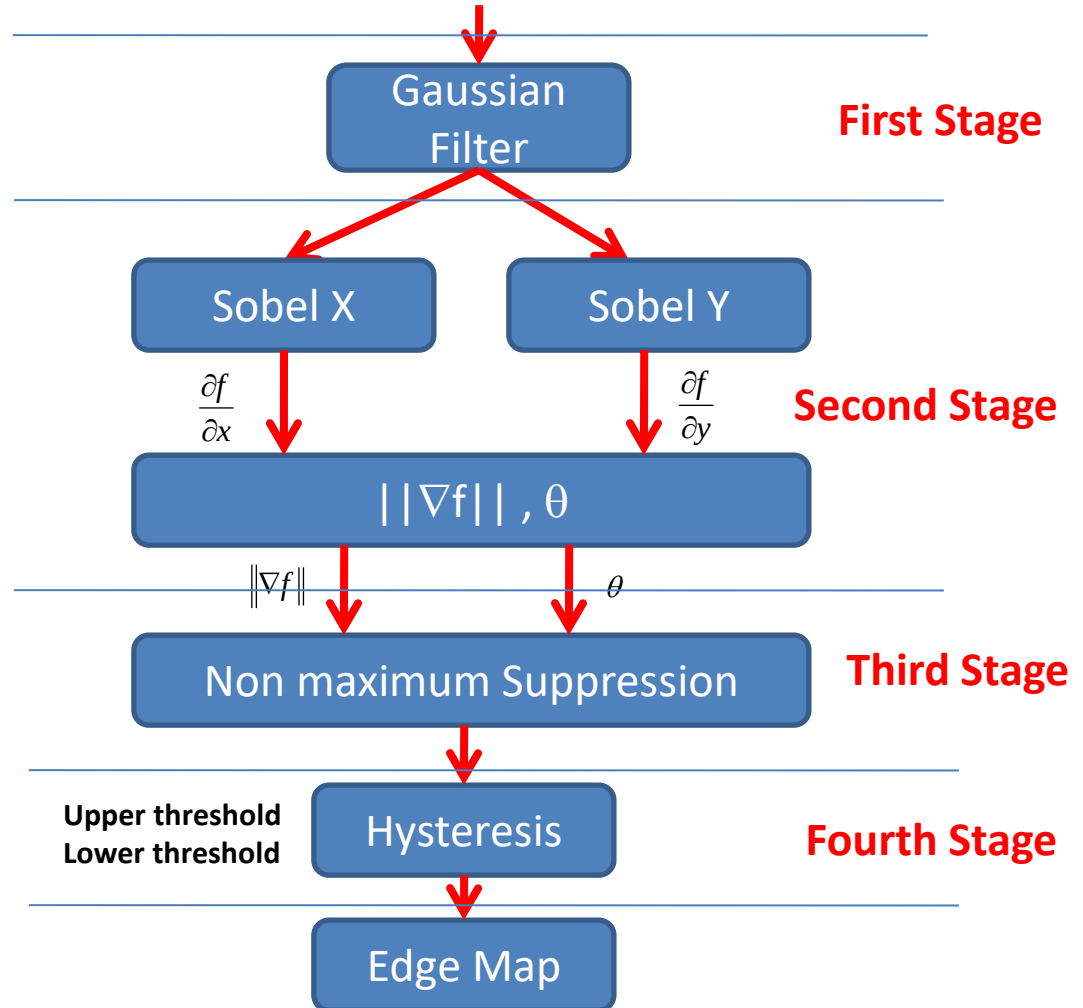


Original image

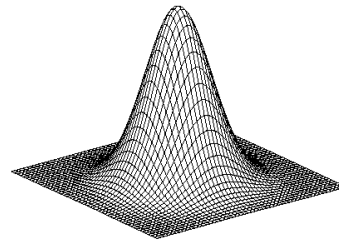
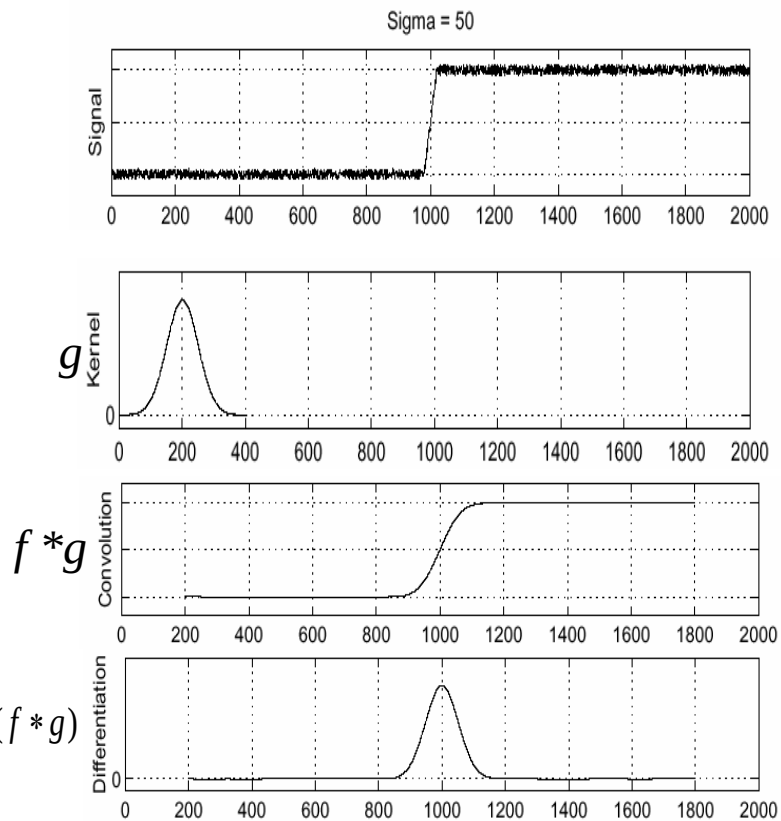
Canny edge detector

This is probably the most widely used edge detector in computer vision

Canny Edge Detector main steps



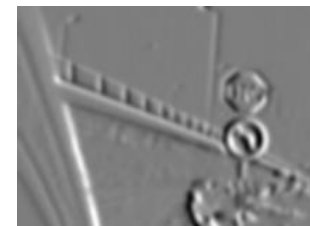
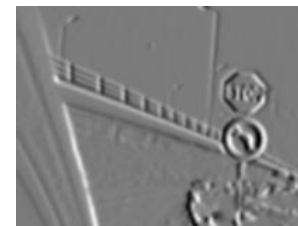
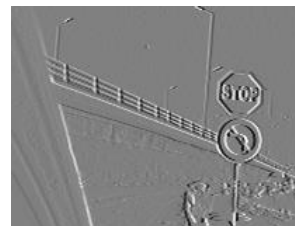
1/ Gaussian filtering



$$\frac{1}{115}$$

2	4	5	4	2
4	9	12	9	4
5	12	15	12	5
4	9	12	9	4
2	4	5	4	2

Discrete approximation to Gaussian function with $\sigma=1,4$



2/ Find magnitude and orientation of gradient

-1	0	+1
-2	0	+2
-1	0	+1

x

+1	+2	+1
0	0	0
-1	-2	-1

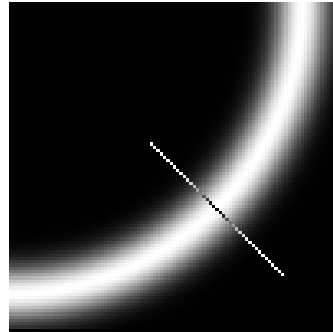
y

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

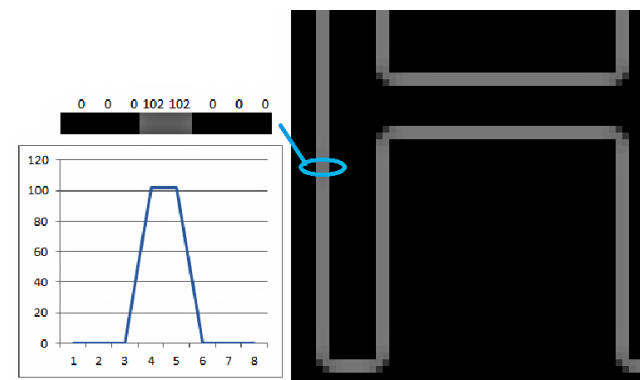
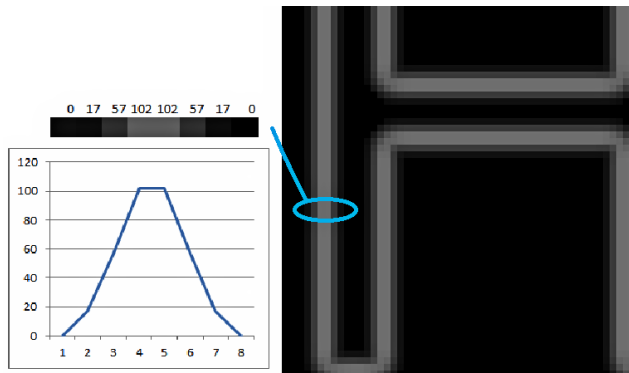
$$\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$$

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

3/ Non maximum Suppression

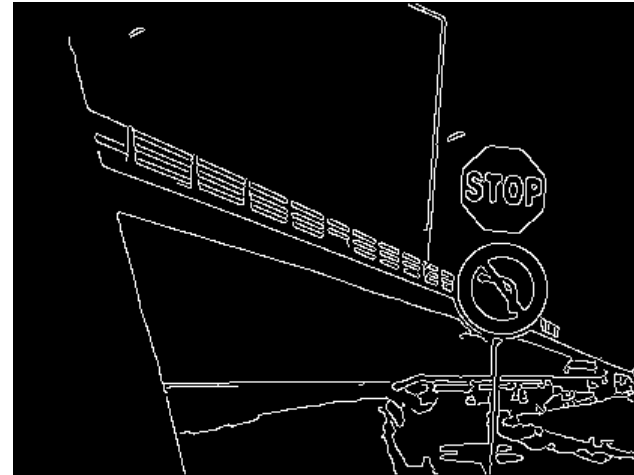


1. Round the gradient direction θ to nearest 45° , (0° , 45° , 90° and 135°) corresponding to the use of an 8-connected neighborhood.
2. Compare the edge strength of the current pixel with the edge strength of the pixel in the positive and negative gradient direction. I.e. if the gradient direction is north ($\theta = 90^\circ$), compare with the pixels to the north and south.
3. If the edge strength of the current pixel is largest; preserve the value of the edge strength. If not, suppress (i.e. remove) the value.



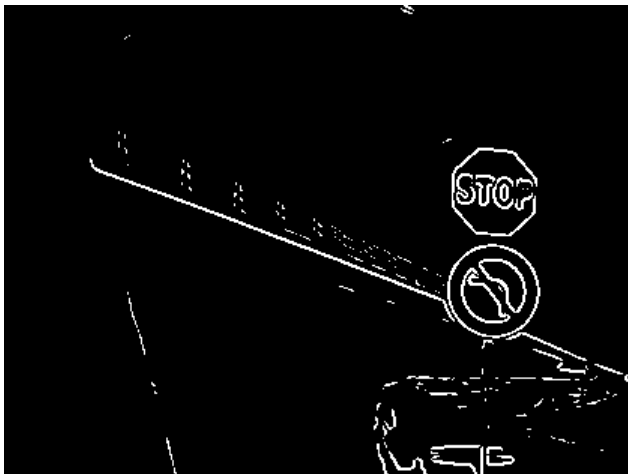
4/ Hysteresis

Original
image



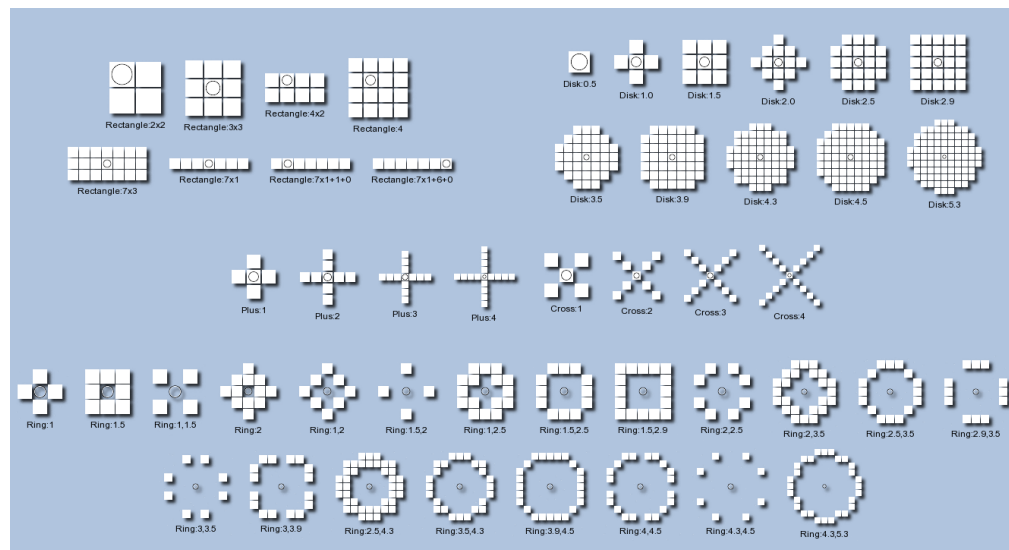
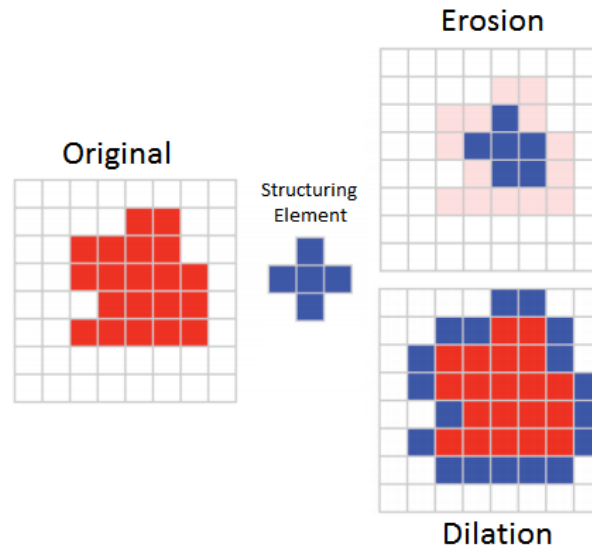
Strong +
connected
weak edges

Strong
edges
only



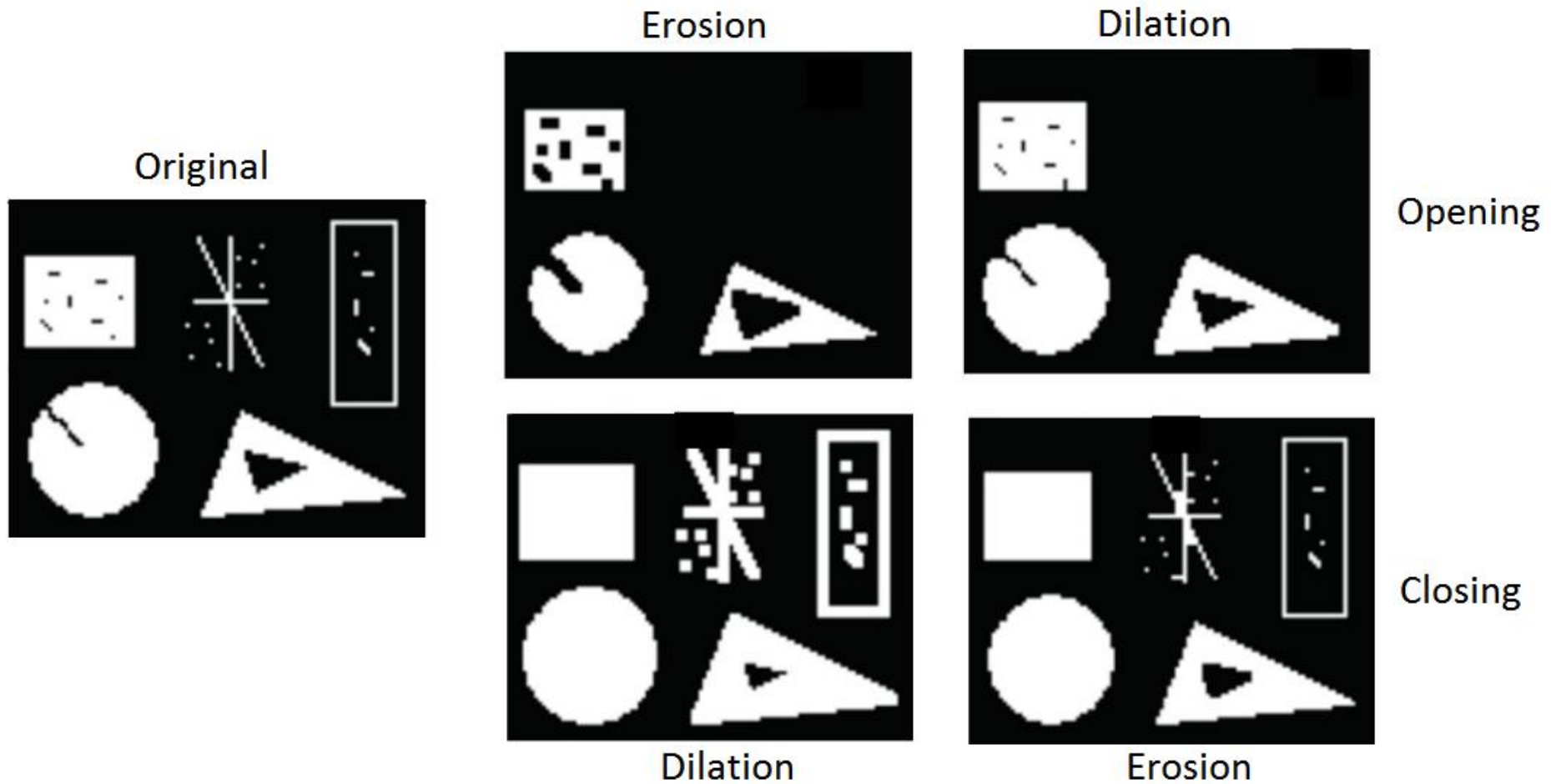
Weak
edges

Edge Detection : Morphological operators

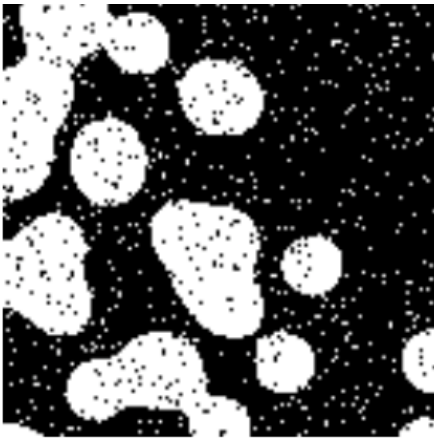


Structuring elements

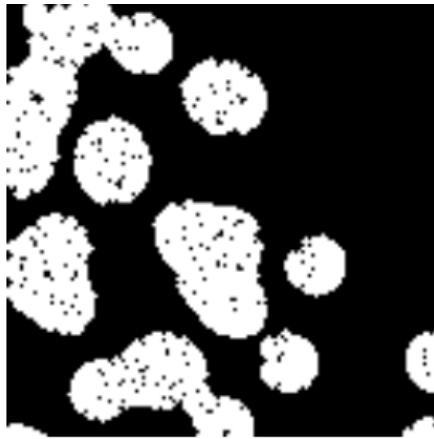
Edge Detection : Morphological operators



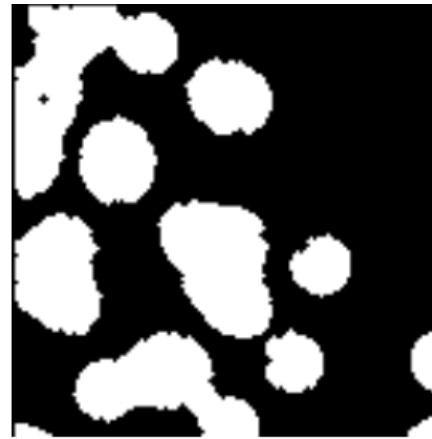
Edge Detection : Morphological operators



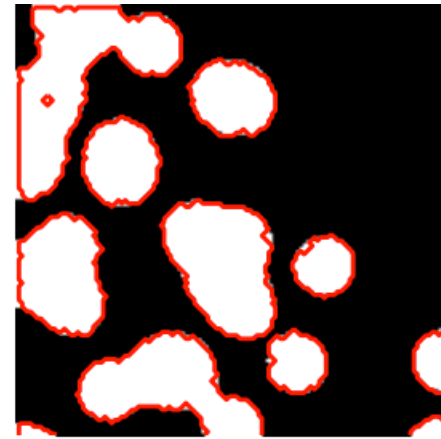
Original



Morphological
operator1



Morphological
operator2



Morphological
operator3

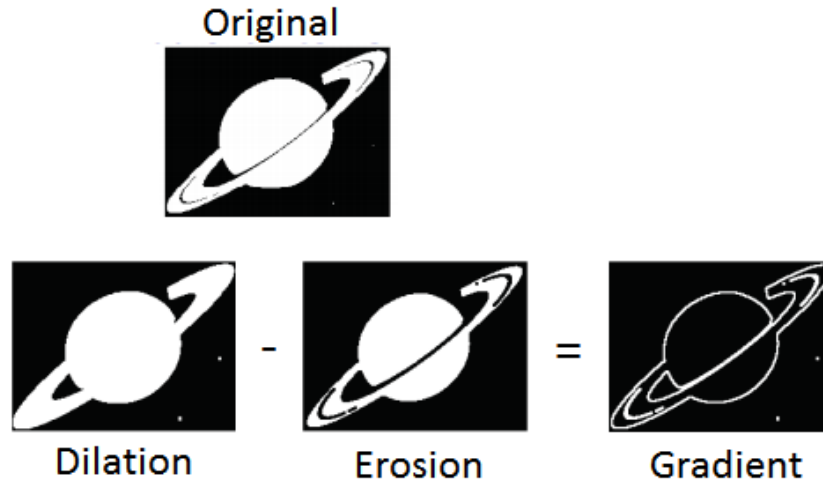
Morphological operator1 =?

Morphological operator2 =?

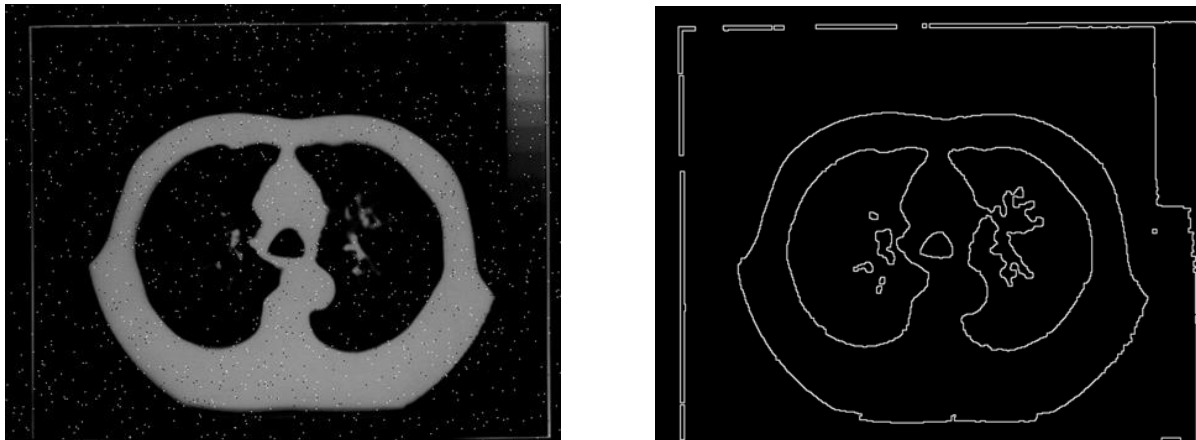
Morphological operator3 =?

Edge Detection : Morphological operators

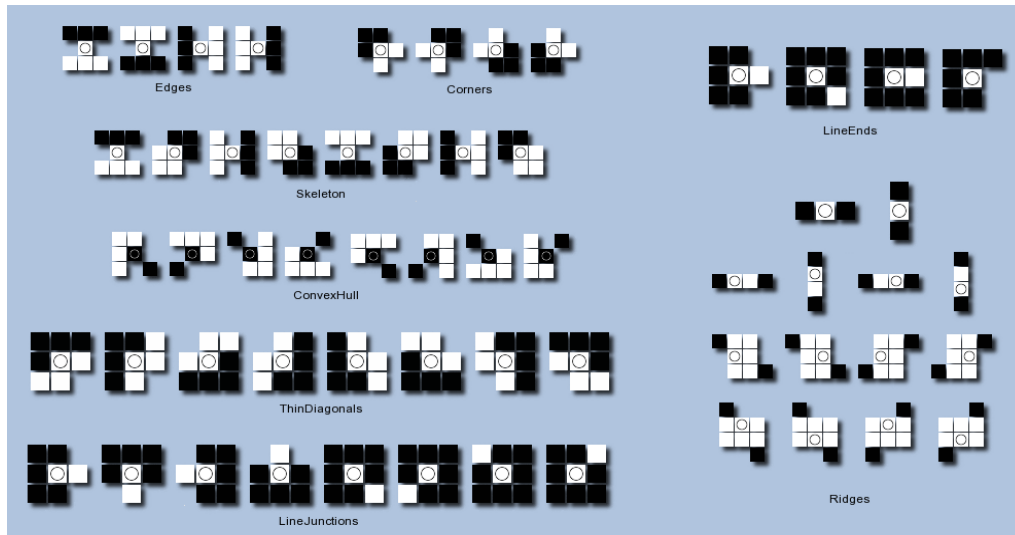
- Morphological Gradient



- Morphological Edge Detection

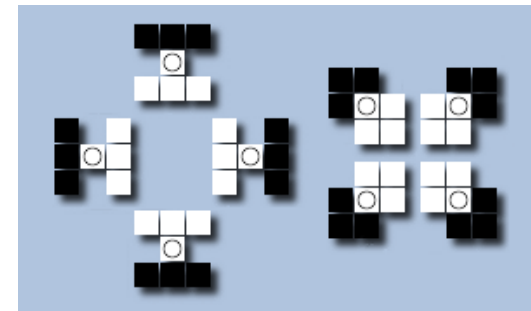
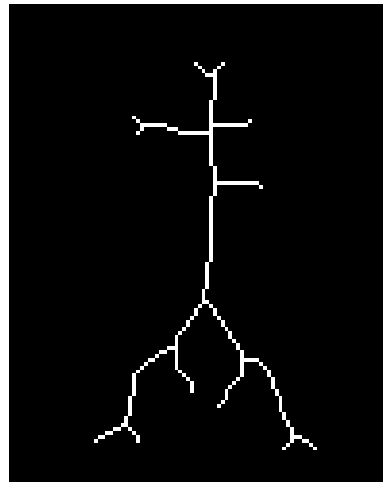
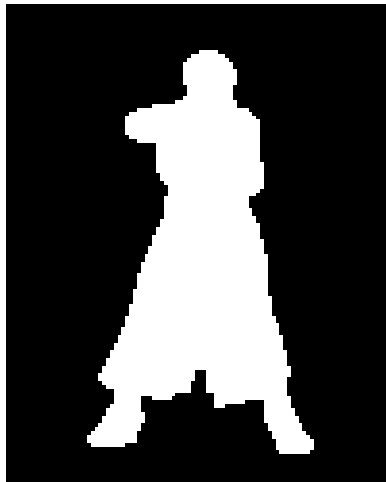


Edge Detection : Skeletal contours



Structuring elements

Skeletal contours



Edge Detection



Reflexion

How we can detect the object to extract it?



Reflexion

How we can detect the object to extract it?



Segmentation for object detection

- Image segmentation is typically used to locate objects and boundaries in images.
- Image segmentation is the process of assigning a label to every pixel in an image such that pixels with the same label share certain visual characteristics.
- **Supervised and Unsupervised segmentation**



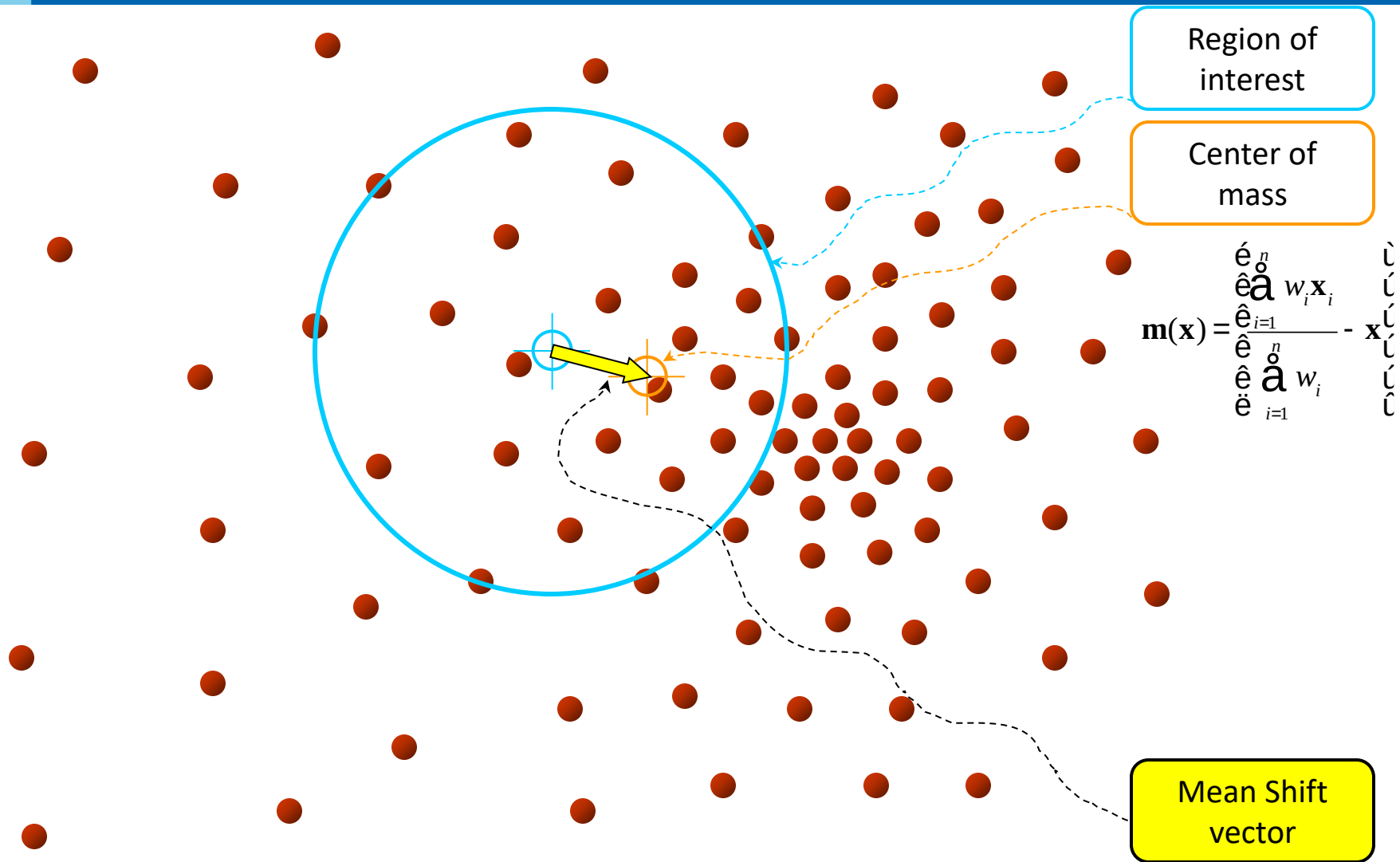
Segmentation for object detection

Operatorc	Accuracy	Complexity and computing time
Unsupervised techniques	Medium	Low
Supervised techniques	High	High

Unsupervised Segmentation techniques

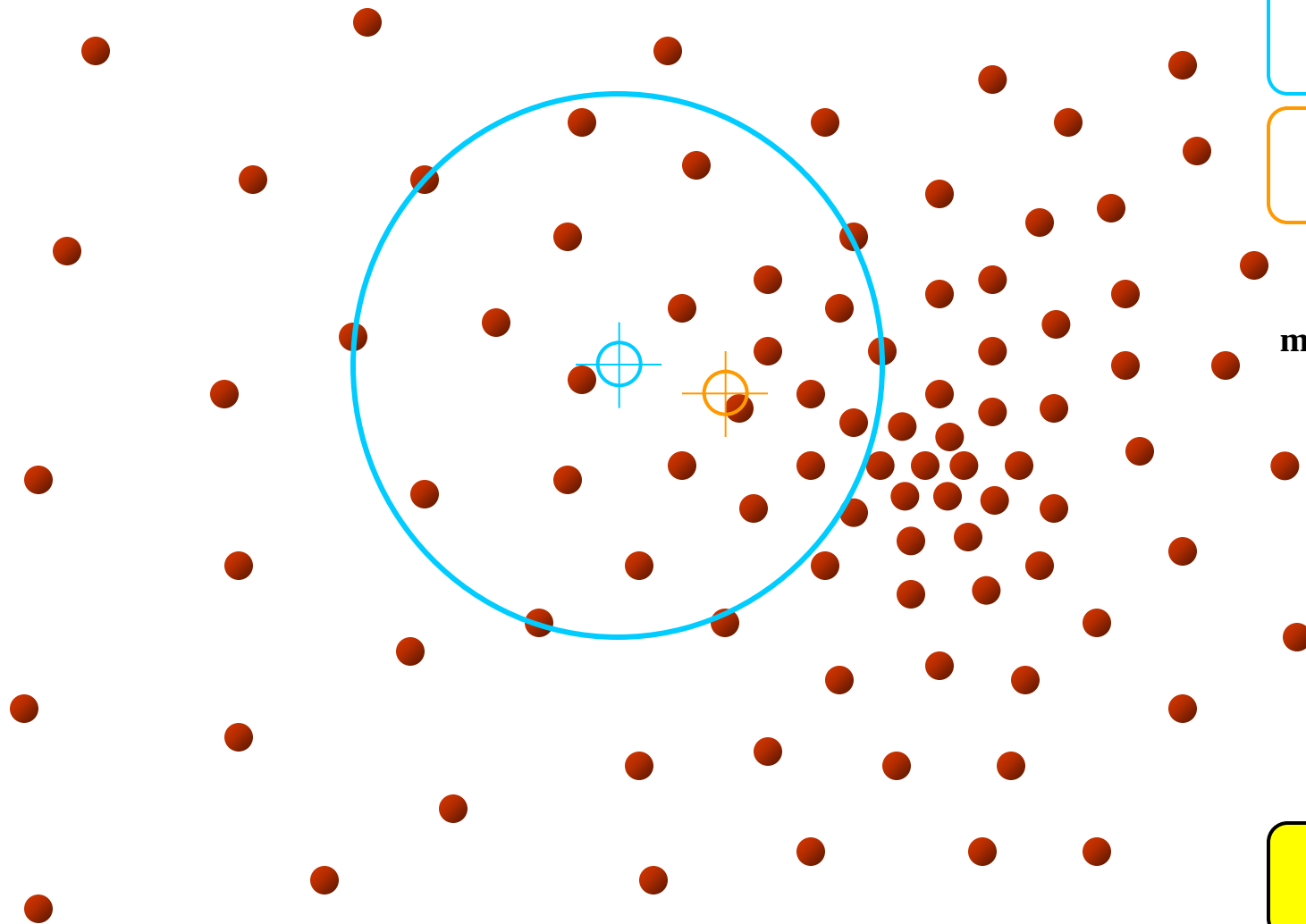
- Unsupervised Segmentation algorithms:
 - Thresholding methods
 - Clustering methods
 - K-means, **MeanShift**, EM, etc.
 - Edge detection based methods
 - Canny edge detector, Snakes, Level sets, etc.
 - Region-growing methods
 - **Region-growing**, λ -connected, etc.
 - Split-and-merge methods
 - Quadtree segmentation, etc.
 - Partial differential equation-based methods
 - Snakes, Level Sets, etc.
 - Graph partitioning methods
 - Graph cuts, Random Walker, etc.
 - Morphological based segmentation
 - Watershed, geodesic segmentation, etc.
 - Model based segmentation
 - Probabilistic segmentation, etc.

Mean shift: Intuitive Description



Objective : Find the densest region
Distribution of identical billiard balls

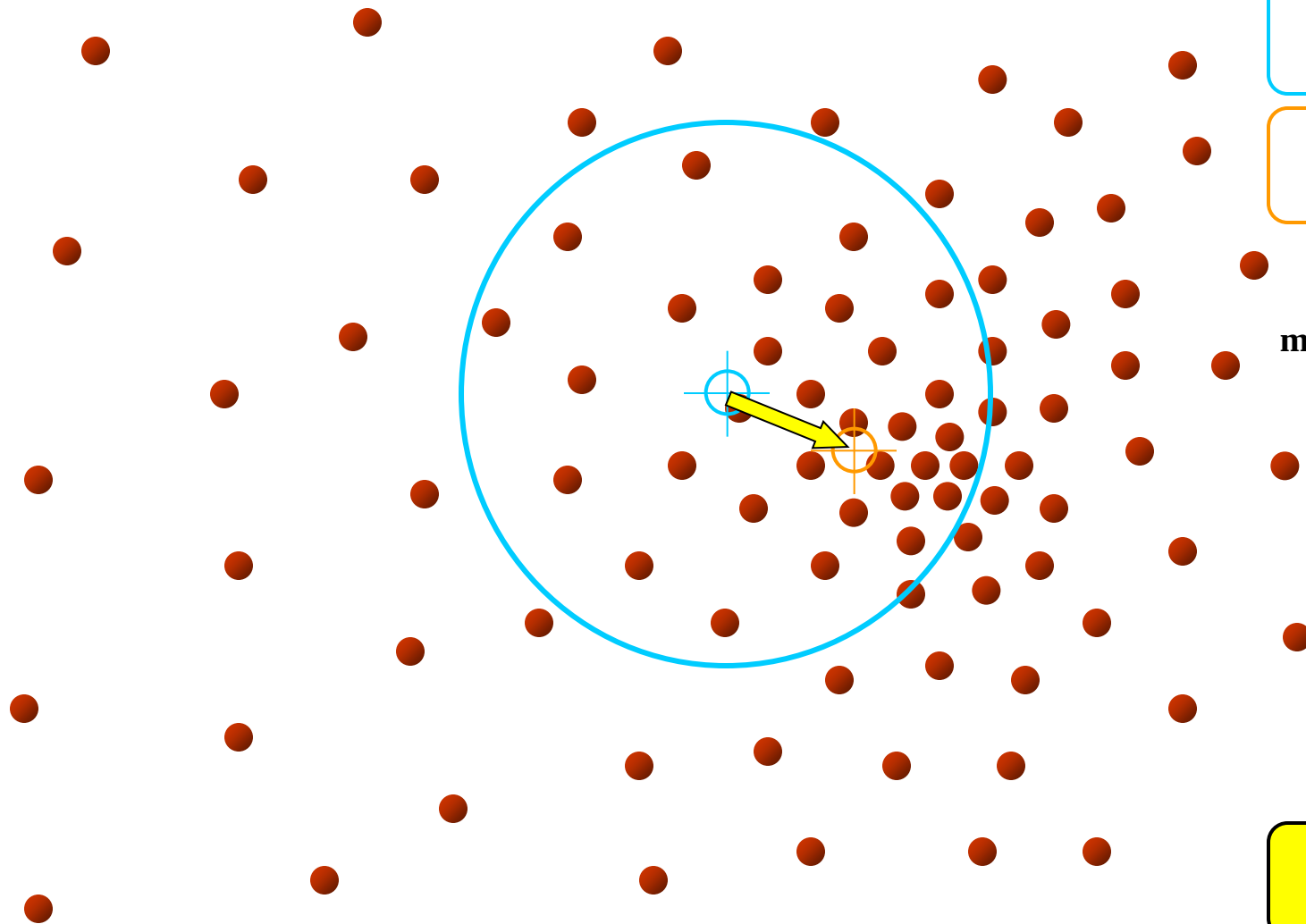
Mean shift: Intuitive Description



$$\mathbf{m}(\mathbf{x}) = \frac{\sum_{i=1}^n w_i \mathbf{x}_i}{\sum_{i=1}^n w_i} - \mathbf{x}$$

Objective : Find the densest region
Distribution of identical billiard balls

Mean shift: Intuitive Description



Region of
interest

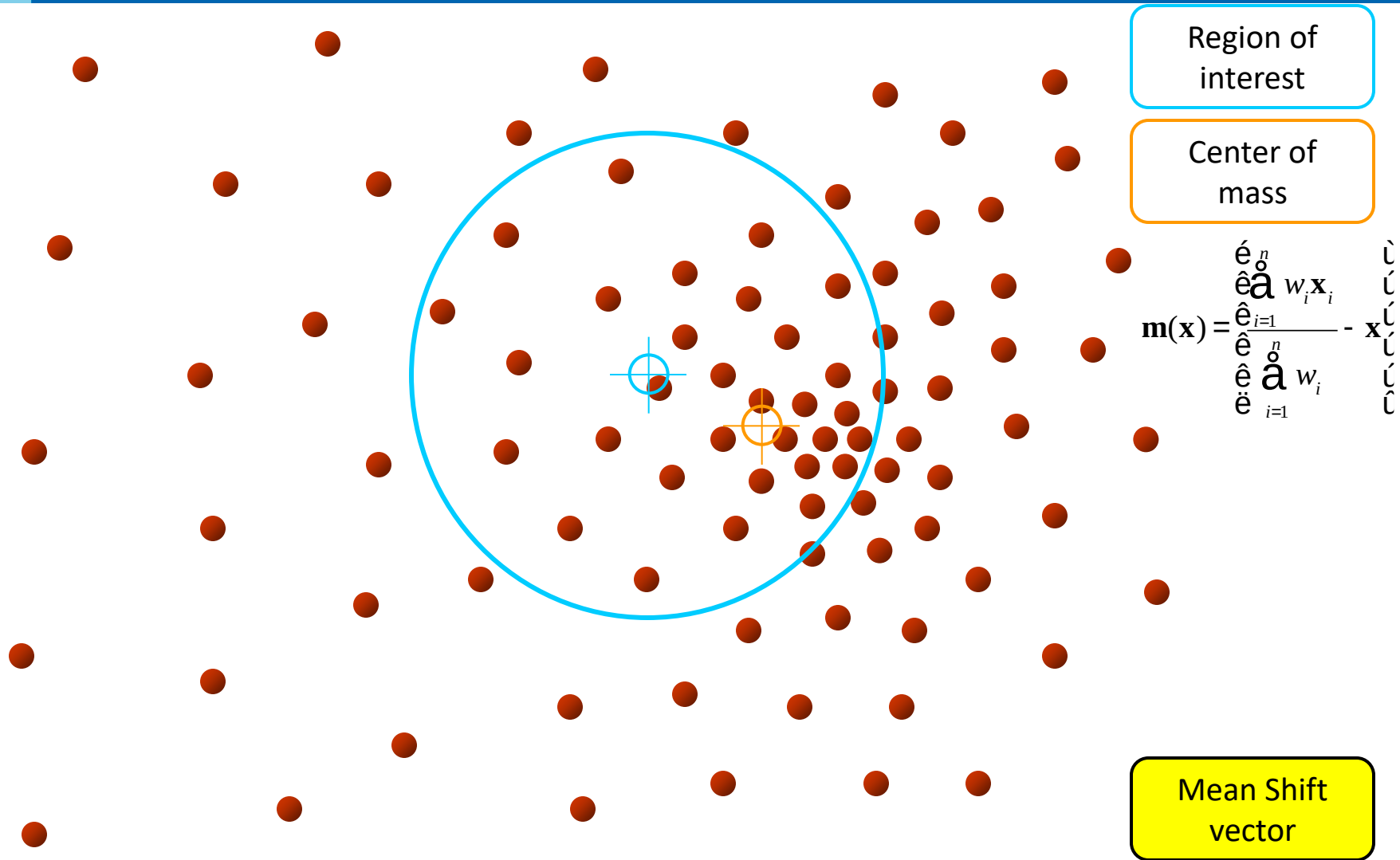
Center of
mass

$$\mathbf{m}(\mathbf{x}) = \frac{\sum_{i=1}^n w_i \mathbf{x}_i}{\sum_{i=1}^n w_i} - \mathbf{x}$$

Mean Shift
vector

Objective : Find the densest region
Distribution of identical billiard balls

Mean shift: Intuitive Description



Objective : Find the densest region
Distribution of identical billiard balls

Mean shift: Intuitive Description

Region of
interest

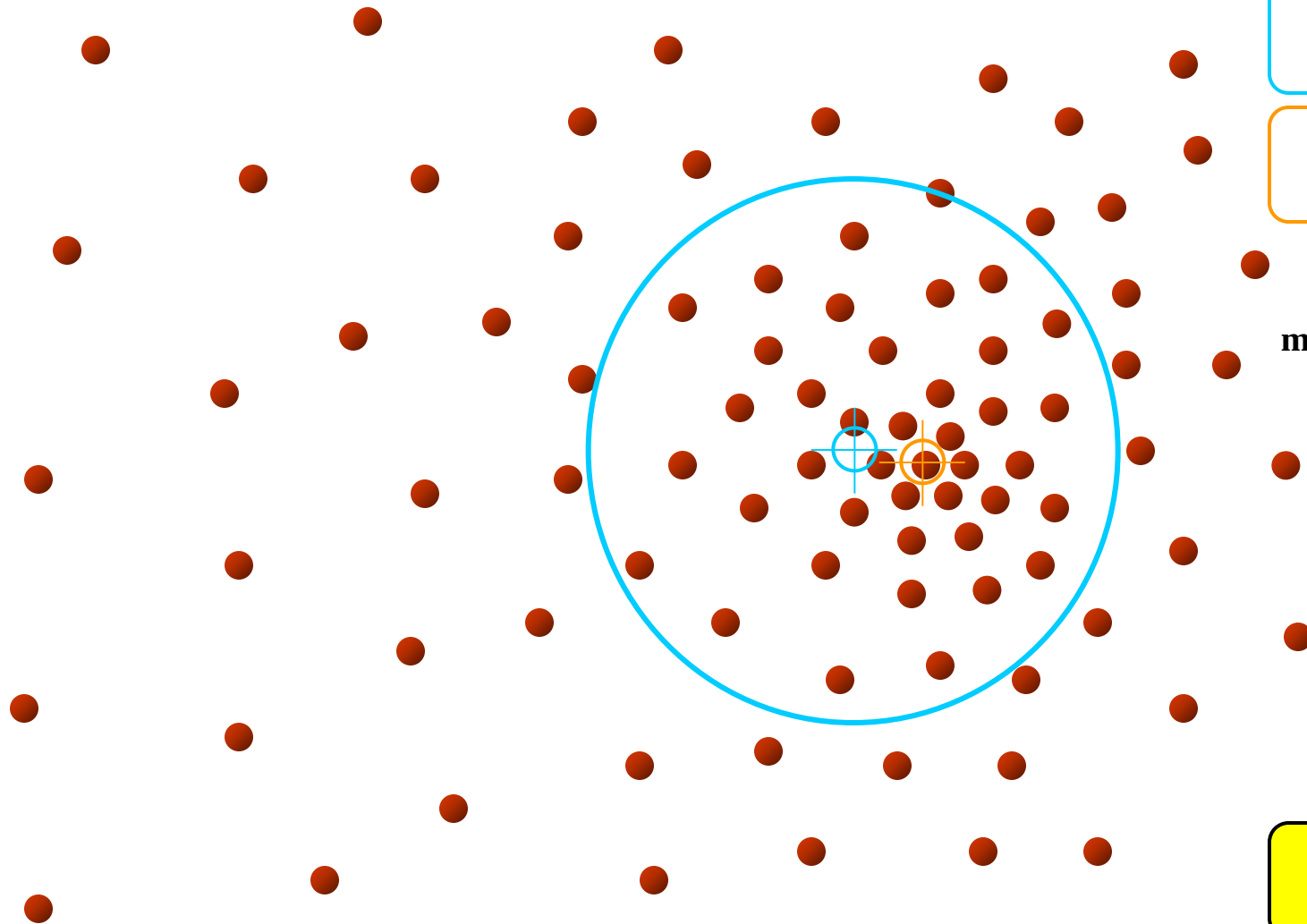
Center of
mass

$$\mathbf{m}(\mathbf{x}) = \frac{\sum_{i=1}^n w_i \mathbf{x}_i}{\sum_{i=1}^n w_i} - \mathbf{x}$$

Mean Shift
vector

Objective : Find the densest region
Distribution of identical billiard balls

Mean shift: Intuitive Description



Region of
interest

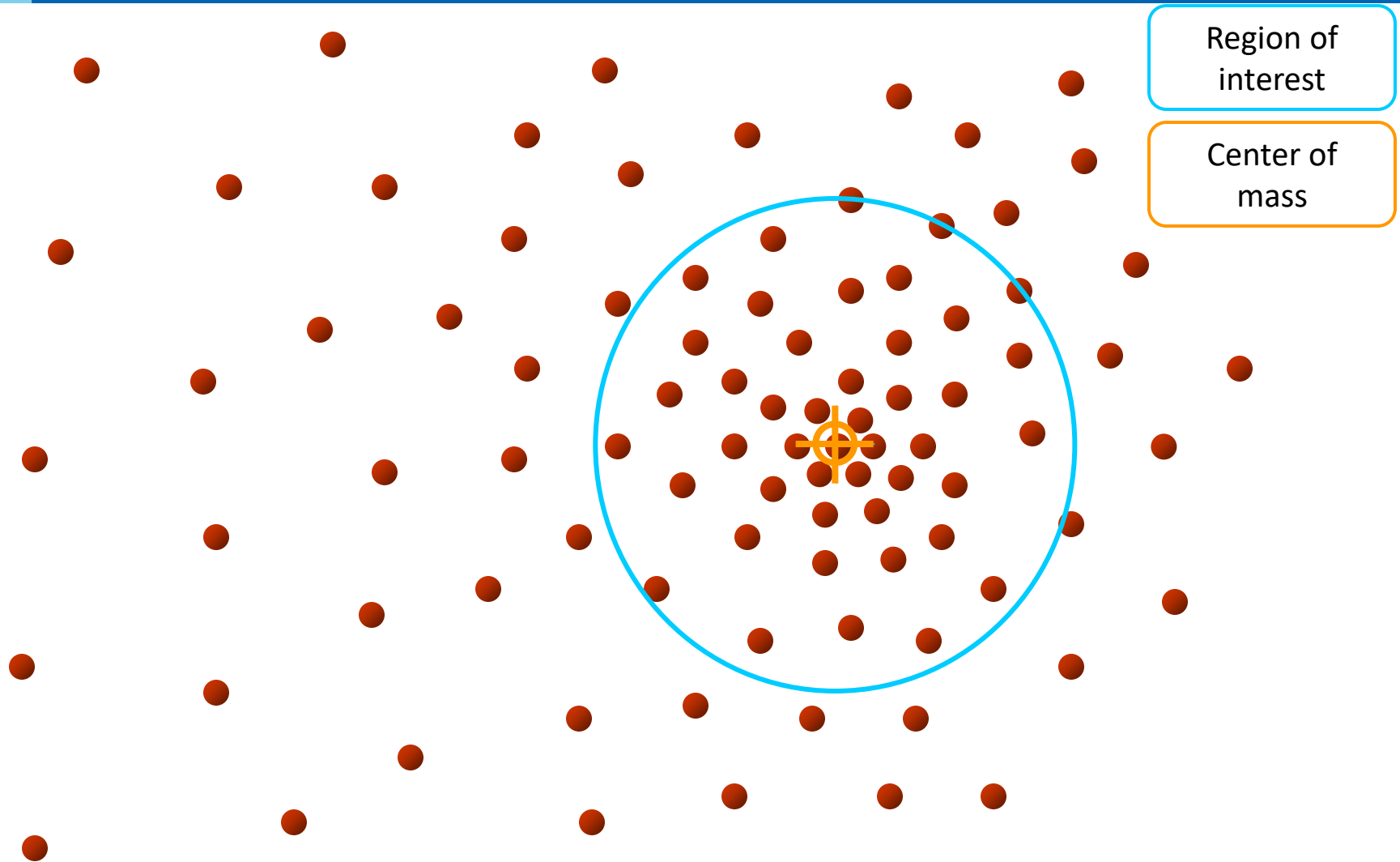
Center of
mass

$$\mathbf{m}(\mathbf{x}) = \frac{\sum_{i=1}^n w_i \mathbf{x}_i}{\sum_{i=1}^n w_i} - \mathbf{x}$$

Mean Shift
vector

Objective : Find the densest region
Distribution of identical billiard balls

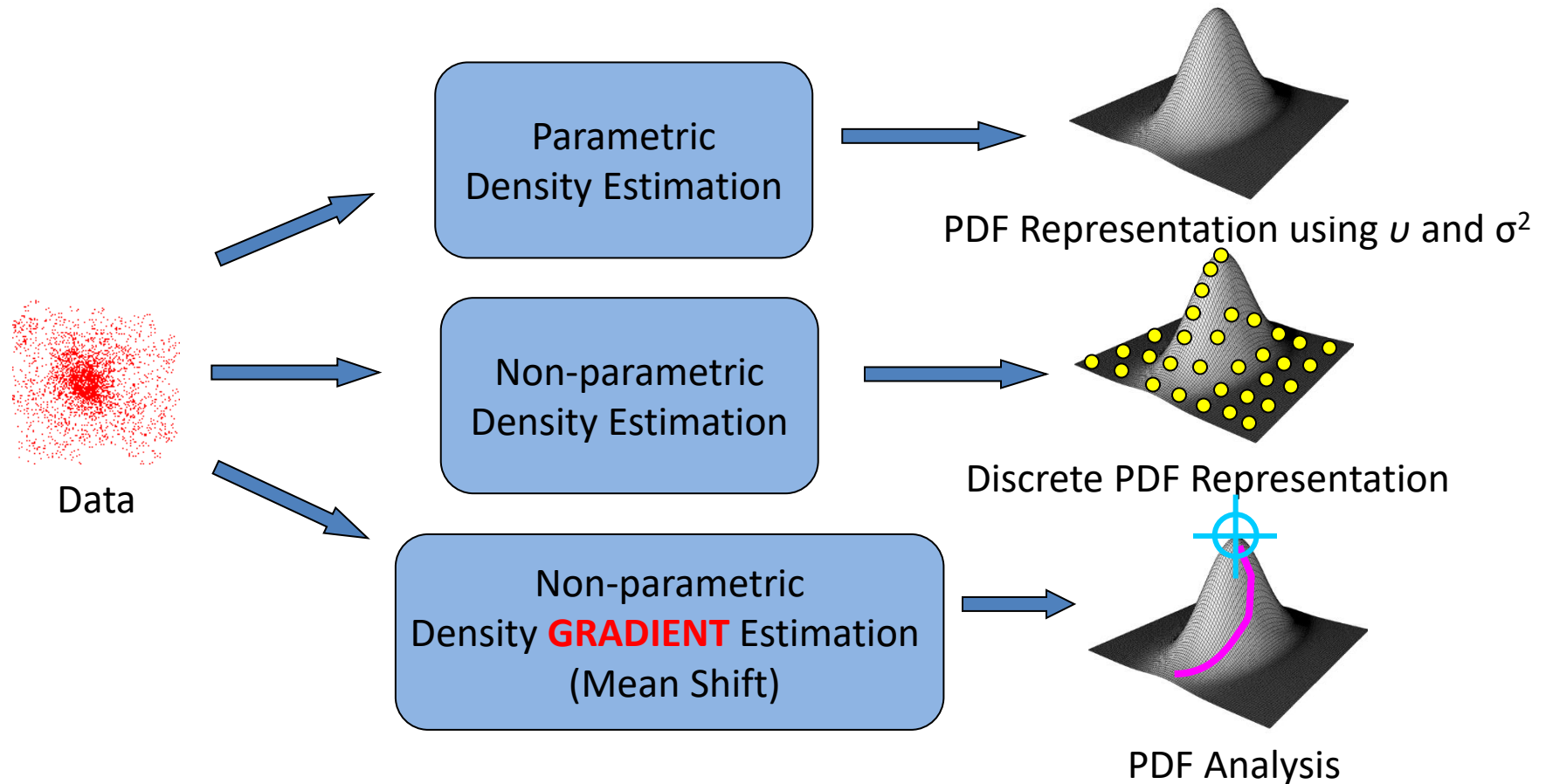
Mean shift: Intuitive Description



Objective : Find the densest region
Distribution of identical billiard balls

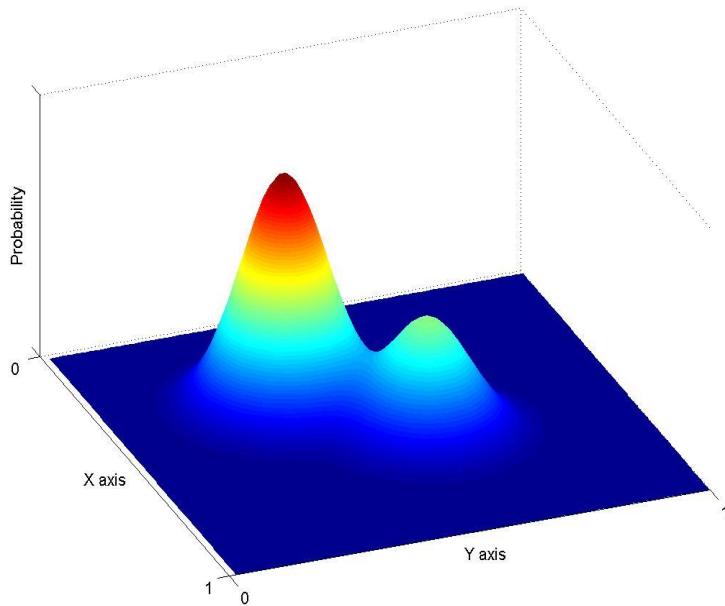
What is Mean Shift ?

A tool for: Finding modes in a set of data samples, manifesting an underlying probability density function (PDF)

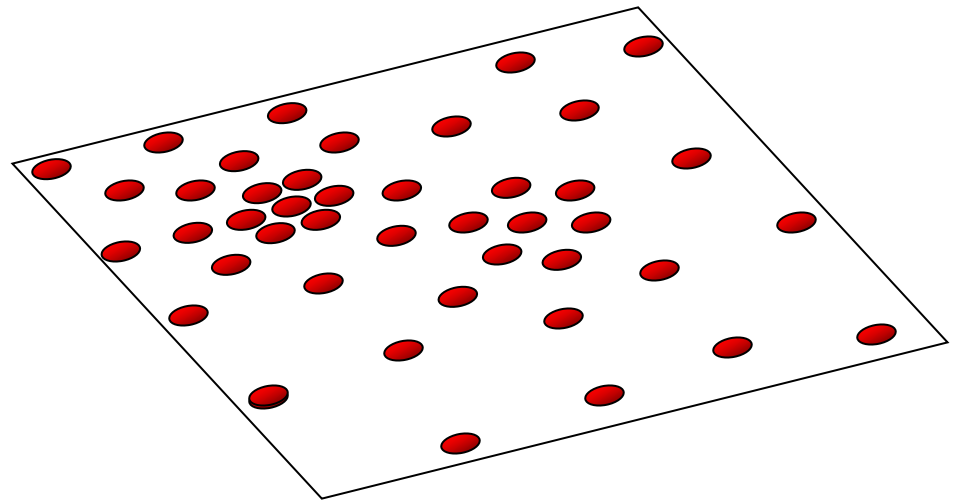


Parametric Density Estimation

Assumption : The data points are sampled from an underlying PDF



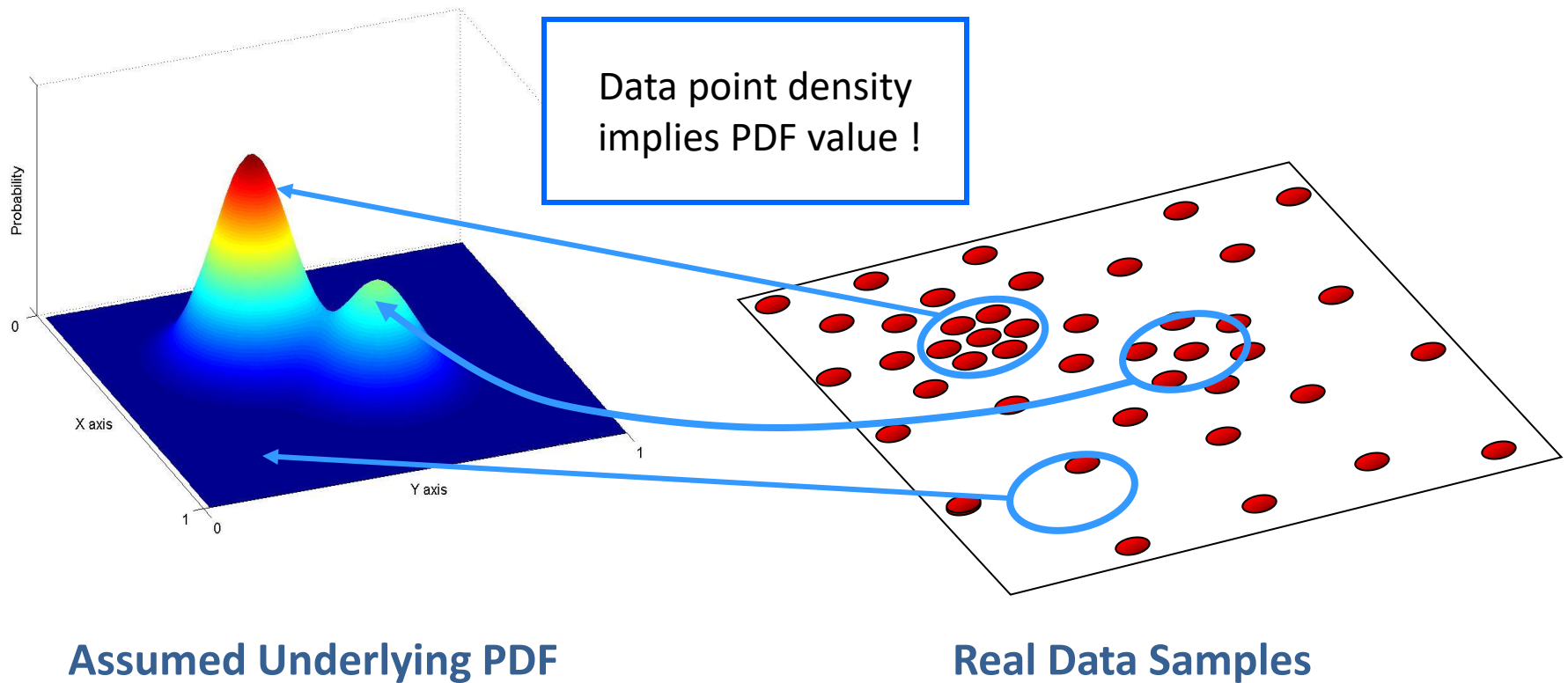
Assumed Underlying PDF



Real Data Samples

Non-Parametric Density Estimation

Assumption : The data points are sampled from an underlying PDF



Kernel Density Estimation: various kernels

$$P(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n K(\mathbf{x} - \mathbf{x}_i)$$

A function of some finite number of data points
 $\mathbf{x}_1 \dots \mathbf{x}_n$

Examples:

- Epanechnikov Kernel

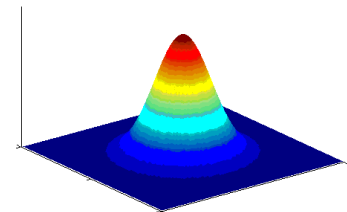
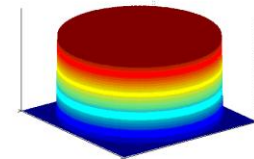
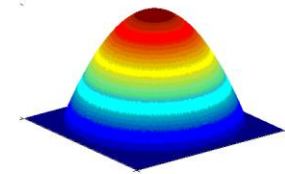
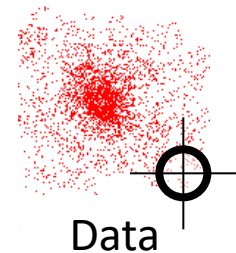
$$K_E(\mathbf{x}) = \begin{cases} c(1 - \|\mathbf{x}\|^2) & \|\mathbf{x}\| \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

- Uniform Kernel

$$K_U(\mathbf{x}) = \begin{cases} c & \|\mathbf{x}\| \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

- Normal Kernel

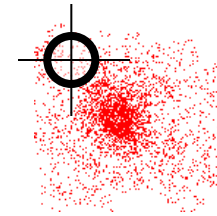
$$K_N(\mathbf{x}) = c \cdot \exp\left(-\frac{1}{2}\|\mathbf{x}\|^2\right)$$



Kernel Density Estimation

$$P(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n K(\mathbf{x} - \mathbf{x}_i)$$

A function of some finite number of data points
 $\mathbf{x}_1 \dots \mathbf{x}_n$



Data

In practice one uses the forms:

$$K(\mathbf{x}) = c \prod_{i=1}^{dx} k(x_i^2) \quad \text{or} \quad K(\mathbf{x}) = ck\left(\|\mathbf{x}\|^2\right)$$

Same function on each dimension

Function of vector length only called profile

Kernel Density Estimation

$$\nabla P(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \nabla K(\mathbf{x} - \mathbf{x}_i)$$

Give up estimating the PDF !
Estimate **ONLY** the gradient

Using the
Kernel form (profile): $K(\mathbf{x} - \mathbf{x}_i) = ck\left(\|\mathbf{x} - \mathbf{x}_i\|^2\right)$

We get :

$$\nabla P(\mathbf{x}) = \frac{c}{n} \left[\sum_{i=1}^n g_i \right] \left[\frac{\sum_{i=1}^n \mathbf{x}_i g_i}{\sum_{i=1}^n g_i} - \mathbf{x} \right]$$
$$g_i = g\left(\|\mathbf{x} - \mathbf{x}_i\|^2\right)$$
$$g(\mathbf{x}) = -k'(\mathbf{x})$$

Computing The Mean Shift

$$\nabla P(\mathbf{x}) = \frac{c}{n} \sum_{i=1}^n \nabla k_i$$

$$= \frac{c}{n} \left[\sum_{i=1}^n g_i \right]$$

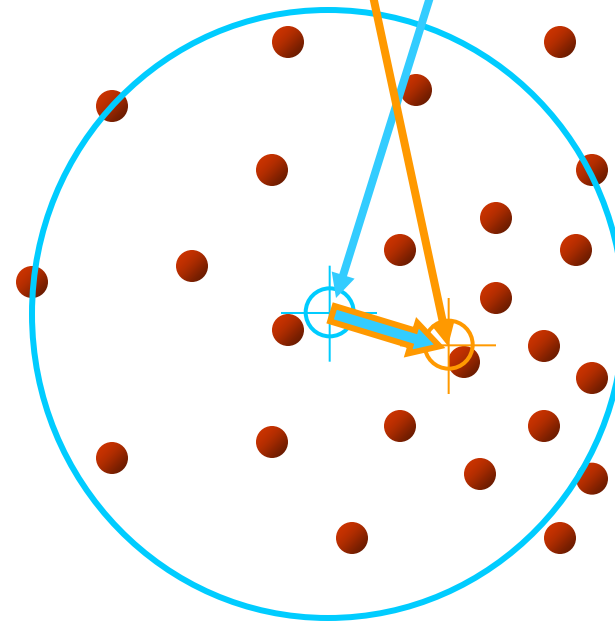
$$\left[\frac{\sum_{i=1}^n \mathbf{x}_i g_i}{\sum_{i=1}^n g_i} - \mathbf{x} \right]$$

Yet another Kernel density estimation !

Simple Mean Shift procedure:

- Compute mean shift vector

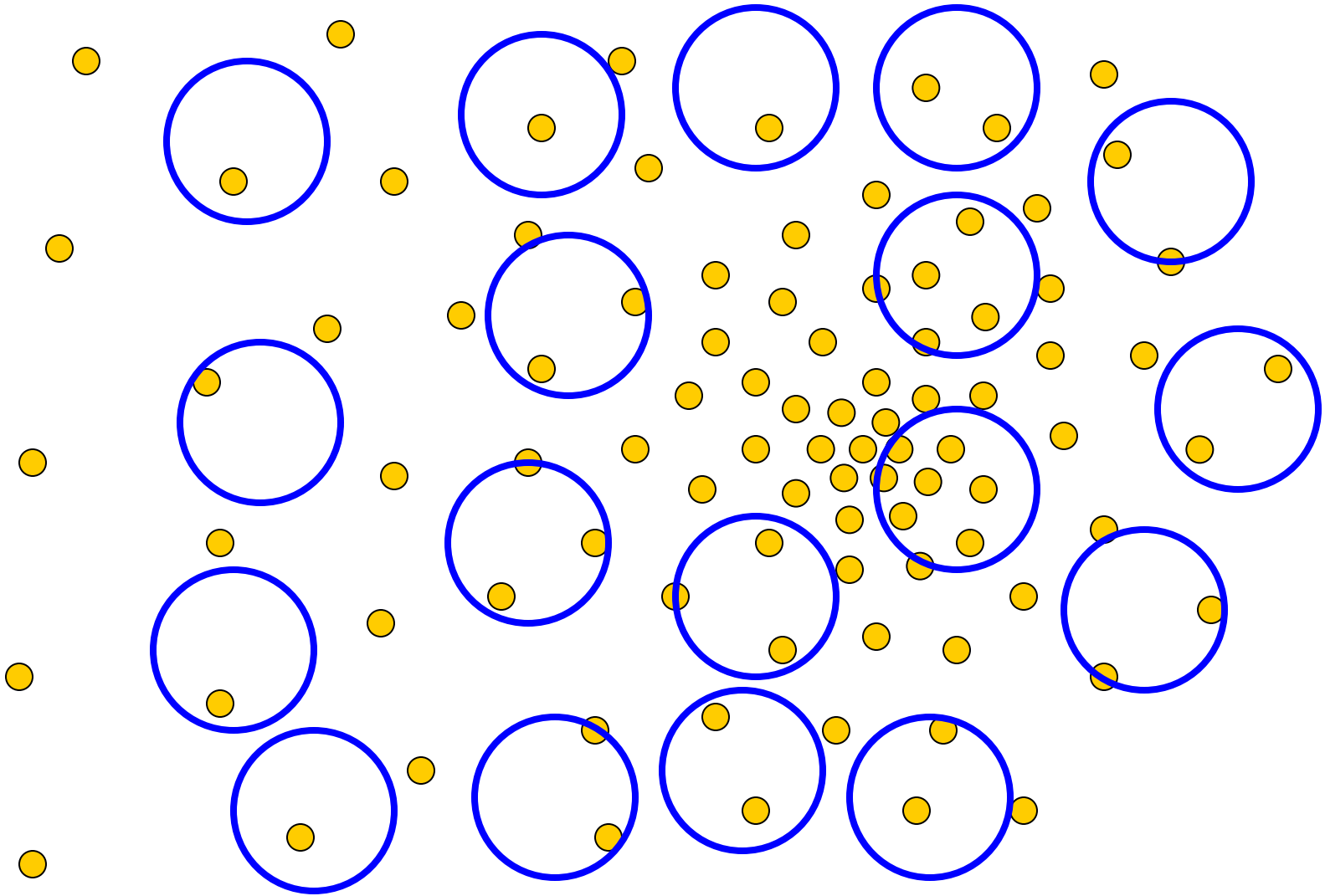
$$\mathbf{m}(\mathbf{x}) = \frac{\sum_{i=1}^n \mathbf{x}_i g(\|\mathbf{x} - \mathbf{x}_i\|^2)}{\sum_{i=1}^n g(\|\mathbf{x} - \mathbf{x}_i\|^2)} - \mathbf{x}$$



- Translate the Kernel window by $\mathbf{m}(\mathbf{x})$

$$g(\mathbf{x}) = -k'(\mathbf{x})$$

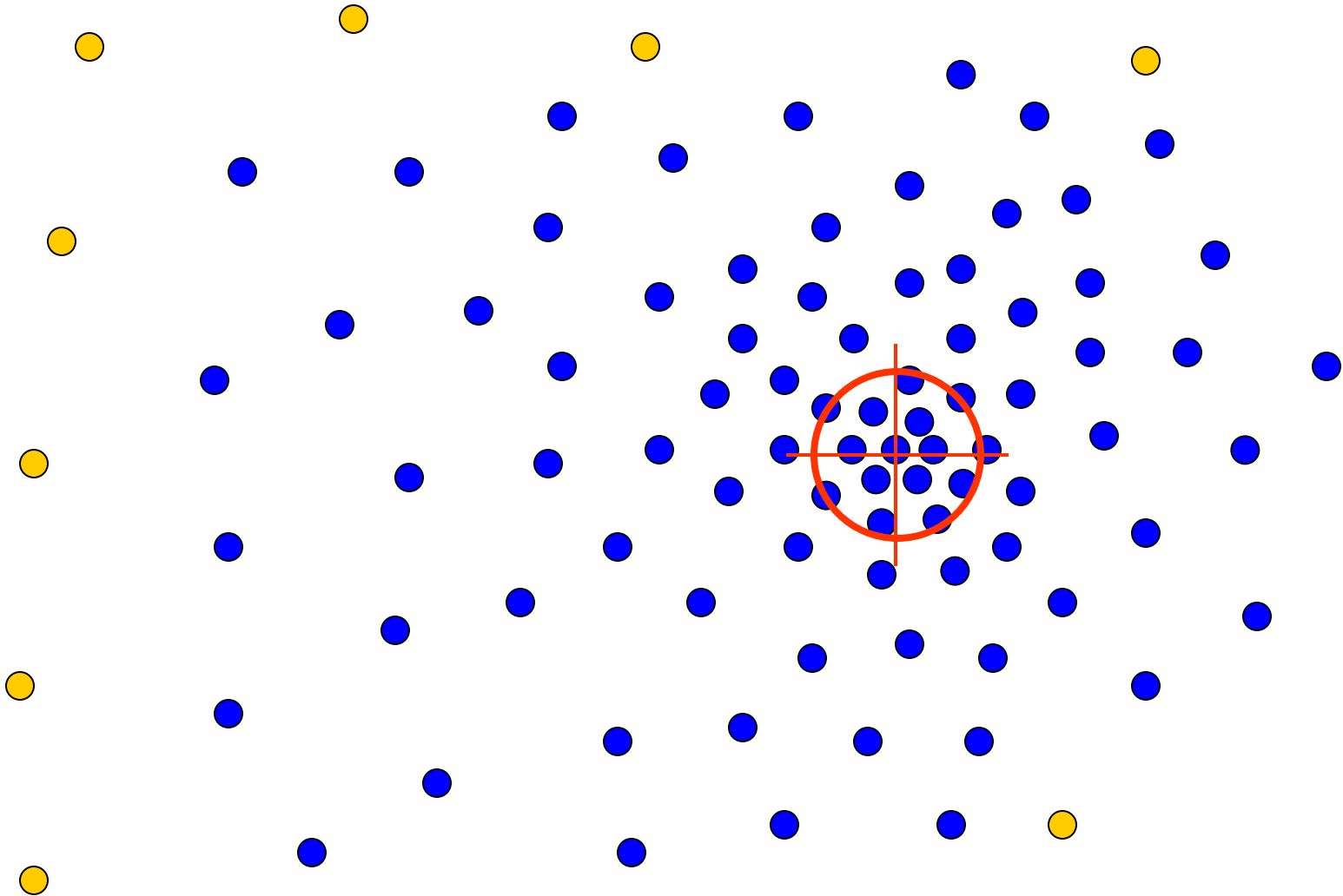
Real Modality Analysis



Tessellate the space
with windows

Run the procedure in parallel

Real Modality Analysis



The blue data points were traversed by the windows towards the mode

Mean shift algorithm main steps

- The mean shift algorithm seeks *modes* of the given set of points
 1. Choose kernel and bandwidth
 2. For each point:
 - a) Center a window on that point
 - b) Compute the mean of the data in the search window
 - c) Center the search window at the new mean location
 - d) Repeat (b,c) until convergence
 3. Assign points that lead to nearby modes to the same cluster

Mean shift segmentation results



Region growing segmentation

- Region growing techniques start with one pixel of a potential region and try to grow it by adding adjacent pixels till the pixels being compared are too dissimilar.

Idea:

- The first pixel selected can be just the first unlabeled pixel in the image or a set of seed pixels can be chosen from the image.
- Usually a statistical test is used to decide which pixels can be added to a region.

Region growing algorithm

- Let R be the N pixel region so far and P be a neighboring pixel with gray tone y .
- Define the mean M and scatter S (sample variance) by:

$$M = \frac{1}{N} \sum_{(r,c) \in R} I(r,c)$$

$$S^2 = \frac{1}{N} \sum_{(r,c) \in R} (I(r,c) - M)^2$$

- The T statistic is defined by

$$T = \frac{(N-1)N}{N+1} \frac{(y - M)^2}{S^2}^{1/2}$$

Region growing algorithm

- For the T distribution, statistical tables give us the probability $Pr(T \leq t)$ for a given degrees of freedom and a confidence level. From this, pick suitable threshold t .
- If the computed $T \leq t$ for desired confidence level, add y to region R and update M and S .
- If T is too high, the value y is not likely to have arise from the population of pixels in R . Start a new region.

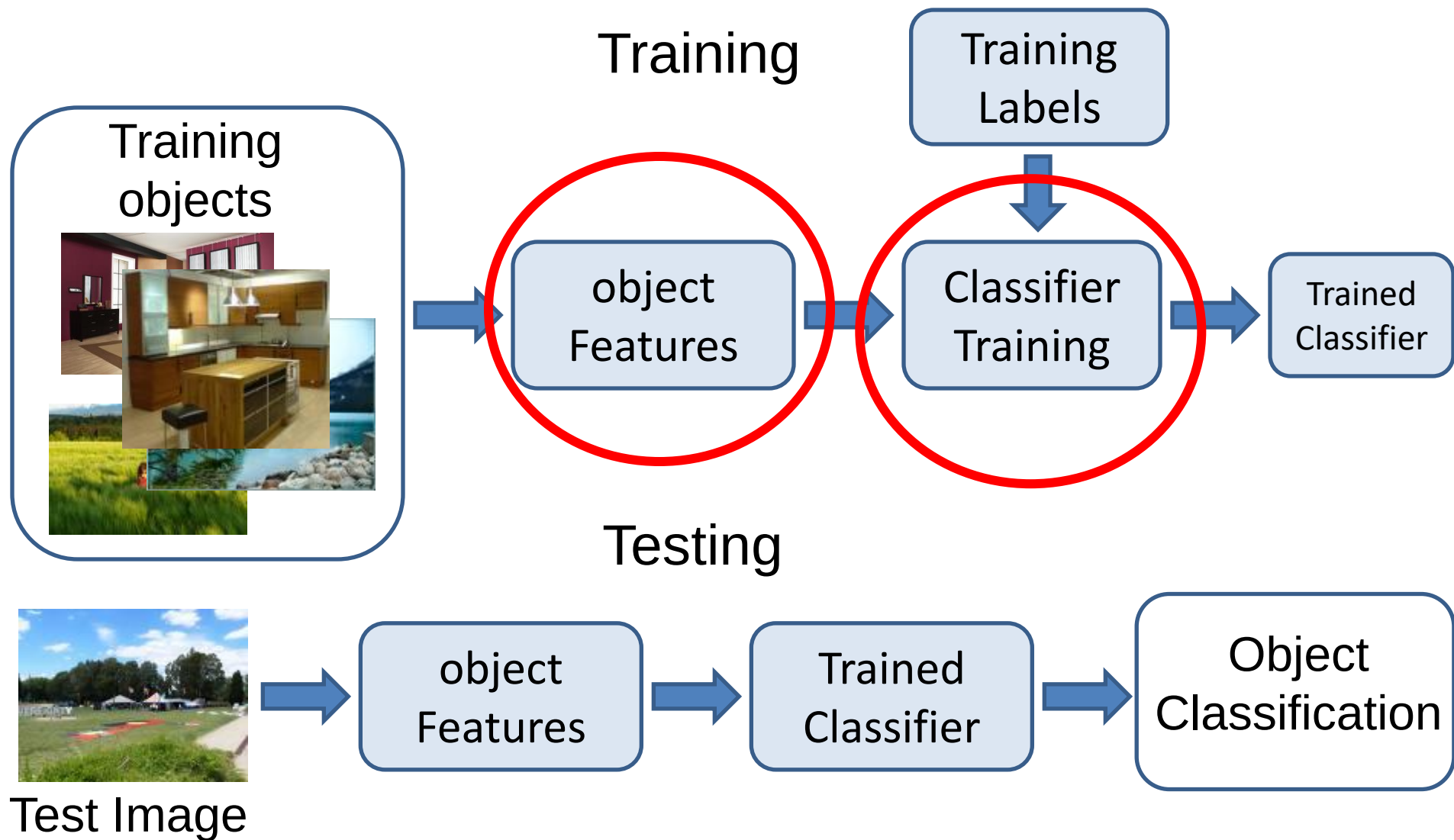
Region growing vs Mean Shift results



Supervised segmentation: learning

- Object detection can be performed by learning different object views automatically from a set of examples by means of a supervised learning mechanism.
- A standard formulation of supervised learning is the classification problem
- The learning approaches include:
 - neural networks
 - **adaptive boosting**
 - K Nearest Neighbor
 - decision trees
 - **support vector machine**
- Supervised learning methods usually require a large collection of samples from each object class (composed of pairs of object features and an associated object class). This collection must be manually labeled.

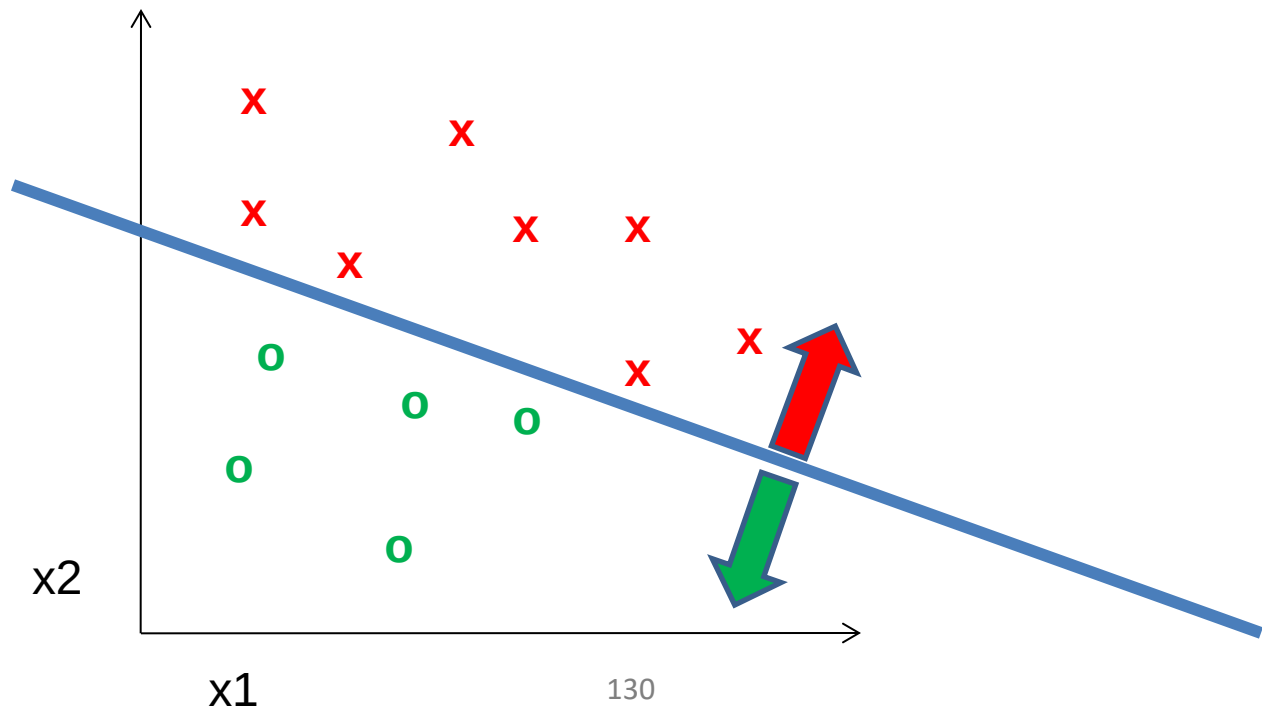
Supervised learning/training



Learning a classifier

Given some set of features with corresponding labels,
learn a function to predict the labels from the features

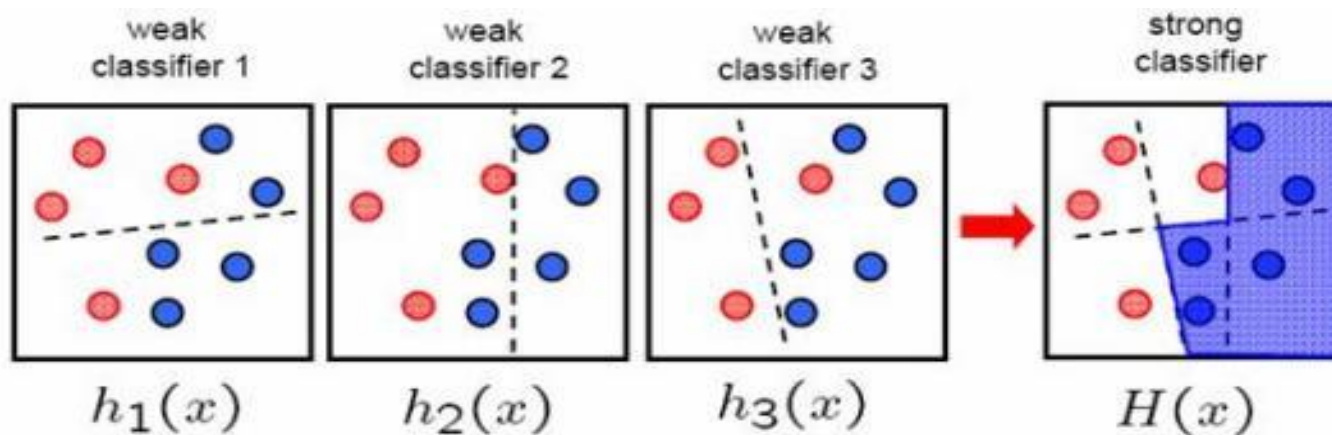
Classifiers try to learn weights or parameters for features
and distance measures so that visual similarity predicts
label similarity



Supervised learning techniques

- Adaptive boosting

- Boosting is an iterative method of finding a very accurate classifier by combining many base classifiers,
- In the training phase of the Adaboost algorithm, the first step is to construct an initial distribution of weights over the training set.
- The boosting mechanism then selects a base classifier that gives the least error according to the misclassified regions.



$$H(x) = \alpha_1 h_1(x) + \alpha_2 h_2(x) + \alpha_3 h_3(x)$$

Case of study: The Viola/Jones Face Detector

- **Needs**
- A seminal, unsupervised, appearance based approach to real-time (at least 2 frames per second must be processed) object detection
- Training is slow, but detection is very fast
- **Key ideas**
- **Feature Computation:** what features? And how can they be computed as quickly as possible
 - *Solution: Haar like features and Integral images*
- **Feature Selection:** select the most discriminating features
 - *Solution: Boosting for feature selection*
- **Real-timeliness:** must focus on potentially positive areas (that contain faces)
 - *Solution: Attentional cascade for fast rejection of non-face windows*

Viola Jones Face detection

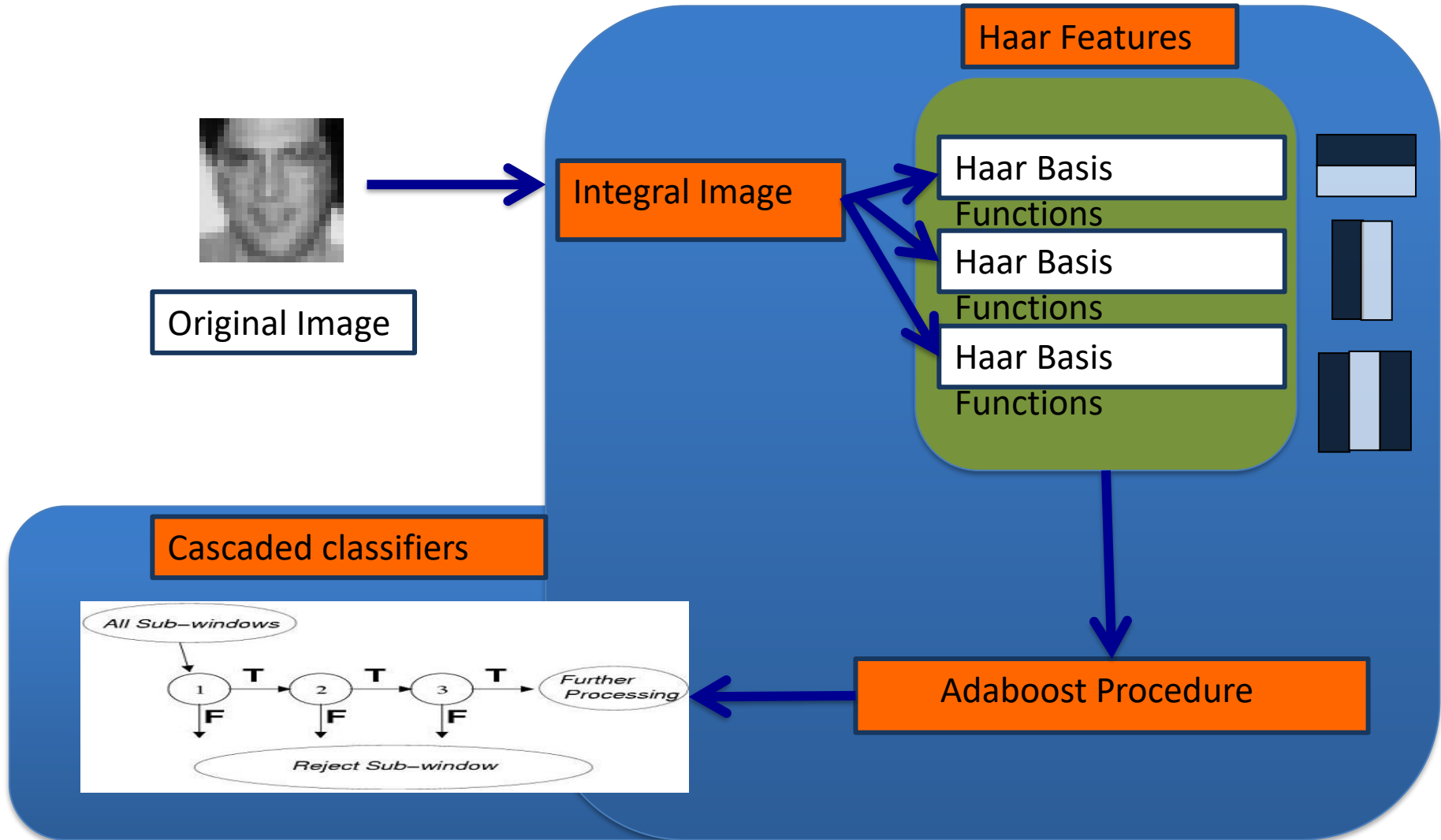
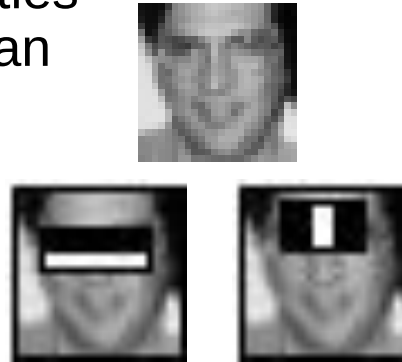


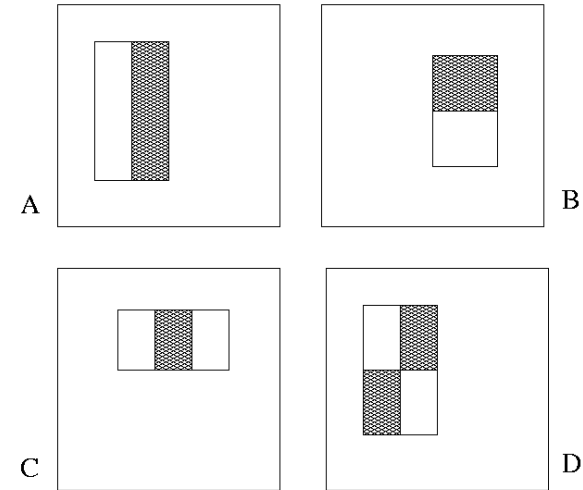
Image Features: Haar Like features

All faces share some similar properties

- The eyes region is darker than the upper-cheeks.
- The nose bridge region is brighter than the eyes.
- **That is useful domain knowledge**



“Rectangle filters”

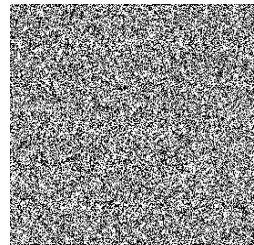


Example:

Value =

$\sum (\text{pixels in white area})$

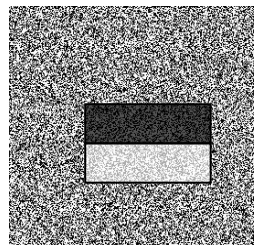
$-\sum (\text{pixels in black area})$



Source

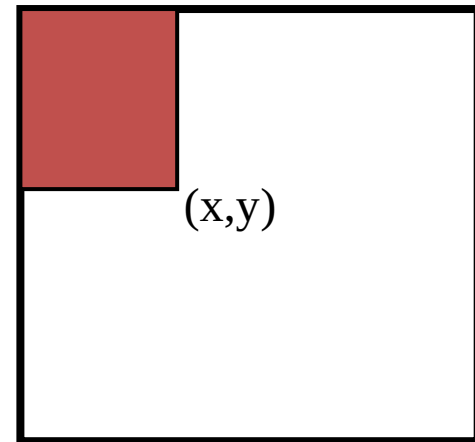


Result

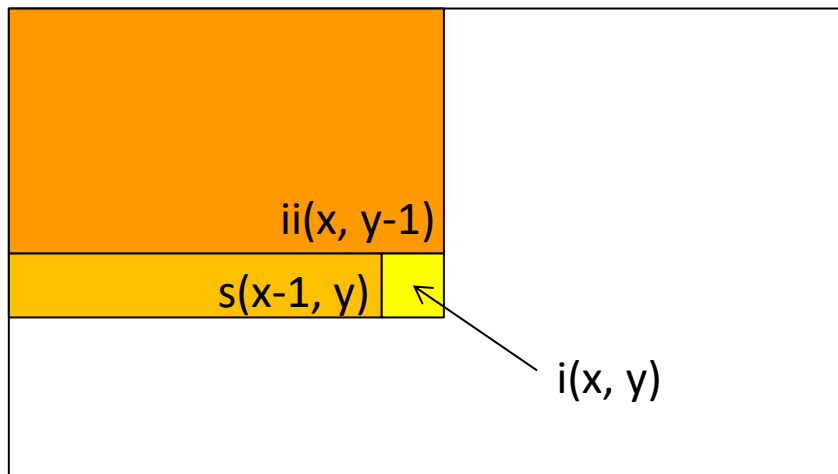


Fast computation with integral images

- The *integral image* computes a value at each pixel (x,y) that is the sum of the pixel values above and to the left of (x,y) , inclusive
- This can quickly be computed in one pass through the image



Computing the integral images



IMAGE

0	1	1	1
1	2	2	3
1	2	1	1
1	3	1	0

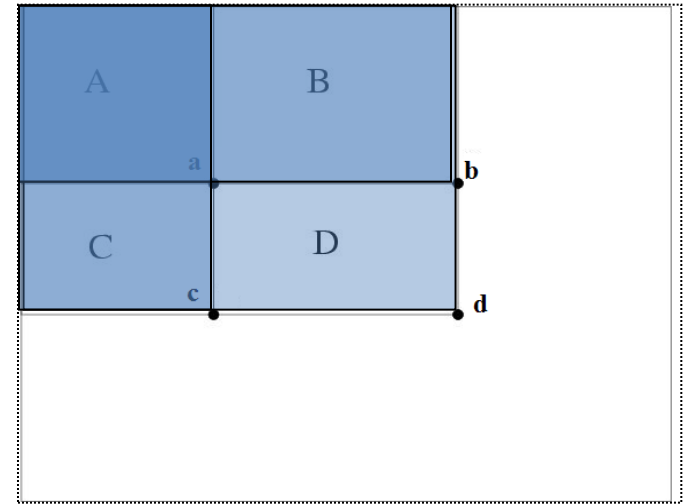
INTEGRAL IMAGE

0	1	2	3
1	4	7	11
2	7	11	16
3	11	16	21

- Cumulative row sum: $s(x, y) = s(x-1, y) + i(x, y)$
- Integral image: $i(x, y) = ii(x, y-1) + s(x, y)$

Fast computation with integral images

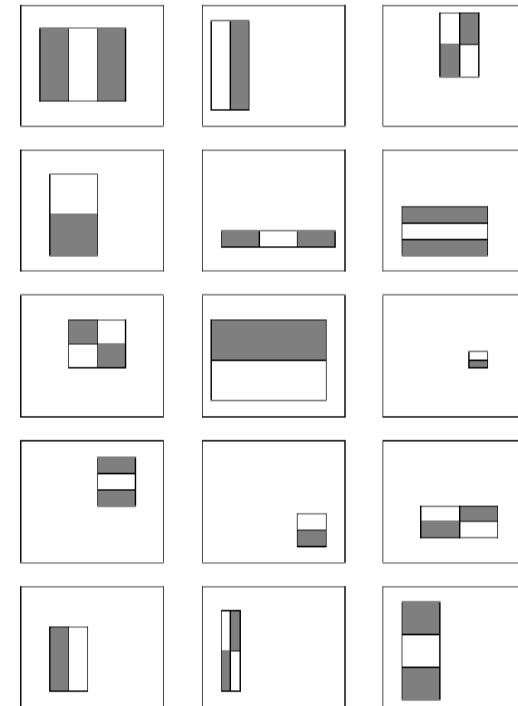
- Using the integral image representation we can compute the value of any rectangular sum (part of features) in **constant time**
 - For example the integral sum inside rectangle D can be computed as:
 $ii(d) + ii(a) - ii(b) - ii(c)$
- two-, three-, and four-rectangular features can be computed with 6, 8 and 9 **array references** respectively.
- As a result: feature computation takes less time



$$\begin{aligned} ii(a) &= A \\ ii(b) &= A+B \\ ii(c) &= A+C \\ ii(d) &= A+B+C+D \\ D &= ii(d) + ii(a) - ii(b) - ii(c) \end{aligned}$$

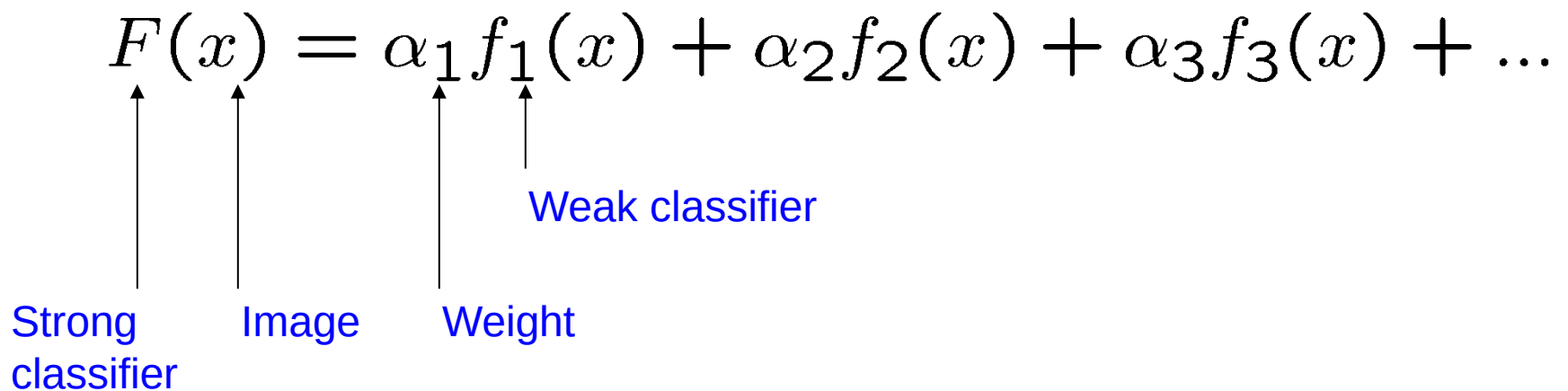
Features selection

- **Problem: Too many features**
 - In a sub-window (24x24) there are ~160,000 features (all possible combinations of orientation, location and scale of these feature types)
 - impractical to compute all of them (computationally expensive)
- We have to select a subset of relevant features – which are informative - to model a face
 - **Hypothesis:** “ A small subset of features can be combined to form an effective classifier”
 - How?
 - AdaBoost algorithm



Relevant feature Irrelevant feature

- Stands for “**Adaptive**” **boost**
- Constructs a “strong” classifier as a linear combination of weighted simple “weak” classifiers

$$F(x) = \alpha_1 f_1(x) + \alpha_2 f_2(x) + \alpha_3 f_3(x) + \dots$$


Strong classifier

Image

Weight

Weak classifier

AdaBoost - *Characteristics*

- Features as weak classifiers
 - Each single rectangle feature may be regarded as a simple weak classifier
- An iterative algorithm
 - AdaBoost performs a series of trials, each time selecting a new weak classifier
- Weights are being applied over the set of the example images
 - During each iteration, each example/image receives a weight determining its importance

AdaBoost - *Getting the idea...*

- Given: example images labeled +/-
 - Initially, all weights set equally
- Repeat T times
 - Step 1: choose the most efficient weak classifier that will be a component of the final strong classifier
 - Step 2: Update the weights to emphasize the examples which were incorrectly classified
 - This makes the next weak classifier to focus on “harder” examples
- Final (strong) classifier is a weighted combination of the T “weak” classifiers
 - Weighted according to their accuracy

$$h(x) = \begin{cases} 1 & \text{if } \sum_{t=1}^T \alpha_t h_t(x) \geq \frac{1}{2} \sum_{t=1}^T \alpha_t \\ 0 & \text{otherwise} \end{cases}$$

$$h_t(x) = \begin{cases} 1 & \text{if } f_t(x) > \theta_t \\ 0 & \text{otherwise} \end{cases}$$

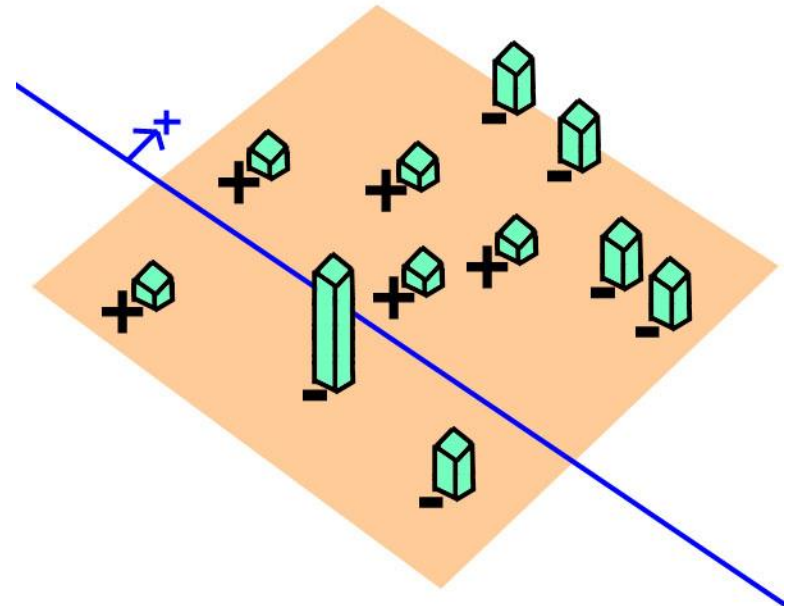
value of rectangle feature

window

threshold

AdaBoost example

- ❑ AdaBoost starts with a uniform distribution of “weights” over training examples.
- ❑ Select the classifier with the lowest weighted error (i.e. a “weak” classifier)
- ❑ Increase the weights on the training examples that were misclassified.
- ❑ (Repeat)
- ❑ At the end, carefully make a linear combination of the weak classifiers obtained at all iterations.



AdaBoost – *Feature Selection*

Problem

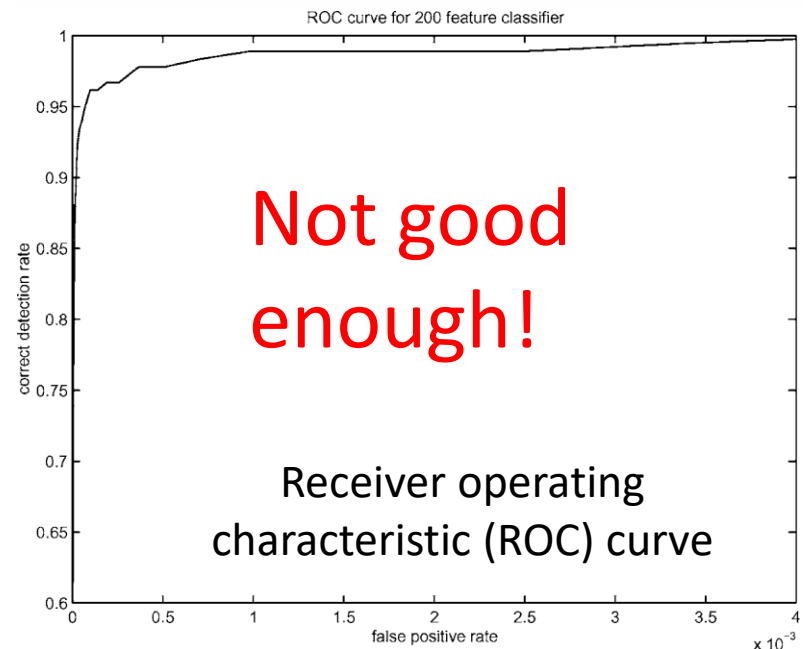
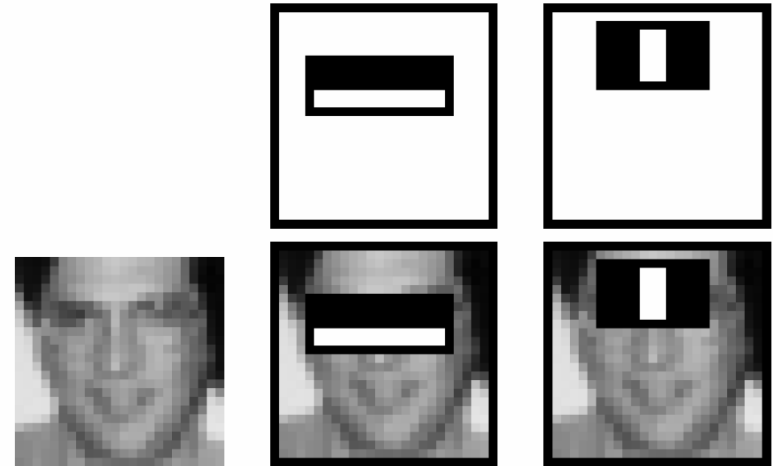
- On each round, large set of possible weak classifiers (each simple classifier consists of a single **feature**) – Which one to choose?
 - choose the most efficient (the one that best separates the examples – the lowest error)
 - choice of a classifier corresponds to choice of a feature
- At the end, the ‘strong’ classifier consists of T features

Conclusion

- AdaBoost searches for a small number of good classifiers – features (feature selection)
- adaptively constructs a final strong classifier taking into account the failures of each one of the chosen weak classifiers (weight appliance)
- AdaBoost is used to **both** select a small set of features and train a strong classifier

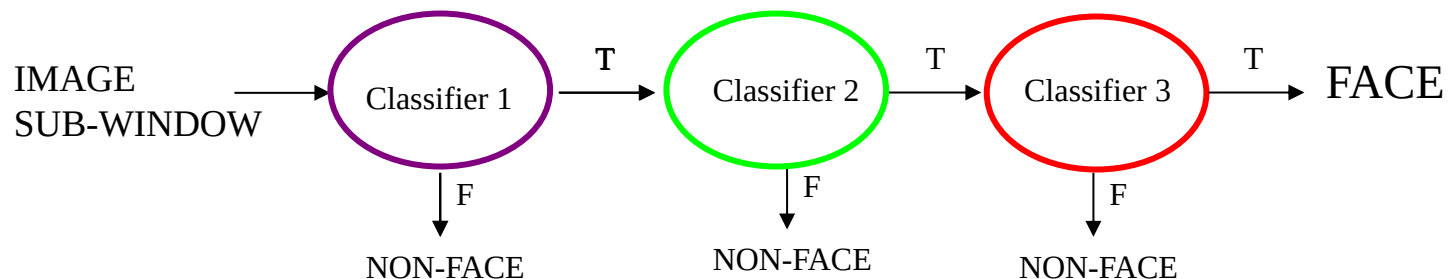
Boosting for face detection

- First two features selected by boosting:
- This feature combination can yield 100% detection rate and 50% false positive rate
- A 200-feature classifier can yield 95% detection rate and a false positive rate of 1 in 14084



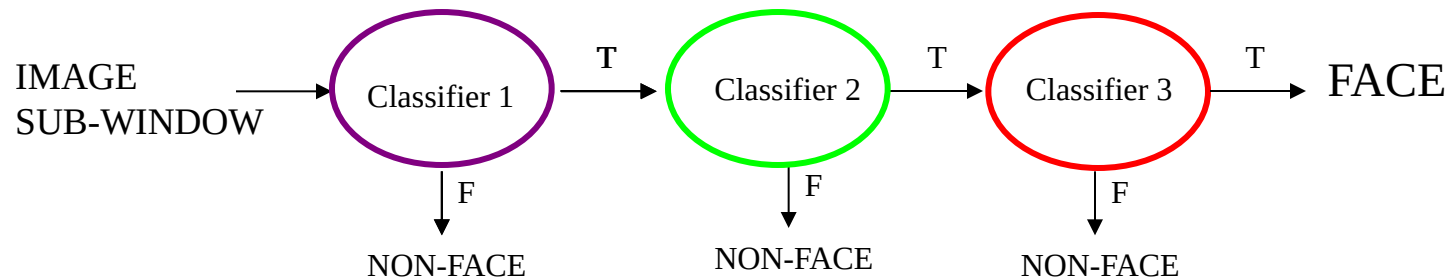
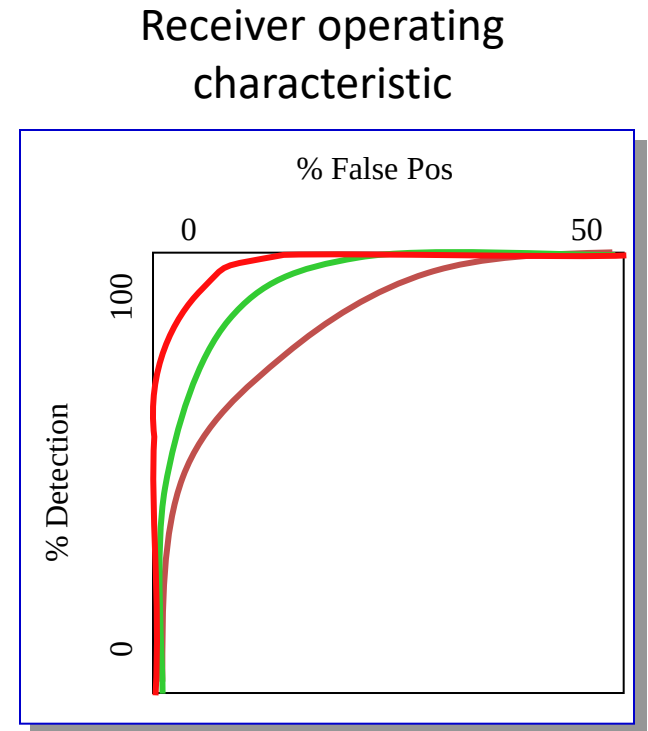
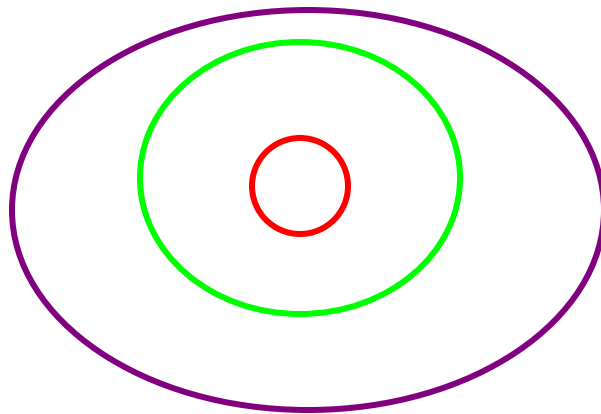
Attentional cascade

- We start with simple classifiers which reject many of the negative sub-windows while detecting almost all positive sub-windows
- Positive response from the first classifier triggers the evaluation of a second (more complex) classifier, and so on
- A negative outcome at any point leads to the immediate rejection of the sub-window



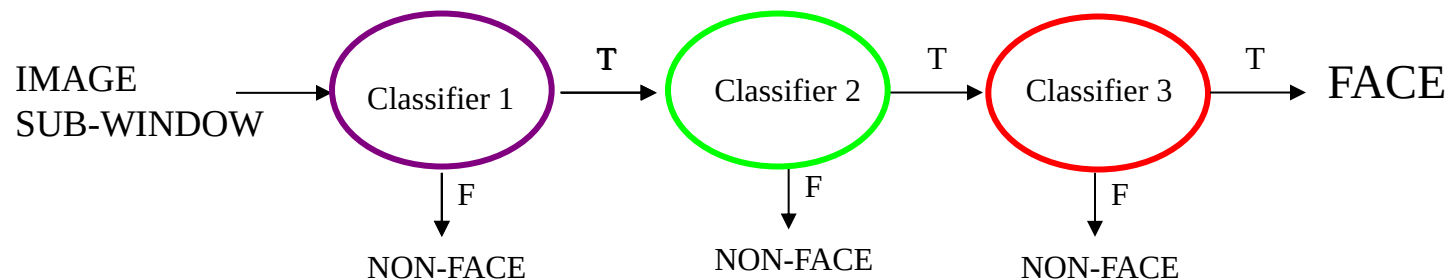
Attentional cascade

- Chain classifiers that are progressively more complex and have lower false positive rates:



Attentional cascade

- The detection rate and the false positive rate of the cascade are found by multiplying the respective rates of the individual stages
- A detection rate of 0.9 and a false positive rate on the order of 10^{-6} can be achieved by a 10-stage cascade if each stage has a detection rate of 0.99 ($0.99^{10} \approx 0.9$) and a false positive rate of about 0.30 ($0.3^{10} \approx 6 \times 10^{-6}$)

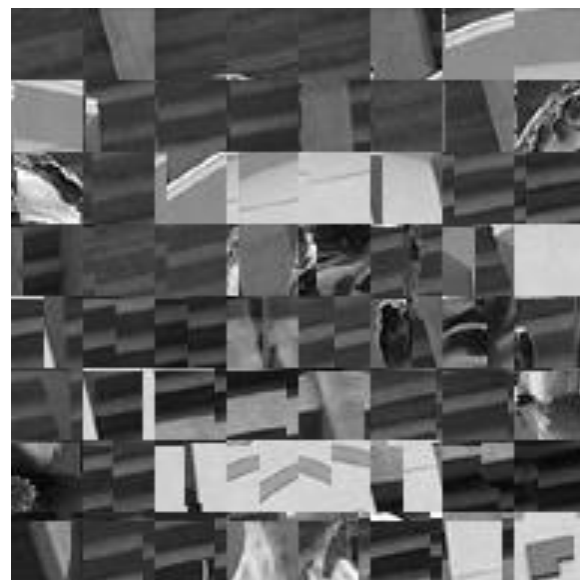


Training the cascade

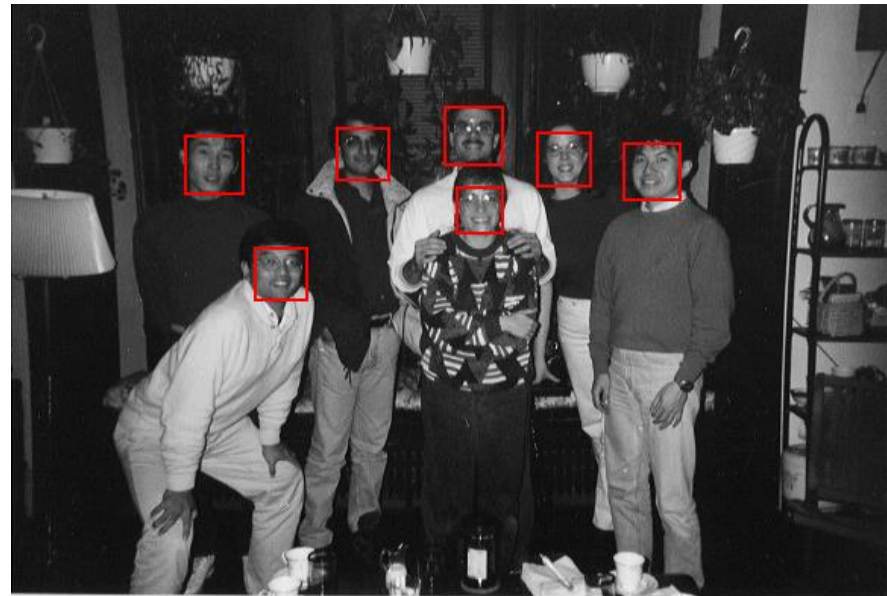
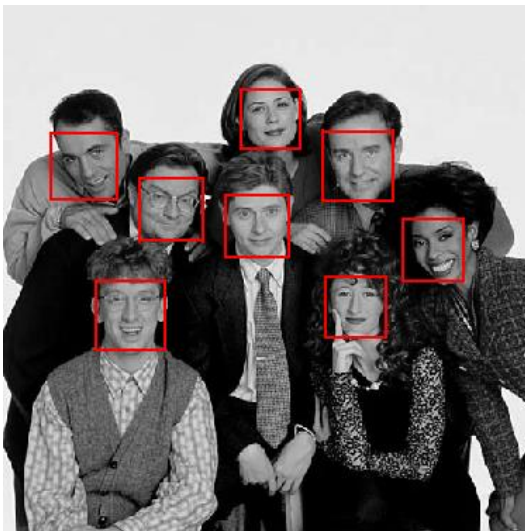
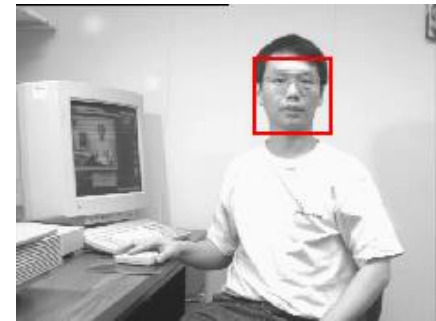
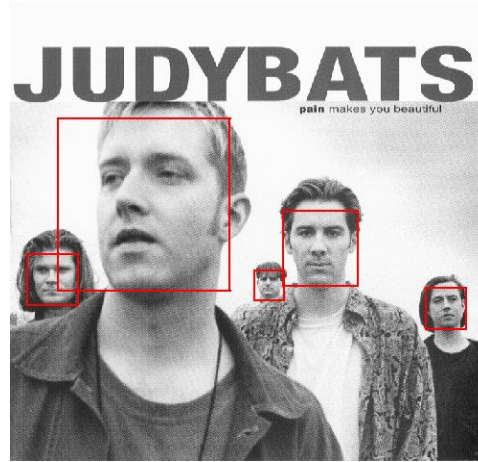
- Set target detection and false positive rates for each stage
- Keep adding features to the current stage until its target rates have been met
 - Need to lower AdaBoost threshold to maximize detection (as opposed to minimizing total classification error)
 - Test on a *validation set*
- If the overall false positive rate is not low enough, then add another stage
- Use false positives from current stage as the negative training examples for the next stage

The implemented system

- Training Data
 - 5000 faces
 - All frontal, rescaled to 24x24 pixels
 - 9500 non-faces
 - Faces are normalized
 - Scale, translation
- Many variations
 - Across individuals
 - Illumination
 - Pose

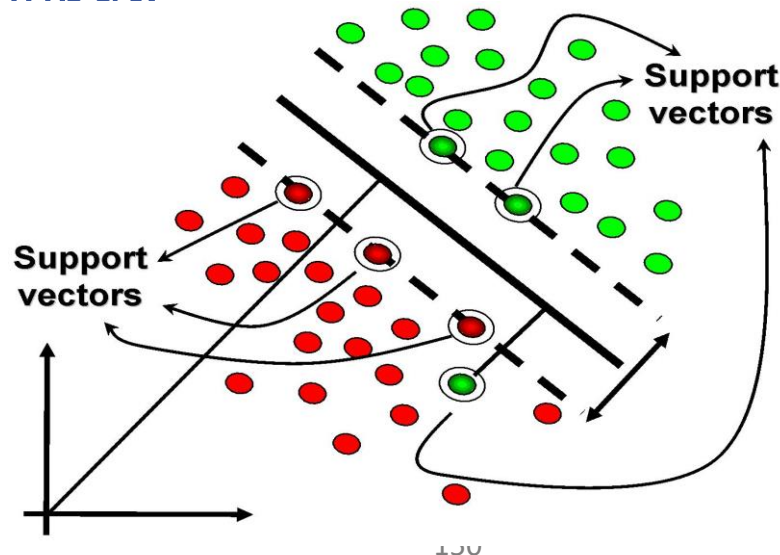


Output of Face Detector on Test Images

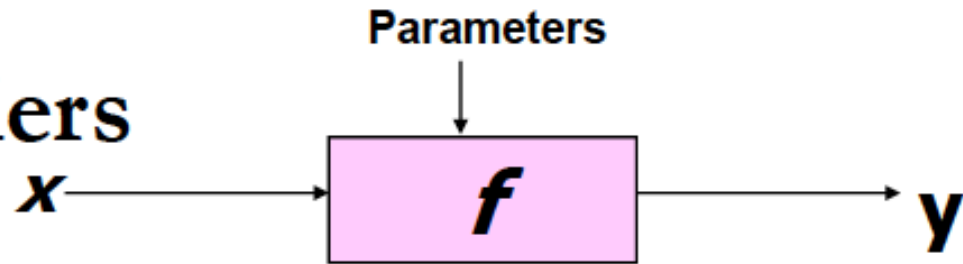


Supervised learning techniques

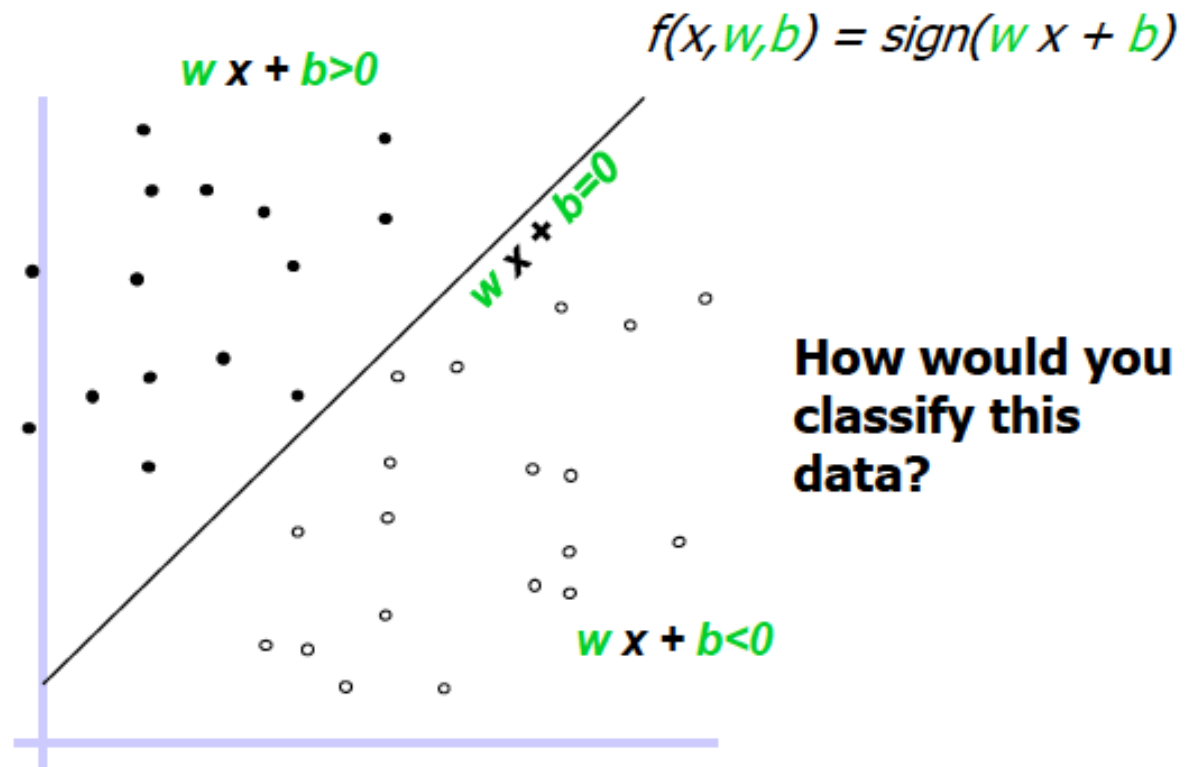
- **Support Vector Machine (SVM)**
- SVM is used to cluster data into two classes by finding the maximum marginal hyperplane that separates one class from the other.
- The margin of the hyperplane, which is maximized, is defined by the distance between the hyperplane and the closest data points.
- Despite being a linear classifier, SVM can also be used as a nonlinear classifier by applying the kernel trick to the input feature vector extracted from the input.



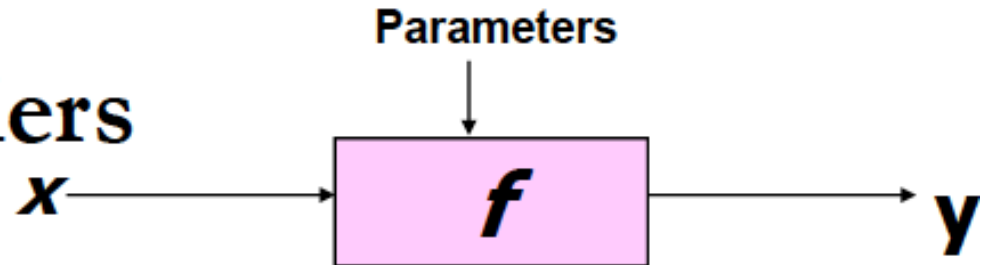
Linear Classifiers



- Label y :
- denotes +1
 - denotes -1

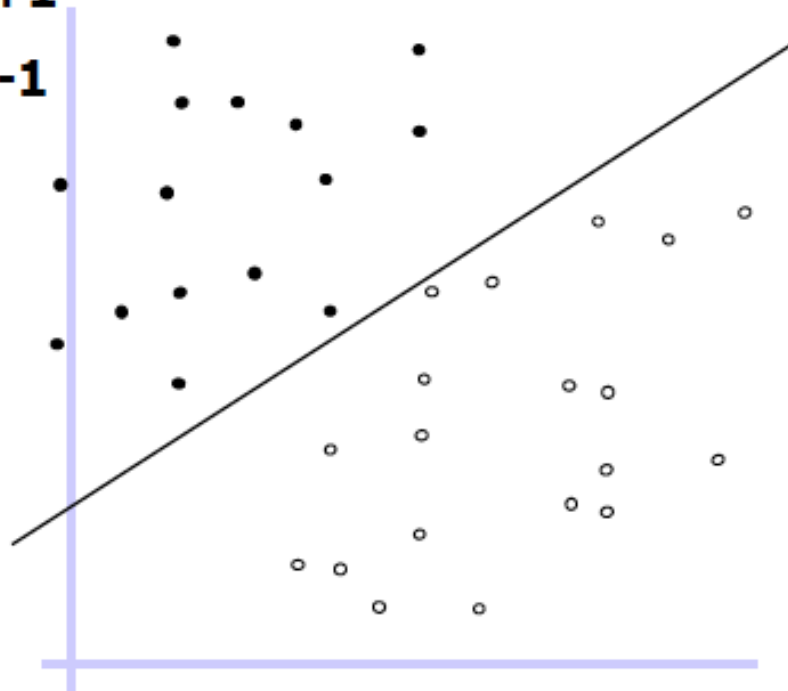


Linear Classifiers



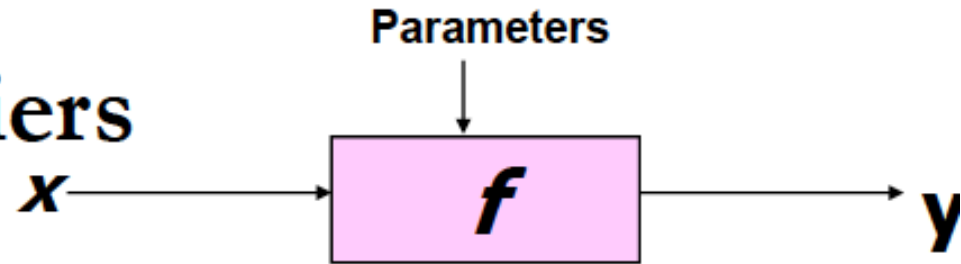
$$f(x, w, b) = \text{sign}(w x + b)$$

- denotes +1
- denotes -1



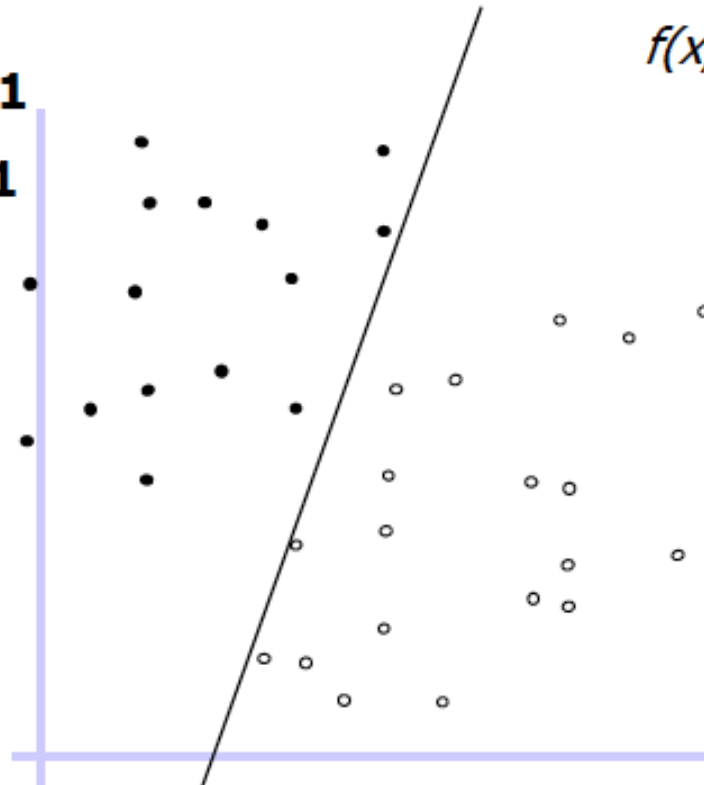
**How would you
classify this
data?**

Linear Classifiers



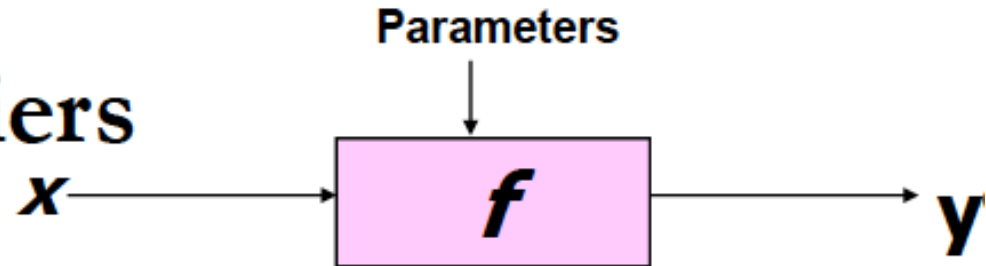
$$f(x, w, b) = \text{sign}(w x + b)$$

- denotes +1
- denotes -1



**How would you
classify this
data?**

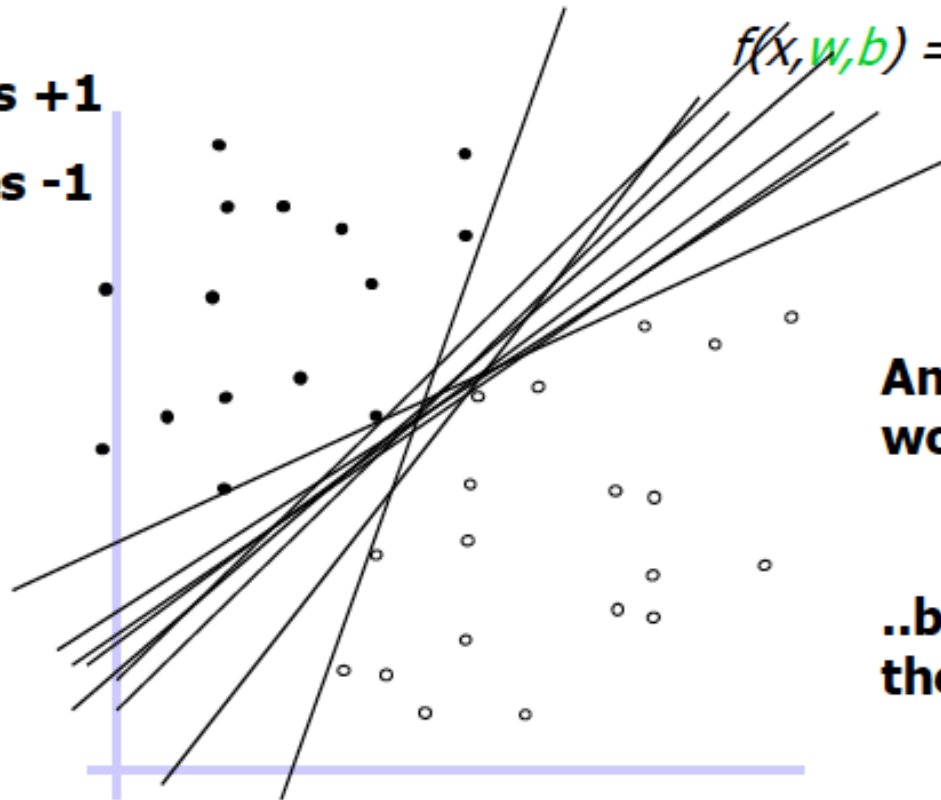
Linear Classifiers



• denotes +1

◦ denotes -1

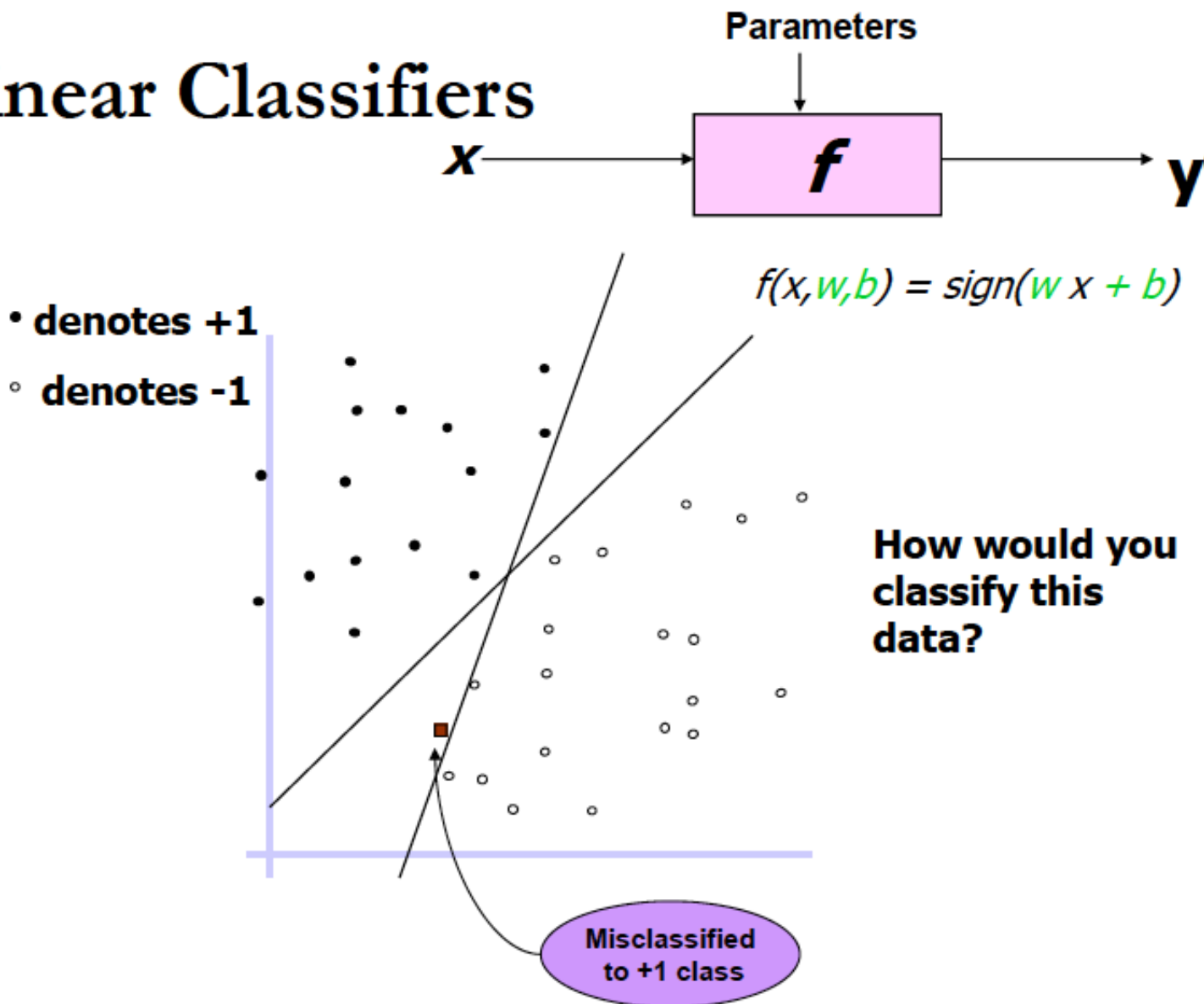
$$f(x, w, b) = \text{sign}(w x + b)$$



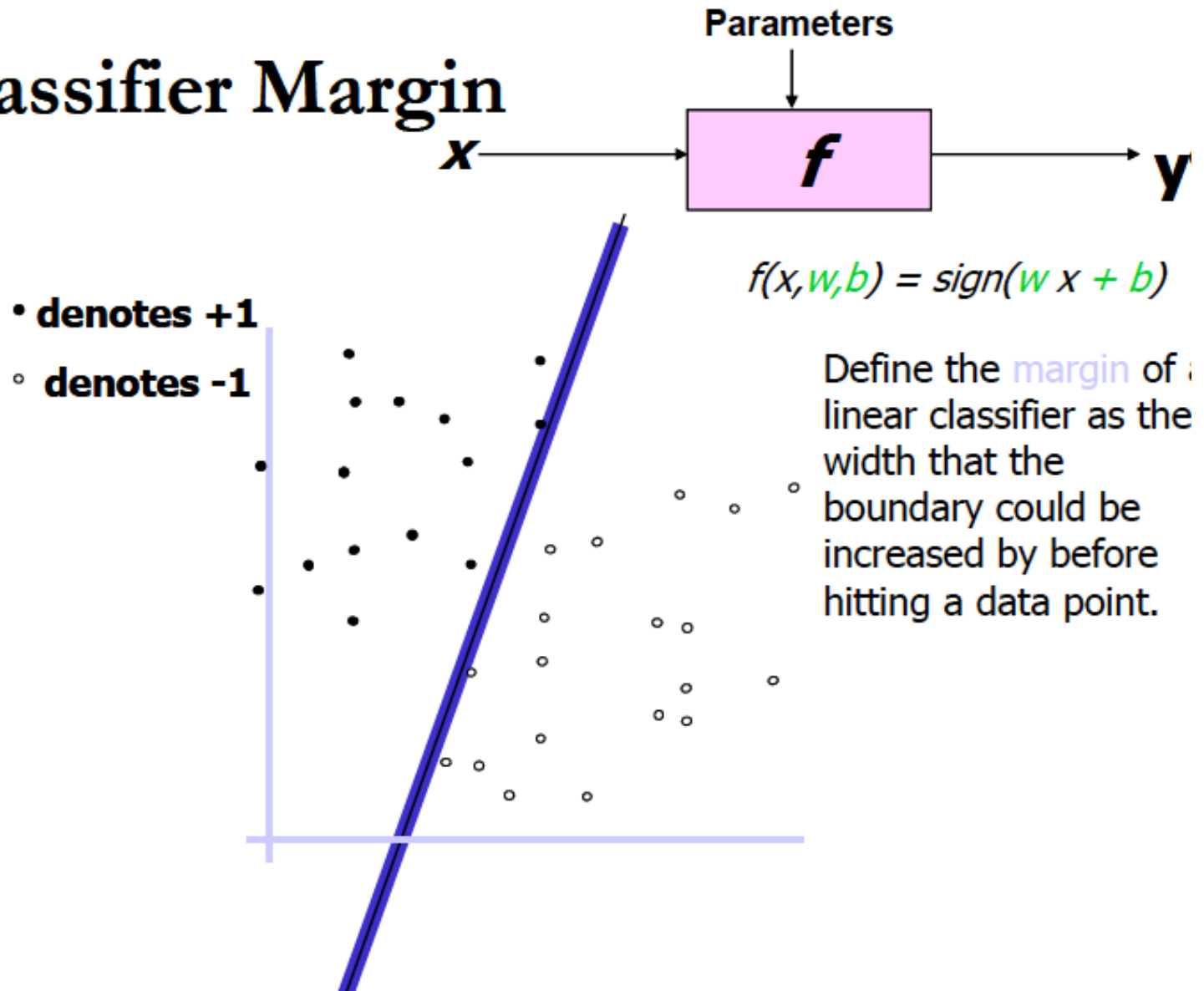
Any of these
would be fine..

..but which is
the best?

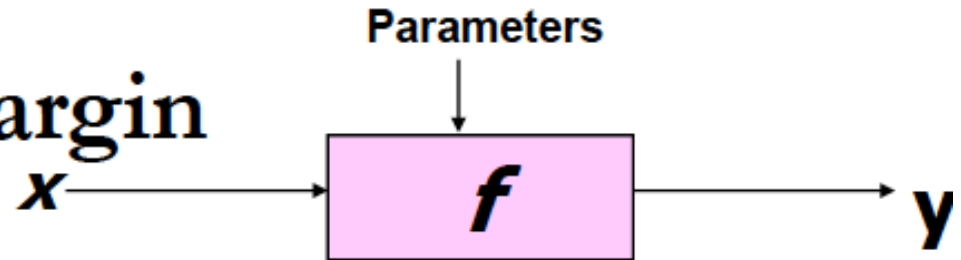
Linear Classifiers



Classifier Margin



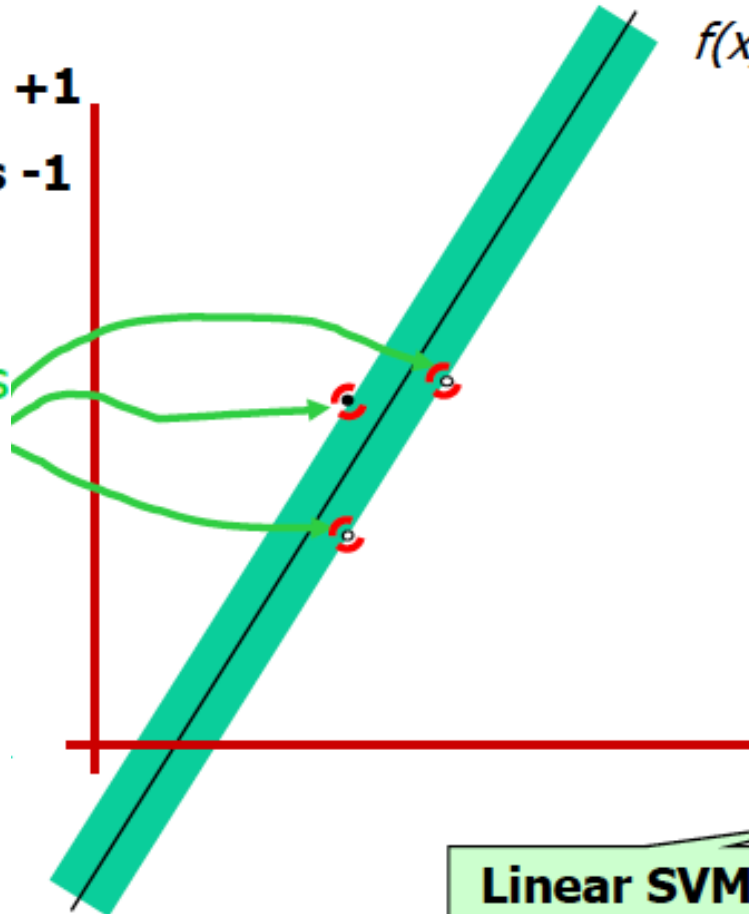
Maximum Margin



$$f(x, w, b) = \text{sign}(w x + b)$$

- denotes +1
- denotes -1

Support Vectors
are those
datapoints that
the margin
pushes up
against



The **maximum margin linear classifier** is the linear classifier with the maximum margin.

This is the simplest kind of SVM (Called an LSVM)

Linear SVM

Linear SVM Mathematically

- **Goal: 1) Correctly classify all training data**

$$\left. \begin{array}{ll} wx_i + b \geq 1 & \text{if } y_i = +1 \\ wx_i + b \leq -1 & \text{if } y_i = -1 \end{array} \right\} \text{for all } i$$


- 2) **Maximize the Margin**

same as minimize

$$M = \frac{2}{|w|}$$
$$\frac{1}{2} w^t w$$

- **We can formulate a Quadratic Optimization Problem and solve for w and b**

- **Minimize** $\Phi(w) = \frac{1}{2} w^t w$

subject to $y_i (wx_i + b) \geq 1 \quad \forall i$

Linear SVM Mathematically

Convert the problem from the primal form into the dual form

$$L(\vec{w}, b, \alpha) = \frac{1}{2} \langle \vec{w} * \vec{w} \rangle - \sum_{i=1}^l \alpha_i [y_i (\langle \vec{w} * \vec{x}_i \rangle + b) - 1]$$

Find $\alpha_1 \dots \alpha_N$ such that

$Q(\alpha) = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$ is maximized and

(1) $\sum \alpha_i y_i = 0$

(2) $\alpha_i \geq 0$ for all α_i

$$\text{maximize } Q(\alpha) = \sum_{i=1} \alpha_i - \frac{1}{2} \sum_{i=1} \sum_{j=1} y_i y_j \alpha_i \alpha_j \langle \vec{x}_i * \vec{x}_j \rangle$$

$$\text{subject to } \sum_{i=1}^l y_i \alpha_i = 0 \quad \alpha_i \geq 0 \quad i = 1, \dots, l.$$

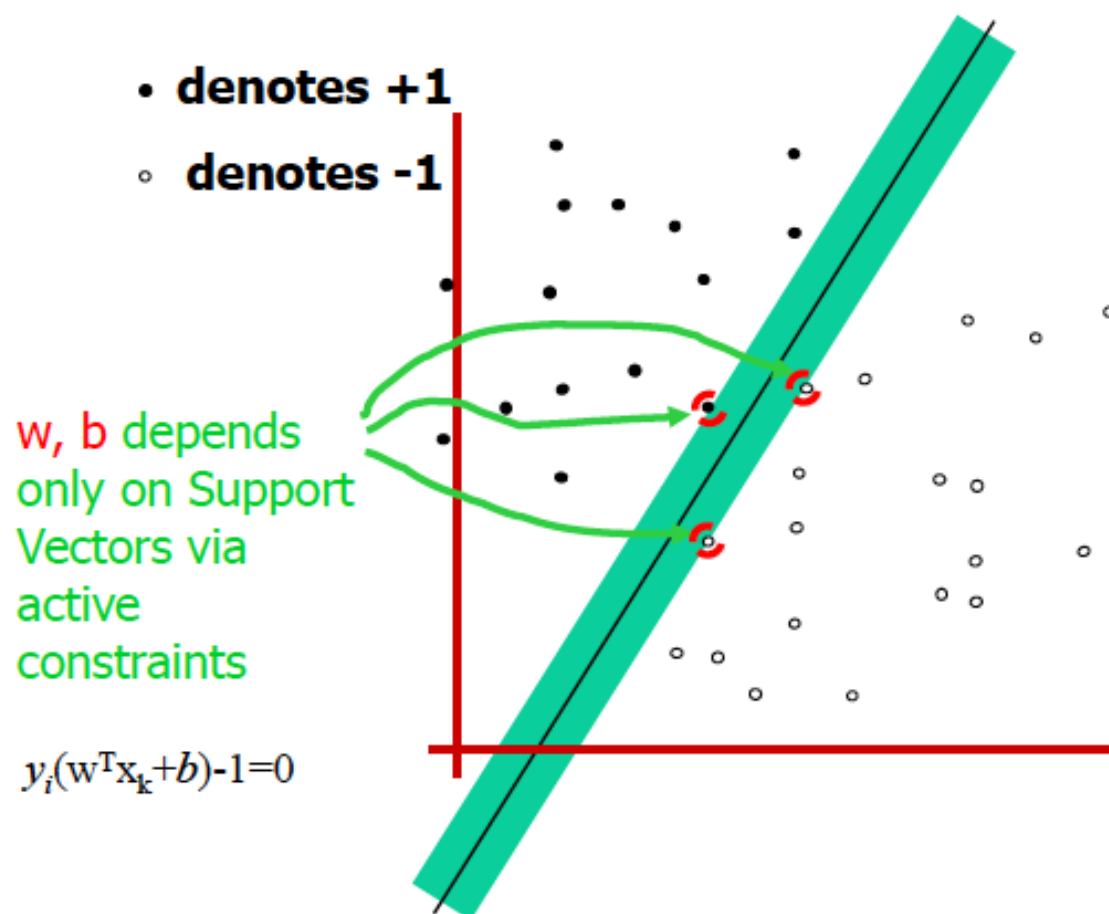
The Optimization Problem Solution

- The solution has the form:

$$w = \sum \alpha_i y_i x_i \quad b = y_k - w^T x_k \quad \text{for any } x_k \text{ such that } \alpha_k \neq 0$$

- α_i must satisfy Karush-Kuhn-Tucker conditions:
 $\alpha_i [y_i(w^T x_i + b) - 1] = 0$, for any i
 - If $\alpha_i > 0$, $y_i(w^T x_i + b) - 1 = 0$, x_i is on the margin
 - If $y_i(w^T x_i + b) > 1$, $\alpha_i = 0$
- Each non-zero α_i indicates that corresponding x_i is a **support vector**.

Maximum Margin



The Optimization Problem Solution

- To classify the new test point x , we use

$$f(x) = wX + b = \sum \alpha_i y_i x_i^T X + b$$

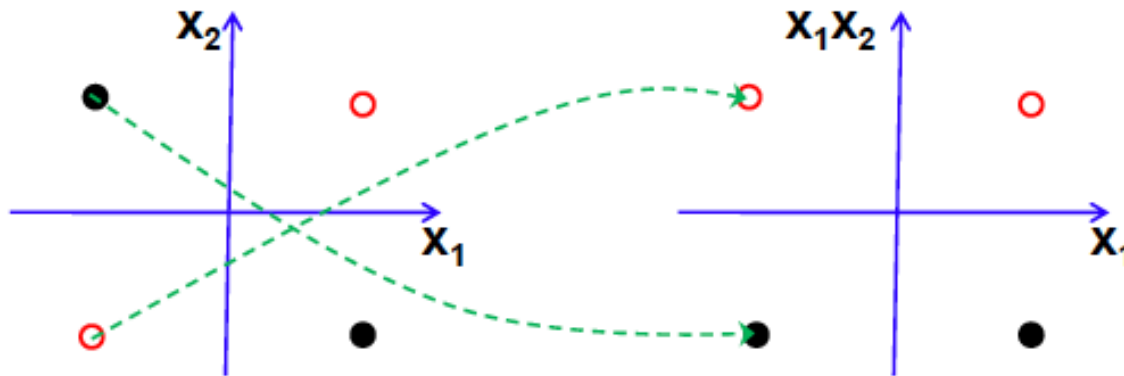
Find $\alpha_1 \dots \alpha_N$ such that

$Q(\alpha) = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j x_i^T x_j$ is maximized and

(1) $\sum \alpha_i y_i = 0$

(2) $\alpha_i \geq 0$ for all α_i

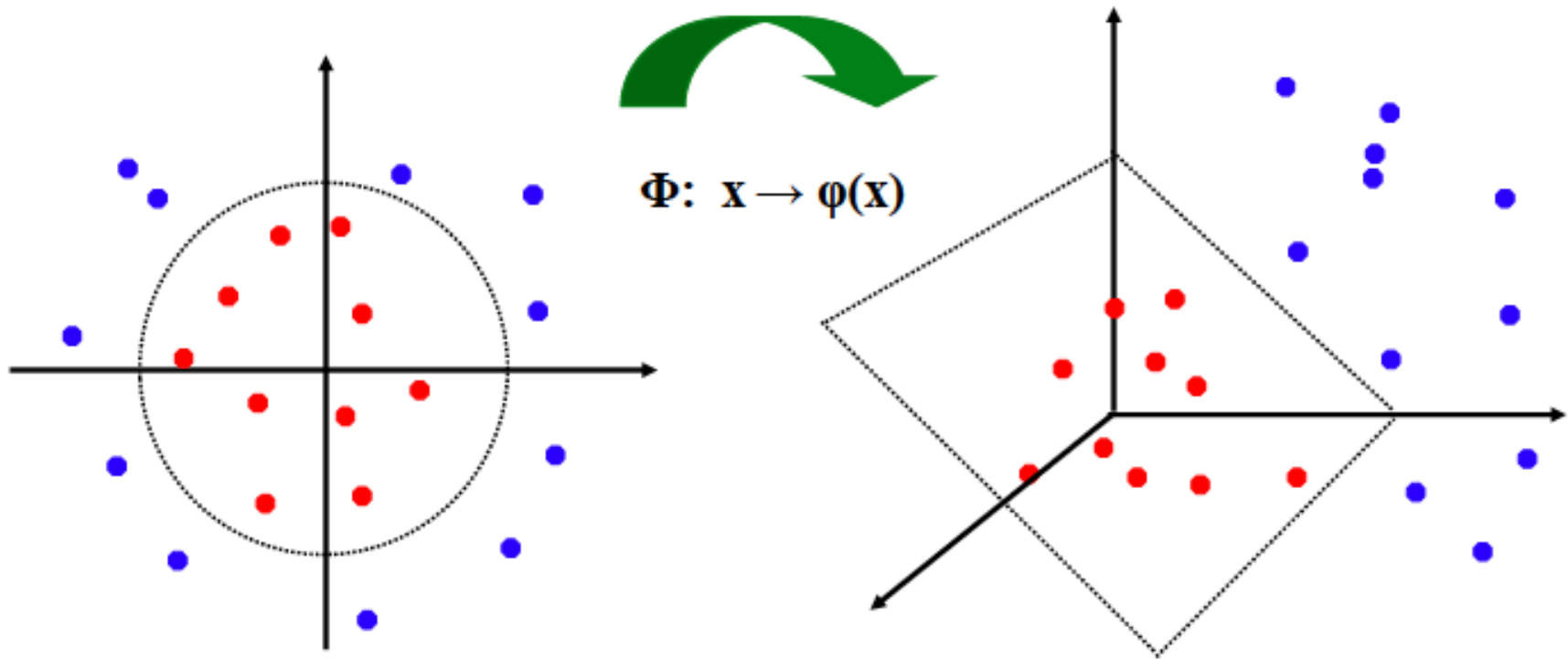
Non Linear SVM: idea



- ❑ XOR data are not linearly separable
- ❑ Mapping (x_1, x_2) to $(x_1, x_1 x_2)$

Non Linear SVM

- General idea: the original input space can always be mapped to some higher-dimensional feature space where the training set is linearly separable:



Non Linear SVM

- If every data point is mapped into high-dimensional space via some transformation $\Phi: \mathbf{x} \rightarrow \phi(\mathbf{x})$, optimization problem is similar:

Find $\alpha_1 \dots \alpha_N$ such that

$Q(\alpha) = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ is maximized

(1) $\sum \alpha_i y_i = 0$

(2) $\alpha_i \geq 0$ for all α_i

- Classifying function is:

$$f(\mathbf{x}) = \sum \alpha_i y_i \phi(\mathbf{x}_i)^T \phi(\mathbf{x}) + b$$

- But relies on inner product $\phi(\mathbf{x}_i)^T \phi(\mathbf{x})$

Examples of Kernel Functions

- **Linear:** $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- **Polynomial of power p :** $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^p$
- **Gaussian (radial-basis function network):**

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$$

- **Sigmoid:** $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\beta_0 \mathbf{x}_i^T \mathbf{x}_j + \beta_1)$

Non Linear SVM Mathematically

- **Dual problem formulation:**

Find $\alpha_1 \dots \alpha_N$ such that

$Q(\alpha) = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j K(x_i, x_j)$ is maximized and

(1) $\sum \alpha_i y_i = 0$

(2) $\alpha_i \geq 0$ for all α_i

- **The solution is:**

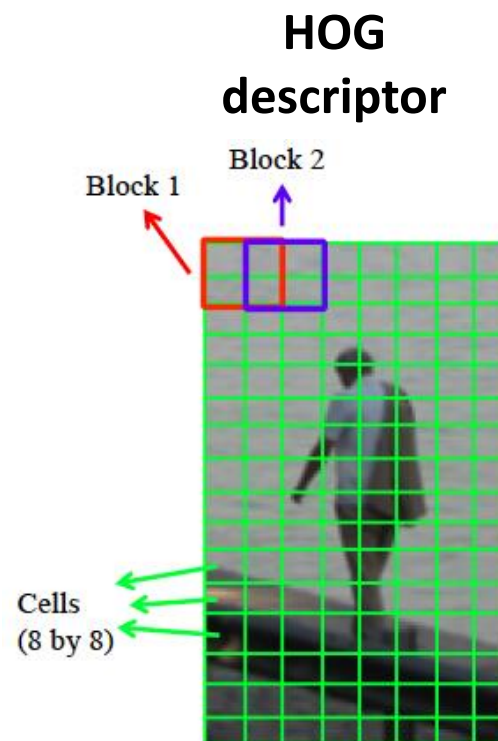
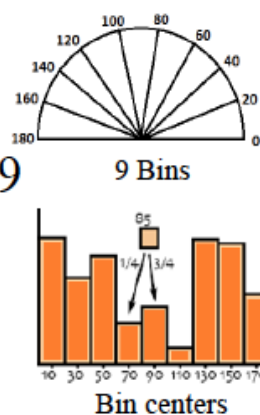
$$f(x) = \sum \alpha_i y_i K(x_i, x_j) + b$$

- **Optimization techniques for finding α_i 's remain the same!**

Case of study: Human detection SVM classification

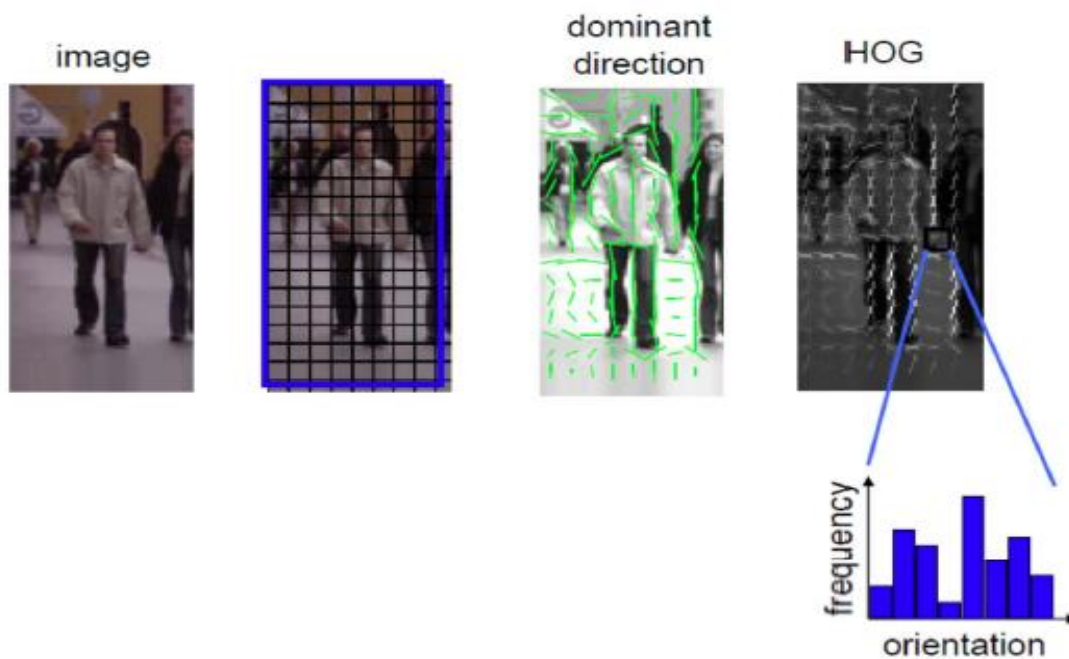
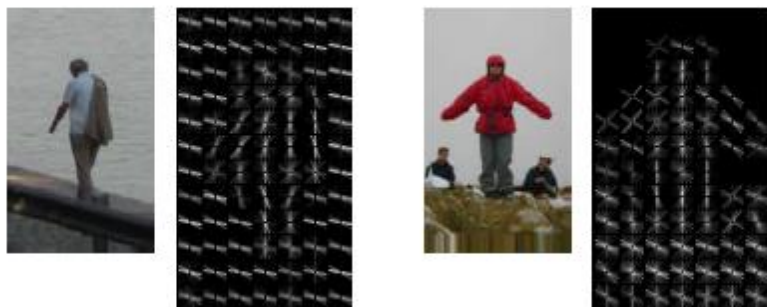


- 16x16 blocks of 50% overlap.
 - 7x15=105 blocks in total
- Each block consists of 2x2 cells with size 8x8
- Quantize the gradient orientation into 9 bins (0-180)
 - The vote is the gradient magnitude



Case of study: Human detection SVM classification

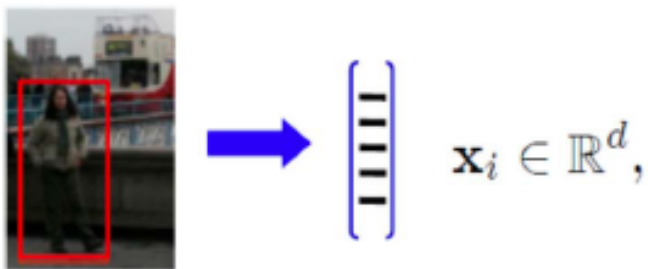
■ Visualization



Case of study: Human detection SVM classification

Training (Learning)

- Represent each example window by a HOG feature vector



- Train a SVM classifier



Testing (Detection)

- Sliding window classifier

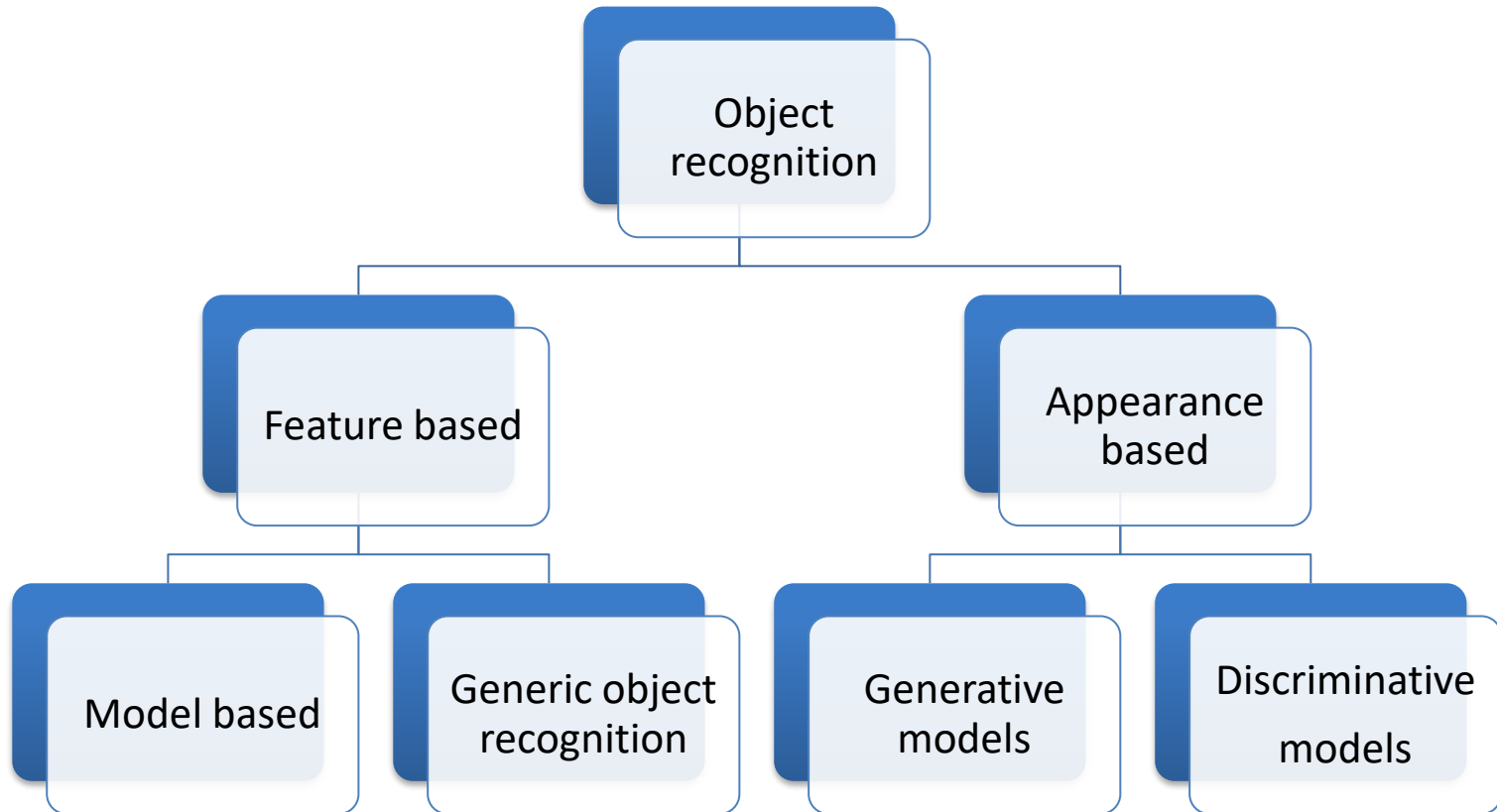
$$f(x) = \mathbf{w}^\top \mathbf{x} + b$$

Case of study: Human detection SVM classification



Object recognition

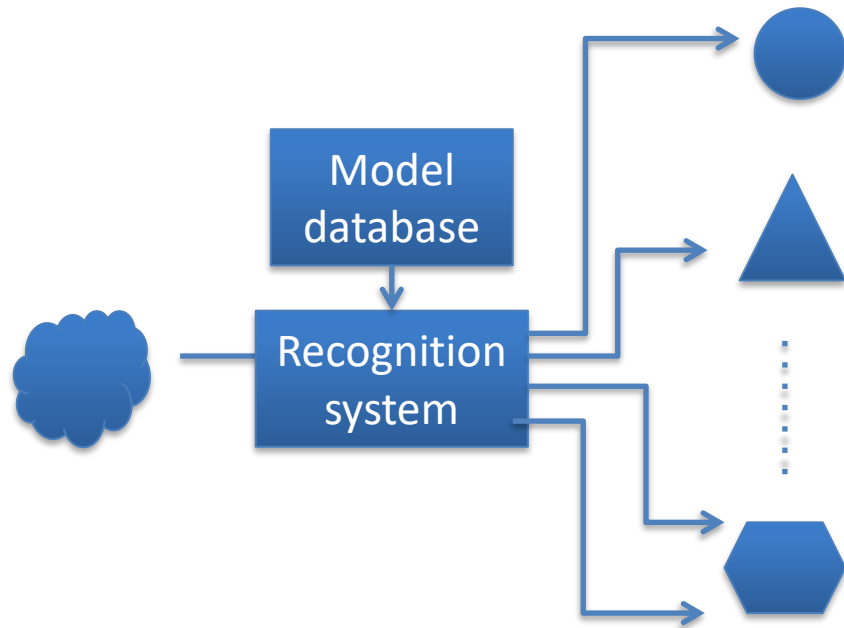
Object Recognition



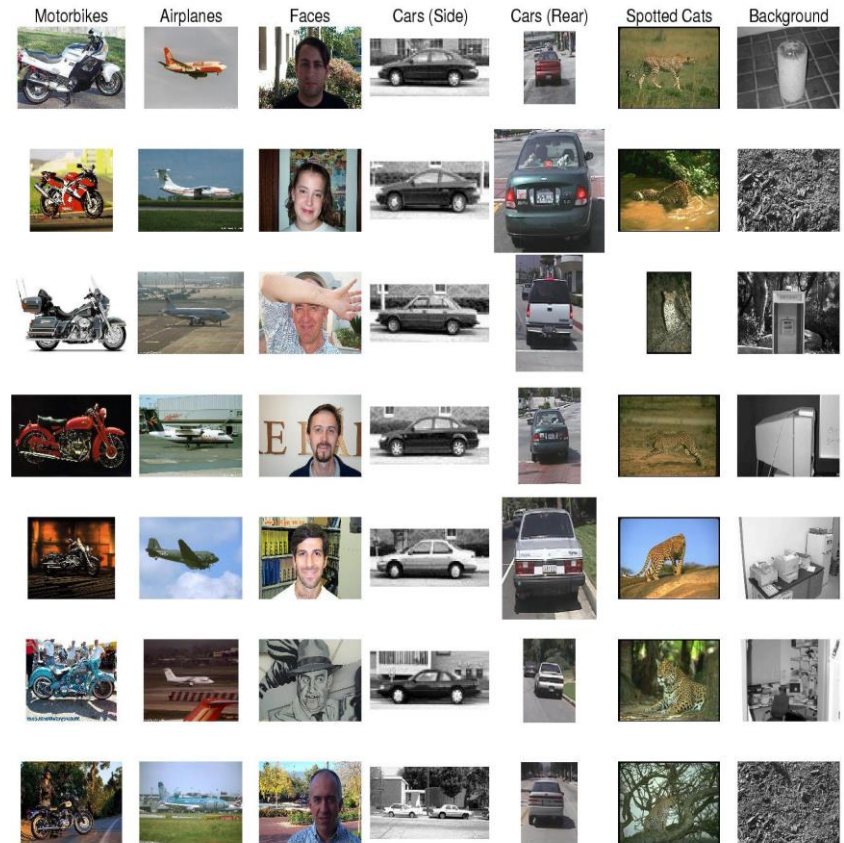
Object Recognition

- Model-based Object Recognition

Recognition relies upon the existence of a set of **predefined** objects.



- Generic Object Recognition



Object Recognition techniques classification

- **Feature based recognition**
 - Model-based Object Recognition
 - Interpretation trees
 - **Hypothesis and test**
 - **Alignment**
 - **Pose clustering**
 - **Geometric Hashing**
 - Recognition using SIFT detector
 - Generic Object Recognition
 - **Bag of features**
- **Appearance based recognition**
 - Generative models
 - Local features
 - Edge recognition
 - Greyscale recognition, texture recognition (**LBP**)
 - Gradient recognition
 - Global features
 - **Large modelbases (Eigenimages)**
 - Discriminative models: Supervised learning

Object Recognition

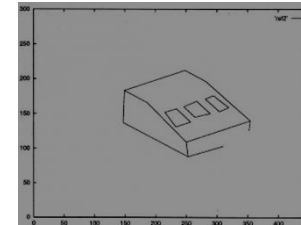
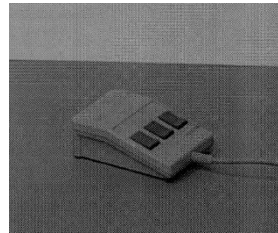
- Main steps :
 - **Preprocessing**
 - A model database is built by establishing associations between features and models.
 - **Recognition**
 - Scene features are used to retrieve appropriate associations stored in the model database.
- Approaches differ according to:
 - Restrictions to the form of the objects (simple vs complex and rigid vs deforming objects)
 - Representation schemes (object centered, viewer centered)
 - Type of features (Local or global)
 - **Matching schemes** (feature based or appearance based)
 - **learning algorithms**

Object Recognition : Matching schemes

- **Features based**

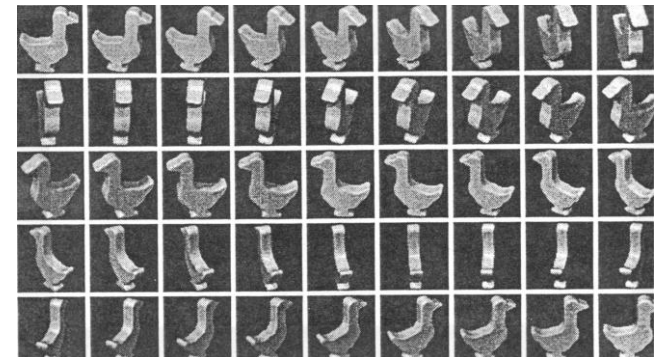
- A search is used to find feasible matches between object features and image features.
- Methods that extract features from the objects to be recognized and the images to be searched, are:

- surface patches
- corners
- linear edges
- Etc.



- **Appearance based**

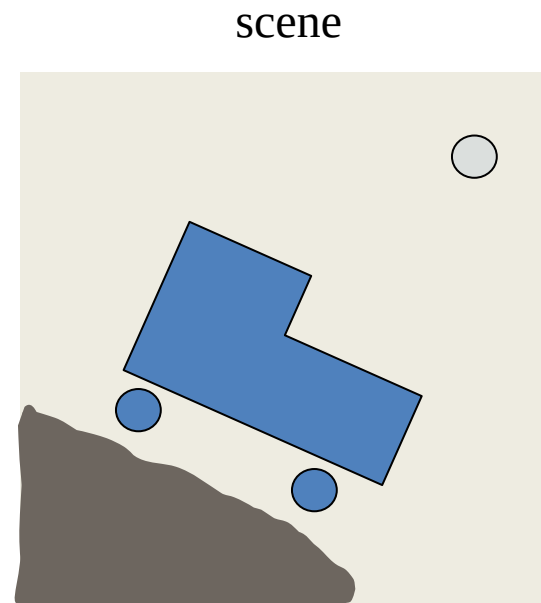
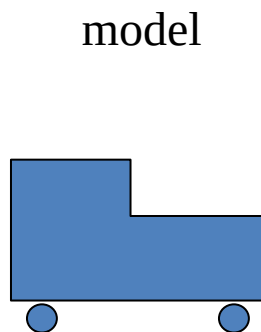
- Use example images (called templates) of the objects to perform recognition
- Objects look different under varying conditions:
 - Changes in lighting or color
 - Changes in viewing direction
 - Changes in size / shape
 - Etc.



- **Feature based recognition**
 - **Model-based Object Recognition**
 - Interpretation trees
 - **Hypothesis and test**
 - Alignment
 - Pose clustering
 - Geometric Hashing
 - Recognition using SIFT detector
 - **Generic Object Recognition**
 - **Bag of features**

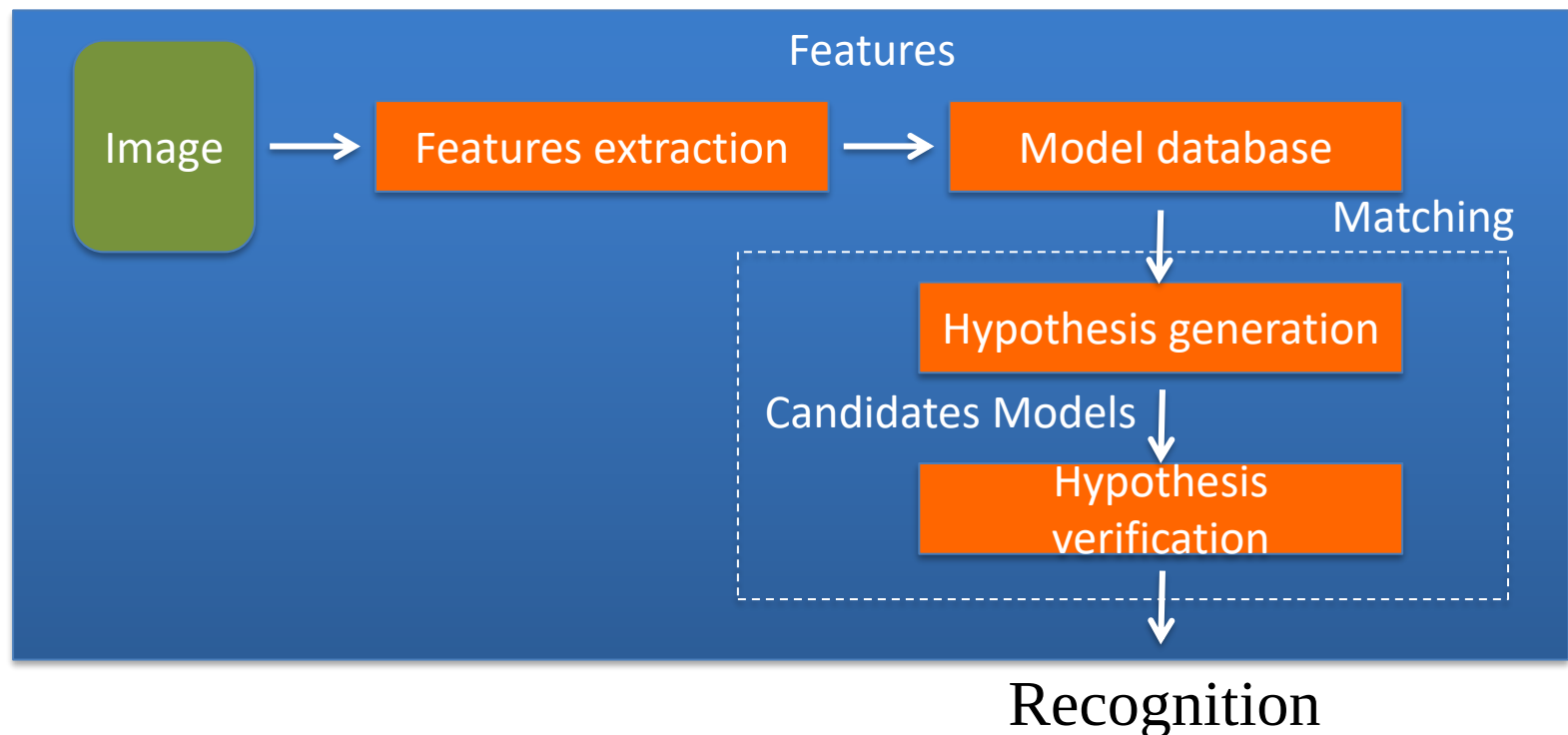
Features based Recognition: Model based

- (1) Identify a group of features from an unknown scene which approximately match a group of features from a model object (i.e., **correspondences**).
- (2) Recover the **geometric transformation** that the model object has undergone and look for additional matches.



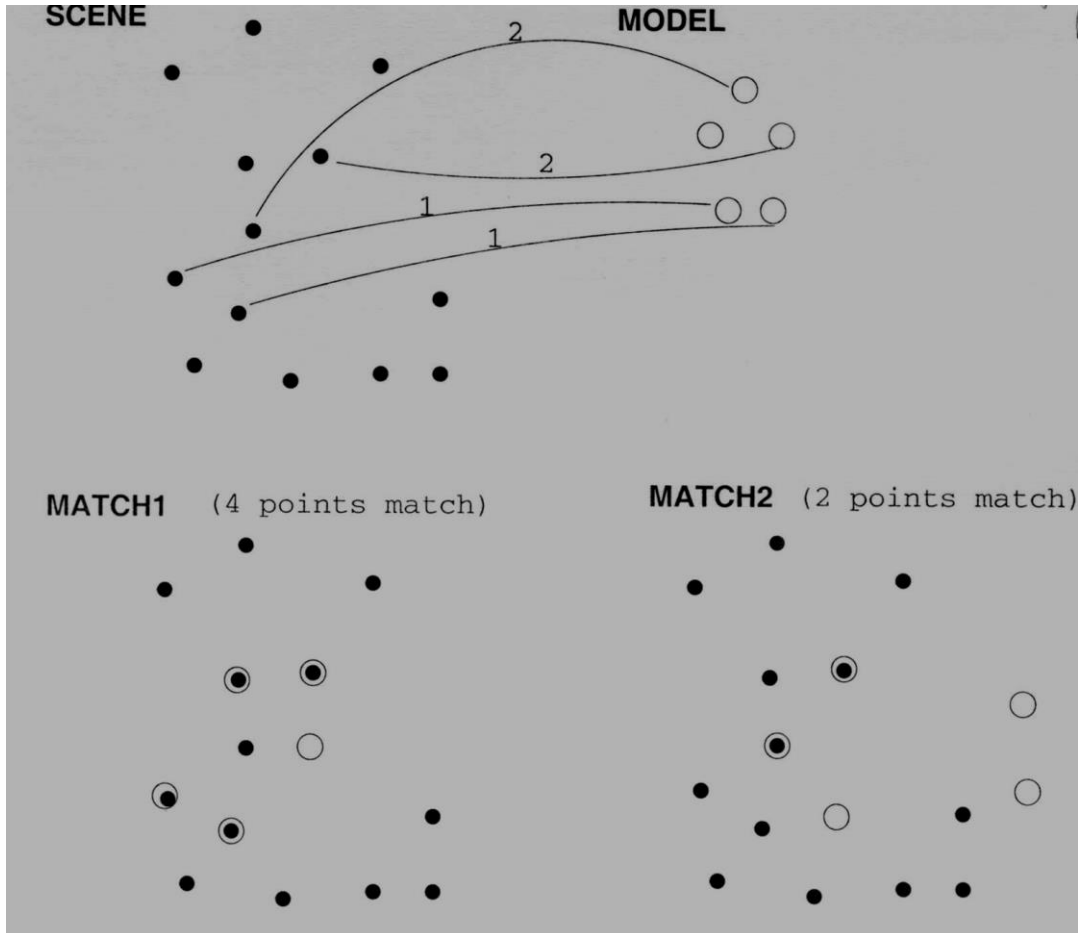
Features based approaches : Hypothesize and test

- **Hypothesis generation**: the identities of one or more models are hypothesized.
- **Hypothesis verification**: tests are performed to check if a given hypothesis is correct or not.



Hypothesis Verification - Example

two hypotheses:



Features (points)
might correspond to:

(1) curvature extrema or
zero-crossings along
the boundary of an object

(2) interest points.

Hypothesis Generation

- General Idea:

- Hypothesize a correspondence between a collection of image features and a collection of object features
- Then use this to generate a hypothesis about the projection from the object coordinate frame to the image frame
- Use this projection hypothesis to generate a rendering of the object. This step is usually known as backprojection
- Compare the rendering to the image, and, if the two are sufficiently similar, accept the hypothesis

- Obtaining Hypothesis:

- There are a variety of different ways of generating hypotheses.
- When camera intrinsic parameters are known, the hypothesis is equivalent to a hypothetical position and orientation – pose – for the object.
- Utilize geometric constraints
- Construct a correspondence for small sets of object features to every correctly sized subset of image points.

Hypothesis Generation

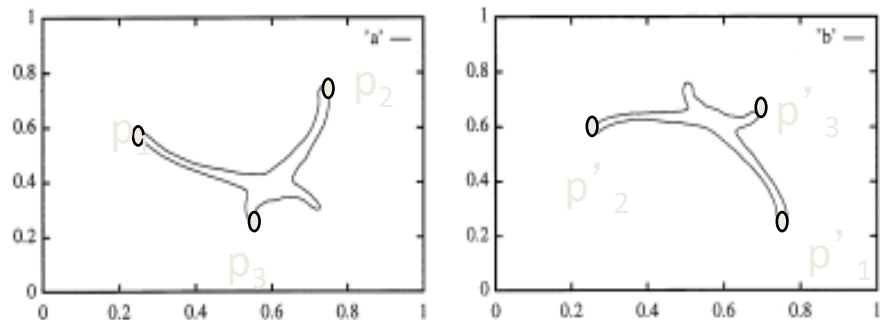
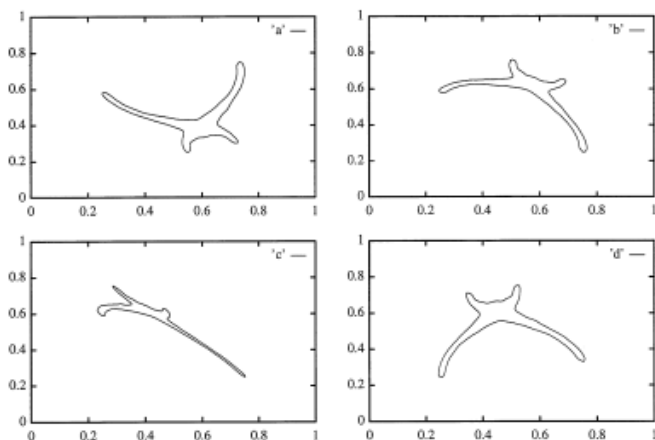
- How to choose the scene groups?
 - Do we need to consider every possible group?
 - How to find groups of features that are likely to belong to the same object?
 - **“Grouping”**: Employs properties based on perceptual organization rules to extract groups of features that are likely to come from the same object (orientation, co-linearity, parallelism, proximity, convexity)
 - **“Randomization”** Examining small sets of image features until likelihood of object presence becomes high. For each small set of image features, all possible matching sets of model features must be considered.
- How to organize and search the model database?
 - Do we need to search the whole database of models?
 - How should we organize the model database to allow for fast and efficient storage and retrieval?
 - **Query Optimization, Gibson’s Optimization, etc.**

Hypothesis Generation

- Three basic approaches:
- Obtaining Hypotheses by **Alignment**
- Obtaining Hypotheses by **Pose Clustering**
- Obtaining Hypotheses by **Geometric Hashing**

Hypothesis Generation by Alignment

- The alignment approach seeks to recover the geometric transformation between the model and the scene using a **minimum** number of correspondences.
- Assuming affine transformations, **three** correspondences are enough.
- Project other model features into image and verify additional correspondences(the others must be consistent with this)
- Under the assumption that objects are “flat” and the camera is not very close to the objects, different 2D views of a 3D object can be related by an **affine transformation**

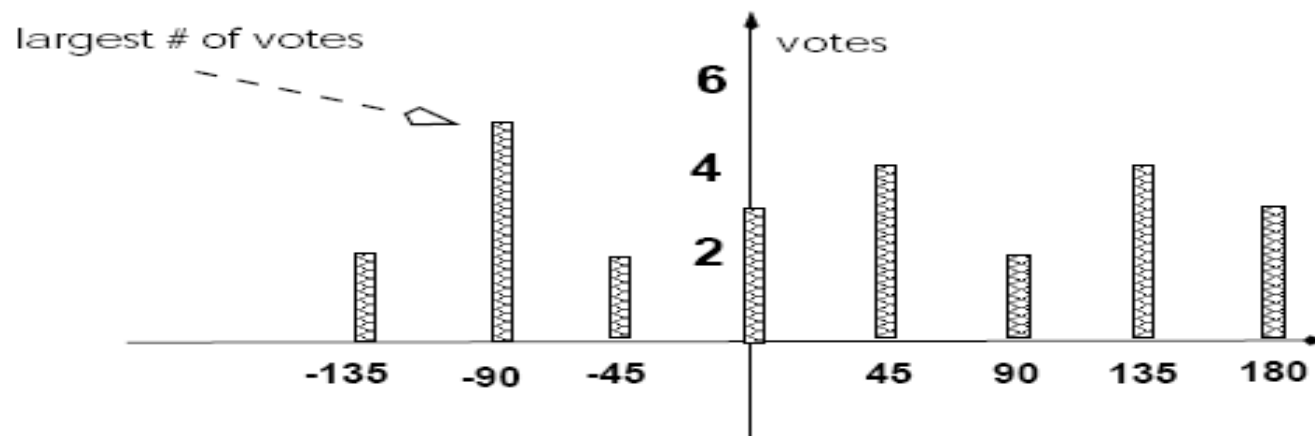
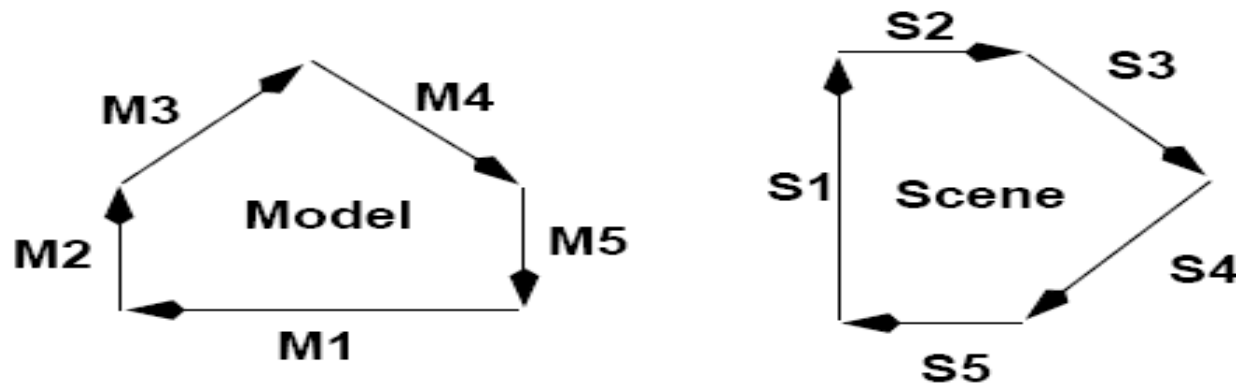


Hypothesis Generation by Pose clustering

- Performs “**voting**” in transformation space:
 - Correct transformations bring into alignment a large number of features.
 - Incorrect transformations bring into alignment a small number of features.
- Transformations that can bring into alignment a large number of features, will receive a large number of votes (i.e., support).
- **Main Steps**
 - (1) Quantize the space of possible transformations.
 - (2) For each hypothetical match, solve for the transformation that aligns the matched features.
 - (3) Cast a “vote” in the corresponding transformation space bin.
 - (4) Find "peak" in transformation space.

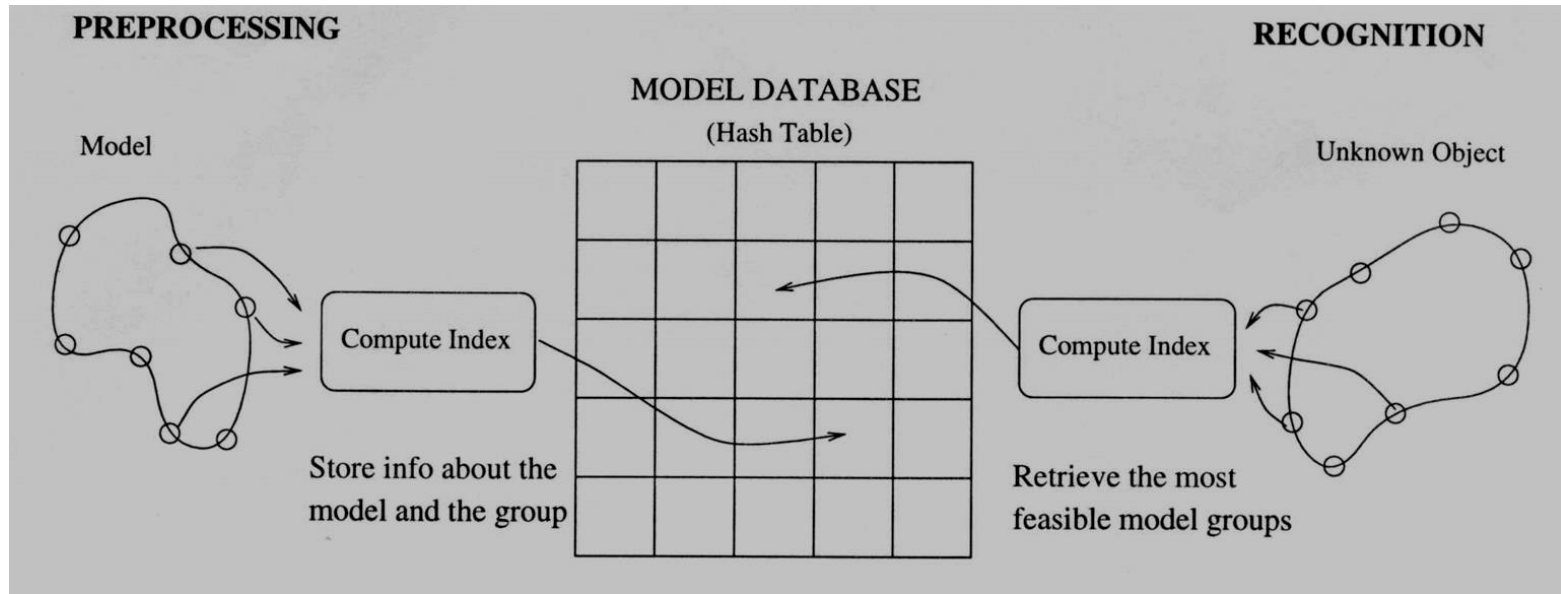
Hypothesis Generation by Pose clustering

- Example



Hypothesis Generation by Geometric Hashing

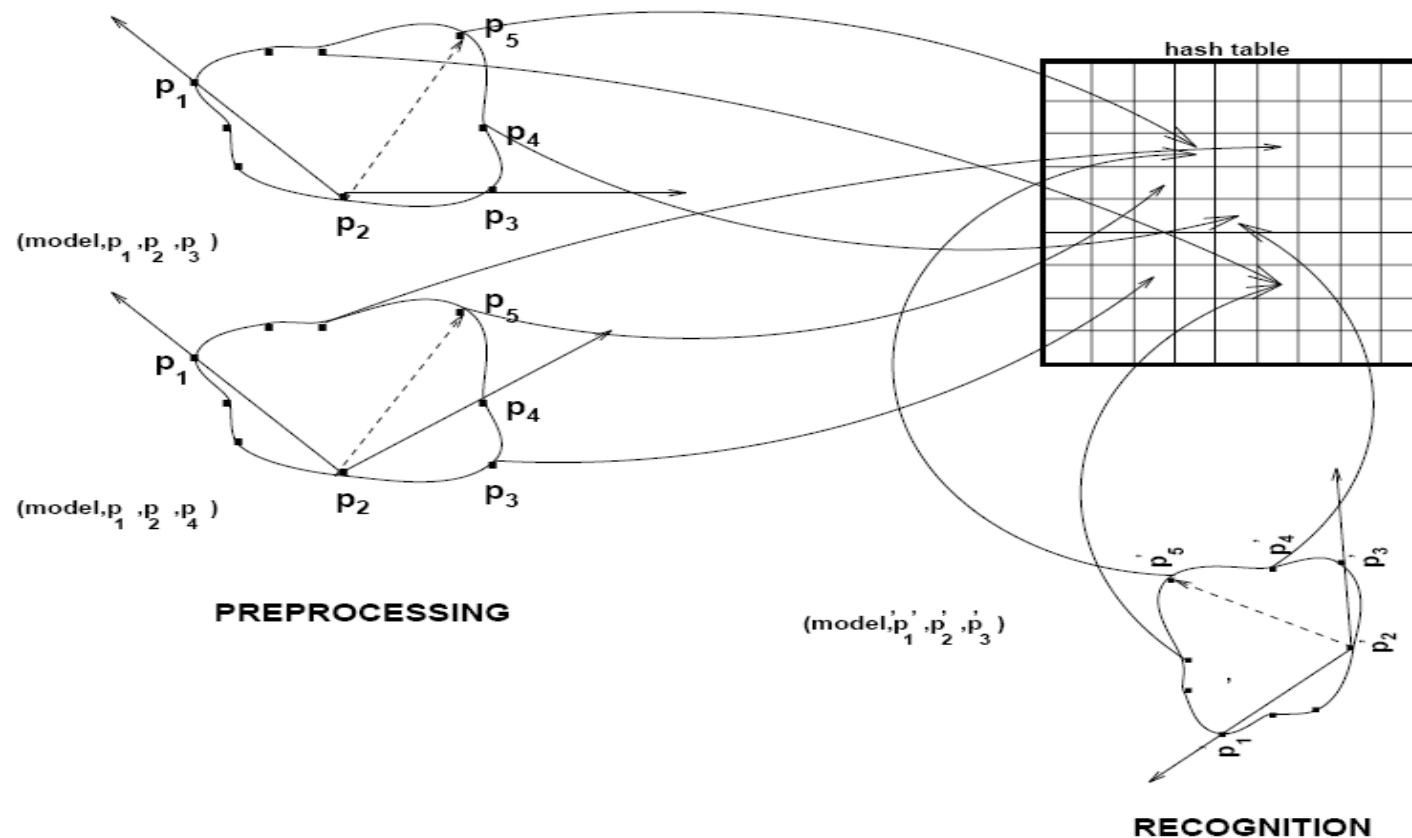
Indexing-Based Recognition



- **Recognition step**
 - (i) Groups of scene features are used to index the database.
 - (ii) The model objects listed in the indexed locations are collected into a list of candidate models (i.e., hypotheses).
- Ideally, we would like the index computed from a group of model features to be **invariant**.

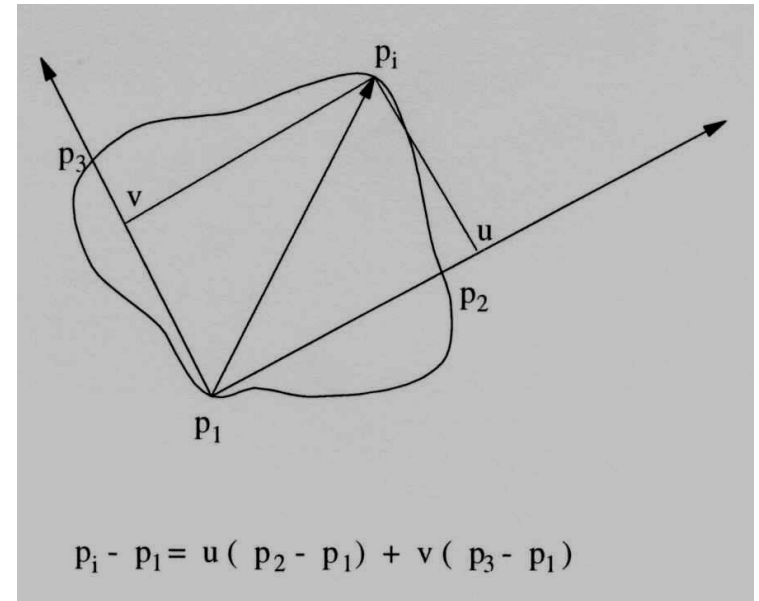
Hypothesis Generation by Geometric Hashing

- Geometric hashing: Indexing using **affine invariants** (Index remains invariant under affine transformations).



Geometric Hashing: Preprocessing

- Each triplet of non-collinear model points forms a basis of a coordinate system that is invariant under **affine transformations**.
- Represent model points in an affine invariant way by rewriting them in terms of this coordinate system.



(u,v) are **affine invariant**!

Geometric Hashing: Preprocessing

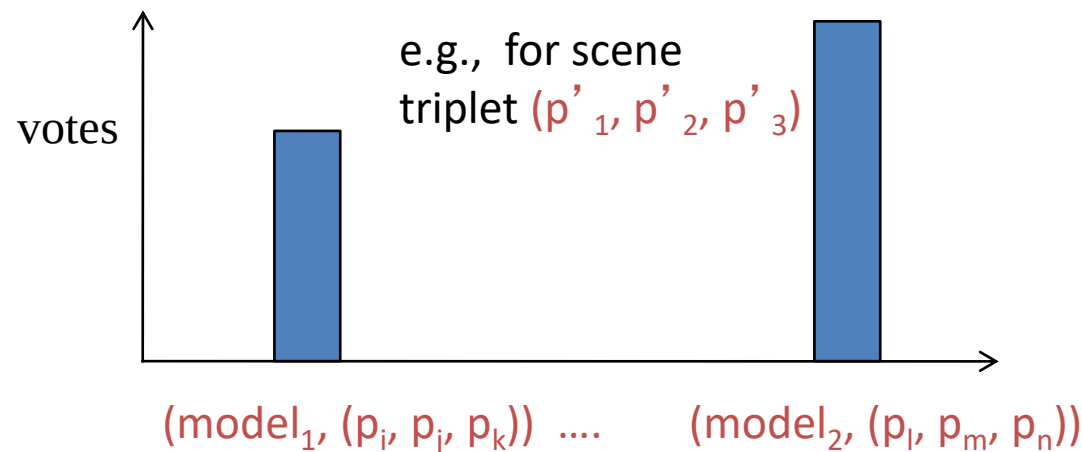
For each model do:

- (1) Extract a set of interest points.
- (2) For each ordered set of three, non-collinear, points (p_1, p_2, p_3)
 - (2a) Compute the coordinates (u, v) of the remaining features in the coordinate frame defined by the model basis (p_1, p_2, p_3)
 - (2b) Use the computed coordinates (u, v) as an index to a two dimensional *hash table*, and record in the corresponding hash table bin the information $(\text{model}, (p_1, p_2, p_3))$

Geometric Hashing: Recognition

- (1) Extract a set of interest points from the scene.
- (2) Choose an arbitrary ordered pair (p'_1, p'_2, p'_3)
- (3) Compute the coordinates (u', v') , of the remaining feature points in the coordinate frame defined by the image basis (p'_1, p'_2, p'_3)
- (4) Use the computed coordinates as an index to the hash table. For every entry **(model, (p_1, p_2, p_3))** found in the corresponding bin, **cast a vote**.

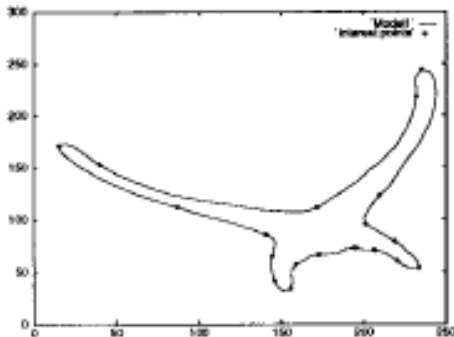
(5) Histogram all hash table entries that received one or more votes. Find those entries that received more than a certain number of votes -- each such entry corresponds to a potential match (**hypothesis generation**).



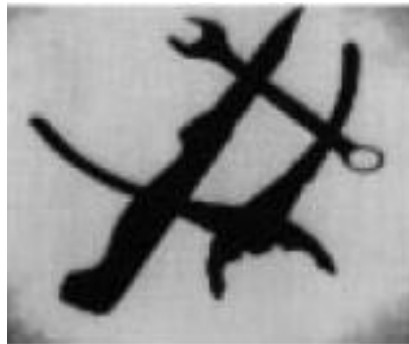
Geometric Hashing: Recognition

- (6) For each potential match (i.e., **hypothesis**), recover the affine transformation $\mathbf{A}$ that aligns the corresponding model-scene interest points.
- (7) Back-project the model onto the scene using the computed transform and compare the model with the scene to find additional matches (**verification step**).
- (8) If verification fails for all hypotheses step (5), go back to step (2) and repeat the procedure using a different group of scene features.

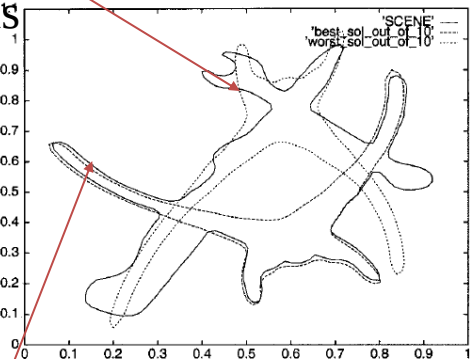
Model



Scene



Bad hypothesis



Good hypothesis

Object Recognition using SIFT features

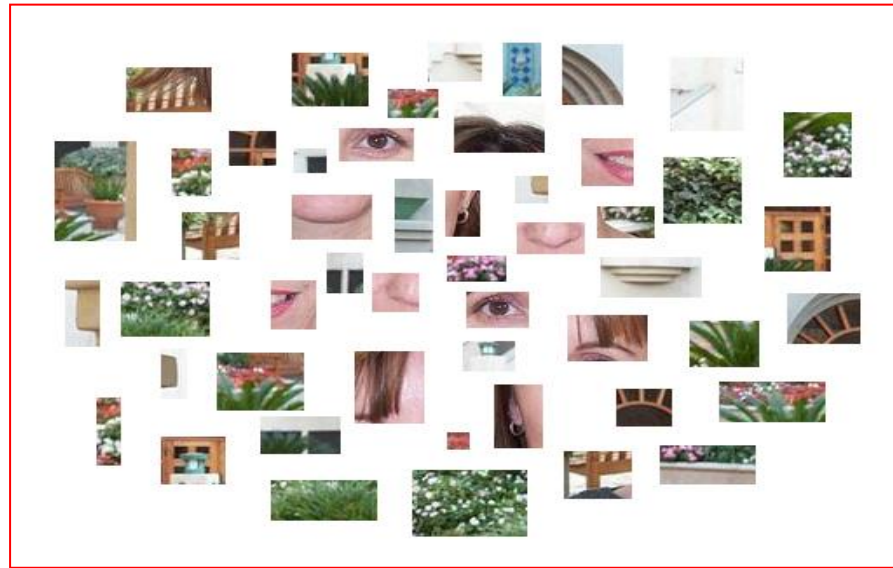
1. Match individual SIFT features from an image to a database of SIFT features from known objects (i.e., find nearest neighbors)
2. Find clusters of SIFT features belonging to a single object (**hypothesis generation**).
3. Estimate object pose (i.e., recover the transformation that the model has undergone) using at least **three** matches.
4. **Verify** that additional features agree on object pose.



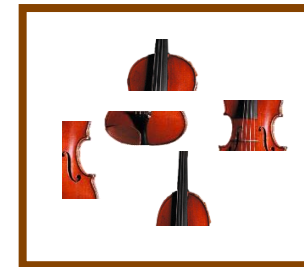
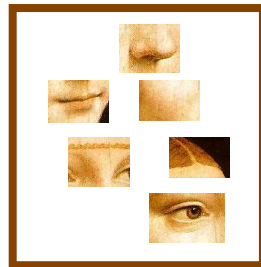
- **Feature based recognition**
 - **Model-based Object Recognition**
 - Interpretation trees
 - **Hypothesis and test**
 - Alignment
 - Pose clustering
 - Geometric Hashing
 - Recognition using SIFT detector
 - **Generic Object Recognition**
 - **Bag of features**

Object Characterization: Bag-of-features (BoF) models

Need a “visual” dictionary!

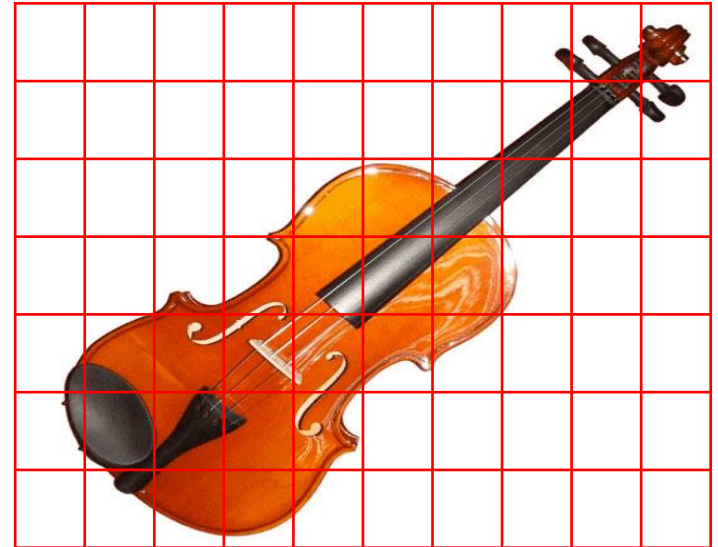
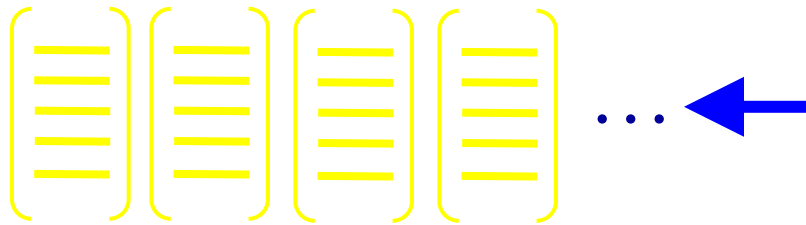


Characterize objects in terms of parts or local features

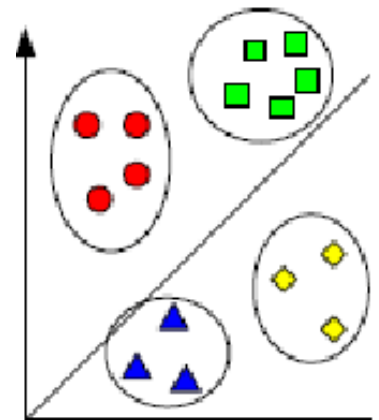


BoF: main steps

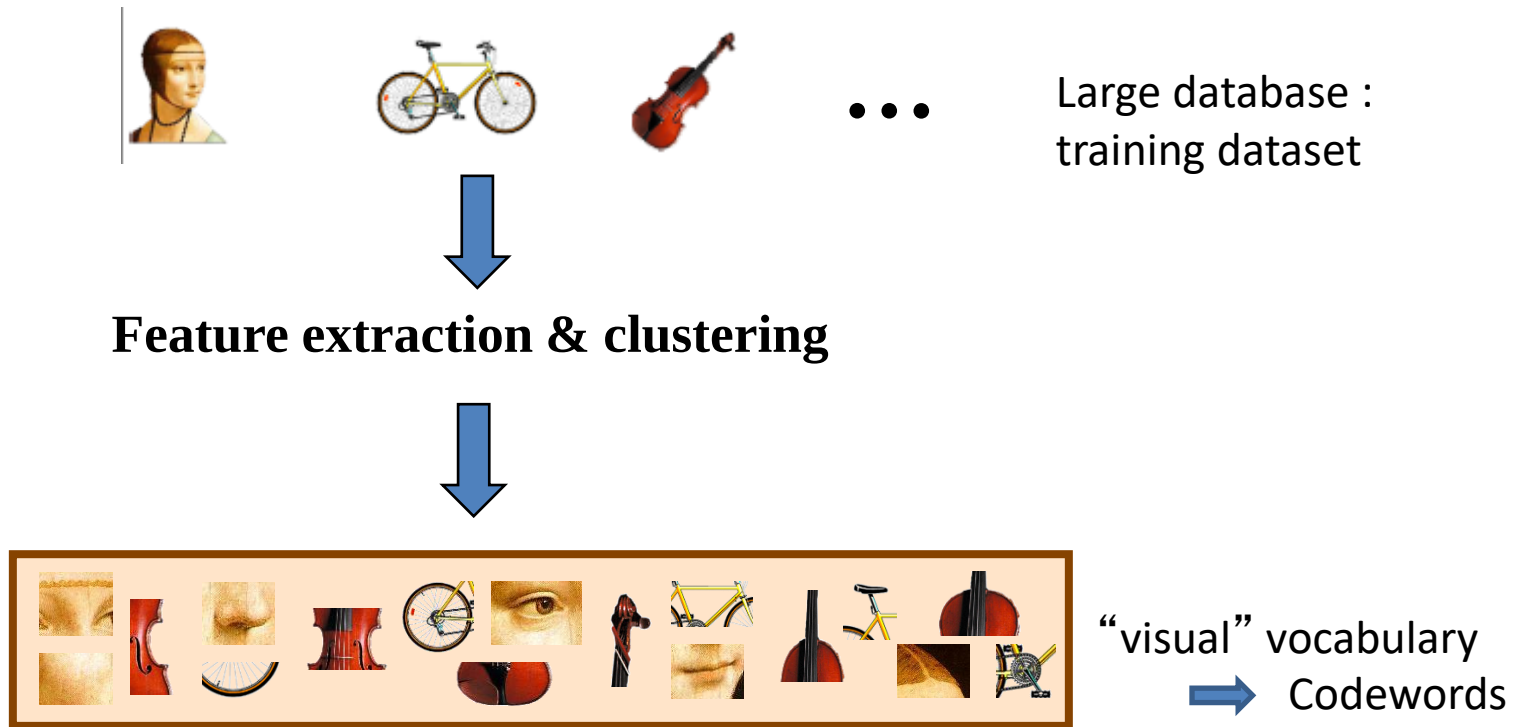
- **Step 1:** Features extraction (e.g., SIFT features, patches, etc.)



- **Step 2:** Features clustering (Kmeans)

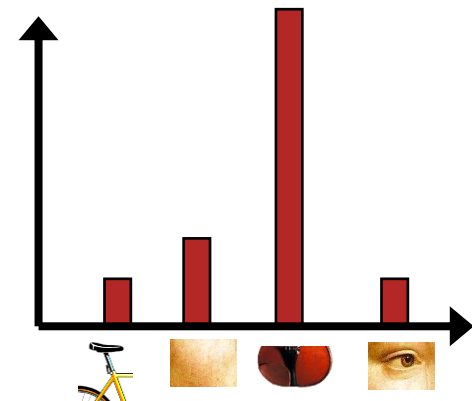
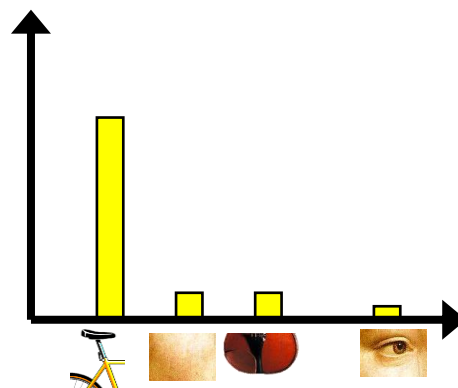
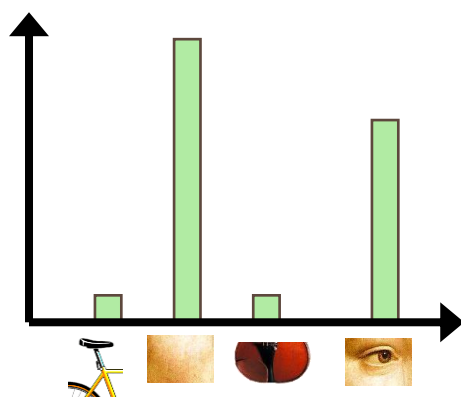


- **Step 3: Learn “visual” vocabulary**

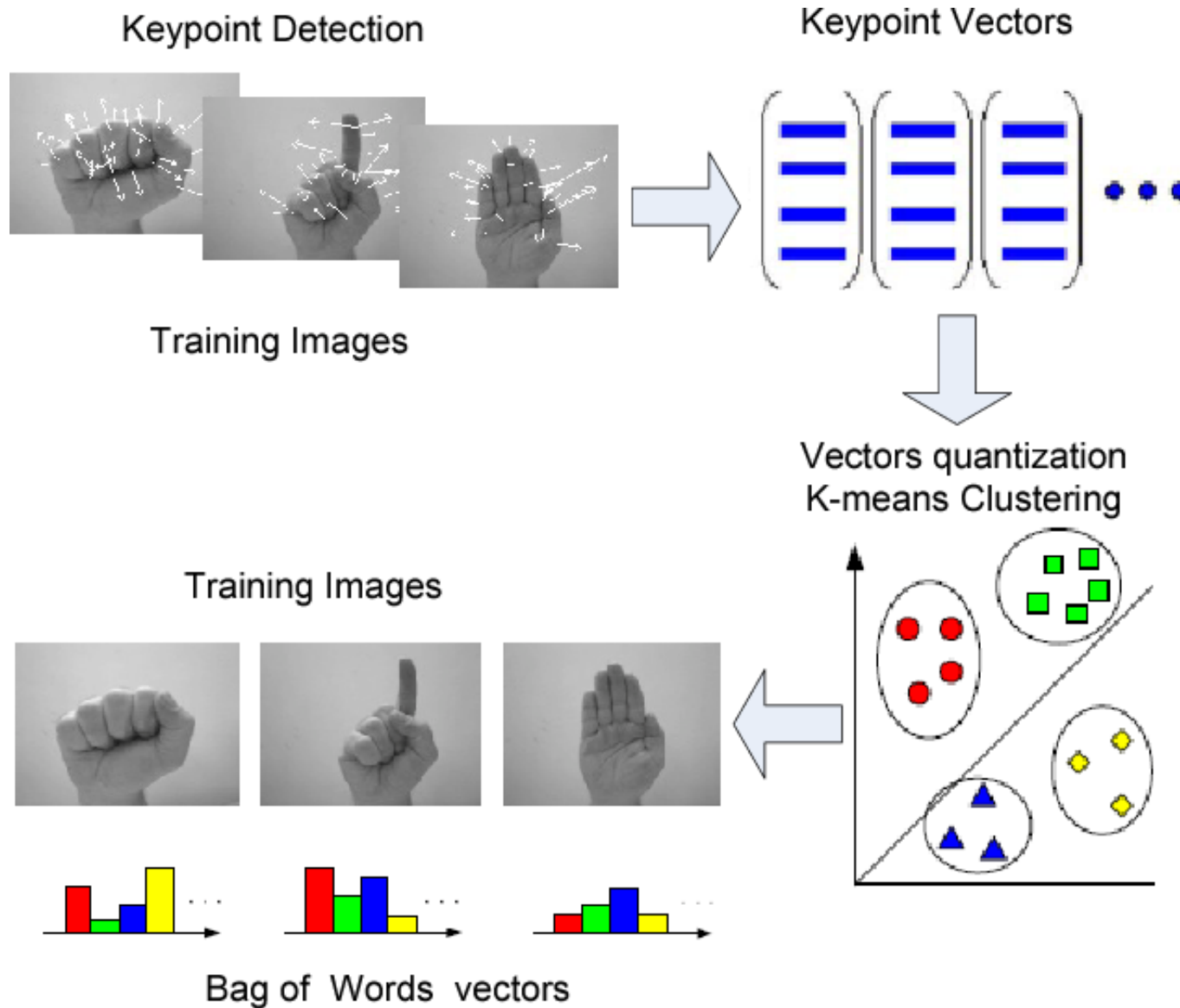


BoF: main steps

- **Step 4:** Quantize features using “visual” vocabulary (i.e., represent each feature by the closest cluster = codeword).
- **Step 5:** Represent images by frequencies of “visual words” (i.e., bags of features)

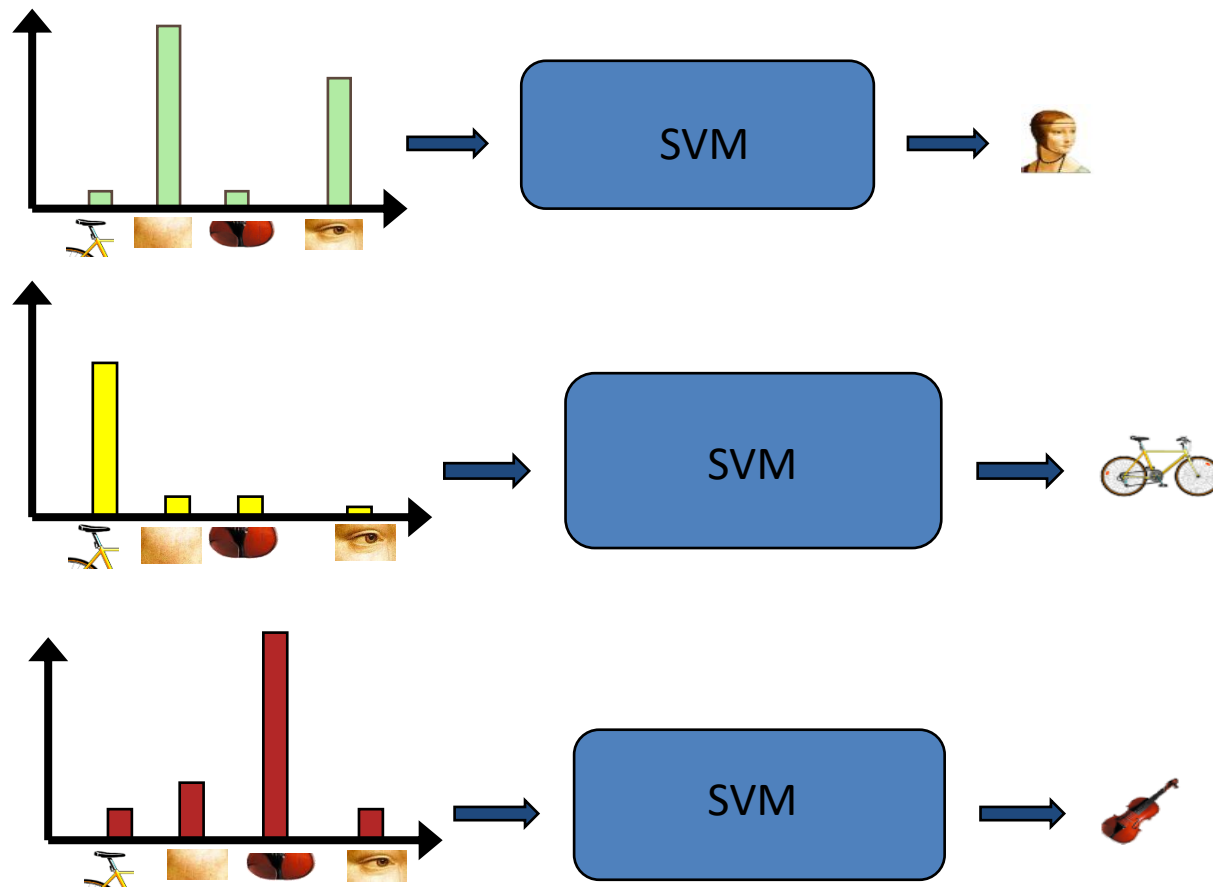


BoF: Example



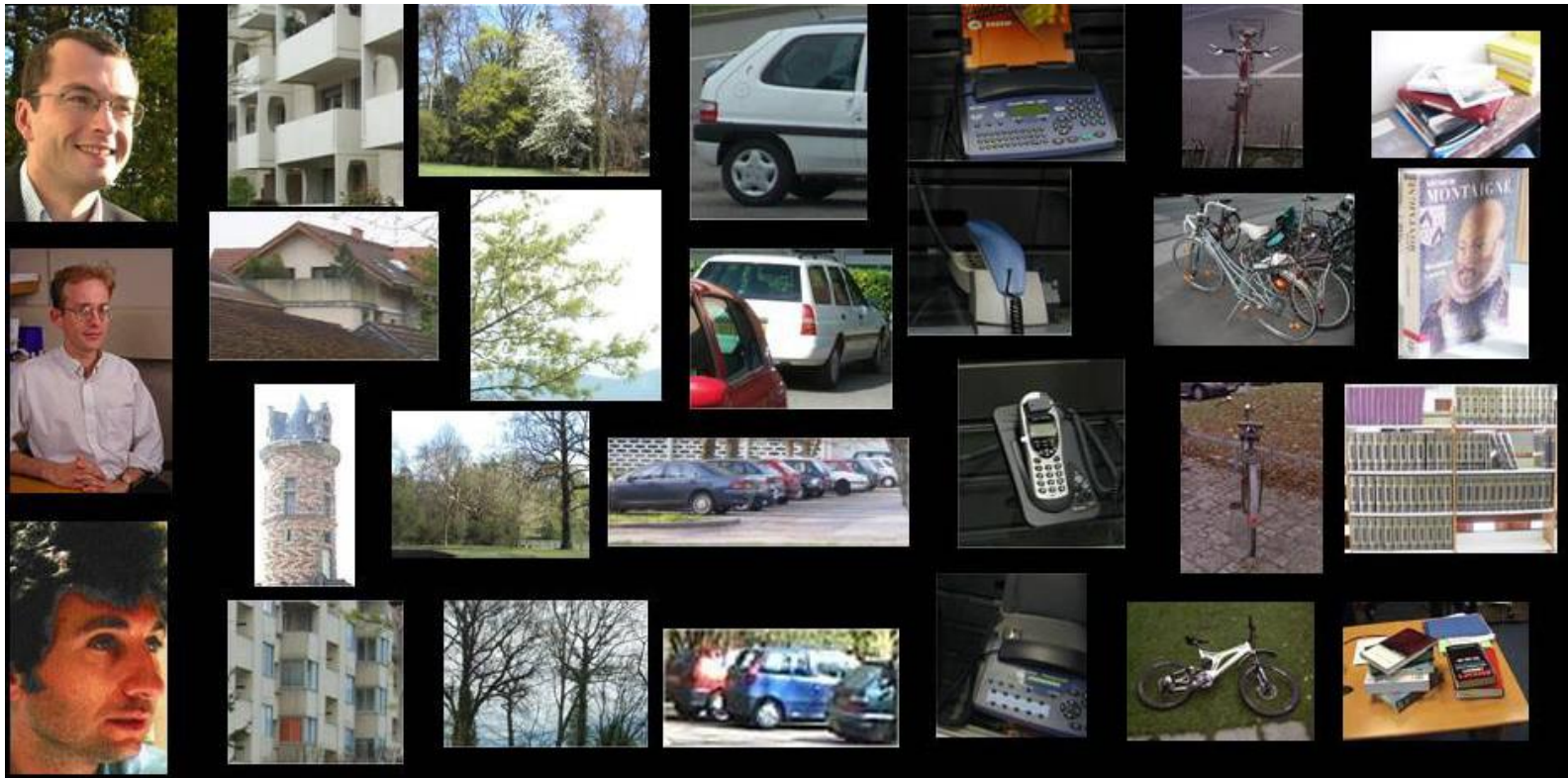
BoF for Object Categorization

- Example : SVM classifier



BoF for Object Categorization: Example

Dataset



BoF for Object Categorization: Example

Table 2. Confusion matrix and mean rank for SVM ($k=1000$, linear kernel).

True classes →	<i>faces</i>	<i>buildings</i>	<i>trees</i>	<i>cars</i>	<i>phones</i>	<i>bikes</i>	<i>books</i>
<i>faces</i>	98	14	10	10	34	0	13
<i>buildings</i>	1	63	3	0	3	1	6
<i>trees</i>	1	10	81	1	0	6	0
<i>cars</i>	0	1	1	85	5	0	5
<i>phones</i>	0	5	4	3	55	2	3
<i>bikes</i>	0	4	1	0	1	91	0
<i>books</i>	0	3	0	1	2	0	73

Dictionary **quality** and **size** are very important parameters!

— Generative models

- Local features

- **Region of interest detectors:** (Interest points detectors, Edge matching, Greyscale matching, etc.)
- **Region of interest descriptors:** (Distribution based, LBP, Moments invariants, etc.)

- Global features

- Large modelbases (Eigenimages)

— Discriminative models: Supervised learning

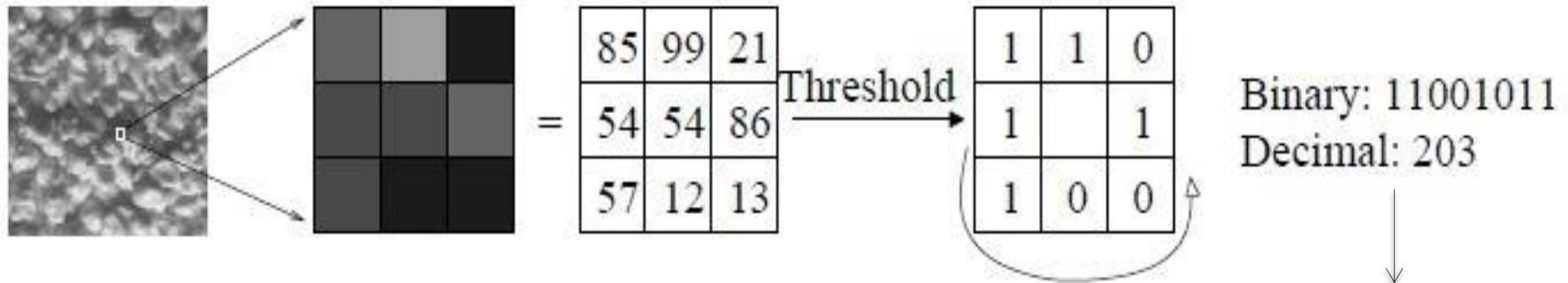
Generative models

Try to find a suitable representation of the original data approximating the original data by keeping as much information as possible

Local generative models

- Region of interest detectors
 - **Interest points detectors:** (Harris)
 - **Edge matching:** Uses edge detection techniques, such as the (Canny edge detector) to find edges.
 - **Greyscale matching:** Must compute pixel distance as a function of both pixel position and pixel intensity
- Region of interest descriptors
 - Distribution based: represent certain region properties by histograms.
 - **LBP:** texture descriptor approach which is based on simple binary coding of thresholds intensity values.
 - Moments invariants: generalized intensity and color moments invariants to geometric and photometric changes

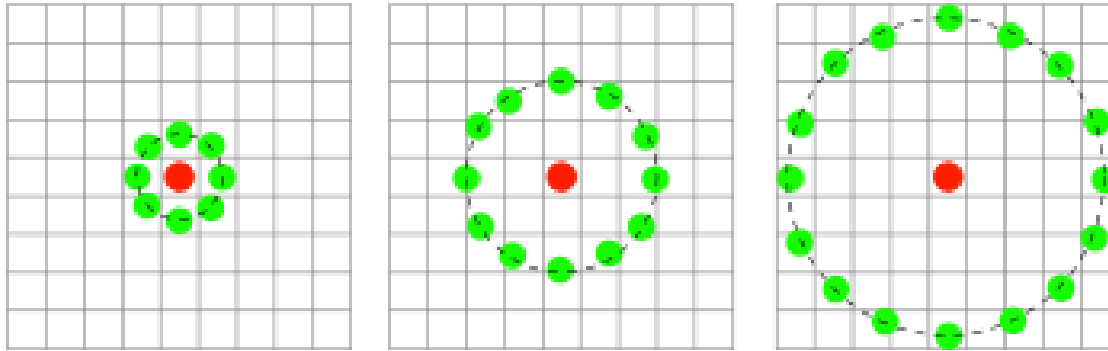
Basic LBP Operator



The histogram of the labels used as a texture descriptor.



The LBP operator was originally designed for texture description. The operator assigns a label to every pixel of an image by thresholding the 3x3-neighborhood of each pixel with the center pixel value and considering the result as a binary number.



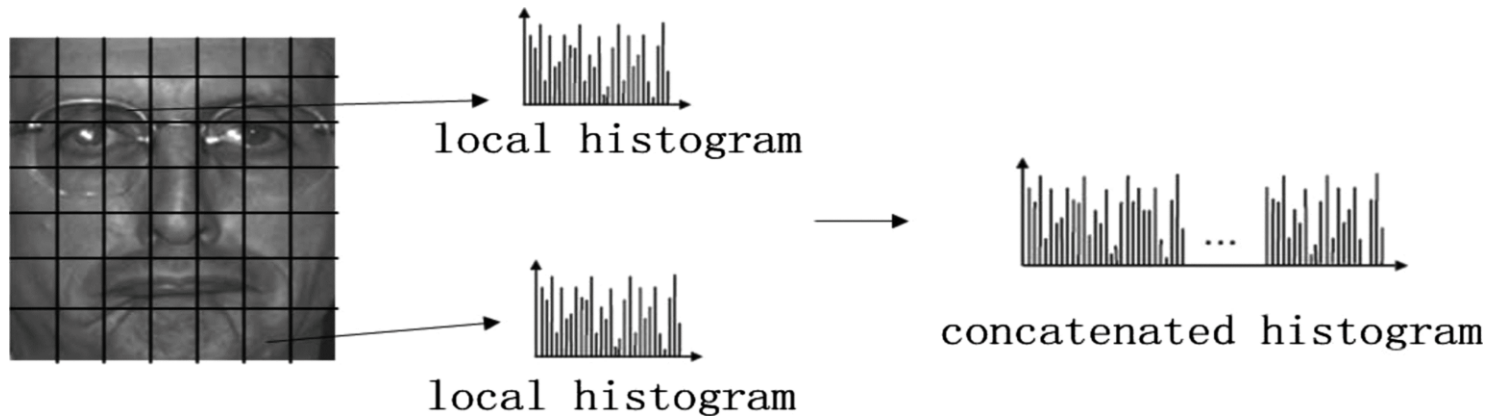
**Circular
neighborhoods.**

Uniform Pattern: A local binary pattern is called uniform if the binary pattern contains at most two bitwise transitions from 0 to 1 or vice versa.

EXP.

- The patterns 00000000 (0 transitions), 01110000 (2 transitions) and 11001111 (2 transitions) are uniform.
- The patterns 11001001 (4 transitions) and 01010011 (6 transitions) are not uniform.

Face description using LBP Operator



- 1: The LBP labels for the histogram contain information about the patterns on a pixel-level
- 2: The labels are summed over a small region to produce information on a regional level
- 3: The regional histograms are concatenated to build a global description of the face.



Face description using LBP Operator

- 1: Psychophysical findings indicate that some facial features (such as eyes) play more important roles in human face recognition than other features
- 2: Assume the regions can be weighted based on the importance of the information they contain.



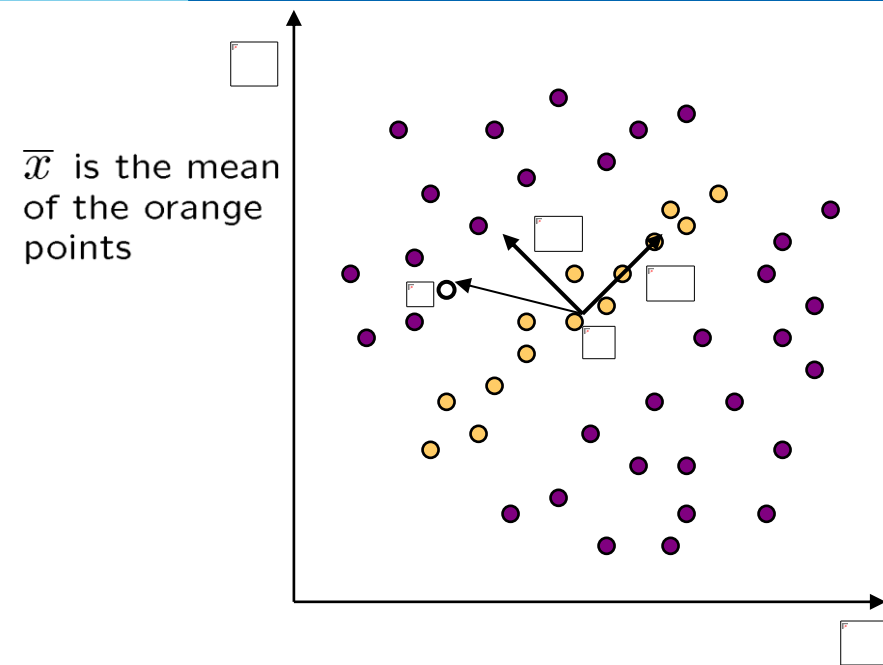
Defining a distance measure: $\chi_w^2(\mathbf{x}, \boldsymbol{\xi}) = \sum_{j,i} w_j \frac{(x_{i,j} - \xi_{i,j})^2}{x_{i,j} + \xi_{i,j}},$

in which $\mathbf{x}$ and $\boldsymbol{\xi}$ are the normalized enhanced histograms to be compared, indices i and j refer to i -th bin in histogram corresponding to the j -th local region and w_j is the weight for region j .

- Global generative models : Large modelbases

- No features are extracted, images are represented by basis functions (eigenvectors) and their coefficients.
- One approach to efficiently searching the database for a specific image using eigenvectors of the templates (called **eigenimages**)
- Images are highly correlated, so “compress” them into a low-dimensional space that captures key appearance characteristics (e.g., use Principal Component Analysis (PCA)). Matching is performed on the compressed image representation

Eigenvectors and Eigenvalues



Consider the sum squared distance of a point $\mathbf{x}$ to all of the orange points:

$$SSD(\mathbf{v}) = \sum_{\text{orange point } \mathbf{x}} \|(\mathbf{x} - \bar{\mathbf{x}})^T \cdot \mathbf{v}\|^2$$

What unit vector $\mathbf{v}$ minimizes SSD?

$$\mathbf{v}_2 = \min_{\mathbf{v}} \{SSD(\mathbf{v})\}$$

What unit vector $\mathbf{v}$ maximizes SSD?

$$\mathbf{v}_1 = \max_{\mathbf{v}} \{SSD(\mathbf{v})\}$$

$$\begin{aligned} SSD(\mathbf{v}) &= \sum_{\mathbf{x}} \|(\mathbf{x} - \bar{\mathbf{x}})^T \cdot \mathbf{v}\|^2 \\ &= \sum_{\mathbf{x}} \mathbf{v}^T (\mathbf{x} - \bar{\mathbf{x}}) (\mathbf{x} - \bar{\mathbf{x}})^T \mathbf{v} \\ &= \mathbf{v}^T \left[\sum_{\mathbf{x}} (\mathbf{x} - \bar{\mathbf{x}}) (\mathbf{x} - \bar{\mathbf{x}})^T \right] \mathbf{v} \\ &= \mathbf{v}^T \mathbf{A} \mathbf{v} \quad \text{where } \mathbf{A} = \sum_{\mathbf{x}} (\mathbf{x} - \bar{\mathbf{x}}) (\mathbf{x} - \bar{\mathbf{x}})^T \end{aligned}$$

Solution: $\mathbf{v}_1$ is eigenvector of $\mathbf{A}$ with *largest* eigenvalue

$\mathbf{v}_2$ is eigenvector of $\mathbf{A}$ with *smallest* eigenvalue

211

Principle Component Analysis

- Suppose each data point is N-dimensional

- Same procedure applies:

$$\begin{aligned} SSD(\mathbf{v}) &= \sum_{\mathbf{x}} \|(\mathbf{x} - \bar{\mathbf{x}})^T \cdot \mathbf{v}\|^2 \\ &= \mathbf{v}^T \mathbf{A} \mathbf{v} \quad \text{where } \mathbf{A} = \sum_{\mathbf{x}} (\mathbf{x} - \bar{\mathbf{x}})(\mathbf{x} - \bar{\mathbf{x}})^T \end{aligned}$$

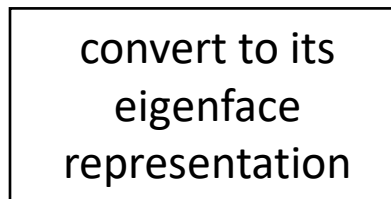
- The eigenvectors of $\mathbf{A}$ define a new coordinate system
 - eigenvector with largest eigenvalue captures the most variation among training vectors $\mathbf{x}$
 - eigenvector with smallest eigenvalue has least variation
- We can compress the data by only using the top few eigenvectors

Turk and Pentland's Eigenimages: Training

- Let $F_1, F_2, \dots, F_M$ be a set of training face images.
Let F be their mean and $\Phi_i = F_i - F$
- Use principal components to compute the eigenvectors and eigenvalues of the covariance matrix of the Φ_i s
- Choose the vector u of most significant M eigenvector to use as the basis.
- Each face is represented as a linear combination of eigenfaces
 $u = (u_1, u_2, u_3, u_4, u_5)$; $F_{27} = a_1 * u_1 + a_2 * u_2 + \dots + a_5 * u_5$

Matching

unknown
face image I



$\Omega = (\Omega_1, \Omega_2, \dots, \Omega_m)$

Find the face class k that minimizes
 $\epsilon_k = ||\Omega - \Omega_k||$

Example: eigenfaces

training
images



mean
image



3 eigen-
images

linear
approx-
imations



Appearance-based Recognition: Discriminative models

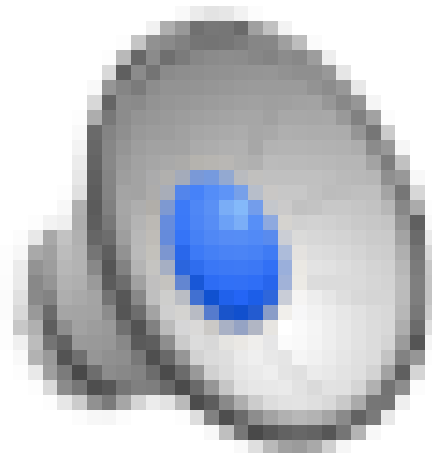
- **Discriminative models:** discriminative classifiers are designed for classification tasks. Given the training data and the corresponding labels, the goal is to find optimal decision boundaries.
 - SVM
 - Adaboost
 - NN
 - KNN
 - Etc.

Motion Analysis

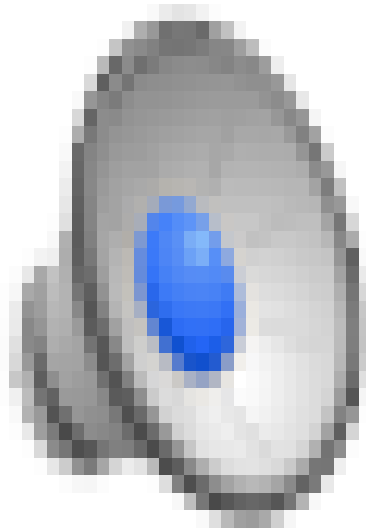
Motion Analysis

- The motion analysis processing can in the simplest case be to detect motion, i.e., find the points in the image where something is moving.
- Processing goals: determine the image regions where significant motion has occurred
- Output: an outline of the motion within the image sequence
- Two main approaches:
 - **Optical Flow**
 - Compute motion within region or the frame as a whole
 - **Change detection: Background subtraction**
 - The idea of background subtraction is to subtract or difference the current image from a reference background model estimated from consecutive frames.

Motion Analysis: Optical Flow

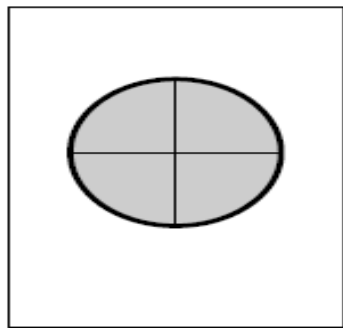


Motion Analysis: Background Substruction

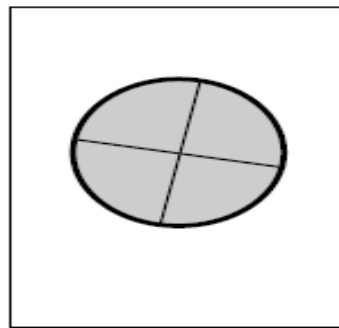


Motion analysis: Optical Flow

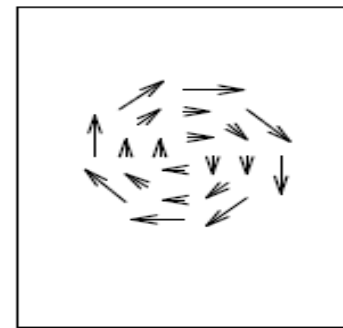
- The optical flow field is the velocity field that represents the three-dimensional motion of object points across a two-dimensional image.
- Optical flow computation is based on two assumptions:
 - The observed brightness of any object point is constant over time.
 - Nearby points in the image plane move in a similar manner (the **velocity smoothness** constraint).



Time t1
Flow

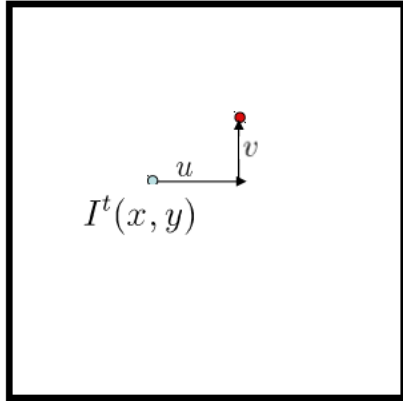


Time t2



Optical

Motion analysis: Optical Flow



- Image Brightness Constancy Equation:

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$$

- Assumes that the scene doesn't change intensity

Taylor-Approximation

- Think of x and y as functions of t

$$I(x + \Delta x, y + \Delta y, t + \Delta t) = I(x, y, t) + \frac{\partial I}{\partial x} \Delta x + \frac{\partial I}{\partial y} \Delta y + \frac{\partial I}{\partial t} \Delta t + \text{higher-order terms}$$

Again, if we assume that the intensity of the scene doesn't change, then

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$$

$$\frac{\partial I}{\partial x} \frac{\partial x}{\partial t} + \frac{\partial I}{\partial y} \frac{\partial y}{\partial t} + \frac{\partial I}{\partial t} \frac{\partial t}{\partial t} = 0 \rightarrow I_x u + I_y v + I_t = 0$$

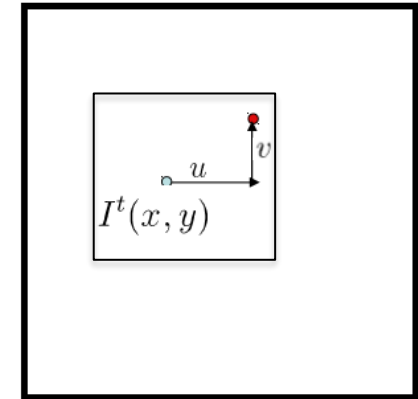
Brightness Constancy Equation

Motion analysis: Estimating Optical Flow

$$I_x u + I_y v + I_t = 0$$

- For only one point, we have 1 equation, 2 unknowns
- Since we can't solve it for one point, we assume that the motion is constant over a window
- Leads to an overcomplete system

$$\begin{aligned} I_x^{(p_1, q_1)} u + I_y^{(p_1, q_1)} v &= -I_t^{(p_1, q_1)} \\ I_x^{(p_2, q_2)} u + I_y^{(p_2, q_2)} v &= -I_t^{(p_2, q_2)} \\ I_x^{(p_3, q_3)} u + I_y^{(p_3, q_3)} v &= -I_t^{(p_3, q_3)} \\ I_x^{(p_4, q_4)} u + I_y^{(p_4, q_4)} v &= -I_t^{(p_4, q_5)} \\ I_x^{(p_5, q_5)} u + I_y^{(p_5, q_5)} v &= -I_t^{(p_5, q_5)} \end{aligned}$$



Assumption2: Nearby points in the window move in a similar manner

Motion analysis: Lucas Kanade Algorithm

$$\begin{matrix} & \mathbf{A} & & \mathbf{h} \\ \begin{bmatrix} I_x^{(p_1,q_1)} & I_y^{(p_1,q_1)} \\ I_x^{(p_2,q_2)} & I_y^{(p_2,q_2)} \\ I_x^{(p_3,q_3)} & I_y^{(p_3,q_3)} \\ I_x^{(p_4,q_4)} & I_y^{(p_4,q_4)} \\ I_x^{(p_5,q_5)} & I_y^{(p_5,q_5)} \end{bmatrix} & \begin{bmatrix} \mathbf{x} \\ u \\ v \end{bmatrix} & = & \begin{bmatrix} I_t^{(p_1,q_1)} \\ I_t^{(p_2,q_2)} \\ I_t^{(p_3,q_3)} \\ I_t^{(p_4,q_4)} \\ I_t^{(p_5,q_5)} \end{bmatrix} \end{matrix}$$

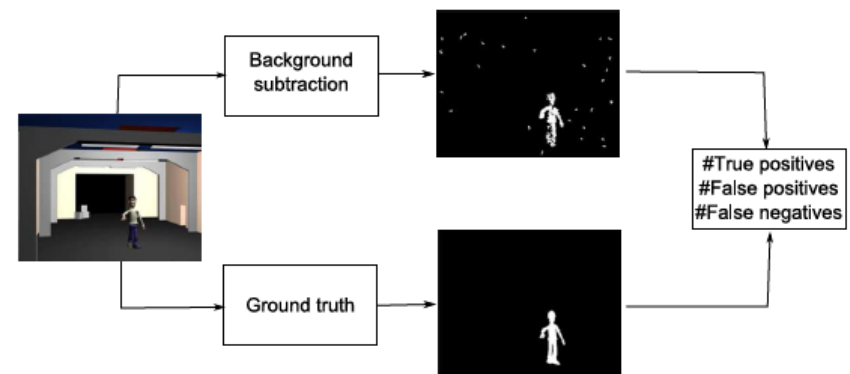
$$\hat{\mathbf{x}} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{h}$$

$$\mathbf{A}^T \mathbf{A} = \begin{bmatrix} \sum (I_x)^2 & \sum I_x I_y \\ \sum I_x I_y & \sum (I_y)^2 \end{bmatrix}$$

The sum is over a window

- Estimate motion using pseudo-inverse
- Warp image according to estimates of motion
- Re-estimate motion
- Iterate

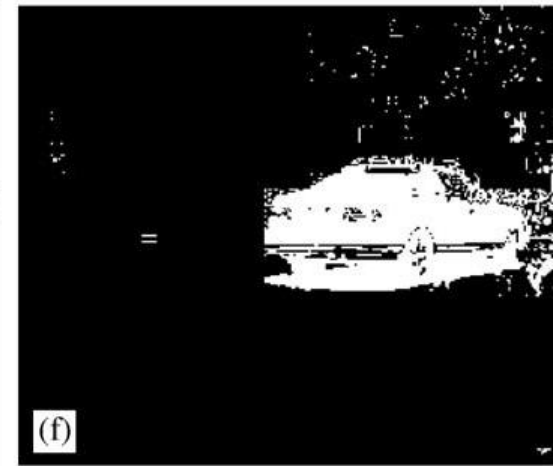
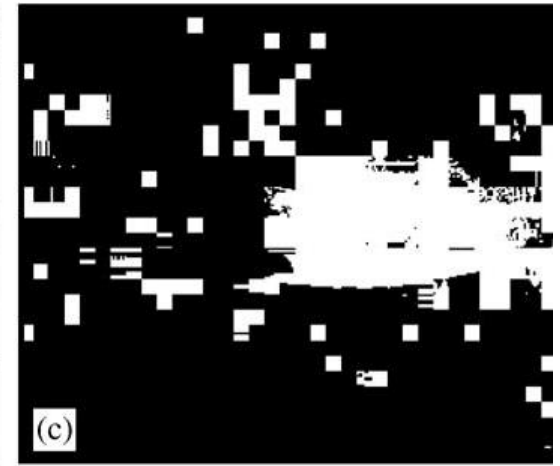
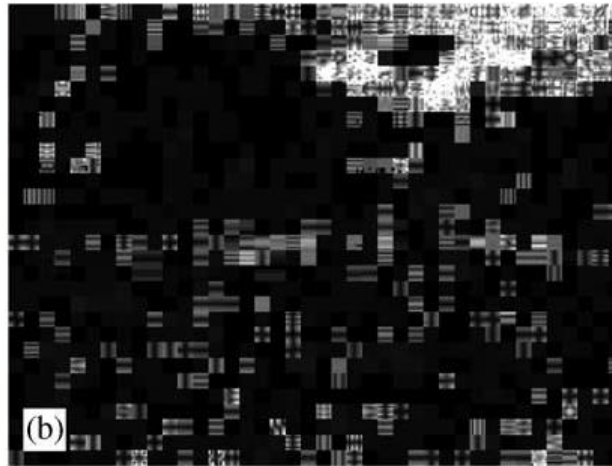
Motion analysis: Background subtraction



Motion analysis: Background subtraction

Operator	Accuracy	Computing time and memory requirements	Noisy backgrounds
Basic method	Medium	High	Low
Min-Max based method	Medium	High	Low
Gaussian Mixture Model	High	Medium	High
Kernel Density estimation	High	Low	High
Codebook based technique	High	Depend on the video	High
Eigen backgrounds	High	Low	High

Background subtraction results



Detection results on a compressed video: (a) original image, (b) standard deviations, (c) One Gaussian model, (d) GMM, (e) Kernel estimation Model, (f) CodeBook.

Background Subtraction: MOG Model

At any time t , what is known about a particular pixel, (x_0, y_0) , is its history:

$$\{X_1, \dots, X_t\} = \{I(x_0, y_0, i) : 1 \leq i \leq t\}$$

This history is modeled by a mixture of K Gaussian distributions:

$$P(X_t) = \sum_{i=1}^K \omega_{i,t} * \mathcal{N}(\mathbf{X}_t | \mu_{i,t}, \boldsymbol{\Sigma}_{i,t})$$

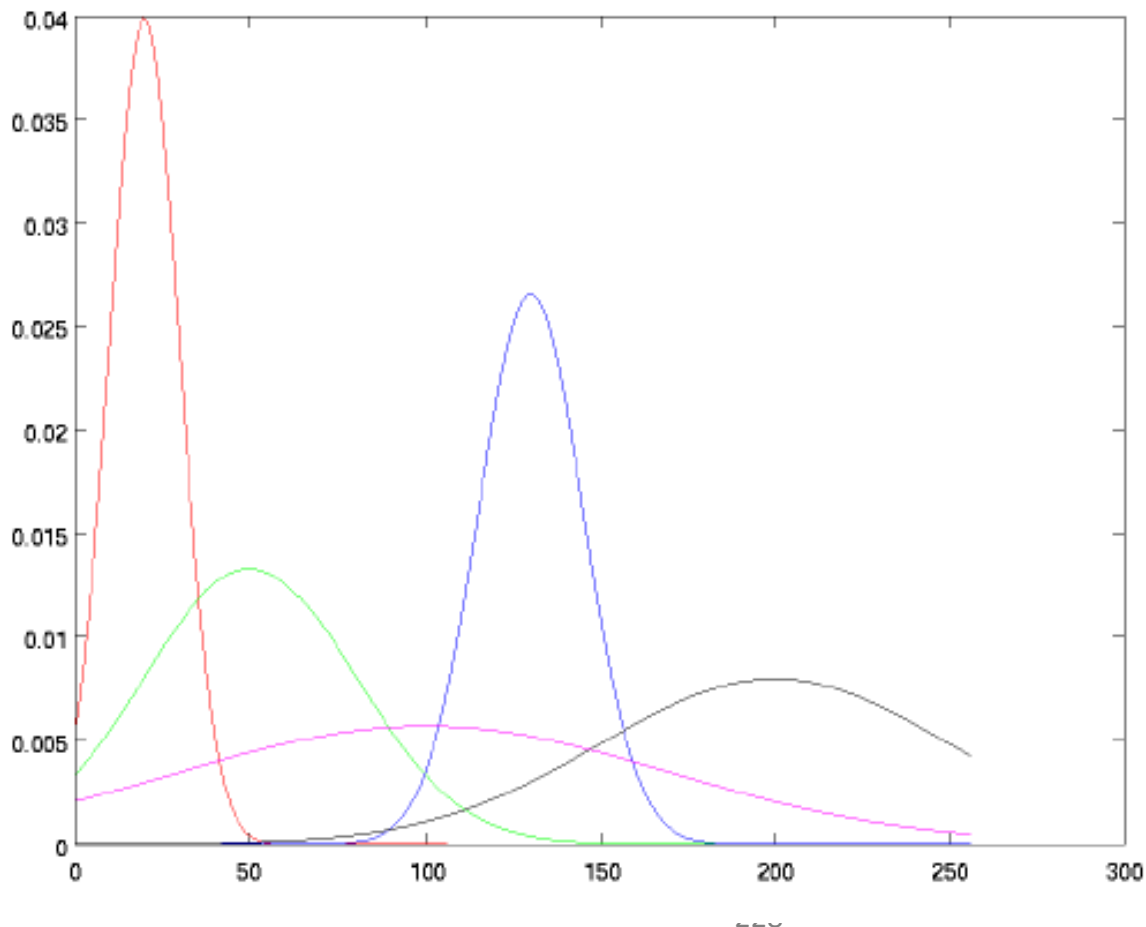
where

$$\mathcal{N}(\mathbf{X}_t | \mu_{i,t}, \boldsymbol{\Sigma}_{i,t}) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\boldsymbol{\Sigma}_{i,t}|^{1/2}} e^{-\frac{1}{2}(\mathbf{X}_t - \mu_{i,t})^T \boldsymbol{\Sigma}_{i,t}^{-1} (\mathbf{X}_t - \mu_{i,t})}$$

What is the dimensionality of the Gaussian?

Background Subtraction: MOG Model

If we assume gray scale images and set $K = 5$, history of a pixel will be something like this:



Background Subtraction: MOG Algorithm

- Model the values of a particular pixel as a mixture of Gaussians.
- We determine which Gaussians may correspond to background colors-Based on the persistence and the variance of each of the Gaussians.
- Pixel values that do not fit the background distributions are considered foreground until there is a Gaussian that includes them.
- Update the Gaussians.
- Pixel values that do not match one of the pixel's “background” Gaussians are grouped using connected components.

Stage 1

- Every new pixel value, X_{t+1} , is checked against the existing K Gaussian distributions until a match is found.
- A match is defined as a pixel value within 2.5 standard deviations of a distribution.

Stage 2 No Match

- If none of the K distributions match the current pixel value, the least probable distribution is go out.
- A new distribution with the current value as its mean value, an initially high variance, and low prior weight, is enter .

Stage 3 Match

- ▶ If a new pixel value, X_{t+1} , can be matched to one of the existing Gaussians (within 2.5σ), that Gaussian's $\mu_{i,t+1}$ and $\sigma_{i,t+1}^2$ are updated as follows:

$$\mu_{i,t+1} = (1 - \rho)\mu_{i,t} + \rho X_{t+1}$$

and

$$\sigma_{i,t+1}^2 = (1 - \rho)\sigma_{i,t}^2 + \rho(X_{t+1} - \mu_{i,t+1})^2$$

where $\rho = \alpha \mathcal{N}(X_{t+1} | \mu_{i,t}, \sigma_{i,t}^2)$ and α is a learning rate.

- ▶ Prior weights of all Gaussians are adjusted as follows:

$$\omega_{i,t+1} = (1 - \alpha)\omega_{i,t} + \alpha(M_{i,t+1})$$

where $M_{i,t+1} = 1$ for the matching Gaussian and $M_{i,t+1} = 0$ for all the others.

Stage 4

- determine which of the Gaussians of the mixture are most likely produced by background processes.
- We are interested in the Gaussian distributions which have the most supporting evidence and the least variance.
- For “background” distributions when a static, persistent object is visible => high weight and relatively low variance.
- New object occludes the background object => creation of a distribution or the increase in the variance of an existing distribution => the variance of the moving object is expected to remain larger than a background pixel until the moving object stop => low weight and relatively high variance.

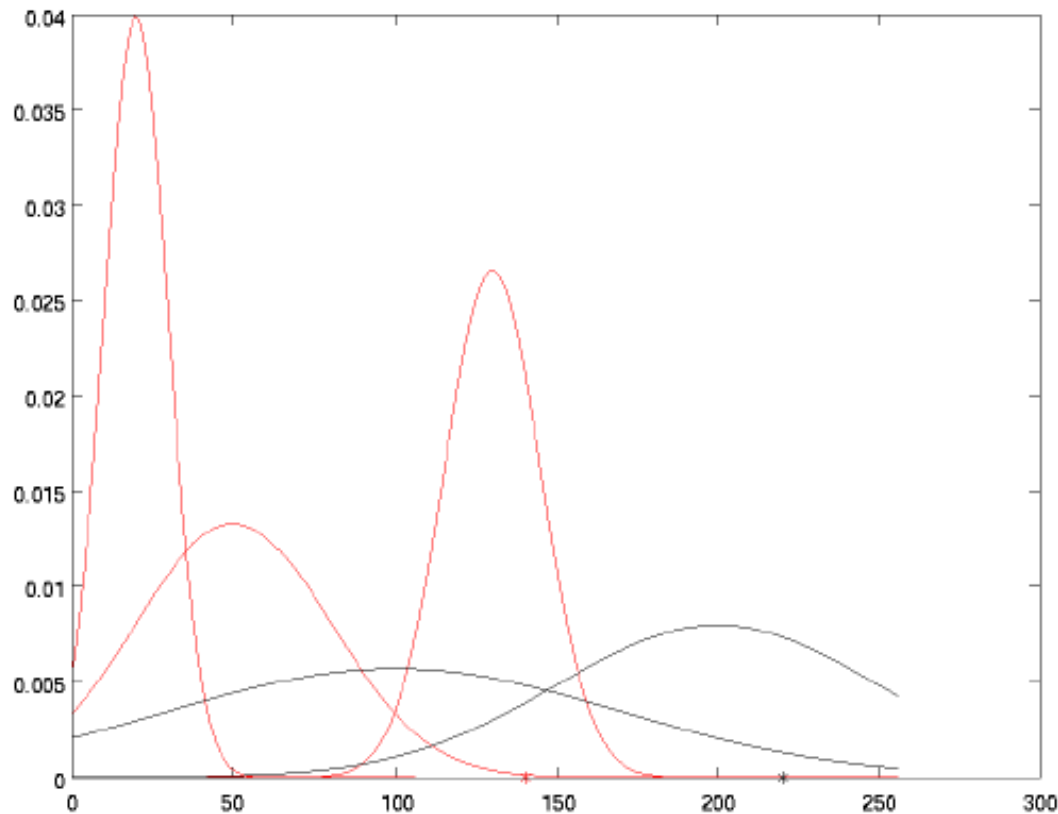
Stage 4

- the Gaussians are ordered by the value of ω/σ .
- Then, the first B distributions are chosen as the background model, where

$$B = \operatorname{argmin}_b \left(\sum_{k=1}^b \omega_k > T \right),$$

- T is a measure of the minimum portion of the data that should be accounted for by the background.

Background Subtraction: MOG



- After background model estimation **red** distributions become the background model and **black** distributions are considered to be foreground.

Background Subtraction: Codebook

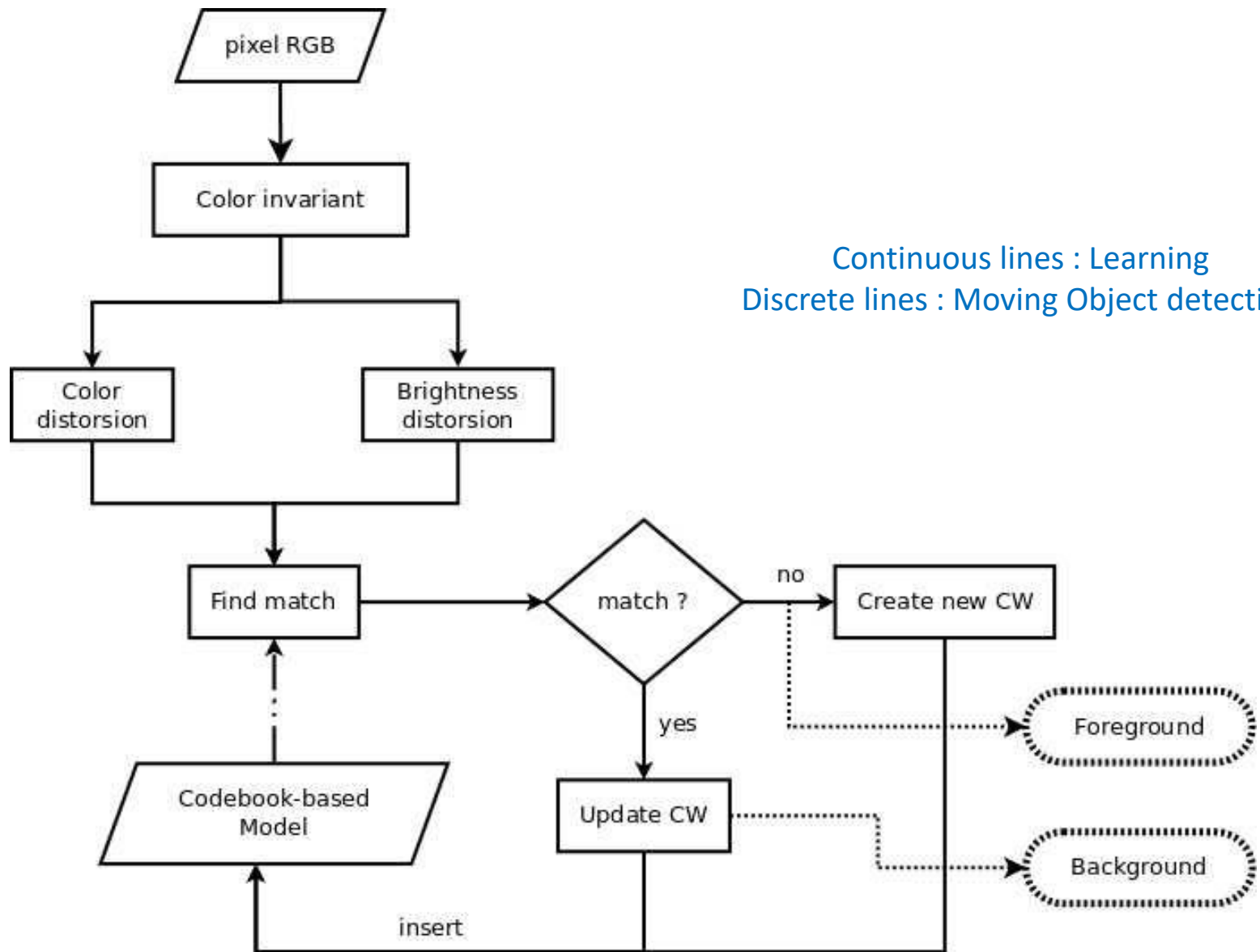
Codebook technique works according to 2 steps :

Learning step & Update step

Learning step : Learning consists of determining a background model used for the comparison of images to be processed.

Update step : The model is then updated with new images to consider changes in the scene

Background Subtraction: Codebook



Background Subtraction: Codebook

Let $\mathbf{x}$ be a training sequence for a single pixel consisting of N RGB-vectors: $\mathbf{x} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$

Let $\mathbf{c} = \{\mathbf{c}_1, \dots, \mathbf{c}_L\}$ represent the codebook for the pixel consisting of L codewords.

Each pixel has a different codebook size based on its sample variation.

Each codeword $\mathbf{c}_i$, $i = 1 \dots L$, consists of an RGB vector $\mathbf{v}_i = (\bar{R}_i, \bar{G}_i, \bar{B}_i)$ and a 6-tuple $\mathbf{aux}_i = \langle \tilde{I}_i, \hat{I}_i, f_i, \lambda_i, p_i, q_i \rangle$

Background Subtraction: Codebook

The tuple aux_i contains intensity (brightness) values and temporal variables described below.

$\tilde{I}, \hat{I}$: the min and max brightness, respectively, that the codeword accepted;
 f : the frequency with which the codeword has occurred;
 λ : the maximum negative run-length (MNRL) defined as the longest interval during the training period that the codeword has NOT recurred;
 p, q : the first and last access times, respectively, that the codeword has occurred.

In the training period, each value, x_t sampled at time t is compared to the current codebook to determine which codeword c , (if any) it matches (m is the matching codeword's index). We use the matched codeword as the sample's encoding approximation. To determine which codeword will be the best match, we employ a color distortion measure and brightness bounds.

Background Subtraction: Codebook

During this learning phase, any new codeword obtained (for a given pixel) is integrated into the background model if it satisfies two conditions expressed by the colordist and brightness functions:

- 1) a brightness distortion constraint
- 2) a color distortion constraint.

These constraints are expressed according to the equations below. For an input pixel : $\mathbf{x}_t = (R, G, B)$ et un codeword c_i possédant un vecteur $\mathbf{v}_i = (\bar{R}_i, \bar{G}_i, \bar{B}_i)$

We have :

$$\begin{aligned}\|\mathbf{x}_t\|^2 &= R^2 + G^2 + B^2 \\ \|\mathbf{v}_i\|^2 &= \bar{R}_i^2 + \bar{G}_i^2 + \bar{B}_i^2 \\ \langle \mathbf{x}_t, \mathbf{v}_i \rangle^2 &= (\bar{R}_i R + \bar{G}_i G + \bar{B}_i B)^2\end{aligned}$$

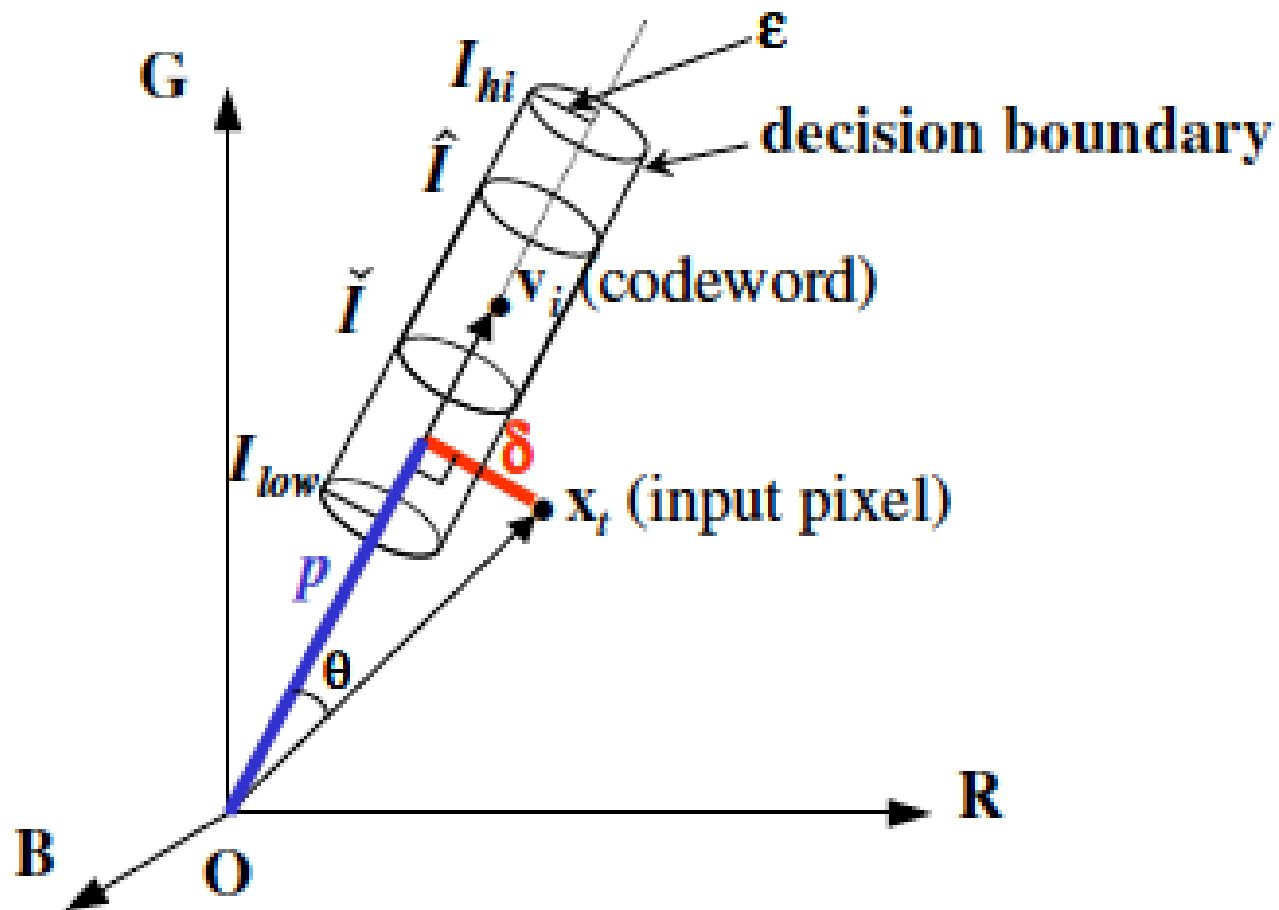
And color distortion can be calculated by:

$$p^2 = \|\mathbf{x}_t\|^2 \cos^2 \theta = \frac{\langle \mathbf{x}_t, \mathbf{v}_i \rangle^2}{\|\mathbf{v}_i\|^2}$$
$$colordist(\mathbf{x}_t, \mathbf{v}_i) = \delta = \sqrt{\|\mathbf{x}_t\|^2 - p^2}$$

The condition for brightness is:

$$brightness(I, \langle \tilde{I}, \hat{I} \rangle) = \begin{cases} \text{true} & \text{if } I_{low} \leq \|\mathbf{x}_t\| \leq I_{hi} \\ \text{false} & \text{otherwise.} \end{cases} \quad \begin{aligned} I_{low} &= \alpha \tilde{I} \\ I_{hi} &= \beta \hat{I} \end{aligned}$$

Background Subtraction: Codebook



Background Subtraction: Codebook

Algorithm for Codebook Construction

- I. $L \leftarrow 0$ ($\leftarrow$ means assignment), $\mathcal{C} \leftarrow \emptyset$ (empty set)
 - II. for $t=1$ to N do
 - i. $\mathbf{x}_t = (R, G, B)$, $I \leftarrow R + G + B$
 - ii. Find the codeword $\mathbf{c}_m$ in $\mathcal{C} = \{\mathbf{c}_i | 1 \leq i \leq L\}$ matching to $\mathbf{x}_t$ based on two conditions (a) and (b).
 - (a) $\text{colordist}(\mathbf{x}_t, \mathbf{v}_m) \leq \epsilon_1$
 - (b) $\text{brightness}(I, \langle \tilde{I}_m, \hat{I}_m \rangle) = \text{true}$
 - iii. If $\mathcal{C} = \emptyset$ or there is no match, then $L \leftarrow L + 1$. Create a new codeword $\mathbf{c}_L$ by setting
 - $\mathbf{v}_L \leftarrow (R, G, B)$
 - $\text{aux}_L \leftarrow \langle I, I, 1, t - 1, t, t \rangle$.
 - iv. Otherwise, update the matched codeword $\mathbf{c}_m$, consisting of $\mathbf{v}_m = (\bar{R}_m, \bar{G}_m, \bar{B}_m)$ and $\text{aux}_m = \langle \tilde{I}_m, \hat{I}_m, f_m, \lambda_m, p_m, q_m \rangle$, by setting
 - $\mathbf{v}_m \leftarrow (\frac{f_m \bar{R}_m + R}{f_m + 1}, \frac{f_m \bar{G}_m + G}{f_m + 1}, \frac{f_m \bar{B}_m + B}{f_m + 1})$
 - $\text{aux}_m \leftarrow \langle \min\{I, \tilde{I}_m\}, \max\{I, \hat{I}_m\}, f_m + 1, \max\{\lambda_m, t - q_m\}, p_m, t \rangle$.
 - end for
 - III. For each codeword $\mathbf{c}_i$, $i = 1 \dots L$, wrap around λ_i by setting $\lambda_i \leftarrow \max\{\lambda_i, (N - q_i + p_i - 1)\}$.
-

Background Subtraction: Codebook

The final decision of the algorithm for a pixel x is then made according to the conditions below :

ϵ_2 is the detection limit and set $\mathcal{M}$ obeys this equation ($T_{\mathcal{M}} = N/2$):

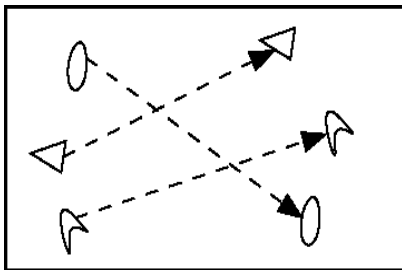
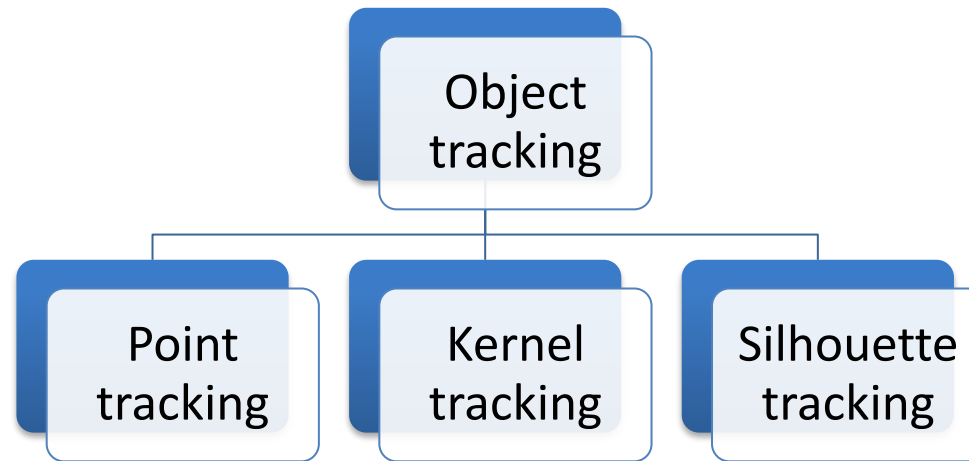
$$\mathcal{M} = \{c_m | c_m \in \mathcal{C} \wedge \lambda_m \leq T_{\mathcal{M}}\}$$

Algorithm for Background Subtraction

- I. $x = (R, G, B), \quad I \leftarrow R + G + B$
 - II. For all codewords in $\mathcal{M}$ in Eq.1, find the codeword c_m matching to x based on two conditions:
 - $colordist(x, v_m) \leq \epsilon_2$
 - $brightness(I, \langle \tilde{I}_m, \hat{I}_m \rangle) = \text{true}$
 - III. $BGS(x) = \begin{cases} \text{foreground} & \text{if there is no match} \\ \text{background} & \text{otherwise.} \end{cases}$
-

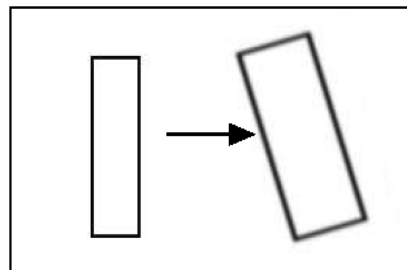
Object tracking

Object tracking



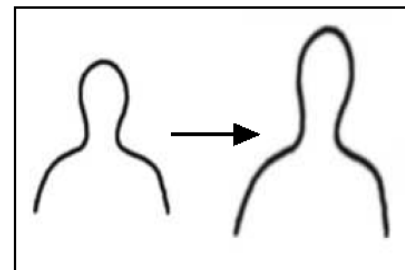
(a)

(a) Points tracking

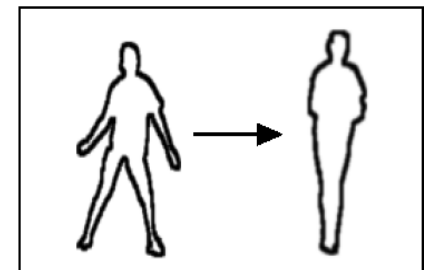


(b)

(b) Kernel tracking



(c)



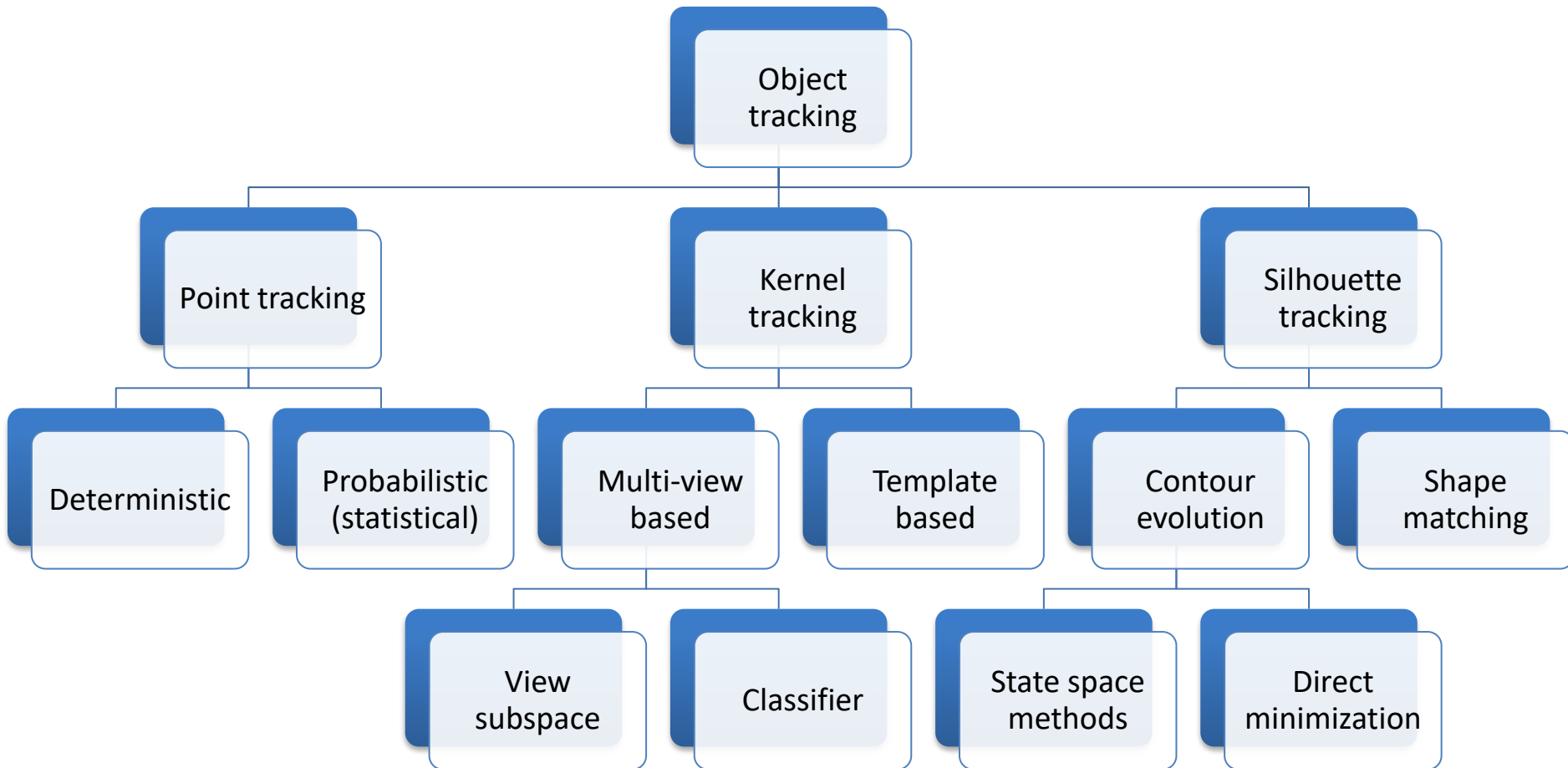
(d)

(c, d) Silhouette tracking

Different tracking approaches

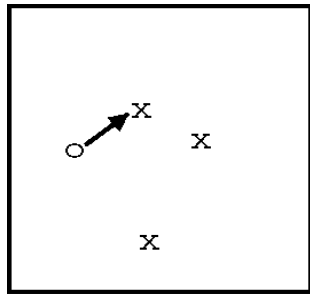
- **Points tracking:** Objects detected in consecutive frames are represented by points, and the association of the points is based on the previous object state (object position and motion).
- **Kernel Tracking:** Kernel refers to the object shape and appearance. Objects are tracked by computing the motion of the kernel in consecutive frames. This motion is usually in the form of a parametric transformation such as translation, rotation, and affine.
- **Silhouette Tracking:** Tracking is performed by estimating the object region in each Frame. Silhouettes are tracked by either shape matching or contour evolution

Object tracking

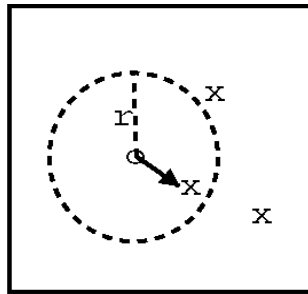


Point tracking: Deterministic methods

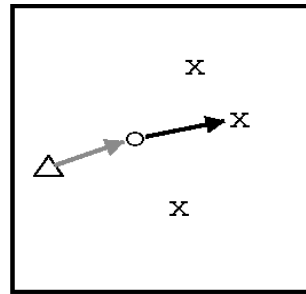
for point correspondence define a cost of associating each object in frame $t - 1$ to a single object in frame t using a set of motion constraints.



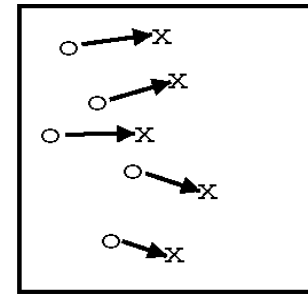
(a)



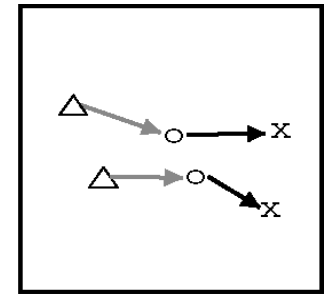
(b)



(c)

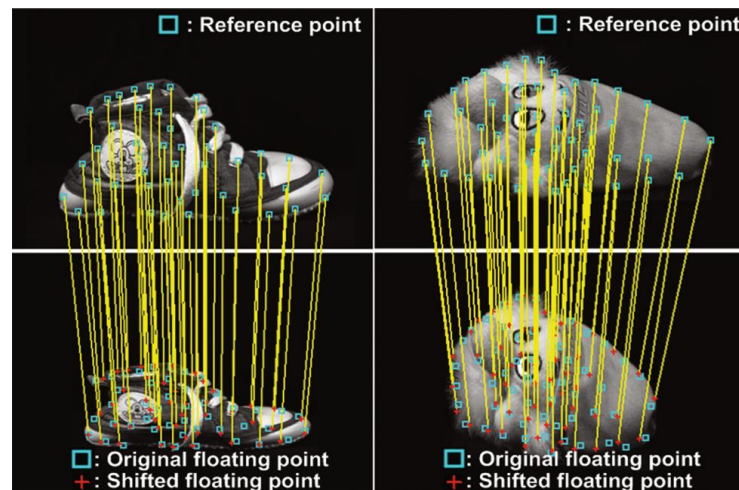


(d)



(e)

- (a) proximity,
- (b) maximum velocity,
- (c) small velocity-change,
- (d) common motion,
- (e) rigidity constraints.

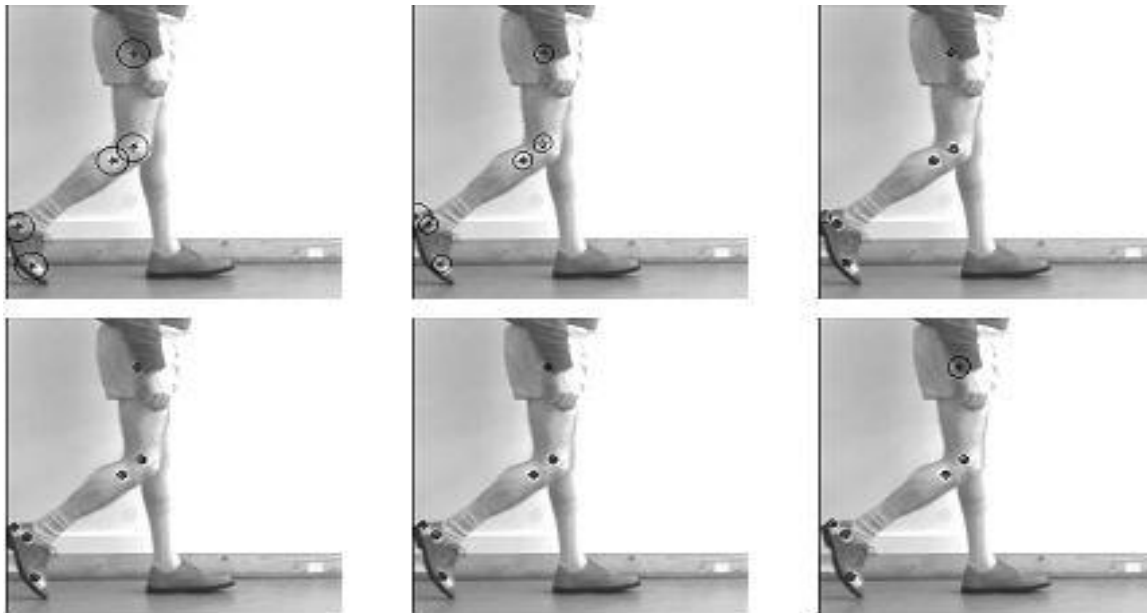


(a)

(b)

Points tracking: Probabilistic methods

The statistical correspondence methods use the state space approach to model the object properties such as position, velocity, and acceleration. This method offer a statistical association between object state spaces. Examples : **the Kalman filter**, **the KLT tracker**, the Particle filter, etc.



Kalman Filter

- Consider motion as a problem where we have to **estimate the values of the variables of some dynamic system**.
- A dynamic system is often described via:
 - a **state vector** $\vec{x}$, also known as the state,
 - a set of equations called the **system model**, which captures the evolution of the state vectors over time.

Kalman Filter : state vector

- The state vector $\mathbf{x}$ is a time-dependent vector $\vec{x}(t) \in R^n$.
- The elements of the vector are variables of the dynamic system.

$$\vec{x}(t) = (q_1(t), q_2(t), \dots, q_n(t))$$

- In case of motion, $\vec{x}(t) = (v_x(t), v_y(t))$.
- How big is n ? As big as necessary in order to capture all the dynamic properties of the system.
- Example1: 3D motion $\vec{x}(t) = (v_x(t), v_y(t), v_z(t))$
- Example2: multiple moving objects, e.g. four objects moving on a plane (2D motion).

$$\begin{aligned}\vec{x}(t) &= (\vec{x}_1(t), \vec{x}_2(t), \vec{x}_3(t), \vec{x}_4(t)) \\ &= (v_{1x}(t), v_{1y}(t), v_{2x}(t), v_{2y}(t), v_{3x}(t), v_{3y}(t), v_{4x}(t), v_{4y}(t))\end{aligned}$$

- Assume that we observe the system at discrete, equally spaced time intervals so that:

$$t_k = t_0 + k\delta t$$

where $k = 0, 1, \dots$ and δt is the sampling interval

- For simplicity $\vec{x}(t_k)$ is denoted as $\vec{x}_k$.
- Assumption: δt is small enough to capture the dynamics of the system. In other words, the system does not change much between consecutive time instants, i.e. during δt .

Kalman Filter : system model

- Key Assumption: The system is **linear**. That means that the relationship between consecutive state-changes is linear.
- Then the system model can be written as:

$$\vec{x}_k = \Phi_{k-1} \vec{x}_{k-1} + \vec{w}_{k-1}$$

- $\vec{w}_{k-1}$ is a vector describing the **random process noise**.
- Φ_{k-1} is the **state transition matrix** that captures the relationship between the current state k and the previous state $k-1$ in the absence of noise.
- Φ_{k-1} is an $n \times n$ matrix, $\vec{w}_{k-1}$ is an n -dimensional vector.

Kalman Filter : measurements

- At any time t_k , we have a vector $\vec{z}_k \in R^m$ of measurements of the system.
- Due to imperfections (e.g. noise) in our sensors, there is uncertainty in our measurements.
- The vector $\vec{\mu}_k$ describes the uncertainty associated with each measurement $\vec{z}_k$.
- The relationship between the true system state $\vec{x}_k$ and our measurements is given by the following equation:

$$\vec{z}_k = \mathbf{H}_k \vec{x}_k + \vec{\mu}_k$$

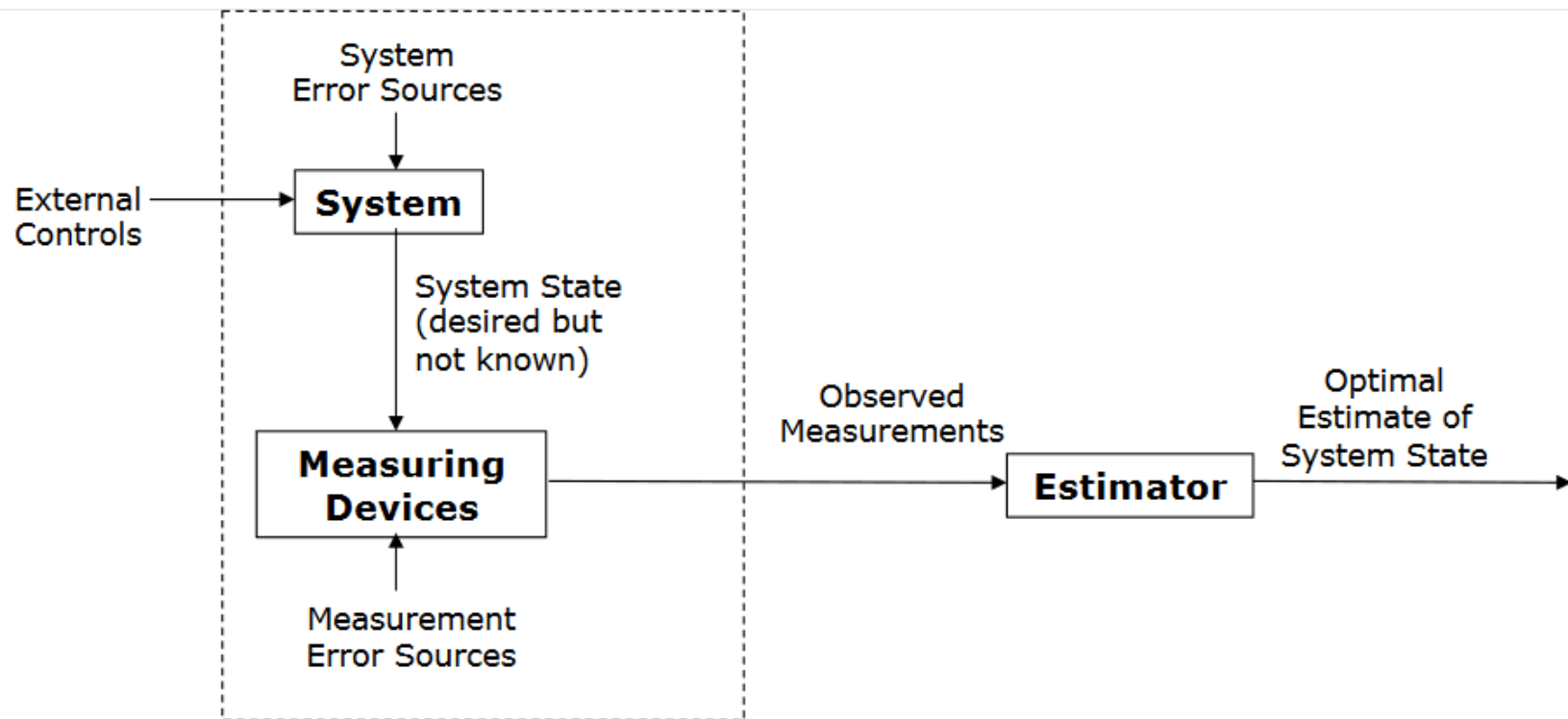
- $\mathbf{H}_k$ is the **measurement matrix** that captures the relationship between our measurements and the real system variables in the absence of noise. It is an $m \times n$ matrix.
- $\vec{\mu}_k$ is an m -dimensional vector known as the **measurement noise**.

- There are two types of noise:
 - Process noise $\vec{w}_k$
 - Measurement noise $\vec{u}_k$
- In Kalman filtering both types of noise are assumed to be white, zero-mean Gaussians.
- As such they are described by their corresponding covariance matrices:
 - Process noise covariance $\mathbf{Q}_k$
 - Measurement noise covariance $\mathbf{R}_k$

Kalman Filter : notations

■ State variable	$\vec{x}_k$
■ State transition matrix	Φ_k
■ Process noise	$\vec{w}_k$
■ Process noise covariance	$\mathbf{Q}_k$
■ Measurement	$\vec{z}_k$
■ Measurement matrix	$\mathbf{H}_k$
■ Measurement noise	$\vec{\mu}_k$
■ Measurement noise covariance	$\mathbf{R}_k$

Kalman Filter



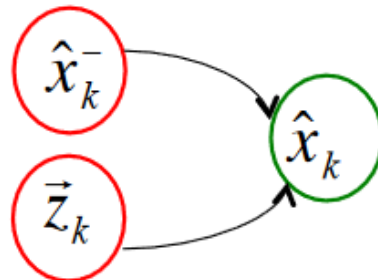
- So far we have setup our variables and equations to describe a linear dynamic system that is measured by some sensors.
- Goal: Compute the best estimate of the system state $\hat{\mathbf{x}}_k$ at time t_k given the previous state estimate $\hat{\mathbf{x}}_{k-1}$ and the current measurements $\mathbf{z}_k$.

Kalman Filter : KF idea

- An estimate of $\hat{\vec{x}}_k$ ($\hat{x}_k$) is obtained from $\hat{\vec{x}}_{k-1}$ ($\hat{x}_{k-1}$) and $\vec{z}_k$ in a 2-step process:
 1. First, obtain an intermediate estimate, $\hat{\vec{x}}_k^-$, based on the previous estimates, but *without* using the newest measurements $\vec{z}_k$.

$$\hat{\vec{x}}_k^- = \Phi_{k-1} \hat{\vec{x}}_{k-1}$$

- It is called the **prediction step**. It predicts what the state variable should be based purely on our model.
- 2. Use the intermediate estimate $\hat{\vec{x}}_k^-$ and combine it with the newest measurements $\vec{z}_k$, to get $\hat{\vec{x}}_k$.



Kalman Filter : KF idea

- This 2-step process is performed as a series of 4 (or 5) recursive equations.
- The 4 (or 5) Kalman Filter equations are characterized by:
 1. The **state covariance matrix** $\mathbf{P}_k$. It is the covariance matrix of the estimate $\hat{x}_k$. It is also known as the **covariance of the estimates**. It is a measurement of the uncertainty in $\hat{x}_k$.
 2. The **state covariance matrix** $\mathbf{P}_k^-$. It is the covariance matrix of the estimate $\hat{x}_k^-$. It is also known as the **covariance of the prediction error**. It is a measurement of the uncertainty in $\hat{x}_k^-$.
 3. The **gain matrix** $\mathbf{K}_k$. It expresses the relative importance of the prediction $\hat{x}_k^-$ and the measurement $\vec{z}_k$.

Kalman Filter

■ State variable	$\vec{x}_k$
■ State transition matrix	Φ_k
■ Process noise	$\vec{w}_k$
■ Process noise covariance	$\mathbf{Q}_k$
■ Covariance of the estimates	$\mathbf{P}_k$
■ Covariance of the prediction	$\mathbf{P}_k^-$
■ Gain Matrix	$\mathbf{K}_k$
■ Measurement	$\vec{z}_k$
■ Measurement matrix	$\mathbf{H}_k$
■ Measurement noise	$\vec{\mu}_k$
■ Measurement noise covariance	$\mathbf{R}_k$

Kalman Filter : equations

■ Prediction equations

$$\hat{\mathbf{x}}_k^- = \Phi_{k-1} \hat{\mathbf{x}}_{k-1}$$

$$\mathbf{P}_k^- = \Phi_{k-1} \mathbf{P}_{k-1} \Phi_{k-1}^T + \mathbf{Q}_k$$

- Project state and covariance estimates forward in time

■ Update equations

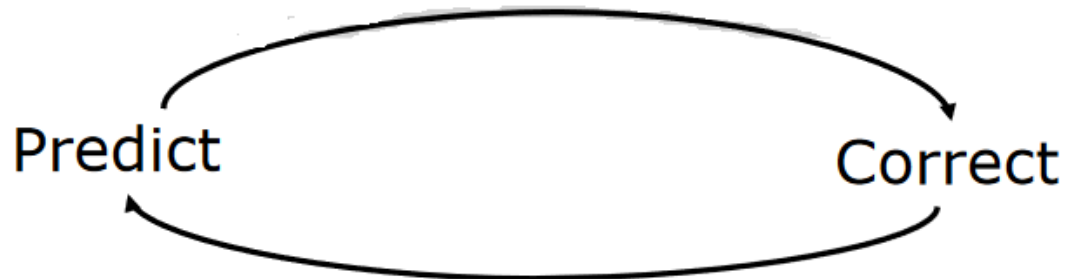
$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^T + \mathbf{R}_k)^{-1}$$

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k (\bar{\mathbf{z}}_k - \mathbf{H}_k \hat{\mathbf{x}}_k^-)$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^- (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^T + \mathbf{K}_k \mathbf{R}_k \mathbf{K}_k^T$$

- Compute Kalman gain $\mathbf{K}$
- Include the measurement
- Compute a posteriori estimate
- Compute a posteriori covariance of the estimate

Kalman Filter : equations



$$\mathbf{P}_k^- = \Phi_{k-1} \mathbf{P}_{k-1} \Phi_{k-1}^T + \mathbf{Q}_k$$

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^T + \mathbf{R}_k)^{-1}$$

$$\hat{\mathbf{x}}_k = \Phi_{k-1} \hat{\mathbf{x}}_{k-1} + \mathbf{K}_k (\vec{z}_k - \mathbf{H}_k \Phi_{k-1} \hat{\mathbf{x}}_{k-1})$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^- (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^T + \mathbf{K}_k \mathbf{R}_k \mathbf{K}_k^T$$

$\Phi_{k-1} \hat{\mathbf{x}}_{k-1}$ is the prediction

$\vec{z}_k - \mathbf{H}_k \Phi_{k-1} \hat{\mathbf{x}}_{k-1}$ is the innovation

$\hat{\mathbf{x}}_k = \Phi_{k-1} \hat{\mathbf{x}}_{k-1} + \mathbf{K}_k (\vec{z}_k - \mathbf{H}_k \Phi_{k-1} \hat{\mathbf{x}}_{k-1})$ is the update

Kalman Filter : notations

- If the measurement noise is much greater than the process noise,

$$\mathbf{R}_k \gg \mathbf{Q}_k$$

$\mathbf{K}_k$ will be small (that is, we won't give much credence to the measurement).

- If the measurement noise is much smaller than the process noise,

$$\mathbf{R}_k \ll \mathbf{Q}_k$$

$\mathbf{K}_k$ will be large (that is, we don't trust our model too much).

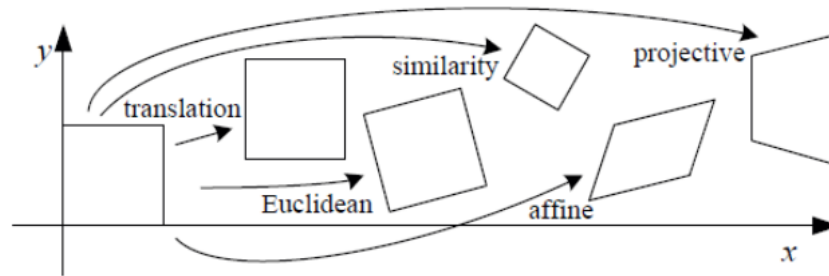
- The method assumes initial estimates of $\mathbf{P}_0$ and $\hat{x}_0$.
- Typically, the entries in $\mathbf{P}_0$ are set to arbitrary high values. We set $\mathbf{P}_0$ to arbitrarily high values because we don't trust our initial estimates. Hence, the estimate error is expected to be high.
- For $\hat{x}_0$, if we have some data, we use it, otherwise we set that, too, to arbitrary values.

KLT(Kanade-Lucas-Tomasi) Tracker

1. Detect Harris corners in the first frame
2. For each Harris corner compute motion (translation or affine) between consecutive frames.
3. Link motion vectors in successive frames to get a track for each Harris point
4. Introduce new Harris points by applying Harris detector at every m (10 or 15) frames
5. Track new and old Harris points using steps 1-3.

The KLT tracker : how to estimate alignment

Basic Set of 2-D Transformation



Summary of Displacement Models (2-D Transformations)

Translation	$x' = x + b_1$ $y' = y + b_2$	Bi-quadratic	$x' = a_1 + a_2x + a_3y + a_4x^2 + a_5y^2 + a_6xy$ $y' = a_7 + a_8x + a_9y + a_{10}x^2 + a_{11}y^2 + a_{12}xy$
Rigid	$x' = x \cos \theta - y \sin \theta + b_1$ $y' = x \sin \theta + y \cos \theta + b_2$	Bi-Linear	$x' = a_1 + a_2x + a_3y + a_4xy$ $y' = a_5 + a_6x + a_7y + a_8xy$
Affine	$x' = a_1x + a_2y + b_1$ $y' = a_3x + a_4y + b_2$	Pseudo-Perspective	$x' = a_1 + a_2x + a_3y + a_4x^2 + a_5xy$ $y' = a_6 + a_7x + a_8y + a_4xy + a_5y^2$
Projective	$x' = \frac{a_1x + a_2y + b_1}{c_1x + c_2y + 1}$ $y' = \frac{a_3x + a_4y + b_2}{c_1x + c_2y + 1}$		

Displacement Models Parameterizations

Translation

$$x' = x + b_1$$

$$y' = y + b_2$$

$$W(\mathbf{x}; \mathbf{p}) = (x + b_1, y + b_2)$$

Rigid

$$x' = x \cos \theta - y \sin \theta + b_1$$

$$y' = x \sin \theta + y \cos \theta + b_2$$

$$W(\mathbf{x}; \mathbf{p}) = (x \cos \theta - y \sin \theta + b_1, x \sin \theta + y \cos \theta + b_2)$$

Affine

$$x' = a_1 x + a_2 y + b_1$$

$$y' = a_3 x + a_4 y + b_2$$

$$W(\mathbf{x}; \mathbf{p}) = (a_1 x + a_2 y + b_1, a_3 x + a_4 y + b_2)$$

Homogenous coordinates



$$W(\mathbf{x}; \mathbf{p}) = \begin{bmatrix} 1 & 0 & b_1 \\ 0 & 1 & b_2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$W(\mathbf{x}; \mathbf{p}) = \begin{bmatrix} c\theta & -s\theta & b_1 \\ s\theta & c\theta & b_2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$W(\mathbf{x}; \mathbf{p}) = [R | t]_{2 \times 3} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$W(\mathbf{x}; \mathbf{p}) = \begin{bmatrix} a_1 & a_2 & b_1 \\ a_3 & a_4 & b_2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$W(\mathbf{x}; \mathbf{p}) = A_{2 \times 3} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

The KLT tracker

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} I & & t \end{bmatrix}_{2 \times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} R & & t \end{bmatrix}_{2 \times 3}$	3	lengths	
similarity	$\begin{bmatrix} sR & & t \end{bmatrix}_{2 \times 3}$	4	angles	
affine	$\begin{bmatrix} A \end{bmatrix}_{2 \times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines	

Derivative & Gradient

Function: $f(x)$

Derivative: $f'(x) = \frac{df}{dx}$, x is a scalar

Function: $f(x_1, x_2, \dots, x_n)$

Gradient: $\nabla f(x_1, x_2, \dots, x_n) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$

Jacobian

$$F(x_1, x_2, \dots, x_n) = (f_1(x_1, x_2, \dots, x_n), f_2(x_1, x_2, \dots, x_n), \dots, f_m(x_1, x_2, \dots, x_n))$$

Vector Valued Function

Derivative?

$$J(F) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \dots & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$



Carl Gustav Jacob Jacobi
10 December 1804—
18 February 1851

Displacement Model Jacobians

Translation

$$W(\mathbf{x}; \mathbf{p}) = (x + b_1, y + b_2) \quad \frac{\partial W}{\partial \mathbf{p}}?$$

Rigid

$$\frac{\partial W}{\partial \mathbf{p}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$W(\mathbf{x}; \mathbf{p}) = (x \cos \theta - y \sin \theta + b_1, x \sin \theta + y \cos \theta + b_2)$$

$$\frac{\partial W}{\partial \mathbf{p}} = \begin{bmatrix} 1 & 0 & -x \sin \theta - y \cos \theta \\ 0 & 1 & x \cos \theta - y \sin \theta \end{bmatrix}$$

Affine

$$W(\mathbf{x}; \mathbf{p}) = (a_1 x + a_2 y + b_1, a_3 x + a_4 y + b_2)$$

$$\frac{\partial W}{\partial \mathbf{p}} = \begin{bmatrix} 1 & 0 & x & y & 0 & 0 \\ 0 & 1 & 0 & 0 & x & y \end{bmatrix}$$

The KLT tracker

Transformation	Matrix	# DoF	Preserves	Icon	Parameters p	Jacobian J
translation	$\begin{bmatrix} I & t \end{bmatrix}_{2 \times 3}$	2	orientation		(t_x, t_y)	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$
rigid (Euclidean)	$\begin{bmatrix} R & t \end{bmatrix}_{2 \times 3}$	3	lengths		(t_x, t_y, θ)	$\begin{bmatrix} 1 & 0 & -s_\theta x - c_\theta y \\ 0 & 1 & c_\theta x - s_\theta y \end{bmatrix}$
similarity	$\begin{bmatrix} sR & t \end{bmatrix}_{2 \times 3}$	4	angles		(t_x, t_y, a, b)	$\begin{bmatrix} 1 & 0 & x & -y \\ 0 & 1 & y & x \end{bmatrix}$
affine	$\begin{bmatrix} A \end{bmatrix}_{2 \times 3}$	6	parallelism		$(t_x, t_y, a_{00}, a_{01}, a_{10}, a_{11})$	$\begin{bmatrix} 1 & 0 & x & y & 0 & 0 \\ 0 & 1 & 0 & 0 & x & y \end{bmatrix}$
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines		$(h_{00}, h_{01}, \dots, h_{21})$	(see Section 6.1.3)

Finding Alignment

Find $\mathbf{p}$ s.t. following is minimized

$$\sum_{\mathbf{x}} [I(W(\mathbf{x}; \mathbf{p})) - T(\mathbf{x})]^2$$

Assume initial estimate of $\mathbf{p}$ is known, find $\Delta \mathbf{p}$

$$\sum_{\mathbf{x}} [I(W(\mathbf{x}; \mathbf{p} + \Delta \mathbf{p})) - T(\mathbf{x})]^2$$

Find Taylor Series

$$\sum_{\mathbf{x}} [I(W(\mathbf{x}; \mathbf{p})) + \nabla I \frac{\partial W}{\partial \mathbf{p}} \Delta \mathbf{p} - T(\mathbf{x})]^2 \quad \nabla I = \begin{bmatrix} I_x & I_y \end{bmatrix}$$

The KLT tracker

Differentiate wrt $\Delta \mathbf{p}$ and equate it to zero

$$2 \sum_{\mathbf{x}} \left[\nabla I \frac{\partial W}{\partial \mathbf{p}} \right]^T [I(W(\mathbf{x}; \mathbf{p})) + \nabla I \frac{\partial W}{\partial \mathbf{p}} \Delta \mathbf{p} - T(\mathbf{x})]$$

And equate it to zero to find

$$2 \sum_{\mathbf{x}} \left[\nabla I \frac{\partial W}{\partial \mathbf{p}} \right]^T [I(W(\mathbf{x}; \mathbf{p})) + \nabla I \frac{\partial W}{\partial \mathbf{p}} \Delta \mathbf{p} - T(\mathbf{x})] = 0$$

$$\Delta \mathbf{p} = H^{-1} 2 \sum_{\mathbf{x}} \left[\nabla I \frac{\partial W}{\partial \mathbf{p}} \right]^T [T(\mathbf{x}) - I(W(\mathbf{x}; \mathbf{p}))]$$

$$H^{-1} = \sum_{\mathbf{x}} \left[\nabla I \frac{\partial W}{\partial \mathbf{p}} \right]^T \left[\nabla I \frac{\partial W}{\partial \mathbf{p}} \right]$$

Hessian for Translation Motion

$$H^{-1} = \sum_{\mathbf{x}} \left[\nabla I \frac{\partial W}{\partial \mathbf{p}} \right]^T \left[\nabla I \frac{\partial W}{\partial \mathbf{p}} \right]$$

$$\frac{\partial W}{\partial \mathbf{p}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$H^{-1} = \sum_{\mathbf{x}} \begin{bmatrix} \left(\frac{\partial I}{\partial x} \right)^2 & \frac{\partial I}{\partial x} \frac{\partial I}{\partial y} \\ \frac{\partial I}{\partial x} \frac{\partial I}{\partial y} & \left(\frac{\partial I}{\partial y} \right)^2 \end{bmatrix}$$

$$\nabla I = \begin{bmatrix} I_x & I_y \end{bmatrix}$$

Harris Corner detector

The KLT tracker

$$\Delta p = H^{-1} \sum_x \left[\nabla I \frac{\partial W}{\partial \mathbf{p}} \right]^T [T(\mathbf{x}) - I(W(\mathbf{x}; \mathbf{p}))]$$

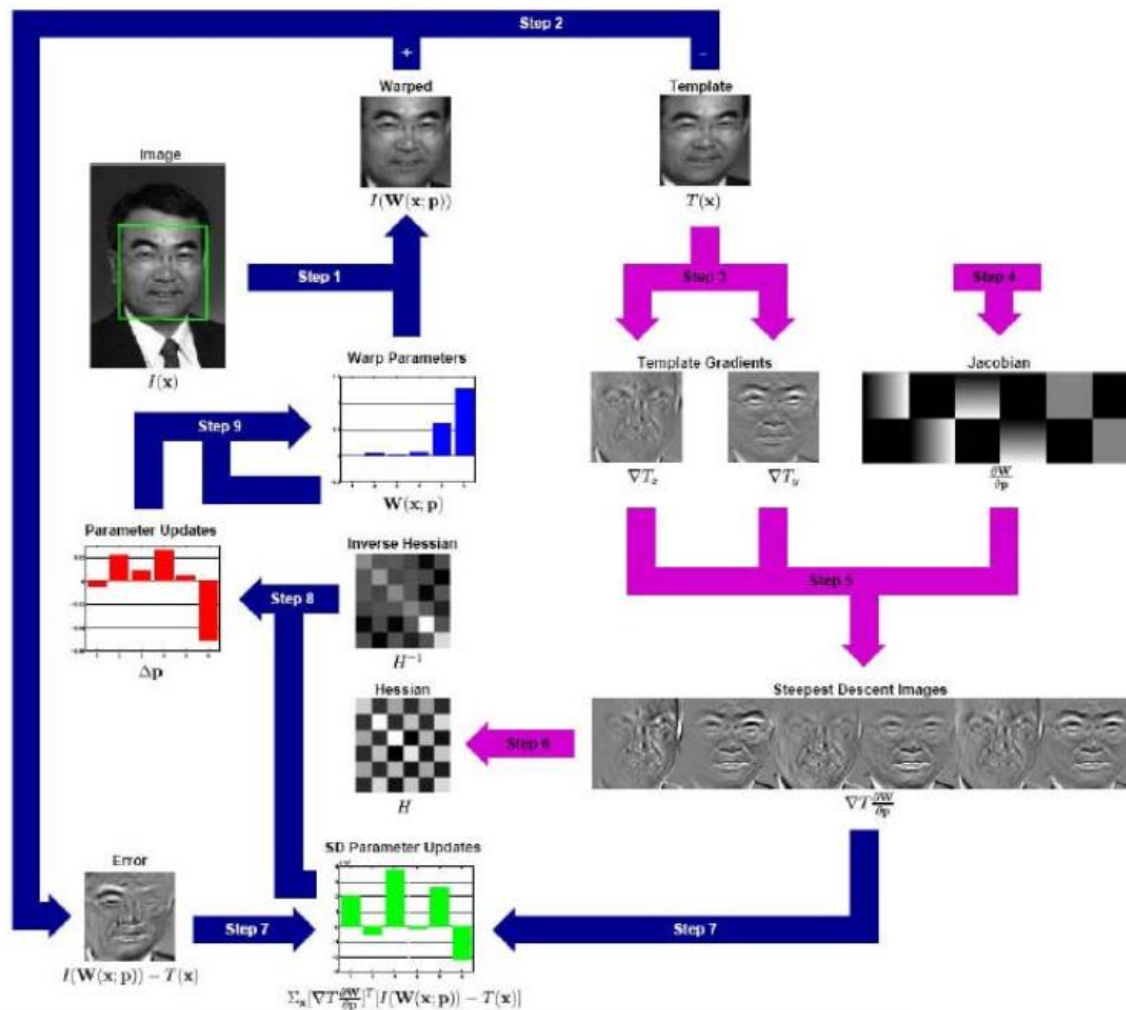
1. Warp I with $W(\mathbf{x}; \mathbf{p})$
2. Subtract I from T $[T(\mathbf{x}) - I(W(\mathbf{x}; \mathbf{p}))]$
3. Compute gradient ∇I
4. Evaluate the Jacobian $\frac{\partial W}{\partial \mathbf{p}}$
5. Compute steepest descent $\nabla I \frac{\partial W}{\partial \mathbf{p}}$ $W(\mathbf{x}; \mathbf{p})$
6. Compute Inverse Hessian H^{-1}
7. Multiply steepest descent with error $\sum_x \left[\nabla I \frac{\partial W}{\partial \mathbf{p}} \right]^T [T(\mathbf{x}) - I(W(\mathbf{x}; \mathbf{p}))]$
8. Compute $\Delta \mathbf{p}$
9. Update parameters $\mathbf{p} \rightarrow \mathbf{p} + \Delta \mathbf{p}$

The KLT tracker

$$\Delta p = H^{-1} \sum_{\mathbf{x}} \left[\nabla T \frac{\partial W}{\partial p} \right]^T [I(W(\mathbf{x}; \mathbf{p})) - T(\mathbf{x})]$$

1. Warp I with $W(\mathbf{x}; \mathbf{p})$
2. Subtract T from I $[I(W(\mathbf{x}; \mathbf{p})) - T(\mathbf{x})]$
3. Compute gradient ∇T
4. Evaluate the Jacobian $\frac{\partial W}{\partial p}$
5. Compute steepest descent $\nabla T \frac{\partial W}{\partial p}$
6. Compute Inverse Hessian H^{-1}
7. Multiply steepest descent with error $\sum_{\mathbf{x}} \left[\nabla T \frac{\partial W}{\partial \mathbf{p}} \right]^T [I(W(\mathbf{x}; \mathbf{p})) - T(\mathbf{x})]$
8. Compute $\Delta \mathbf{p}$ $\Delta p = H^{-1} \sum_{\mathbf{x}} \left[\nabla T \frac{\partial W}{\partial p} \right]^T [I(W(\mathbf{x}; \mathbf{p})) - T(\mathbf{x})]$
9. Update parameters $\mathbf{p} \rightarrow \mathbf{p} + \Delta \mathbf{p}$

Finding Alignment



Object tracking: kernel tracking

- Kernel tracking is typically performed by computing the motion of the object, which is represented by a primitive object region, from one frame to the next.
- The object motion is generally in the form of parametric motion (translation, affine, etc.) or the dense flow field computed in subsequent frames.
- These algorithms differ in terms of the
 - appearance representation used,
 - the number of objects tracked,
 - the method used to estimate the object motion.
- We divide these tracking methods into two subcategories
 - based on the appearance representation (templates) or density-based appearance models,
 - multiview appearance models (The objects may appear different from different views)

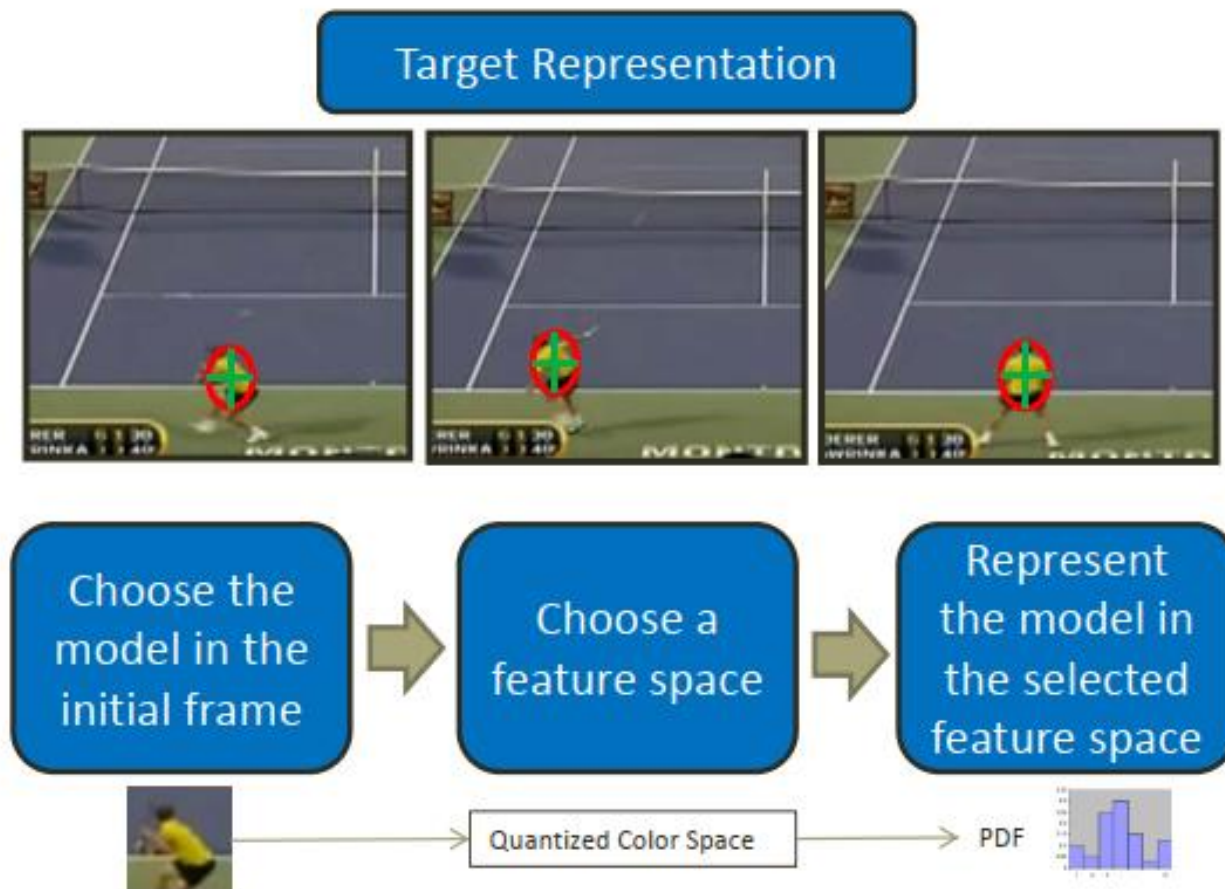
Kernel tracking: template based methods

- Template matching is a method of searching in the image, I_w , for a region similar to the object template, O_t defined in the previous frame. The position of the template in the current image is computed by a similarity measure (cross correlation).
- Instead of templates, other object representations can be used for tracking, color histograms or mixture models can be computed by using the appearance of pixels inside the rectangular or ellipsoidal regions.
- Once the new location of the interest point is obtained, the the affine Transformation is computed between the corresponding patches in consecutive frames.



Kernel tracking: density based method

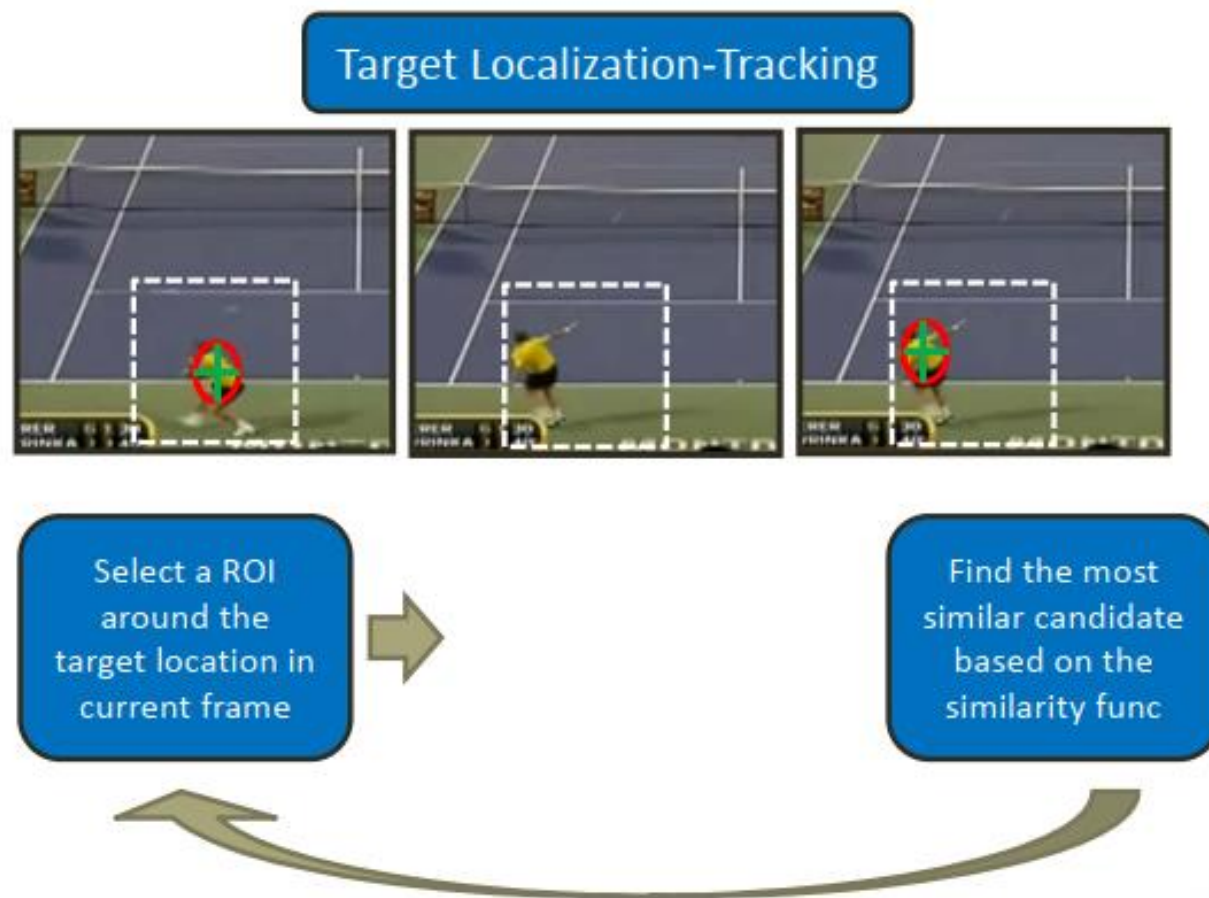
- General Framework



- The object is being modeled using color probability density

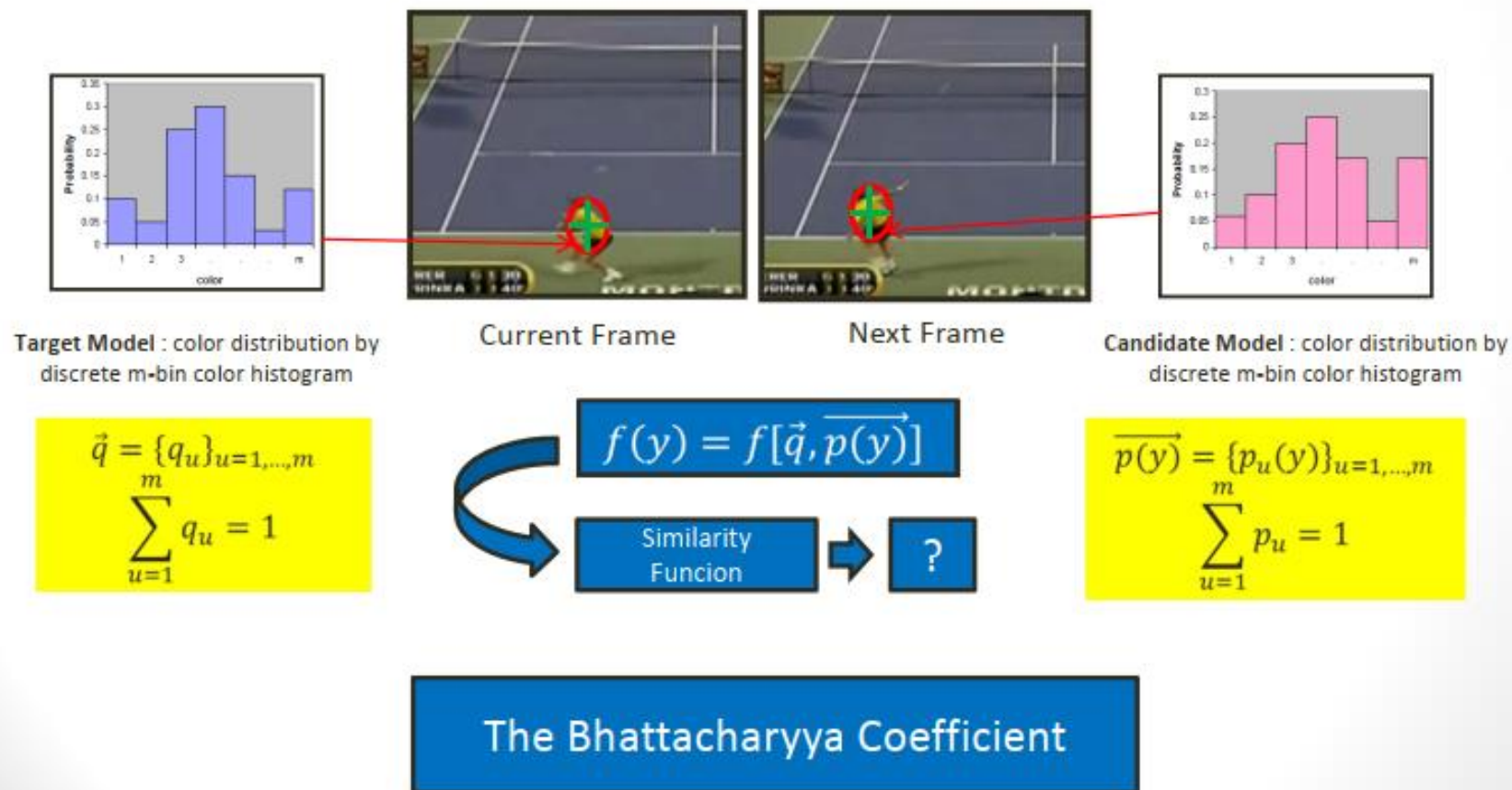
Kernel tracking: density based method

- General framework



Kernel tracking: density based method

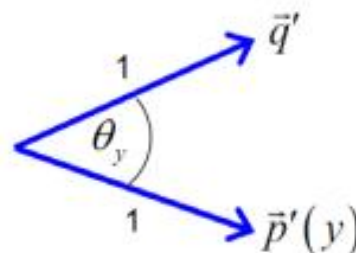
- PDF Representation



Kernel tracking: density based method

- The Bhattacharyya Coefficient
 - Measures similarity between object model q and color p of target at location y

$$\rho(p(y), q) = \sum_{u=1}^m \sqrt{p_u(y) q_u}$$



- ρ is the cosine of vectors $(\sqrt{p_1}, \dots, \sqrt{p_m})^T$ and $(\sqrt{q_1}, \dots, \sqrt{q_m})^T$.
- Large ρ means good match between candidate and target model
- In order to find the new target location we try to maximize the Bhattacharyya coefficient



Kernel tracking: densitybased method

Target : $\mathbf{q}_u$.
Candidate : $\hat{\mathbf{p}}_u$.

$$d(\mathbf{y}) = \sqrt{1 - \rho(\mathbf{y})}$$

$$\rho(\mathbf{y}) = \rho[\hat{\mathbf{p}}(\mathbf{y}), \mathbf{q}] = \sum_{u=1}^m \sqrt{\hat{p}_u(\mathbf{y}) q_u}$$

$\rho(\mathbf{y})$: Bhattacharya coefficient.

Minimizing the distance corresponds to *maximizing* Bhattacharya coefficient.

$$\rho[\hat{\mathbf{p}}(\mathbf{y}), \mathbf{q}] = \sum_{u=1}^m \sqrt{\hat{p}_u(\mathbf{y}) q_u}$$

Taylor expansion around $\hat{\mathbf{p}}(\mathbf{y}_0)$

$$\rho[\hat{\mathbf{p}}(\mathbf{y}), \mathbf{q}] \cong \rho[\hat{\mathbf{p}}(\mathbf{y}_0), \mathbf{q}] + \frac{1}{2} \sum_{i=1}^m \hat{p}_u(\mathbf{y}) \sqrt{\frac{q_u}{\hat{p}_u(\mathbf{y}_0)}}$$

Maximizing Bhattacharya coefficient can be obtained by *maximizing the blue term*.

Kernel tracking: density based method

$$\rho[\hat{\mathbf{p}}(\mathbf{y}), \mathbf{q}] \cong \rho[\hat{\mathbf{p}}(\mathbf{y}_0), \mathbf{q}] + \frac{1}{2} \sum_{i=1}^m \hat{p}_u(\mathbf{y}) \sqrt{\frac{q_u}{\hat{p}_u(\mathbf{y}_0)}}$$



$$\frac{C_h}{2} \sum_{i=1}^{n_x} \left[\sum_{u=1}^m \delta[S(\mathbf{x}_i) - u] \sqrt{\frac{q_u}{\hat{p}_u(\mathbf{y}_0)}} \right] k\left(\left\| \frac{\mathbf{y} - \mathbf{x}_i}{h} \right\|\right)$$



likelihood maximization
depends on maximizing w_j .

h : radius of sphere
 C_h : normalization constant
 $S(\mathbf{x}_i)$: gray level at $\mathbf{x}_i$
 $\mathbf{y}$: kernel center
 m : number of bins

Kernel tracking: density based method

Maximization of the likelihood of target and candidate depends on the weights:

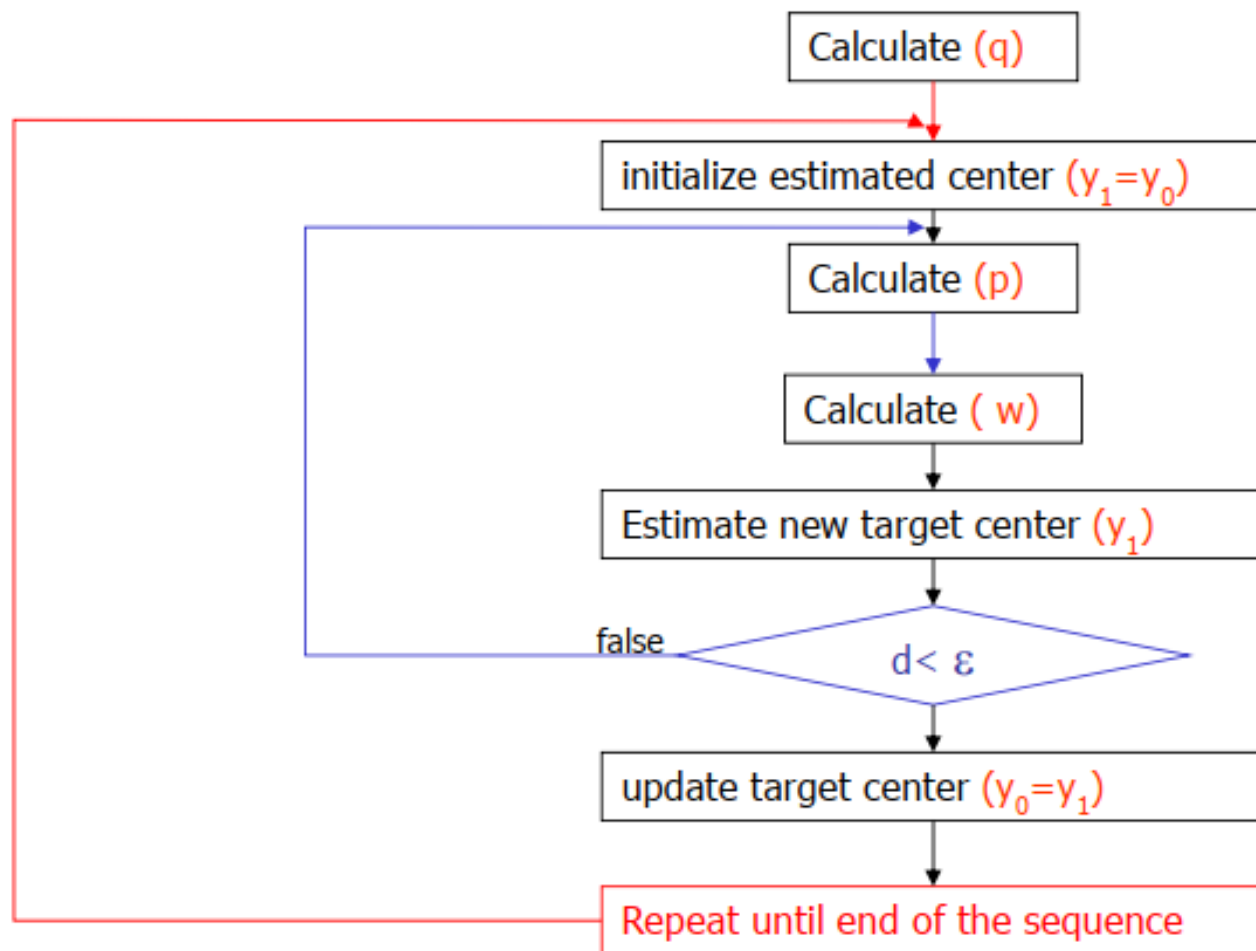
$$w_i(\mathbf{y}_o) = \sum_{u=1}^m \delta[S(\mathbf{x}_i) - u] \sqrt{\frac{q_u}{\hat{p}_u(\mathbf{y}_o)}} \quad \text{where } 0 \leq w_i \leq 1$$

Since $\sum_{i=1}^{n_x} w_i(\mathbf{y}_o)$ is strictly positive, mean shift vector can be written as

$$M_h(\mathbf{y}_o) = \frac{\sum_{i=1}^{n_x} w_i(\mathbf{y}_o) \mathbf{x}_i}{\sum_{i=1}^{n_x} w_i(\mathbf{y}_o)} - \mathbf{y}_o$$

Thus, new target center is $\hat{\mathbf{y}} = \mathbf{y}_o + M_h(\mathbf{y}_o)$

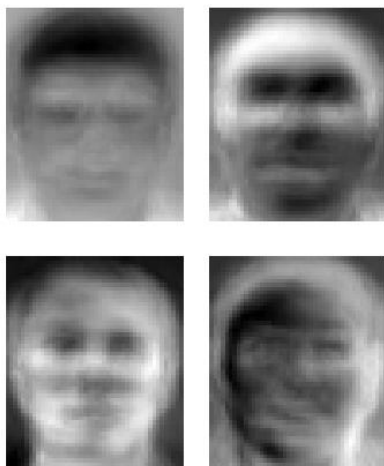
Kernel tracking: density based method



Kernel tracking: multi-view appearance methods

View subspaces

- A subspace is defined, for example eigenspace
- The use of eigenspace for similarity computation is a useful alternative to standard template matching techniques such as normalized correlation.
- The eigenspace-based similarity computation is equivalent to matching with a linear combination of eigen templates. This allows for distortions in the templates, for example, distortion caused by illumination changes in images.



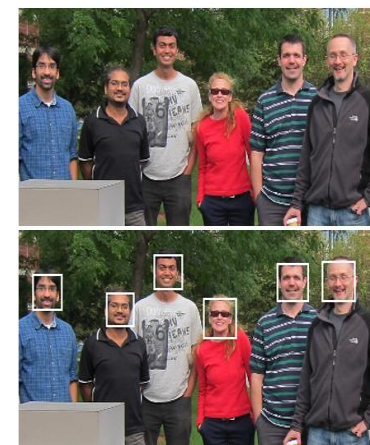
Eigenfaces

Classifier

- Positive samples consists of the images of the object to be tracked
- Negative samples consists of images of background objects
- The algorithm maximizes the SVM classification score over image region to estimate the object position
- Knowledge about the background objects is explicitly integrated in the tracker.



Positive samples



Object tracking: silhouette tracking

- Objects may have complex shapes, for example, hands, head, and shoulders that cannot be well described by simple geometric shapes.
- Silhouette based methods provide an accurate shape description for these objects.
- The goal of a silhouette-based object tracker is to find the object region in each frame by means of an object model generated using the previous frames. This model can be in the form of a color histogram, object edges or the object contour.
- We divide silhouette trackers into two categories,
 - shape matching: Shape matching approaches search for the object silhouette in the current frame.
 - contour tracking: Contour tracking approaches evolve an initial contour to its new position in the current frame by either using the state space models or direct minimization of some energy

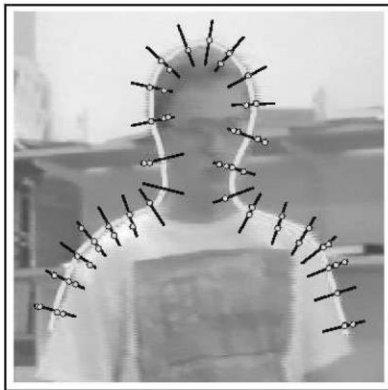
Silhouette tracking: Shape tracking

- Shape matching can be performed similar to tracking based on template matching
- The Hausdorff distance is used to construct a correlation surface from which the minimum is selected as the new object position.
- Emphasize parts of the object that are not drastically affected by object motion. Examples of a person walking: head and torso vs. legs and arms

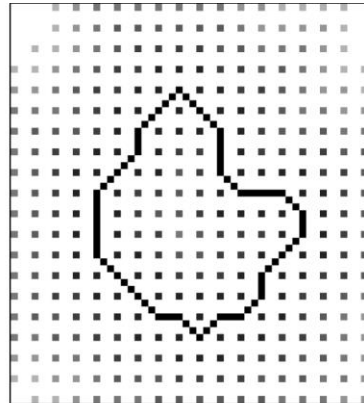
Silhouette tracking: contour tracking

State space model

- State in term of shape and motion parameters
- Control points of the contour moves on the contour on the string stiffness parameters
- Measurements consist of the image edges computed in the normal direction of the contour



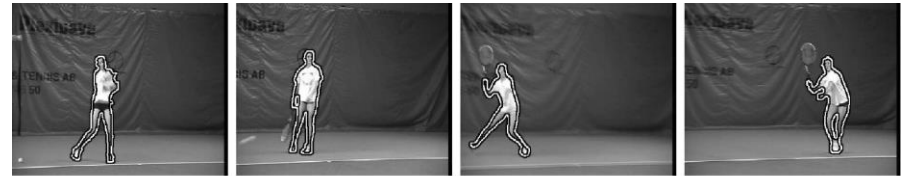
(a)



(b)

Direct minimization

- Direct minimization algorithm : gradient descend
- To find a local minimum of a function using gradient descent, one takes steps proportional to the negative of the gradient of the function



(a)



(b)